O'REILLY®

Evals for AI Engineers

Systematically Measuring and Improving AI Applications



Shreya Shankar
& Hamel Husain

Preface 前言

Large language models have moved from research curiosity to production reality with remarkable speed. Organizations now routinely embed LLMs in customer service systems, content generation pipelines, decision support tools, and information extraction workflows. Yet this rapid adoption has outpaced our ability to systematically evaluate whether these systems actually work.

大型语言模型以惊人的速度从研究热点走向了实际应用。如今，各组织机构已将大型语言模型常规地嵌入到客户服务系统、内容生成流程、决策支持工具和信息提取工作流程中。然而，这种快速普及的速度已经超过了我们系统性评估这些系统实际效果的能力。

This book addresses that gap.

本书旨在填补这一空白。

Unlike traditional software with deterministic outputs, LLM pipelines produce responses that are often subjective, context-dependent, and multifaceted. A response might be factually accurate yet inappropriate for the context. It might sound persuasive while conveying incorrect information. It might also address most but not all parts of the user's question. These ambiguities make evaluation fundamentally different from conventional software testing or even traditional machine learning validation.

与输出结果确定的传统软件不同，LLM 流程产生的响应往往是主观的、依赖于上下文的，并且是多方面的。响应可能在事实上是准确的，但却不适用于当前上下文；可能听起来很有说服力，但却传递了错误的信息；也可能回答了用户问题的大部分内容，但并非全部。这些模糊性使得 LLM 评估与传统的软件测试，甚至与传统的机器学习验证有着本质的区别。

The challenge we tackle is straightforward to state but difficult to solve: How do you assess whether an LLM pipeline is performing adequately? How do you diagnose where it's failing? And how do you systematically improve it?

我们面临的挑战说起来很简单，但解决起来却很困难：如何评估 LLM 流程是否运行良好？如何诊断其故障所在？以及如何系统地改进它？

What This Book Covers

本书涵盖的内容

This book introduces a practical framework for evaluating LLM pipelines: the *Analyze-Measure-Improve* lifecycle. This iterative approach helps you:

本书介绍了一种评估生命周期管理（LLM）流程的实用框架： *分析-测量-改进*生命周期。这种迭代方法可以帮助您：

- *Analyze* your pipeline's behavior on representative data to identify failure modes

  *分析*管道在代表性数据上的运行情况，以识别故障模式
- *Measure* those failures quantitatively using automated evaluators

  使用自动化评估器对这些失败进行定量*测量*
- *Improve* your system based on concrete evidence rather than guesswork

  根据确凿证据而非猜测*来改进*你的系统。

We cover the complete evaluation journey, from initial error analysis through production monitoring:

我们涵盖了完整的评估流程，从最初的错误分析到生产监控：

- Understanding the unique challenges of LLM evaluation (the "Three Gulfs" model)

  了解法学硕士评估的独特挑战（"三重困境"模型）
- Systematic error analysis using qualitative coding techniques

  利用定性编码技术进行系统误差分析
- Collaborative evaluation practices for building shared understanding across teams

  团队协作评估实践有助于建立跨团队的共识
- Implementing automated evaluators, including LLM-as-judge approaches

  实施自动化评估工具，包括以 LLM 作为评判者的方法
- Evaluating specific architectures: multi-turn conversations, RAG systems, tool calling, and agentic systems

  评估特定架构：多轮对话、RAG 系统、工具调用和代理系统

- Continuous integration and deployment for LLM pipelines

  LLM 管道的持续集成和部署

- Building interfaces for ongoing human review and error analysis

  构建用于持续人工审核和错误分析的界面

- Strategies for improvement: accuracy optimization, cost reduction, and model cascades

  改进策略：精度优化、成本降低和模型级联

Each chapter combines conceptual frameworks with practical implementation details. You'll find code examples, real-world case studies, common pitfalls to avoid, and exercises to test your understanding.

每一章都将概念框架与实际实现细节相结合。您将找到代码示例、真实案例研究、需要避免的常见陷阱以及用于检验您理解程度的练习。

The material is based on a live course we teach for engineers and product managers who build production AI systems. Scan this QR code to follow the link to the course:

本课程内容基于我们为构建生产级人工智能系统的工程师和产品经理开设的在线课程。扫描此二维码即可访问课程链接：

Who Should Read This Book

谁应该读这本书?

This book is written for engineers and technical product managers building software products with generative AI components. We assume you have:

本书面向使用生成式人工智能组件构建软件产品的工程师和技术产品经理。我们假设您已具备以下条件：

- Familiarity with large language models and how they work

  熟悉大型语言模型及其工作原理。
- Experience building (or planning to build) applications that use LLM APIs

  具备构建（或计划构建）使用 LLM API 的应用程序的经验
- A practical need to know whether your LLM pipeline is actually working

  了解您的 LLM 流程是否真正有效，这是一个实际的需求。

You don't need to be an expert in machine learning or statistics. While we include code examples and some mathematical notation, we always explain the underlying concepts. If you're not comfortable with code or math, focus on the high-level frameworks and practical patterns; you'll still gain tremendous value from understanding the evaluation process, forming the right mental models, and recognizing common pitfalls.

你无需成为机器学习或统计学专家。虽然我们会提供代码示例和一些数学符号，但始终会解释其背后的概念。如果你不熟悉代码或数学，可以专注于高级框架和实用模式；理解评估过程、构建正确的思维模型以及识别常见陷阱，仍然会让你受益匪浅。

Product managers will find particular value in the chapters on error analysis, collaborative evaluation, and the strategic framework for thinking about evaluation across the product lifecycle. Engineers will appreciate the detailed implementation guidance, code examples, and production deployment strategies.

产品经理会发现书中关于错误分析、协作评估以及贯穿产品生命周期的评估战略框架的章节尤为宝贵。工程师则会欣赏书中详尽的实施指南、代码示例和生产部署策略。

A Note on the Pace of Change

关于变革速度的说明

Large language models change fast. By the time you read this, new models and tools will have come out. It helps to separate the parts of evaluation that last from the parts that move.

The first part is a set of principles that hold across models and tools:

- Define what good output means for your use case.
- Break quality into specific dimensions you can observe.
- Build datasets that match real use.
- Analyze errors before you change the system.
- Measure each change against a baseline.
- Combine automated checks with human judgment.
- Close the loop from evaluation back to improvement.

This book teaches those principles, and they stay useful when the models underneath them change.

The second part is the techniques that carry out those principles, and this part moves quickly:

- Using AI to help find and group errors
- Sampling traces for a person to label
- Writing and validating an LLM judge for a specific task
- Having an agent propose evaluation cases
- Deciding where a person reviews or overrides the system

These change as models and tooling improve. When we wrote this book, using AI to help find errors in a system was less reliable than it is now. That kind of gap is normal for a printed book. We expect the book to run six to twelve months behind current practice, and we plan to fold the advances that prove durable into later editions.

We work on evals almost every day, and we keep our live course current as we meet new failure modes and tools. Check it out if you want to see how these ideas are being applied with current models and tools.

What This Book Doesn't Cover

This book focuses exclusively on *application-centric evaluation*, or assessing whether your specific LLM pipeline works for your specific use case. We deliberately don't cover:

- General-purpose foundation model benchmarking (like MMLU, HellaSwag, or other academic benchmarks)
- Training or fine-tuning foundation models from scratch
- Comparative model selection based solely on benchmark performance

If you're trying to decide which foundation model to use, start by prototyping with your actual application and data, then use the techniques in this book to evaluate whether each model meets your requirements.

How This Book Is Organized

The book follows a logical progression through the evaluation lifecycle:

Part I: Foundations (Chapters 1–2)

Establishes why LLM evaluation is uniquely challenging and introduces the core concepts you'll need

Part II: The Analyze Phase (Chapters 3–4)

Teaches systematic error analysis and collaborative evaluation practices for understanding what your pipeline is actually doing

Part III: The Measure Phase (Chapters 5–8)

Covers implementing automated evaluators and handling specific architectures like RAG, multi-turn conversations, and agentic systems

Part IV: Production and Improvement (Chapters 9–12)

Addresses continuous integration, monitoring, interfaces for ongoing review, and systematic improvement strategies

You can read the book linearly or jump to chapters relevant to your immediate needs. We strongly recommend reading Chapters 1–2 before diving into later sections; they

establish the foundational framework that everything else builds upon.

Conventions Used in This Book

The following typographical conventions are used in this book:

Italic

Indicates new terms, URLs, email addresses, filenames, and file extensions.

Constant width

Used for program listings, as well as within paragraphs to refer to program elements such as variable or function names, databases, data types, environment variables, statements, and keywords.

TIP

This element signifies a tip or suggestion.

NOTE

This element signifies a general note.

WARNING

This element indicates a warning or caution.

Example P-1.

This element indicates a text-based example we return to throughout the book.

O'Reilly Online Learning

NOTE

For more than 40 years, O'Reilly Media has provided technology and business training, knowledge, and insight to help companies succeed.

Our unique network of experts and innovators share their knowledge and expertise through books, articles, and our online learning platform. O'Reilly's online learning platform gives you on-demand access to live training courses, in-depth learning paths, interactive coding environments, and a vast collection of text and video from O'Reilly and 200+ other publishers. For more information, visit *https://oreilly.com*.

How to Contact Us

Please address comments and questions concerning this book to the publisher:

O'Reilly Media, Inc.

141 Stony Circle, Suite 195

Santa Rosa, CA 95401

800-889-8969 (in the United States or Canada)

707-827-7019 (international or local)

707-829-0104 (fax)

support@oreilly.com

https://oreilly.com/about/contact.html

We have a web page for this book with additional information. You can access this page at *https://oreil.ly/evals-for-ai-eng*.

For news and information about our books and courses, visit *https://oreilly.com*.

Find us on LinkedIn: *https://linkedin.com/company/oreilly*.

Watch us on YouTube: *https://youtube.com/oreillymedia*.

Acknowledgments

This book would not exist without the engineers and product managers who have participated in our course, and the companies we've partnered with through consulting engagements. Your real-world challenges, questions, feedback, and shared experiences have shaped every chapter.

We're grateful to our technical reviewers Eugene Yan, Panos Alexopoulos, Bryan Bischof, Lorenzo Toni, Rudrendu Paul, Zsolt Szekeres, and Willis Vandevanter for their thorough feedback and for catching our mistakes.

Thanks to our editor, Angela Rufino, at O'Reilly for believing in this project and for patience as we translated course materials into book form.

Part I. Foundations

第一部分： 基础知识

Chapter 1. Introduction

第一章引言

The past several years have seen rapid advances in the development and deployment of large language models (LLMs). Organizations are embedding LLMs into software products for customer service, content creation, decision support, and information extraction, among many others (Bommasani et al. 2021; Zaharia et al. 2024). We call these software products *LLM applications*, and the LLM-driven component inside each one (the part that makes decisions, calls tools, and produces output) the *LLM agent*.

过去几年，大型语言模型（LLM）的开发和部署取得了快速进展。各组织正在将 LLM 嵌入到用于客户服务、内容创作、决策支持和信息提取等诸多领域的软件产品中（Bommasani 等人，2021； Zaharia 等人，2024）。我们将这些软件产品称为 *LLM 应用程序*，而每个应用程序中由 LLM 驱动的组件（即做出决策、调用工具和生成输出的部分）则称为 *LLM 代理*。

WHAT IS AN AGENT?
什么是代理人？

We use "agent" broadly throughout the book. It covers the full range from a single LLM call with no tools (the simplest case) to a system that runs in a loop, calls many tools, and makes many decisions per request. Chapter 2 walks through this range with concrete examples.

我们在本书中广泛使用了"代理人"一词。 它涵盖了所有范围 仅通过一次 LLM 调用，不使用任何工具（最简单的情况） 对于一个循环运行的系统， 调用多种工具， 并根据要求做出许多决定。 第二章将通过具体例子来探讨这一范围。

The distinction between agent and application is useful for evaluation. The *agent* is the LLM-driven component that reasons, calls tools, and produces output. The *application* is everything around it: the user interface, the database, the API layer, the deployment infrastructure. Some failures come from the agent itself (e.g., wrong tool call, incorrect reasoning, bad output format). Others come from the application around it (e.g., broken user interface, missing data, slow response times). This book focuses on agent-level evaluation.

区分代理和应用程序对于评估至关重要。 *代理*是由逻辑层模型（LLM）驱动的组件，负责推理、调用工具并生成输出。 *应用程序*则涵盖了代理周围的一切：用户界面、数据库、API 层以及部署基础设施。有些故障源于代理本身（例如，错误的工具调用、错误的推理、错误的输出格式），而另一些故障则源于其周围的应用程序（例如，损坏的用户界面、缺失的数据、响应速度慢）。本书重点关注代理级别的评估。

Adoption is outpacing our ability to systematically evaluate these LLM applications (Ward and Feldstein 2024). The consequences of shipping an unevaluated agent can be significant. In one illustrative story, in late 2022, a passenger named Jake Moffatt visited the Air Canada website to book a last-minute flight after his grandmother died. He asked the airline's customer support chatbot whether he could get a *bereavement fare* (a discounted ticket that many airlines offer to people traveling for a funeral or death in the family). The chatbot incorrectly told him he could book the regular fare and apply for the bereavement discount as a refund within 90 days of the flight. Air Canada's actual policy was that bereavement discounts could only be requested *before* the flight, not after. Later, when Moffatt applied for the refund, the airline refused. He took the case to the British Columbia Civil Resolution Tribunal, which ruled in his favor in February 2024 (*Moffatt v. Air Canada* 2024). The tribunal rejected the airline's argument that the chatbot was a "separate legal entity" and held Air Canada liable for the

hallucinated policy. The airline was ordered to pay damages, and removed the chatbot from its website soon after.

LLM（生命周期管理）应用的普及速度已经超过了我们系统评估这些应用的能力（Ward 和 Feldstein，2024）。未经评估就部署代理的后果可能非常严重。举个例子，2022 年末，一位名叫 Jake Moffatt 的乘客在祖母去世后访问加拿大航空公司的网站预订最后一刻的航班。他询问航空公司客服聊天机器人是否可以购买*丧假票价*。（许多航空公司提供的折扣机票）对于因参加葬礼或家人去世而出行的人来说）。聊天机器人错误地告诉他可以预订普通票价。并申请丧假折扣作为退款 航班起飞后90天内。加拿大航空的实际政策是，丧亲折扣 只能在航班起飞*前*提出申请，不能在起飞后提出。后来，当莫法特申请退款时，航空公司拒绝了。他将此案提交至不列颠哥伦比亚省民事纠纷解决法庭，该法庭于 2024 年 2 月作出有利于他的裁决（*莫法特诉加拿大航空，2024*）。法庭驳回了航空公司关于聊天机器人是"独立法律实体"的辩解，并认定加拿大航空对这项虚假政策负有责任。航空公司被责令支付赔偿金，并在不久后从其网站上移除了聊天机器人。

No one knows exactly why the chatbot produced the wrong answer. It could have been trained or prompted on policy text that it then paraphrased incorrectly. It could have incorrectly referenced language from other airlines' policies that do allow retroactive bereavement claims. What we do know is that nobody caught the failure before a customer relied on it. This is the kind of issue that evaluation is designed to surface: a systematic check of the chatbot's answers on common policy questions could have flagged the discrepancy before the chatbot went live. Cases like this illustrate why evaluation is necessary. Agents that give wrong answers in production can lead to serious legal and reputational costs.

没有人确切知道聊天机器人为何会给出错误答案。它可能接受过相关政策文本的训练或提示，然后错误地对其进行了转述。它也可能错误地引用了其他航空公司允许追溯性丧亲抚恤金索赔的政策条款。我们所知道的是，在客户依赖该机器人之前，没有人发现这个错误。这正是评估机制旨在发现的问题：如果在聊天机器人上线之前，对其常见政策问题的答案进行系统性检查，就能发现其中的差异。这类案例说明了评估的必要性。在实际工作中给出错误答案的客服人员可能会导致严重的法律和声誉损失。

Evaluation doesn't stop at launch. Prelaunch evaluation tests a system before users see it; production monitoring checks the system continuously once it's live. Both are part of the evaluation process, but they catch different problems. Prelaunch evaluation finds failures before they reach users, while production monitoring finds failures that only appear in the real world, where input distributions shift and user behavior is

unpredictable. Chapter 9 covers production monitoring in depth.

评估不会在产品发布后就结束。 预发布评估是在用户看到系统之前对其进行测试； 生产监控会在系统上线后持续进行检查。 两者都是评估过程的一部分，但它们发现的问题不同。 预发布评估可以在产品到达用户之前发现缺陷。 生产监控能够发现那些只在现实世界中才会出现的故障， 输入分布发生变化且用户行为不可预测。 第九章深入探讨了生产监控。

Unlike traditional software, LLM applications do not produce consistent, repeatable responses the way traditional software does, and they have no formal specifications for the logic they should and should not perform. The same input can produce different outputs on different runs, and the outputs can be wrong in ways that are hard to anticipate. Sometimes, as in the airline story, they are obviously incorrect. Other times, the issue is harder to describe. An output can be factually right but inappropriate (the "vibes are off"), or sound convincing while being totally wrong. These ambiguities make evaluation fundamentally different from conventional software testing and even from traditional machine learning (ML).[1]

与传统软件不同，LLM 应用无法像传统软件那样产生一致且可重复的响应，也没有关于其逻辑执行方式的正式规范。相同的输入在不同的运行中可能会产生不同的输出，而且输出结果的错误方式难以预料。有时，就像航空公司案例中那样，错误显而易见。而有时，问题则难以描述。输出结果可能在事实上是正确的，但却不恰当（"感觉不对劲"），或者听起来很有说服力，但实际上完全错误。这些模糊性使得 LLM 评估与传统的软件测试，甚至与传统的机器学习 (ML) 都存在根本性的差异。

The core challenge of evaluation is this: how do we assess whether an LLM agent is performing adequately, and how do we diagnose where it is failing? This book builds a practical foundation for answering those questions, with methods you can apply to your own projects.

评估的核心挑战在于：我们如何评估 LLM 代理的性能是否令人满意，以及如何诊断其不足之处？本书为解答这些问题奠定了实用的基础，并提供了一些您可以应用于自身项目的方法。

In this chapter, you will learn:

本章你将学习：

- What evaluation means for LLM applications

  评估对 LLM 申请意味着什么

- The Three Gulfs framework for understanding where applications fail

  用于理解应用程序失败原因的“三大障碍”框架
- The Analyze-Measure-Improve lifecycle that structures the rest of this book

  本书其余部分将围绕“分析-测量-改进”生命周期展开。

What Is Evaluation? 什么是评估?

Evaluation is the *systematic process of understanding and measuring quality in an LLM application*. It includes both qualitative analysis of failure patterns and quantitative measurement of their frequency. A good evaluation produces results that are interpretable and actionable.

评估是*对 LLM 应用中质量进行理解和衡量的系统过程*。它包括对故障模式的定性分析和对其发生频率的定量测量。良好的评估能够产生可解释且可操作的结果。

In some cases evaluation produces a quantitative metric. For example, if you are building a travel planning assistant, you might measure how often the system returns valid flight and hotel options within budget, or whether itineraries satisfy the user's constraints. In other cases, the "metric" is generated with more judgment involved. A set of experts, or sometimes the product team itself, may review outputs and rate them for qualities such as clarity, tone, or usefulness. Both approaches fall on a spectrum of rigor: from quick, informal checks that guide early iteration, to carefully defined rubrics and systematic sampling that support production systems.

在某些情况下，评估会产生量化指标。例如，如果您正在开发一个旅行规划助手，您可能会衡量系统返回符合预算的有效航班和酒店选项的频率，或者行程是否满足用户的限制条件。而在其他情况下，“指标”的生成则更多地依赖于主观判断。一组专家，有时甚至是产品团队本身，可能会审查输出结果，并根据清晰度、语气或实用性等质量对其进行评分。这两种方法的严谨程度各不相同：从指导早期迭代的快速、非正式检查，到支持生产系统的精心定义的评估标准和系统抽样。

A single application will usually require multiple evaluations, each designed to capture a different dimension of performance. Evaluations can then be integrated into the application in several ways:

单个应用程序通常需要进行多次评估，每次评估旨在捕捉性能的不同维度。评估结果可以通过多种方式集成到应用程序中：

Monitoring 监测

Continuous checks that run in the background to detect drift or degradation over time

持续在后台运行的检查，用于检测随时间推移发生的漂移或性能下降。

Guardrails 护栏

Evaluations placed in the critical path, allowing the system to block, retry, or fall back when outputs fail predefined criteria

评估环节位于关键路径上，当输出结果不符合预定义标准时，系统可以阻塞、重试或回退。

Improvement mechanisms 改进机制

Evaluations used to generate feedback, such as labeled examples for training the model further, curated example inputs and outputs included directly in the prompt (known as few-shot examples), or systematic identification of failure cases that motivate architectural changes

用于生成反馈的评估方法包括：提供带标签的示例以进一步训练模型；提供直接包含在提示中的精选示例输入和输出（称为少样本示例）；或者系统地识别失败案例，从而推动架构变更。

Model migration 模型迁移

Evaluations allowing you to compare the performance of different model versions, and to make informed decisions about when to migrate to a new model

评估功能允许您比较不同模型版本的性能，并帮助您就何时迁移到新模型做出明智的决策。

We rely on evaluations to detect errors, maintain user trust, enforce safety constraints, monitor ongoing behavior, and guide systematic improvement.

我们依靠评估来发现错误、维护用户信任、执行安全约束、监控持续行为并指导系统改进。

Evaluation Throughout the Lifecycle of an LLM

法学硕士（LLM）全生命周期评估

Evaluation appears at every stage of the LLM lifecycle. At each stage (pretraining, post-training, and then any applications we build on top) the role of evaluation is different, reflecting distinct goals and constraints. Understanding these stages helps clarify the spectrum of capabilities being tested, and why there can be no single evaluation framework that applies everywhere.

评估贯穿 LLM 生命周期的每个阶段。在每个阶段（预训练、后训练以及我们基于此构建的任何应用程序），评估的作用各不相同，反映了不同的目标和限制。理解这些阶段有助于明确所测试能力的范围，以及为什么不存在适用于所有情况的单一评估框架。

Pretraining 预训练

Pretraining is the first stage in the lifecycle, where a foundation model is built from scratch. A large neural network is exposed to massive text (and sometimes code) collections with a single objective, predicting the next *token* in a sequence. A token is the smallest unit of text the model reads or writes at a time. It is usually a short sequence of characters, roughly corresponding to a word or part of a word. For example, the sentence “The chatbot answered the question” might be broken up into tokens like `The`, `chat`, `bot`, `answered`, `the`, `question`. Models read and generate one token at a time, and the cost of an API call is usually measured in the number of tokens in and out. During pretraining, the model learns broad linguistic knowledge and some capacity for reasoning and world knowledge by doing this next-token prediction over billions of examples.

预训练是生命周期的第一阶段，在此阶段，基础模型从零开始构建。大型神经网络会接触海量的文本（有时也包括代码）集合，其唯一目标是预测序列中的下一个*词元*。词元是模型一次读取或写入的最小文本单元。它通常是一小段字符序列，大致对应于一个单词或单词的一部分。例如，句子“The chatbot answers the question”可能会被拆分成 `The`、`chat`”、`bot`、`answered`、`the`”、`question` 这样的词元。模型一次读取和生成一个词元，API 调用的成本通常以输入和输出的词元数量来衡量。在预训练期间，模型通过对数十亿个示例进行下一个词元的预测，学习广泛的语言知识以及一定的推理能力和世界知识。

Pretraining is carried out by large research labs and commercial model providers. The process requires substantial computational resources, often costing hundreds of millions of dollars. Evaluation at this stage is *intrinsic*, meaning we measure whether the model predicts the next token correctly on text it wasn’t trained on. One common

metric is *perplexity*, the average negative log-likelihood per token on held-out text. Lower perplexity means the model is less surprised by unseen data. Pretraining evaluation also includes scaling-law analysis, which tracks how loss decreases as compute and training data increase. These metrics tell us whether the model has absorbed the statistical patterns of language, but they do not tell us whether the model is useful for the downstream tasks that people actually build with LLMs.

预训练由大型研究实验室和商业模型提供商进行。该过程需要大量的计算资源，通常耗资数亿美元。此阶段的评估是*内在的*，这意味着我们衡量模型在未训练过的文本上能否正确预测下一个词元。一个常用的指标是*困惑度（perplexity）*，即每个词元在保留文本上的平均负对数似然值。困惑度越低，意味着模型对未见过的数据越不适应。预训练评估还包括尺度律分析，该分析跟踪损失如何随着计算量和训练数据的增加而降低。这些指标告诉我们模型是否已经吸收了语言的统计模式，但它们并不能告诉我们模型对于人们实际使用语言学习模型（LLM）构建的下游任务是否有用。

Post-training 训练后

Post-training takes a pretrained model and adapts it for practical use. It turns out that models trained only on next-token prediction can already generalize to many unseen tasks, or tasks that LLMs weren't explicitly pretrained on. Post-training builds on this ability by steering the model toward behaviors that users care about, e.g., following instructions, staying on topic, and avoiding unsafe responses.

后训练是指对预训练模型进行调整，使其更适用于实际应用。事实证明，仅针对下一个词元预测进行训练的模型已经能够泛化到许多未见过的任务，或者说，是逻辑学习模型（LLM）未明确预训练的任务。后训练在此基础上，通过引导模型朝着用户关心的行为方向发展，例如遵循指令、保持主题相关以及避免不安全的回复，从而进一步增强了这种能力。

Post-training usually has two parts. First, the model provider fine-tunes the model on curated examples of prompts paired with good responses, so the model learns to produce answers in the format people want. Second, the provider uses preference data (pairs of responses where a human picked which one was better) to further nudge the model toward the preferred style.

训练后通常分为两个部分。首先，模型提供商会使用精心挑选的提示示例（这些示例与优秀的回答配对）对模型进行微调，使模型学习生成用户期望的格式的答案。其次，提供商会使用偏好数据（由用户选择哪个答案更好的配对）进一步引导模型朝着用户偏好的风格发展。

More recent post-training methods include direct preference optimization (DPO) and process reward models, which change how the model reasons internally. Some models also use reinforcement learning with verifiable rewards (RLVR) to train long chains of thought with step-by-step reasoning (DeepSeek-AI 2025). The post-training method affects what kinds of errors the model makes in applications; it's useful to know which method was used even if you don't run the training yourself.

较新的训练后方法包括直接偏好优化（DPO）和过程奖励模型，这些方法会改变模型内部的推理方式。一些模型还使用可验证奖励的强化学习（RLVR）来训练具有逐步推理能力的长思维链（ DeepSeek-AI 2025 ）。训练后方法会影响模型在应用中出现的错误类型；即使您不亲自运行训练，了解使用了哪种方法也很有用。

We do not cover these techniques in detail in this book because most engineers interact with the model after these steps are done. Evaluation at this stage shifts from measuring raw next-token accuracy to running the model on benchmarks that approximate user-facing tasks like question-answering, reasoning challenges, safety and bias tests, and coding. These evaluations are designed and run by the model providers themselves, and their results are often reported as headline benchmarks.

本书并未详细介绍这些技术，因为大多数工程师在这些步骤完成后才会与模型进行交互。此阶段的评估重点不再是测量原始的下一个词元准确率，而是将模型运行在模拟用户实际任务的基准测试中，例如问答、推理挑战、安全性和偏差测试以及编码。这些评估由模型提供商自行设计和运行，其结果通常作为重要的基准测试结果发布。

Application 应用

The application stage is where most engineers and product managers engage with LLMs. At this point the model has already been trained and aligned by a model provider, and the task is to integrate it into a concrete workflow, e.g., triaging emails, assisting customer support, or analyzing documents. Evaluation at this stage must be tailored to the goals and data of the specific application. In some cases, existing post-training evaluations overlap directly with the task. For example, a coding assistant can draw on established programming benchmarks. In most cases, however, the application is not represented in public leaderboards. A system for reviewing contracts, summarizing medical notes, or checking compliance will require its own evaluation meth-

ods.

应用阶段是大多数工程师和产品经理与生命周期模型 (LLM) 接触的阶段。此时，模型提供商已经对模型进行了训练和调整，任务是将其集成到具体的流程中，例如，电子邮件分类、客户支持或文档分析。此阶段的评估必须根据特定应用的目标和数据进行定制。在某些情况下，现有的训练后评估与任务直接相关。例如，编码助手可以借鉴已建立的编程基准。然而，在大多数情况下，应用程序不会出现在公开排行榜上。用于审查合同、汇总医疗记录或检查合规性的系统需要有其自身的评估方法。

Responsibility therefore falls to the application development team to define what success means, design appropriate metrics, and implement monitoring to verify that the system behaves reliably for its intended use.

因此，应用程序开发团队有责任定义成功的含义，设计适当的指标，并实施监控，以验证系统是否能够可靠地运行，从而实现其预期用途。

Evaluation in Applications

应用评估

At the application stage, the model has already been trained. The work is in building around it. You have to decide what the product should achieve, translate that goal into prompts and agent logic, and then verify that the model's outputs are acceptable. Each of these steps introduces its own difficulties:

在应用阶段，模型已经训练完成。接下来的工作是围绕它进行构建。你需要确定产品应该实现的目标，将该目标转化为提示和代理逻辑，然后验证模型的输出是否可接受。每个步骤都会带来各自的挑战：

Objectives are vague at first.

目标起初比较模糊。

A goal like "summarize emails" sounds simple, but only when you inspect outputs do the ambiguities appear: how detailed, in what format, and with what exclusions?

"总结电子邮件"这样的目标听起来很简单，但只有在检查输出结果时才会出现歧义：详细程度如何，格式如何，以及排除哪些内容？

Instructions might have gaps.

说明书可能存在遗漏。

> Prompts and agent logic expressed in natural language rarely capture the full intent, which leads to inconsistent or incomplete behavior.
>
> 用自然语言表达的提示和代理逻辑很少能完全捕捉到意图，这会导致行为不一致或不完整。

Behavior changes on new data.

基于新数据的行为改变。

> A setup that works on a handful of test cases may fail once it encounters the variability of real inputs.
>
> 一套在少数测试用例中运行良好的系统，一旦遇到实际输入的各种变化，就可能失效。

In practice, these issues show up again and again when building LLM applications. To reason about these challenges more systematically, we use the *Three Gulfs* framework, which names three places where development tends to break down—comprehension, specification, and generalization. Each gulf highlights a different point where development typically goes wrong, and each requires a different strategy to address, a theme we will return to throughout this book.

在实践中，这些问题在构建 LLM 应用程序时反复出现。为了更系统地分析这些挑战，我们采用了 "*三重障碍*" 框架。该框架指出了开发过程中容易出现问题的三个环节：理解、规范和泛化。每个障碍都突出了开发过程中通常出错的不同环节，并且每个环节都需要不同的应对策略，我们将在本书中反复探讨这一主题。

The Three Gulfs of LLM Application Development

法学硕士申请开发的三个难关

As mentioned, a useful way to understand the challenges of building with LLMs is the Three Gulfs model (Figure 1-1), adapted from Shankar et al. (2025) and inspired by Norman (1988). The model highlights three common gaps that developers face when building and evaluating applications. We call these gaps comprehension, specification,

and generalization.

如前所述，理解使用 LLM 进行建设所面临的挑战的一个有用方法是采用“三海湾模型”（图 1-1），该模型改编自 Shankar 等人 (2025) 的研究。 该模型借鉴了 Norman (1988) 的研究成果，着重指出了开发者在构建和评估应用程序时面临的三个常见差距。我们将这三个差距分别称为理解、规范和泛化。

Throughout this book, we will use this framework to diagnose why an application is failing, and select the most appropriate strategy to fix it.

在本书中，我们将使用此框架来诊断应用程序失败的原因，并选择最合适的策略来修复它。

Consider an agent that processes incoming emails sent to an organization. The goal is to extract the sender's name, summarize the key requests, and categorize the email. At first glance, this seems straightforward. But each stage of development exposes different challenges across the Three Gulfs.

设想一个代理程序，用于处理发送到某个组织的电子邮件。其目标是提取发件人姓名、概括关键请求并对邮件进行分类。乍一看，这似乎很简单。但开发的每个阶段都会在三大挑战中暴露出不同的问题。

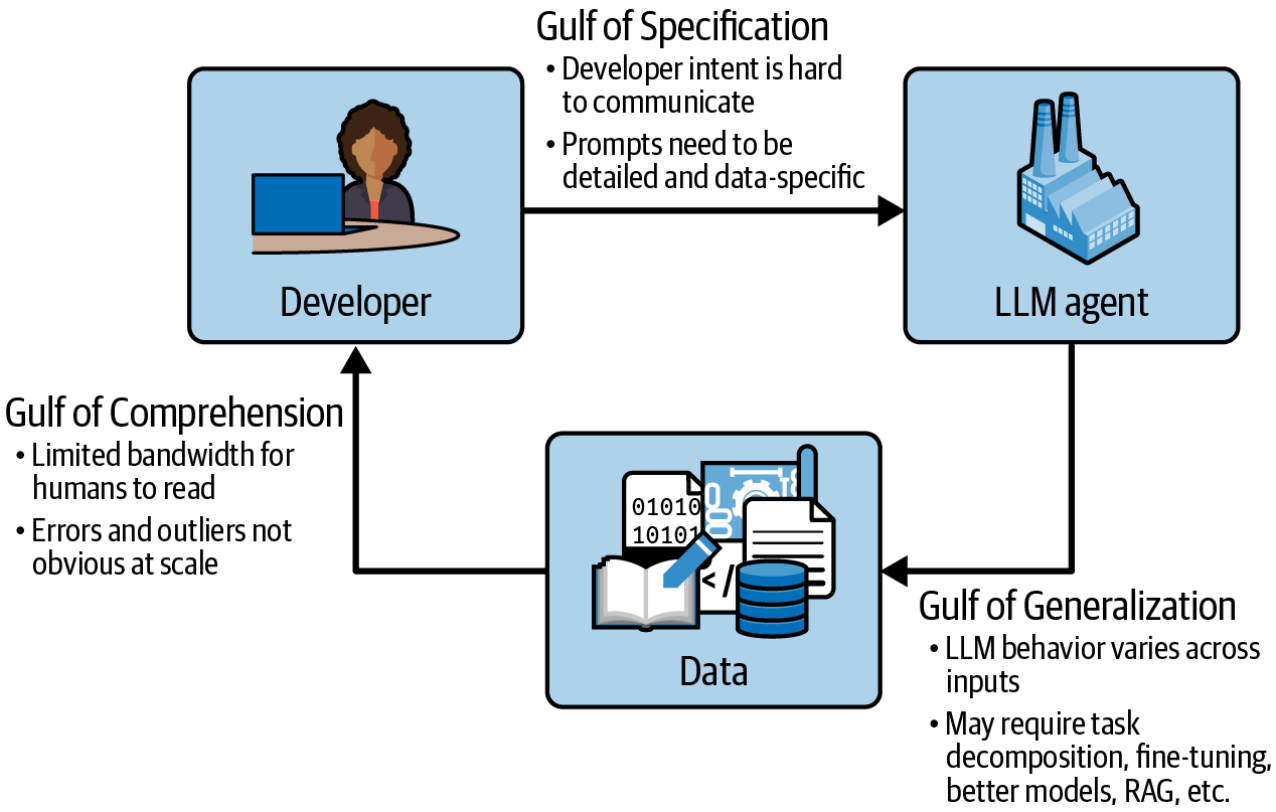


Figure 1-1. Three Gulfs model of challenges in LLM agent development, adapted from Shankar et al. (2025)

图 1-1. LLM 代理开发中面临的三大挑战模型，改编自 Shankar 等人 (2025)

The Gulf of Comprehension (Between the Developer and Data)

理解鸿沟（开发者与数据之间）

The first gulf is between the developer and the data. To build effective prompts and agents, you need a solid grasp of what your inputs actually look like. Without that, you will design against an imagined version of the task, not the real one.

第一个鸿沟存在于开发者和数据之间。要构建有效的提示和代理，你需要对输入的实际形式有透彻的了解。否则，你的设计将基于对任务的想象，而非真实的任务。

Consider the email processing example. If you only look at a handful of clean, well-written emails when drafting your prompt, you might write instructions that work fine on those cases. But the production data will also include forwards, replies with long quoted threads, messages with signatures in multiple languages, and poorly formatted text copied from PDFs. A prompt that ignores these realities might lead the LLM to, for example, summarize an entire quoted thread instead of the new message, or mistake a signature block for the main request.

以电子邮件处理为例。如果您在编写提示时只查看少量格式规范、文笔流畅的邮件，那么您编写的说明可能适用于这些情况。但实际生产环境中的数据还会包含转发邮件、带有长篇引用的回复、使用多种语言签名的邮件，以及从 PDF 文件复制的格式混乱的文本。如果提示忽略了这些实际情况，可能会导致语言学习模型 (LLM) 错误地总结整个引用的邮件内容，而不是新邮件本身，或者将签名块误认为是主要请求。

The Gulf of Comprehension applies to outputs too, not just inputs. We also need a clear sense of how the LLM behaves on those inputs. You cannot read every output it generates, but you still need to recognize the common ways it succeeds and the common ways it fails. Otherwise, it's easy to be misled by a few good examples and miss the patterns that only show up at scale. In the email agent, for example, the LLM might usually extract the sender and write a decent summary. But when you look closer, you may notice recurring mistakes. The LLM might pull the wrong name when several are mentioned, drop key details in longer messages, or copy boilerplate like confidentiality notices into the summary. These failures may only happen occasionally,

but if they occur, the system will feel unreliable to users.

理解鸿沟不仅适用于输入，也适用于输出。我们还需要清楚地了解 LLM（生命周期管理）如何处理这些输入。你不可能阅读它生成的每一个输出，但你仍然需要识别它成功和失败的常见方式。否则，很容易被几个好的例子误导，而忽略那些只有在大规模处理时才会出现的模式。例如，在电子邮件代理中，LLM 通常会提取发件人信息并生成一份不错的摘要。但仔细观察后，你可能会注意到一些反复出现的错误。例如，当出现多个发件人时，LLM 可能会提取错误的名称；在较长的邮件中遗漏关键细节；或者将类似保密声明的模板内容复制到摘要中。这些错误可能只是偶尔发生，但如果出现，用户就会觉得系统不可靠。

Bridging the Gulf of Comprehension means developing strategies to characterize both your inputs and your outputs at scale, so you know what the data really looks like, and how the system actually behaves, without having to inspect everything by hand.

弥合理解鸿沟意味着制定策略来大规模地描述你的输入和输出，这样你就能知道数据的真实面貌以及系统的实际运行情况，而无需手动检查所有内容。

The Gulf of Specification (Between the Developer and the LLM Agent)

规范鸿沟（开发商与 LLM 代理商之间）

The second gulf is between what we want and what we actually tell the LLM to do. Our intent (the task we have in mind) is often only loosely captured by the prompts or agent logic we write. Natural language is imprecise, and small gaps in how we phrase things can lead to large differences in behavior.

第二个差距存在于我们想要的结果和我们实际指示 LLM 执行的操作之间。我们的意图（我们心中设想的任务）通常只能通过我们编写的提示或智能体逻辑粗略地表达出来。自然语言本身就不精确，措辞上的细微差别就可能导致行为上的巨大差异。

Take the instruction：请按照指示操作：

> **Extract the sender's name and summarize the key requests in this email.**

At first glance it looks specific, but important details are left unstated. Should the summary be a paragraph or a bulleted list? Should the sender be the display name, the full email address, or both? Should implicit requests be included, or only explicit ones?

How concise should the summary be?

乍看之下似乎很具体，但一些重要细节却未提及。摘要应该用一段文字还是一个项目符号列表？发件人应该是显示名称、完整邮箱地址还是两者都用？是否应该包含隐含请求，还是只包含明确请求？摘要应该有多简洁？

The LLM cannot fill in these choices for you. If you don't make them explicit, the system will guess, and those guesses will not always line up with what you expect. That's why two runs of the same agent might produce outputs in different formats, with varying levels of detail, or with inconsistent handling of edge cases.

LLM 无法替你填写这些选项。如果你不明确指定，系统会进行猜测，而这些猜测并不总是符合你的预期。这就是为什么同一个代理运行两次可能会产生格式不同、细节程度不同，或者对特殊情况处理不一致的输出结果。

The Gulf of Specification is this gap between our intent and our instructions. Closing it requires making expectations explicit and aligning them closely with the data the agent will actually see.

"规范鸿沟"指的是我们的意图和指令之间的差距。弥合这一鸿沟需要明确预期，并使其与代理实际看到的数据紧密匹配。

The Gulf of Generalization (Between the Data and the LLM Agent)

概括化鸿沟（数据与 LLM 代理之间）

The third gulf is between the data your system sees and how the LLM generalizes to it. Even if you have a solid grasp of your inputs and have written a clear, well-scoped prompt, the LLM is still prone to errors. LLMs are not perfect, and they will make mistakes.

第三个差距存在于系统接收到的数据和语言学习模型（LLM）如何对其进行概括之间。即使你对输入数据有透彻的理解，并且编写了清晰明确的提示，语言学习模型仍然容易出错。语言学习模型并非完美无缺，它们会犯错。

For example, imagine you've clearly specified that the system should extract the sender's name from the email header. Most of the time the LLM follows this instruction. But occasionally, when the body of the email mentions another person (say, a public figure), the LLM will incorrectly return that name instead. Nothing about your understanding of the data or your prompt was wrong. The LLM simply failed to apply

the instructions consistently. Maybe the LLM is too biased in favor of names it has seen in its training data.

例如，假设您已明确指定系统应从电子邮件标头中提取发件人姓名。大多数情况下，LLM 都能遵循此指令。但有时，当电子邮件正文提及其他人（例如公众人物）时，LLM 会错误地返回该人名。您对数据的理解或您的指令均无误。LLM 只是未能始终如一地执行指令。或许 LLM 对训练数据中已出现过的姓名存在过度偏好。

This is the Gulf of Generalization: the fact that LLMs can behave unpredictably even when comprehension and specification are handled well. These errors usually become visible only when the system is exposed to the full variety of real-world data. And they are unavoidable: no matter how advanced the models become, there will always be cases where they produce the wrong output.

这就是泛化鸿沟：即使理解和规范处理得当，逻辑学习模型（LLM）也可能出现不可预测的行为。这些错误通常只有在系统接触到各种各样的真实世界数据时才会显现出来。而且，这些错误是不可避免的：无论模型多么先进，总会有模型产生错误输出的情况。

APPLYING THE THREE GULFS TO OTHER DOMAINS
将这三个海湾应用于其他领域

The Three Gulfs apply beyond email processing. For example, in a *customer support chatbot*, the Gulf of Comprehension is about the full variety of customer queries, including multilingual messages, typos, and multi-part questions. The Gulf of Specification is about defining what a "good" response looks like; a bot that apologizes when it should escalate to a human is following the wrong instructions, and without explicit rules the bot will guess. The Gulf of Generalization shows up when the bot handles billing questions well but fails on product-specific questions it has seen fewer examples of.

三大鸿沟不仅适用于电子邮件处理。例如，在*客户支持聊天机器人*中，"理解鸿沟"指的是理解各种各样的客户查询，包括多语言消息、拼写错误和多部分问题。"规范鸿沟"指的是定义"良好"回复的标准；如果机器人本应将问题转接给人工客服却道歉，那就说明它遵循了错误的指令，如果没有明确的规则，机器人只能靠猜测。"泛化鸿沟"则体现在机器人能够很好地处理账单问题，但却无法回答它很少遇到的产品特定问题。

In a *code generation assistant*, the Gulf of Comprehension is about how developers actually describe what they want, which often includes vague requests, incomplete context, and references to undocumented internal APIs. The Gulf of Specification is about defining output format, error handling, and style conventions. The Gulf of Generalization shows up when the model produces correct Python but fails on less common languages or complex multi-file changes.

在*代码生成助手*领域，"理解鸿沟"指的是开发者如何实际描述他们的需求，这通常包括模糊的请求、不完整的上下文以及对未公开的内部 API 的引用。"规范鸿沟"指的是如何定义输出格式、错误处理和风格规范。"泛化鸿沟"则体现在模型能够生成正确的 Python 代码，但在处理不太常见的语言或复杂的多文件更改时却会出错。

The evaluation strategy for each application depends on which gulf is the primary source of failure.

每个应用的评估策略取决于哪个鸿沟是主要的故障根源。

Why LLM Agent Evaluation Is Challenging

为什么 LLM 代理人评估具有挑战性

Evaluating LLM applications is challenging because every application requires bridging all Three Gulfs (Comprehension, Specification, and Generalization) again in its own context. Evaluation is not a one-time check. It is an iterative process where requirements become clear only after you interact with real outputs.

评估 LLM 应用极具挑战性，因为每个应用都需要在其自身背景下再次跨越理解、规范和概括这三大鸿沟。评估并非一次性检查，而是一个迭代过程，只有在与实际输出互动后，需求才会变得清晰。

NOTE 笔记

The Three Gulfs framework provides a shared vocabulary for discussing LLM application failures. When debugging with your team, you can quickly identify whether a problem is a "specification failure" (the prompt didn't make requirements clear enough) or a "generalization failure" (the LLM isn't applying instructions consistently). This common language helps teams diagnose issues faster and choose the right fixes, rather than debating vague notions of the model "not working."

Three Gulfs 框架为讨论 LLM 应用故障提供了一套通用词汇。在与团队一起调试时，您可以快速识别问题是"规范错误"（提示信息不够清晰）还是"泛化错误"（LLM 未能始终如一地应用指令）。这种通用语言有助于团队更快地诊断问题并选择正确的修复方案，而不是纠结于模型"无法正常工作"这种模糊的概念。

When you start examining outputs (closing the Gulf of Comprehension), you will inevitably find mistakes. The next question is whether these mistakes are due to incomplete specification or to failures of generalization. If the prompt does not make the requirement explicit, you may fix it with better instructions or agent design. If the prompt is already well specified and the LLM still fails, then the issue lies in generalization, and automated evaluations can help measure how often that failure occurs.

当你开始检查输出结果（弥合理解鸿沟）时，不可避免地会发现错误。接下来的问题是，这些错误是由于规范不完整还是泛化失败造成的。如果提示没有明确表达需求，你可以通过改进指令或智能体设计来解决这个问题。如果提示已经明确，但 LLM 仍然失败，那么问题就出在泛化上，而自动化评估可以帮助衡量这种失败发生的频率。

Applications also differ in how much they can rely on users to bridge the gulfs on their own. In a coding assistant, maybe we can assume programmers have high agency. They

know how to specify requirements precisely, they can read and debug generated code (comprehension), and they can run tests that expose when the LLM has failed to generalize (e.g., generating a working React app in some cases but not others).

应用程序之间的差异还在于它们对用户自主弥合差距的依赖程度。例如，在编码助手方面，我们可以假设程序员拥有高度的自主性。他们知道如何精确地定义需求，能够阅读和调试生成的代码（理解能力），并且能够运行测试来发现 LLM 何时无法泛化（例如，在某些情况下生成可运行的 React 应用，但在其他情况下则不能）。

In contrast, consider an LLM-powered personal stylist. A user asking for an outfit may not know how to specify requirements clearly. The application itself may need to ask clarifying questions, such as about weather or formality, to reduce specification gaps. Users also cannot easily detect generalization failures: if the LLM produces good suggestions for casual winter wear, they may assume it will also perform well for winter weddings, an assumption that may not hold.

相比之下，考虑一下基于 LLM（逻辑逻辑模型）的个人造型师。用户在寻求服装搭配建议时，可能并不清楚如何清晰地表达自己的需求。应用程序本身可能需要询问一些澄清性问题，例如关于天气或正式程度，以减少需求上的偏差。用户也很难发现概括性建议的不足：如果 LLM 为休闲冬季服装提供了不错的建议，他们可能会认为它同样适用于冬季婚礼，但这种假设可能并不成立。

Effective evaluation requires thinking about the stakes in each gulf for your application. Where the stakes are higher, because users cannot easily catch mistakes themselves or because the cost of errors is significant, you will need to design more rigorous processes to close the gaps.

有效的评估需要考虑应用程序中每个环节的利害关系。如果利害关系较高，例如用户难以自行发现错误，或者错误代价重大，则需要设计更严格的流程来弥补这些差距。

The LLM Evaluation Lifecycle: Bridging the Gulfs with Evaluation

法学硕士评估生命周期：衔接 海湾评估

The Three Gulfs (Comprehension, Specification, and Generalization) capture the main ways LLM applications go wrong. In this book, we approach evaluation through the

Analyze-Measure-Improve lifecycle (Figure 1-2).

理解、规范和概括这三大障碍概括了学习领导力模型（LLM）应用中出现问题的主要途径。本书采用分析-测量-改进生命周期（ 图 1-2 ）进行评估。

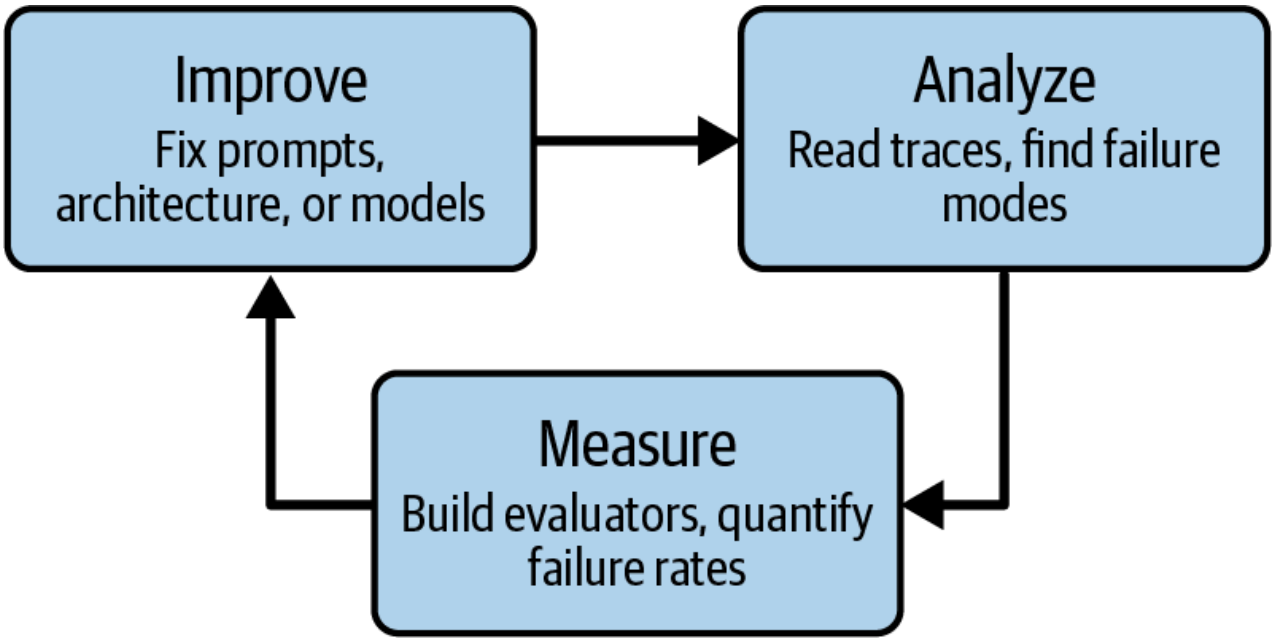


Figure 1-2. The Analyze-Measure-Improve evaluation lifecycle

图 1-2. 分析-测量-改进评估生命周期

This iterative framework bridges the Three Gulfs by using evaluation to systematically understand failures (Analyze), quantify them (Measure), and fix them (Improve). This lifecycle is the framework we will use to show how evaluation can make these gulfs visible and help you decide how to address them.

这个迭代框架通过评估系统地理解（分析）、量化（衡量）和改进（改进）失败，从而弥合三大鸿沟。我们将使用这个生命周期框架来展示评估如何使这些鸿沟显现出来，并帮助您决定如何应对它们。

Analyze 分析

The cycle begins with *Analyze*. At this stage you look at real inputs and outputs to understand how the system actually behaves. The goal is to identify failure modes, not to quantify them yet. While Analyze connects most directly to the Gulf of Comprehension (understanding your data and your system's behavior), it also surfaces specification gaps and generalization issues; you often discover all three types of problems during

the same trace-reading sessions.

整个流程始于*分析阶段*。在这个阶段，你需要查看实际的输入和输出，以了解系统的实际运行情况。目标是识别故障模式，而不是对其进行量化。分析阶段与理解鸿沟（理解你的数据和系统行为）最为直接相关，但它也会暴露出规范缺陷和概括问题；你通常会在同一次跟踪读取过程中发现所有这三种类型的问题。

In practice, this means reading traces (the full record of what the agent did for each request), talking to users or domain experts about what "correct" looks like, and building up a list of the ways the system fails. Chapter 3 covers this in detail.

实际上，这意味着读取跟踪信息（代理对每个请求所执行操作的完整记录），与用户或领域专家讨论"正确"的定义，并列出系统失效的各种方式。第三章对此有详细介绍。

Imagine you review a batch of email summaries and notice two recurring problems: the LLM sometimes includes irrelevant text from signatures, and sometimes omits the most important request. These observations tell you where things break down and offer clues about whether the issue is poor specification or a generalization failure.

假设你审阅了一批邮件摘要，发现两个反复出现的问题：LLM（邮件摘要管理）有时会包含签名中的无关文本，有时会遗漏最重要的请求。这些观察结果可以告诉你问题出在哪里，并为你判断问题是规范不完善还是泛化失败提供线索。

Measure 措施

After Analyze, you will know the kinds of mistakes the system makes. Some will clearly be specification problems: the prompt did not say enough, so the fix is obvious. Others will look like generalization errors: the prompt was clear, but the LLM still misbehaves.

分析之后，您将了解系统会犯哪些类型的错误。有些错误显然是规范问题：提示信息不足，因此解决方法显而易见。另一些错误则像是概括错误：提示信息清晰明了，但 LLM 仍然运行异常。

The *Measure* step is about quantifying those generalization issues. The key question during Measure is: how often does each failure mode actually occur, and is the rate low enough that the system is usable? Without measurement, you're relying on impressions from a handful of traces, which can be misleading in either direction (a few bad examples can make a good system look broken, and a few good examples can make a broken system look fine). To measure at scale, you design evaluators (automated checks) that can score traces without human review. These can be code-based rules, or

they can use another LLM as a judge (a pattern we cover in Chapter 5).

*测量*步骤旨在量化这些概括性问题。测量的关键问题是：每种故障模式实际发生的频率是多少？其发生率是否足够低，以确保系统可用？如果没有测量，您只能依赖于少量跟踪记录的印象，而这些印象可能会产生误导（少数糟糕的例子会让一个原本良好的系统看起来像是坏的，而少数良好的例子也会让一个原本坏的系统看起来像是正常的）。为了进行大规模测量，您需要设计评估器（自动化检查），以便在无需人工审核的情况下对跟踪记录进行评分。这些评估器可以是基于代码的规则，也可以使用另一个 LLM（生命周期管理模型）作为评判标准（我们将在第 5 章中介绍这种模式）。

Imagine you are working with an LLM that sometimes pulls signature blocks into summaries even after you clarified the prompt. You build a rule-based check that flags boilerplate phrases across thousands of outputs and find it happens in 8% of cases. Measurement tells you whether a failure is an occasional annoyance or a systematic problem.

假设你正在使用一款语言学习管理系统 (LLM)，即使你已经明确了提示，它有时仍然会将签名块提取到摘要中。你构建了一个基于规则的检查机制，用于标记数千个输出中的样板短语，结果发现这种情况在 8% 的情况下都会发生。通过测量可以判断这种错误是偶尔出现的小问题，还是系统性问题。

Improve 提升

The final step is *Improve*: decide how to adjust the system based on what Analyze and Measure have uncovered. The path you follow depends on the source of the errors:

最后一步是*改进*：根据分析和测量的结果，决定如何调整系统。具体采取的路径取决于错误的根源：

- If they stem from specification gaps, the fix is usually to tighten the prompt or adjust agent logic.

  如果问题源于规范缺陷，解决方法通常是收紧提示或调整代理逻辑。

- If they stem from generalization failures, you may need stronger interventions such as adding more examples, refining retrieval, or fine-tuning on labeled data.

  如果问题源于泛化失败，您可能需要采取更强有力的干预措施，例如增加更多示例、改进检索或对标记数据进行微调。

Chapter 12 covers these improvement strategies in detail, organized by effort level from quick prompt edits to model-level changes.

第 12 章详细介绍了这些改进策略，并按工作量级别进行了分类，从快速及时的编辑到模型级别的更改。

Imagine your measurements show that missing key requests happens because the prompt never specified the level of detail. This is a specification fix: you revise the instruction to say, “Summarize the sender’s requests in three concise bullet points.” But the name confusion issue persists even after a clear prompt. That is a generalization problem, and fixing it may require adding labeled training data, or redesigning the application to extract the sender from email metadata instead of relying on the LLM to parse the body.

假设您的测量结果表明，关键请求缺失的原因是提示信息没有明确说明详细程度。这可以通过修改说明来解决：您可以将指令修改为“用三个简洁的要点概括发件人的请求”。但是，即使有了清晰的提示，名称混淆的问题仍然存在。这是一个泛化问题，解决它可能需要添加带标签的训练数据，或者重新设计应用程序，使其从电子邮件元数据中提取发件人信息，而不是依赖 LLM 来解析邮件正文。

Each pass through this cycle helps you understand the gulfs you are facing and choose the right level of evaluation rigor for your application.

每次经历这个循环都能帮助你了解你所面临的差距，并为你的申请选择合适的评估严格程度。

Putting It Together 整合

When you encounter a failing LLM application, ask which gulf the failure falls into:

当你遇到一个失败的 LLM 应用程序时，问问自己：这个失败属于哪一类情况？

Comprehension 理解

> You don’t yet understand what your inputs look like or how the system actually behaves on them; look at more traces before changing anything.
>
> 你还不了解你的输入是什么样的，也不清楚系统实际是如何处理这些输入的；在做任何更改之前，请查看更多的跟踪信息。

Specification 规格

> Your requirements aren’t explicit enough for the LLM to follow; revise the prompt or agent logic to close the gap.

你的要求不够明确，LLM 无法遵循；请修改提示或代理逻辑以弥补差距。

Generalization 概括

The requirements are clear, but the LLM still fails on some inputs. Measure how often the failure occurs, and consider stronger interventions like more examples, better retrieval, or fine-tuning.

要求很明确，但 LLM 模型在某些输入数据上仍然失效。需要统计失效发生的频率，并考虑采取更有效的干预措施，例如增加示例数量、改进检索方法或进行微调。

The Analyze-Measure-Improve lifecycle structures this diagnosis into a repeatable process. You analyze traces to surface the failure modes, measure how common they are, and then apply a fix and feed the results back into the next round of analysis.

分析-测量-改进生命周期将诊断过程构建成一个可重复的流程。您分析跟踪数据以发现故障模式，测量故障模式的发生频率，然后应用修复方案并将结果反馈到下一轮分析中。

In the next chapter, we turn to the first practical step: building a prototype application and writing initial prompts. You need something concrete in place, even a rough version, before you can analyze how it behaves.

下一章，我们将进入第一个实践步骤：构建原型应用程序并编写初始提示。在分析其运行情况之前，你需要一些具体的东西，哪怕只是一个粗略的版本。

1 In traditional ML, outputs are well defined (numbers, classes, or structured predictions), and we often have ground truth labels. We can use standard metrics like accuracy or F1 score. In contrast, LLM output can be open-ended, unstructured, and often subjective.

在传统机器学习中，输出结果定义明确（数值、类别或结构化预测），并且通常有真实标签。我们可以使用准确率或 F1 分数等标准指标。相比之下，LLM 的输出结果可能是开放式的、非结构化的，并且往往带有主观性。

Chapter 2. LLMs and Evaluation Basics

第 2 章. 大语言模型与评估基础

In Chapter 1, we introduced evaluation and its role. Before we can evaluate, we need something to evaluate. This chapter walks through building a first application and understanding its moving parts.

在第一章中，我们介绍了评估的概念及其作用。在能够进行评估之前，我们首先需要一些可供评估的对象。这一章将介绍如何构建第一个应用程序，以及理解其各个组成部分的工作原理。

In this chapter, you will learn:

在本章中，你将学到：

- How to build a simple LLM agent from a single API call

  如何仅通过一次 API 调用就构建一个简单的 LLM 代理？
- The components of LLM agents, from single calls to complex agent loops

  LLM 代理的各个组成部分，从单次调用到复杂的代理循环都有涵盖。
- How to write initial prompts

  如何编写初始提示信息
- The two main *evaluation modes* we use throughout this book:

  在本书中，我们采用了两种主要的评估方式：

  Absolute evaluation *绝对评价*

  Judging whether one configuration (e.g., a specific prompt, model, or tool setup) is correct enough to ship

  判断某个配置（例如特定的提示语、模型或工具设置）是否足够完善，可以正式发布。

  Comparative evaluation *比较评估*

Comparing multiple options (A/B tests, pairwise, ranking, leaderboards) to decide which is better

比较多个选项（A/B 测试、成对比较、排序、排行榜等），以确定哪个方案更优。

By the end of this chapter, you will have a clear mental model of what makes up an LLM agent and some of the types of evaluation you might do for your application.

到本章结束时，你应该已经清楚地了解了构成 LLM 代理的要素，以及为你的应用可能需要进行的一些评估类型。

Components of LLM Agents

LLM 代理的组成部分

In Chapter 1, we defined an LLM agent broadly: it takes input, calls a language model one or more times, possibly invokes tools or retrieves documents, and produces output. This section walks through the components that make up an agent, from the simplest case to the most complex.

在第一章中，我们广泛定义了 LLM 代理：它接收输入，调用一次或多次语言模型，可能调用工具或检索文档，并产生输出。本节将介绍构成代理的组件，从最简单的情况到最复杂的情况。

The word "agent" sometimes implies autonomy or multi-step reasoning, but we use it to cover the full range. A single LLM call with no tools is an agent (the degenerate case). A chatbot that maintains conversation history is an agent. A system that calls five tools in a loop to complete a task is also an agent. What changes across these examples is the number of components and how they are wired together.

"代理"这个词有时意味着自主性或多步骤的推理过程，但实际上我们用它来涵盖所有情况。一个仅依赖单一大语言模型的案例就可以被视为一个代理（最简单的情形）。一个能够保存对话历史的聊天机器人也是代理。而一个需要调用五个工具来完成任务的系统同样也是代理。这些例子之间的区别在于各个组件的数量以及它们之间的连接方式。

We cover the components in increasing order of complexity. Each component brings new capabilities, and each comes with its own failure modes to watch for.

我们按照复杂度递增的顺序来介绍各个组件。每个组件都带来了新的功能，但同时也存在各自独特的故障模式，需要特别注意。

One design choice applies to all of these components: *how much autonomy to give the agent*. At one end, you can constrain the agent to a fixed sequence of steps with no discretion. At the other, you can let the agent decide what to do at each turn, including which tools to call and when to stop. More autonomy makes the agent more flexible, but also harder to evaluate, because the space of possible behaviors grows with each decision the agent can make. The right level of autonomy depends on the task and the cost of errors; it's good to decide this early, because it affects what you need to evaluate.

在所有这些组件中，有一个设计上的选择是至关重要的：那就是给智能体多少自主权。一方面，可以限制智能体只能按照固定的步骤序列行事，无法自行决策。另一方面，可以让智能体在每一轮决策中自行决定该做什么，包括使用哪些工具以及何时停止行动。更多的自主权会让智能体更加灵活，但也更难以评估其表现，因为随着智能体做出每一个决策，其可能的行为方式就会不断增加。适当的自主权水平取决于任务的性质以及出错的成本；这一点需要尽早确定，因为它会影响到你需要评估哪些内容。

Before diving into the components, here is how we interact with LLMs programmatically. All the code examples in this chapter use *LiteLLM*, a popular Python library that provides a unified interface for calling different LLM APIs (OpenAI, Anthropic, Google Gemini, and others) with consistent syntax.[1]

在深入组件之前，以下是我们在程序中如何与大型语言模型（LLM）交互。本章中的所有代码示例都使用了 LiteLLM，这是一个流行的 Python 库，它提供了一个统一的接口来调用不同的 LLM API（如 OpenAI、Anthropic、Google Gemini 等），并使用一致的语法。[1]

LLM APIs expect input as a *list of message objects*, where each message has two fields: `role` and `content`. The `role` field specifies who is speaking and can be one of four values:

LLM API 期望输入为消息对象列表，其中每个消息有两个字段：`role` 和 `content`。`role` 字段指定说话者，可以是四个值之一：

system

Sets the overall behavior and context for the assistant (e.g., "You are a helpful assistant that summarizes emails"). This is optional but helps establish consistent behavior across requests.

设定助手的整体行为和上下文（例如："你是一个乐于助人的助手，能够总结电子邮件的内容"。）这是可选的，但有助于在多次请求中保持一致的行为模式。

user

Represents input from the human or application making the request. This is where you provide the actual task or query.

表示由发起请求的人类或应用程序提供的输入信息。在这里，你可以提供具体的任务或查询内容。

assistant

Represents previous responses from the LLM. You include these when maintaining conversation history across multiple turns.

表示大语言模型之前的回应内容。在持续进行多轮对话时，会包含这些回应信息。

tool

Used to pass the results of a tool invocation back to the LLM. As of 2026, every major provider's API (OpenAI, Anthropic, Google, Mistral, Cohere, and others) supports the `tool` role. We discuss tool calling later in this chapter.

这个参数通常用于将工具调用的结果返回给大语言模型。到 2026 年，所有主要的服务提供商（如 OpenAI、Anthropic、Google、Mistral、Cohere 等）都支持 `tool` 这一角色。我们将在本章的后面部分讨论如何调用工具。

For a single request, you typically send a list with one or two messages: an optional `system` message to set context, and a `user` message with your query. For multi-turn conversations, you append each exchange to the list as alternating `user` and `assistant` messages.

对于单个请求，通常需要发送一两条消息：一条可选的 `system` 消息用于提供上下文信息，另一条 `user` 消息则包含具体的查询内容。在多轮对话中，每条消息都被添加到列表中，形式为交替出现的 `user` 和 `assistant` 消息。

When debugging conversation issues, check that `user` and `assistant` messages alternate correctly. Mismatched roles are a common source of errors.

在调试对话问题时，请检查 `user` 和 `assistant` 这两条消息是否能够正确交替出现。角色分配不当是常见的错误来源。

To make each component concrete, we use a running example throughout this chapter: imagine you are building a simple *email summarizer* for your organization. It takes an incoming email and produces a short summary that includes the sender's name and

their key requests.

为了让每个组件都变得具体可行，我们在本章中通过一个实际案例来加以说明：假设您正在为您的组织开发一个简单的邮件摘要生成工具。该工具接收一封邮件，然后生成一份简短的摘要，其中包含了发件人的姓名以及他们的主要需求。

Single LLM Call 单独使用的大语言模型调用

The simplest case is a single call to a model. You provide an input prompt, the model generates a response, and that response is passed back to the user or to the next stage of the system. No tools, no retrieval, no memory, no loops. Just one prompt in, one response out.

最简单的情况就是只对模型进行一次调用。用户提供输入提示，模型生成响应，然后该响应会被传递回给用户或继续传递给系统的下一阶段。无需工具、无需检索、无需记忆，也无需循环处理。只需一个提示，一个响应即可。

For the email summarizer, this could be one prompt that takes the body of an email and asks the model to extract the sender's name and summarize the requests. In code, a single-step call might look like this:

对于邮件摘要生成功能来说，可以这样设计：输入邮件的正文后，让模型提取发件人的姓名并总结邮件中的请求内容。在代码中，这一过程的实现可能如下所示：

```
from litellm import completion

response = completion(
  model="gpt-4o-2024-08-06",
  messages=[
    {
      "role": "system",
      "content": "You are an assistant that summarizes emails."
    },
    {
      "role": "user",
      "content": "Extract the sender's name and summarize "
            "the key requests:\n\n" + email_text
    }
  ]
)

print(response["choices"][0]["message"]["content"])
# Example output:
```

```
# "Sender: Alice Johnson
#  Key requests:
#  - Reset her account password
#  - Verify email on file"
```

One thing to keep in mind: with the default temperature setting (usually above 0), LLM outputs are nondeterministic. The same input can produce different outputs on each run. When evaluating, either set `temperature=0` for reproducibility or run multiple samples and report confidence intervals. We cover this in more detail in Chapter 5.

需要注意的一点是：在默认温度设置下（通常高于 0 度），大语言模型的输出是不确定的。同样的输入在不同运行中的输出可能会有所不同。在进行评估时，为了确保结果的重复性，可以将 `temperature=0` 设置为固定值，或者运行多个样本并报告其置信区间。这一点在第五章中有更详细的说明。

Single LLM calls like this are a natural starting point when building applications. But modern applications often combine additional components, e.g., retrieval to bring in related messages, tools to query internal systems, or agent loops that coordinate multiple steps. Each component brings new capabilities, but also introduces its own potential failure modes.

在构建应用程序时，像这样的独立 LLM 调用自然是一个良好的起点。不过，现代应用程序通常会结合更多的组件，比如检索功能来获取相关信息，使用工具来查询内部系统，或者利用代理循环来协调多个步骤的执行。每个组件都带来了新的功能，但同时也带来了潜在的故障模式。

Conversations 对话/交谈

Many applications don't stop at a single request-response exchange. They allow the user and the model to interact over multiple turns. This conversational setting is what most people associate with chatbots, customer support assistants, or copilots.

许多应用程序不仅仅满足于简单的请求与响应模式。它们允许用户与模型在多次交互中相互交流。这种对话式的交互方式，正是大多数人所认为的聊天机器人、客户支持助手或副驾驶的功能。

In our email example, instead of just producing a summary given a single user message, the system could support follow-up questions:

在我们的电子邮件示例中，该系统不仅可以根据用户的单条消息生成摘要，还可以回答后续的疑问。

User: Summarize the email.
Assistant: Sender: Alice Johnson. Key requests: reset her account password; verify email on file.
User: Does she mention her preferred contact method?
Assistant: Yes. She asks to be contacted by phone.

Technically, a conversation is just a series of LLM calls where the prompt includes the full dialogue history (or at least the recent turns). Here is a minimal illustration:

从技术上讲，对话其实只是一系列由大语言模型发起的“调用”而已。在这些调用中，提示信息会包含整个对话的历史记录（或者至少是最近的几轮对话内容）。以下是一个简单的示例：

```
from litellm import completion

messages = [
  {
    "role": "system",
    "content": ("You are an assistant that summarizes and "
          "answers questions about emails.")
  },
  {"role": "user", "content": "Summarize this email:\n\n" + email_text}
]

response1 = completion(model="gpt-4o-2024-08-06", messages=messages)
print(response1["choices"][0]["message"]["content"])
# "Sender: Alice Johnson. Key requests: reset her account password;
#  verify email on file."

# Add the model's reply to the conversation and continue
messages.append({
  "role": "assistant",
  "content": response1["choices"][0]["message"]["content"]
})
messages.append({
  "role": "user",
  "content": "Does she mention her preferred contact method?"
})

response2 = completion(model="gpt-4o-2024-08-06", messages=messages)
print(response2["choices"][0]["message"]["content"])
# "Yes. She asks to be contacted by phone."
```

Conversation brings the system closer to how people naturally interact with assistants. But it also introduces new design choices: how much dialogue history to keep, how to manage context length, and how to ensure consistency across turns. These are all aspects we will return to when we discuss evaluation.

对话方式使得系统更接近于人们与自然助手之间的互动方式。不过，这也带来了新的设计挑战：需要保留多少对话记录？如何管理对话的上下文长度？又如何确保在不同对话轮次中保持一致性？这些都是我们在讨论评估问题时需要再次考虑的方面。

Retrieval 检索/获取

Sometimes, the LLM needs background knowledge that isn't in the immediate input. Retrieval-augmented generation (RAG) is a common pattern: the system searches a knowledge base or document store, pulls in relevant passages, and then gives both the query and the retrieved context to the model.

有时候，大语言模型需要一些并非直接输入中的背景知识。增强检索生成是一种常见的做法：系统会搜索知识库或文档存储中相关的段落，然后将这些段落作为上下文信息提供给模型，同时也会将查询内容一并传递给模型。

In the email summarizer, retrieval might bring in prior messages from the same sender or related support tickets. That way, the model can understand the new message in light of past correspondence, for example, recognizing that "I still can't log in" refers to an ongoing password reset issue.

在邮件摘要生成器中，检索功能可能会带回来自同一发件人或其他相关支持工单的先前消息。这样，模型就能基于过去的通信记录来理解新消息的含义。例如，当收到"我还是无法登录"这样的消息时，模型就能意识到这可能是密码重置问题仍在持续的情况。

There are two common approaches to retrieval:

有两种常见的检索方法：

Keyword-based methods 基于关键词的方法

These rank documents by how many words they share with the query. The standard algorithm is *BM25* (Robertson and Zaragoza 2009), a classic keyword-matching method from information retrieval that has been around since the 1990s and still works well. Keyword methods are fast and effective when the terminology is stable, but they miss anything that is phrased differently

from the query.

这些文档按照与查询的词汇重复次数进行排序。标准算法是 BM25 算法（Robertson 和 Zaragoza 于 2009 年提出），这是一种经典的关键词匹配方法，源自信息检索领域，自 20 世纪 90 年代以来一直被广泛应用，且效果依然很好。当术语较为稳定时，关键词匹配方法既快速又有效，但这种方法可能会遗漏那些与查询表述方式不同的内容。

Embedding-based methods 基于嵌入的方法

These turn each piece of text (the query and each document) into a list of numbers called an *embedding*, where similar meanings map to similar lists of numbers. Ranking is then done by comparing the numbers, which lets the system match meaning even when wording differs (e.g., "reset my login" versus "can't sign in").

这些操作将每段文本（包括查询和每个文档）转化为一系列数字列表，这些数字列表被称为"嵌入"。其中，相似的含义对应着相似的数字列表。然后通过比较这些数字来进行排序，这样即使措辞不同，系统也能准确识别含义的差异（例如："重置我的登录信息"与"无法登录"）。

A typical retrieval flow has two steps: (1) search for relevant items, then (2) insert those items into the LLM prompt. Here is an example:

典型的检索流程包含两个步骤：(1) 寻找相关的项目，然后(2) 将这些项目插入到 LLM 的提示中。以下是一个示例：

```
# Small archive of past support emails
past_emails = [
    "Ticket #4521: Please reset my password. I cannot log in.",
    "Ticket #4605: Could you verify my email address on file?",
    "Ticket #4710: I need help updating my phone number."
]

# Embedding-based retrieval
from litellm import embedding, completion
import numpy as np

# Cosine similarity scores two embedding vectors from -1 (opposite)
# to 1 (identical). Higher means the two pieces of text are closer
# in meaning.
def cosine_similarity(a, b):
    return np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b))
```

```
corpus_response = embedding(
    model="text-embedding-3-small",
    input=past_emails
)
corpus_embeddings = [item["embedding"] for item in corpus_response["data"]]

query_response = embedding(
    model="text-embedding-3-small",
    input=email_text
)
query_embedding = query_response["data"][0]["embedding"]
scores = [cosine_similarity(query_embedding, e) for e in corpus_embeddings]
retrieved = past_emails[int(np.argmax(scores))]

# Combine retrieved context with the new email
messages = [
    {
        "role": "system",
        "content": "You are an assistant that summarizes emails with "
                   "context."
    },
    {
        "role": "user",
        "content": (f"Email:\n{email_text}\n\n"
                    f"Related past email:\n{retrieved}")
    }
]

response = completion(model="gpt-4o-2024-08-06", messages=messages)
print(response["choices"][0]["message"]["content"])
# Example output:
# "Sender: Alice Johnson.
#  Key requests: reset her account password and confirm contact information.
#  Context: this continues a previous unresolved login issue."
```

Keyword-based retrieval could be swapped in here using a library like `whoosh` or `elasticsearch`, returning the top BM25 match instead of computing embeddings.

可以使用类似 `whoosh` 或 `elasticsearch` 这样的库来进行关键词检索，从而返回最匹配的 BM25 模型的结果，而非计算嵌入向量。

Whichever method you choose, retrieval changes the system's behavior: the quality of the output now depends both on what gets retrieved and on how the model uses it.

These are issues we return to when we discuss RAG evaluation in Chapter 7.

无论选择哪种方法，检索都会改变系统的行为：输出的质量现在取决于被检索到的内容以及模型如何处理这些信息。这些问题将在第 7 章中讨论 RAG 评估时再次被提及。

Tools 工具/器具

Some tasks are better handled by software than by the model itself. Think about doing exact math, pulling information from a system of record, or reformatting something like a date or a phone number. In these cases, it is easier and more reliable to let the model call a tool instead of trying to produce the result on its own.

有些任务更适合由软件来处理，而不是由模型本身来完成。比如进行精确的数学计算、从数据库中提取信息，或者对日期或电话号码等数据进行格式化处理。在这些情况下，让模型调用外部工具来完成这些任务会更加简单且可靠，而不是试图让模型自行生成结果。

A tool is a function that the model can call. In practice, tools can be implemented in any language or as API endpoints; we use Python here because our code examples are in Python. We describe the function to the model, along with the arguments it expects, so the model knows how to invoke it. When the model calls the tool, our runtime executes the function and feeds the result back into the conversation.

工具是模型可以调用的功能。实际上，这些工具可以用任何语言实现，也可以作为 API 端点提供。在这里，我们使用 Python 作为实现语言，因为我们的代码示例都是用 Python 编写的。我们会向模型描述这个功能以及它所需的参数，这样模型就知道如何调用这个工具。当模型调用这个工具时，运行环境会执行这个功能，并将结果返回到对话中。

In the email summarizer, suppose the sender includes a callback number like “Call me at (415) 555-9021 ext. 7”. If you want to store this in your customer relationship management (CRM) or pass it into an automated dialer, you'll need the number in a standard format. Rather than relying on the model to rewrite phone numbers consistently, you can give it access to a `normalize_phone` tool that always produces the same output format:

在邮件摘要生成器中，假设发件人提供了类似“请拨打(415) 555-9021 ext. 7”这样的回拨号码。如果你想将这一信息存储在客户关系管理系统中，或者将其传递给自动拨号程序，就需要以标准格式存储该号码。而不是依赖模型来持续重新格式化电话号码，你可以允许该系统使用 `normalize_phone` 工具，该工具始终能生成相同格式的输出结果。

```
from litellm import completion
import json
```

```python
import phonenumbers

# Define the tool as a Python function
def normalize_phone(number: str):
    parsed = phonenumbers.parse(number, "US")
    return phonenumbers.format_number(
        parsed, phonenumbers.PhoneNumberFormat.E164
    )

# Tell the model what the tool is and how to call it
tools = [
    {
        "type": "function",
        "function": {
            "name": "normalize_phone",
            "description": "Convert a phone number to "
                           "standardized E.164 format",
            "parameters": {
                "type": "object",
                "properties": {"number": {"type": "string"}},
                "required": ["number"]
            }
        }
    }
]

messages = [
    {
        "role": "system",
        "content": "You summarize emails and may call tools when helpful."
    },
    {"role": "user", "content": f"Summarize this email:\n\n{email_text}"}
]

resp = completion(model="gpt-4o-2024-08-06", messages=messages, tools=tools)
msg = resp["choices"][0]["message"]

# If the model calls the tool, run it and send back the result
if "tool_calls" in msg:
    # First, append the assistant's own message (which carries the tool_calls)
    messages.append(msg)
    for call in msg["tool_calls"]:
        if call["function"]["name"] == "normalize_phone":
            args = json.loads(call["function"]["arguments"])
            result = normalize_phone(**args)
            messages.append({
```

```
            "role": "tool",
            "tool_call_id": call["id"],
            "name": "normalize_phone",
            "content": json.dumps(result)
        })

# Continue the conversation until the model produces a final summary
resp2 = completion(model="gpt-4o-2024-08-06", messages=messages, tools=tools)
print(resp2["choices"][0]["message"]["content"])
# Example:
# "Sender: Alice Johnson. Key requests: reset her account password.
#  Preferred contact: +14155559021"
```

This approach is not limited to small utilities. In practice, applications often expose tools such as calculators, calendar lookups, search engines, or CRM APIs. These let the model handle parts of the task that require precision, data access, or integration with existing systems.

这种方法并不局限于小型公用事业领域。实际上，许多应用程序都会使用到各种工具，比如计算器、日历查询功能、搜索引擎，或是客户关系管理相关的 API。这些工具使得模型能够处理那些需要精确计算、数据访问，或者需要与现有系统集成的工作任务。

WARNING 警告

Tools give LLMs the ability to take actions. Always validate tool inputs and consider the security implications before allowing an LLM to call functions that modify data or external systems.

这些工具使得大语言模型能够执行各种操作。在让大语言模型调用那些可能修改数据或外部系统的函数之前，务必先验证其输入参数，并考虑相关的安全问题。

Retrieval can also be seen as a tool: instead of returning a phone number, the function returns a set of documents given a query. We treat it separately here because retrieval is such a common building block.

检索也可以被视为一种工具：它并非只是返回电话号码，而是根据查询返回一组文档。在这里，我们将检索单独处理，因为检索是系统中非常常见的功能之一。

A recent effort to make this easier is the *Model Context Protocol* (MCP). MCP defines a standard way for a tool server to describe available functions (including their names, arguments, and authentication requirements) and for a client to discover and call them. Instead of writing custom glue code for every service, you can point your appli-

cation at an MCP server and the model immediately has a catalog of tools it can use.

最近有一种尝试来简化这一过程的方案，那就是模型上下文协议（Model Context Protocol，MCP）。MCP 定义了工具服务器描述可用功能的标准方式——包括这些功能的名称、参数以及认证要求等信息。而客户端则可以通过该协议发现并调用这些功能。这样一来，就无需为每项服务编写专门的代码，只需将应用程序连接到 MCP 服务器后，模型就能立即获得可供使用的工具列表。

MCP also introduces evaluation challenges that don't arise with hardcoded tool definitions. Tool servers can add, remove, or change their schemas between calls, so an agent that worked yesterday may break today because a tool signature changed. Permission scoping becomes important too: if an MCP server exposes 50 tools but your agent should only use 3 of them, you need to verify that the agent stays within its authorized set. We return to evaluating tool use in Chapter 8.

MCP 还引入了一些评估挑战，这些问题在使用固定化的工具定义时并不会出现。工具服务器可以在调用之间添加、删除或更改自己的模式，因此，昨天还有效的代理今天可能会因为工具签名的变化而失效。权限范围的控制也非常重要：如果 MCP 服务器提供了 50 种工具，而你的代理只被允许使用其中的 3 种，那么就需要确保代理使用的工具数量始终在授权的范围内。关于如何评估工具的使用情况，我们将在第八章中进行讨论。

Agent Loops 特工循环

So far we have seen single calls, conversations, retrieval, and tools as separate components. These are all agents in the broad sense we defined in "What Is an Agent?", but they are relatively simple. The most complex pattern is an *agent loop*. In this pattern, the LLM receives the current state, decides what action to take, executes it, observes the result, and repeats until the task is complete. The defining feature is that the model decides each step as it goes, rather than following a predetermined sequence.

到目前为止，我们已经将单个调用、对话、数据检索以及工具视为独立的组件。这些都属于我们在"什么是代理？"一文中定义的各种代理类型，不过它们相对较为简单。最复杂的模式是代理循环。在这种模式下，大语言模型会接收当前状态，决定要执行什么操作，执行该操作后观察结果，然后重复这一过程，直到任务完成。这种模式的特点在于，模型的每一步决策都是随着实际情况而自动决定的，而不是按照预先设定的顺序进行。

In the email summarizer example, an agent loop might start by reading the new message and deciding that more context is needed. It retrieves past correspondence from the same sender, looks at the result, and then calls a tool to normalize a callback number. Only after those steps does it decide that it now has enough information to pro-

duce a final summary:

在邮件摘要示例中，一个代理循环可能从读取新消息开始，并决定需要更多上下文。它检索来自同一发件人的过去通信，查看结果，然后调用一个工具来规范化回调号码。只有完成这些步骤后，它才决定现在拥有足够的信息来生成最终摘要：

```
# Minimal sketch of an agent loop
from litellm import completion

messages = [
  {
    "role": "system",
    "content": ("You are an agent that can retrieve past emails, "
          "call tools, and summarize.")
  },
  {
    "role": "user",
    "content": f"Summarize this new email:\n\n{email_text}"
  }
]

for step in range(10):  # cap the number of steps
  resp = completion(model="gpt-4o-2024-08-06", messages=messages, tools=tools)
  msg = resp["choices"][0]["message"]

  # If the model requests a tool, run it and append the result
  if "tool_calls" in msg:
    messages.append(msg)  # append the assistant turn with tool_calls
    # (execute retrieval or tool functions here)
    messages.append({
      "role": "tool",
      "tool_call_id": msg["tool_calls"][0]["id"],
      "content": "Tool result goes here"
    })
    continue

  # If the model produces a final answer, stop
  if msg.get("content"):
    print("Final output:", msg["content"])
    break
```

Frameworks like LangChain, LlamaIndex, and AutoGPT wrap this pattern with additional features such as memory, planning strategies, and error handling. When reading blog posts or papers about agents, it is easy to feel overwhelmed with jargon and

seemingly complex ideas. But the idea is simple: an agent loop is just repeated LLM calls that decide, step by step, whether to generate text, retrieve information, call a tool, or stop when the task is complete. (Recall that we defined "agent" broadly in "What Is an Agent?" to include even a single LLM call; the loop pattern is the most complex form an agent can take.)

像 LangChain、LlamaIndex 和 AutoGPT 这样的框架为这种模式增加了一些额外的功能，比如内存管理、规划策略以及错误处理。在阅读关于智能体的博客文章或论文时，很容易会被各种专业术语和看似复杂的概念弄得不知所措。但实际上，这个概念很简单：智能体循环其实就是不断地调用大语言模型，逐步决定是生成文本、检索信息、调用工具，还是在任务完成之后停止操作。（回想一下，我们在"什么是智能体？"这一章节中就已经广义地定义了"智能体"的概念，甚至包括单个大语言模型的调用行为；而智能体循环则是智能体所能采取的最复杂的形态。）

NOTE 注意/备注

Agent loops can, in theory, run indefinitely if not bounded. Always include a maximum step count (as shown in the example with `range(10)`) and consider implementing timeouts in production systems.

理论上，如果不存在限制条件，代理循环可以无限运行下去。不过，始终需要设置一个最大步数限制（如 `range(10)` 中的示例所示），并且在实际系统中考虑引入超时机制。

The examples in this chapter assume a single agent, but some applications use multiple agents that coordinate with each other. Multi-agent architectures introduce additional evaluation challenges, such as whether agents pass the right information between them and whether the overall system produces coherent results. The evaluation principles in this book still apply; the main difference is that you need to evaluate both individual agents and their interactions.

本章中的例子都假设只有一个代理节点，但有些应用程序会使用多个代理节点，它们需要相互协作。多代理架构带来了额外的评估挑战，比如如何确保各个代理节点能够准确传递信息，以及整个系统是否能够产生一致性的结果。本书中的评估原则仍然适用；主要的区别在于，你需要同时评估各个代理节点本身以及它们之间的交互情况。

Fundamentals of Prompting

提示的基本技巧

So far we have covered the architectural components of an LLM agent, including single calls, conversations, retrieval, tools, and the agent loop. Another set of fundamentals is how to write the prompts that drive each of these components. A good prompt is specific enough to guide the model and consistent enough to give predictable outputs. Over time you will refine it, but a strong starting point has a few recurring pieces.

到目前为止，我们已经介绍了 LLM 代理的架构组成部分，包括单次调用、对话处理、信息检索、工具使用以及代理的循环机制。另一项基础技能则是如何编写能够驱动这些组件的提示语。一个好的提示语应该具有足够的具体性以指导模型的行为，同时又要保持一致性，从而输出可预测的结果。随着时间的推移，你可以不断改进这些提示语，但一个好的起点通常包含一些反复使用的元素。

Prompts for Single LLM Calls
提示进行单次大语言模型调用

A good first draft prompt usually includes:

一个好的初稿提示通常包括：

Role 角色

Tell the model who it is meant to be. This keeps it anchored in the right domain.

告诉模型它应该属于哪个领域。这样就能确保它始终适用于正确的领域。

Objective 目标

State clearly what the task is and what success looks like.

明确说明任务的内容以及成功的标准是什么。

Instructions 指示/说明

Spell out dos and don'ts. For example, if you need three bullet points, say so. If quoted text should be excluded, make that explicit.

请明确列出可以做什么和不能做什么。例如，如果有三个要点需要提及，请明确指出。如果不需要引用其他文本，也请说明这一点。

Context 背景

Provide the data the model should act on, like the email, the document, or any other text snippets. Without context, it can only guess.

提供模型应处理的数据，比如电子邮件、文档或其他文本片段。如果没有上下文信息，模型只能进行猜测而已。

Reasoning 推理

For harder tasks, you can ask the model to explain its reasoning before returning the answer. Older models benefit from explicit "think step by step" instructions; newer reasoning models (such as OpenAI's o-series or DeepSeek R1) generate internal chains of thought automatically, so adding these instructions can actually hurt performance by interfering with the model's own reasoning process.

对于难度较大的任务，可以要求模型在给出答案之前先解释其推理过程。较旧的模型可以通过明确的"逐步思考"指令来辅助其推理过程；而较新的推理模型（如 OpenAI 的 o 系列或 DeepSeek R1）则能够自动生成内部的思维链。因此，添加这些指令可能会干扰模型的推理过程，从而降低其性能。

Output format 输出格式

Describe how the answer should be structured so it's easy to consume downstream.

描述一下这个答案的结构应该如何设计，以便后续能够轻松地被理解。

Examples 示例/例子

Optional but powerful. A short input-output pair shows format, style, and level of detail.

可选但功能强大。简短的输入输出对可以展示格式、风格以及详细程度。

Delimiters 分隔符

Use section headers, code fences, or tags to keep instructions, context, and examples separate.

使用段落标题、代码框或标签来区分说明、上下文和示例。

In practice, you typically organize these elements into a *prompt template*, a reusable structure with placeholders for variable content that will be filled in at runtime. For example, the *[PASTE EMAIL TEXT HERE]* placeholder would be replaced with the actual email text when the application runs. Using templates makes prompts easier to maintain, debug, and version control, since you can modify specific sections without

rewriting the entire prompt each time.

在实践中，通常会将这些元素组织成一个模板结构，这种结构可以重复使用，其中的占位符可以随着运行环境的改变而填充具体的内容。例如， *[PASTE EMAIL TEXT HERE]* 这样的占位符在应用程序运行时会被实际的电子邮件文本所替换。使用模板可以使得提示信息更易于维护、调试和版本控制，因为你可以只修改特定的部分，而不需要每次都重写整个提示信息。

Example 2-1 shows how the aforementioned elements come together in a prompt template for our email summarizer.

示例 2-1 展示了上述各个元素如何组合在一起，形成我们的电子邮件摘要生成器的模板。

Example 2-1. Prompt template for email summarizer

示例 2-1：用于邮件摘要生成器的提示模板

```
### Role
You are an assistant for our internal support team.

### Objective
Extract the sender's name and summarize the key requests in an incoming email.

### Instructions
- Produce up to three concise bullet points for the requests.
- Do not include quoted text from prior threads or signatures.
- Keep the tone plain and professional.

### Context
[PASTE EMAIL TEXT HERE]

### Reasoning
Think through the steps needed to extract the sender and requests,
but return only the final formatted output.

### Output format
- Sender: <name>
- Requests:
  - <bullet 1>
  - <bullet 2>
  - <bullet 3>

### Example
Input: "My account was locked after travel. Please reset it and
call me at (415) 555-9021. – Alice Johnson"
```

Output:
- Sender: Alice Johnson
- Requests:
  - Reset account access
  - Call back at (415) 555-9021

Another benefit of using a template is that it makes prompts easier to refine. Suppose you discover the model sometimes includes quoted email threads in its summaries, and you don't want the summaries to include too much information from previous emails in a thread. With a structured template, you know exactly where to fix this. Add a line in the instructions section that says, "Do not include quoted text from prior threads." Or imagine you realize the summaries don't always include a clear call to action for the support agent. You can add a new field in the format section, for example, "Next action for agent: <short instruction>." Overall, instead of piling ad hoc notes into one long block, the template keeps changes organized and easy to maintain.

使用模板的另一个好处是，它使得提示的优化变得更加容易。例如，你可能会发现模型在总结中偶尔会引用之前的邮件内容，而你不希望总结中包含过多的这些邮件信息。有了结构化的模板，你就知道该在哪里进行修改。可以在说明部分添加一条提示："不要引用之前邮件中的文本。"或者，如果你发现总结中并不总是包含明确的行动提示，可以在格式部分添加一个新字段，例如："代理的下一步行动：<简短提示>。"总的来说，与随意记录笔记相比，使用模板可以让更改更加有组织，也更容易维护。

TIP 提示/窍门

Store your prompt templates as separate files or configuration rather than hardcoding them. Commit them to version control, and tag or hash each version so you can trace any output back to the exact prompt that produced it. When logging traces, record the prompt version alongside the model version and input; this makes it possible to tell whether a behavior change came from a prompt edit, a model update, or a data shift.

将提示模板存储在单独的文件中或配置中，而不是硬编码在代码中。将这些模板提交到版本控制系统中，并为每个版本添加标签或哈希值，这样就能将任何输出结果追溯到具体的提示语句。在记录跟踪时，同时记录提示版本和模型版本及输入内容；这样就能判断是提示编辑、模型更新还是数据变化导致了行为的改变。

Retrieval Fundamentals 检索基础

When implementing retrieval for your application, we recommend starting with keyword search. It is fast to set up, easy to interpret, and often a very strong baseline.

With keyword methods like BM25, you can see exactly why a document was retrieved, because the query terms overlap with it. This transparency makes debugging straightforward, and in many domains with stable vocabulary (like support tickets), keyword search performs surprisingly well.

在您的应用程序中实现检索功能时，我们建议首先从关键词搜索开始。关键词搜索的设置非常快速，且易于理解，通常也能提供非常可靠的基础数据。通过使用像 BM25 这样的关键词算法，您可以清楚地了解为什么某个文档被检索到了，因为查询条件与文档内容有重叠。这种透明性使得调试过程变得简单，而在许多词汇表相对稳定的领域（如支持请求等），关键词搜索的表现相当出色。

Here is a toy example using the `rank_bm25` library on a small list of past support emails:

这里有一个使用 `rank_bm25` 库的玩具示例，适用于一小份过去支持的邮件列表：

```
from rank_bm25 import BM25Okapi
import numpy as np

# Small archive of past emails
past_emails = [
    "Ticket #4521: Please reset my password. I cannot log in.",
    "Ticket #4605: Could you verify my email address on file?",
    "Ticket #4710: I need help updating my phone number."
]

# Build BM25 index
tokenized_corpus = [doc.lower().split() for doc in past_emails]
bm25 = BM25Okapi(tokenized_corpus)

query = "I still can't log in."
tokenized_query = query.lower().split()

# Score and select the best match
scores = bm25.get_scores(tokenized_query)
best_doc = past_emails[int(np.argmax(scores))]

print(best_doc)
# Example output:
# "Ticket #4521: Please reset my password. I cannot log in."
```

This is only a toy demonstration. In practice, your corpus may be thousands of emails or tickets stored in a database, and you would use the database's own retrieval fea-

tures or a dedicated search library to handle indexing and ranking. The important part is that you begin with a simple, interpretable baseline. From there, if you find it misses relevant cases, you can move on to embedding search or to *hybrid search*, which combines keyword and dense retrieval scores using a method like reciprocal rank fusion (RRF; Cormack et al. 2009). Hybrid search has become the default in many production RAG systems because keyword matching catches exact terms that embeddings miss, while embeddings catch semantic similarity that keywords miss. Most vector databases (e.g., Pinecone, Weaviate, and Qdrant) support hybrid search natively.

这只是一个玩具级的演示而已。在实际应用中，你的数据可能包含数千封电子邮件或票据，这些数据存储在数据库中。此时，你可以利用数据库自带的检索功能或专门的搜索库来处理索引和排序问题。重要的是，要从一个简单且易于理解的基准模型开始。如果发现模型未能处理一些相关案例，那么就可以采用混合搜索的方式，即结合关键词检索结果和嵌入检索结果，通过类似互惠排名融合算法（RRF；Cormack 等人，2009 年）来实现。由于关键词匹配能够捕捉到嵌入检索未能捕捉到的精确术语，而嵌入检索则能够捕捉到关键词无法实现的语义相似性，因此混合搜索已成为许多生产型 RAG 系统的默认选择。大多数向量数据库（如 Pinecone、Weaviate 和 Qdrant）都原生支持混合搜索功能。

Tools and Agents 工具与代理人

When you add tools to an application for the first time, the main step is to describe them clearly to the model. Most modern APIs expect a specification in the style of OpenAPI: a function name, a short description, and a JSON schema for its arguments. Precision matters. Write descriptions that explain exactly what the function does, define arguments unambiguously, and think through edge cases. For example, if a number must be a string, say so. If the tool only works for US phone numbers, make that explicit. The clearer the specification, the less room there is for the model to make mistakes while using the tool.

当你首次向应用程序添加工具时，最关键的步骤就是向模型清晰地描述这些工具。大多数现代 API 都要求使用类似 OpenAPI 的规范来描述工具：包括函数名、简短的描述以及参数的 JSON 格式描述。精确性非常重要。请编写能够准确说明函数功能的描述，明确定义参数，并考虑好边缘情况。例如，如果某个参数必须是一个字符串，那就明确说明；如果工具仅适用于美国电话号码，也请明确指出。规范越清晰，模型在使用工具时出错的可能性就越小。

Here is an example tool spec for normalizing phone numbers:

以下是用于规范电话号码的工具规格说明的一个例子：

```
{
  "type": "function",
  "function": {
    "name": "normalize_phone",
    "description": "Convert a phone number to standardized E.164 format",
    "parameters": {
      "type": "object",
      "properties": {
        "number": {
          "type": "string",
          "description": "Phone number to normalize, assumed to be US "
                         "if no country code is provided"
        }
      },
      "required": ["number"]
    }
  }
}
```

A first-pass tool prompt can reuse the same template we developed earlier (role, objective, context, instructions, format). The only addition is a section that lists the available tools and explains when they should be used. For example: "You summarize emails. You may call `normalize_phone` whenever a phone number is present. Always include the tool's result in the final summary." Combined with the tool spec, this is enough to get a simple tool-enabled agent running.

在初次使用工具提示时，可以重复使用我们之前开发的模板结构：角色、目标、上下文、说明和格式。唯一的补充是添加一个部分，列出可用的工具以及说明何时需要使用这些工具。例如："您负责总结电子邮件内容。当遇到电话号码时，可以调用 `normalize_phone` 工具。务必在最终总结中包含该工具的结果。"结合工具说明，这就足以让简单的工具驱动代理开始运行了。

Once you have prompts, retrieval, and tools in place, you can combine them into an *agent*. An agent is simply a system that loops through the following cycle: the model reasons about the current state, decides whether to act (by calling a tool or retrieving context), observes the result, and continues until it produces a final answer. This pattern is often called *ReAct* (short for "Reason + Act"; Yao et al. 2023). Modern LLM APIs now handle much of the ReAct loop internally; tool calling APIs automatically interleave reasoning and action without requiring explicit ReAct prompts. The pattern is still useful to understand, because the underlying structure is the same. Example 2-2

shows a ReAct-style agent prompt for our email summarizer.

一旦你有了提示、检索工具和相应的机制，你就可以将它们结合起来形成一个智能体。智能体本质上是一个不断循环运行的系统：首先，模型对当前状态进行分析，然后决定是否需要采取行动（通过调用工具或检索上下文信息），接着观察行动的结果，并继续这个过程，直到得出最终答案。这种模式通常被称为 ReAct 模式（即“推理+行动”的缩写；Yao 等人，2023 年）。如今，现代大语言模型 API 已经能够内部处理 ReAct 模式中的大部分逻辑；工具调用 API 可以自动结合推理和行动，而无需额外的 ReAct 提示。这种模式仍然很有用，因为其底层结构是相同的。示例 2-2 展示了一个适用于我们的电子邮件摘要生成器的 ReAct 风格智能体提示。

Example 2-2. Agent prompt (ReAct-style) for email summarizer

示例 2-2. 用于邮件摘要生成器的 ReAct 风格提示语

```
### Role
You are an agent that helps summarize support emails.

### Objective
Read a new email, use retrieval and tools when helpful, and produce
a final summary for the support team.

### Instructions
- You may retrieve past tickets if needed.
- You may call `normalize_phone` when a phone number is present.
- Work step by step. At each step, decide on one action
(retrieve, call a tool, or stop with a final answer).
- After you take an Action, you will receive an Observation,
with the result. Use it to decide your next step.
- End with a final answer in the specified format.
- If the user's request is ambiguous, [choose one]:
**(a)** ask a clarifying question before proceeding;
**(b)** infer the most likely intent and continue.

### Reasoning (ReAct pattern)
Follow the Thought → Action → Observation cycle:
- **Thought:** (optional, for development) a brief statement of what
you intend to do.
- **Action:** the tool to call, with JSON arguments.
- **Observation:** will be provided after each action.
- Repeat until you produce the final answer.

### Output format (for Final)
Final answer:
- Sender: <name>
```

```
- Requests:
  - <bullet 1>
  - <bullet 2>
  - <bullet 3>
- Phone: <normalized number if present>

### Step format example
Thought: I should check prior tickets for context.
Action: retrieve_tickets
Args: {"query": "can't log in; password reset", "sender": "<sender_email>"}

Observation: [{"id":"4521","text":"Please reset my password. I cannot log in."}]

Thought: A phone number is present; I will normalize it.
Action: normalize_phone
Args: {"number": "(415) 555-9021 ext. 7"}

Observation: "+14155559021"

Thought: I have enough information to write the summary.
Final answer:
- Sender: Alice Johnson
- Requests:
  - Reset account access
  - Verify email on file
  - Call back at
  - Phone: +14155559021
```

One important design choice in building an agent is how much freedom it should have. For example, if you want it to be cautious and avoid guessing on behalf of the user, include some instruction such as: "If anything is ambiguous, always ask the user to clarify before proceeding." If you want it to be more autonomous, instead say: "If anything is ambiguous, infer the most likely intent and continue." Making this decision explicit in the prompt will make the agent more predictable and easier to evaluate later.

在构建智能体时，一个重要的设计决策是给智能体提供多大的自由度。例如，如果你希望智能体保持谨慎，避免为用户做出猜测，那么可以在指令中加上这样的提示："如果有什么内容不清楚，请务必先询问用户以得到明确说明。"如果你希望智能体更加自主地行动，那么可以改为："如果有什么内容不清楚，请推断出最可能的意图并继续操作。"在指令中明确说明这一点，可以让智能体的行为更加可预测，也便于后续评估。

Evaluation Setups 评估设置

A wide variety of evaluation setups is described in the literature and used in practice. We can bucket them all into two types of high-level questions: “Is this system good enough to ship?” and “Which of these options works better?” The first question requires *absolute evaluation*, or judging whether your system meets some standard of correctness. The second requires *comparative evaluation*, or deciding which of several alternatives performs best.

在文献中描述了多种评估方法，并且这些方法在实际应用中也得到了广泛使用。我们可以将这些问题归结为两类高层次的问题：“这个系统是否足够好，可以投入使用？”以及“在这些选项中，哪个方案表现得更好？”第一个问题需要绝对评估，即判断系统的正确性是否达到某种标准。第二个问题则需要比较评估，即决定几个选项中哪个表现最好。

Absolute Evaluation 绝对评价

This is where most teams start. You've built a system (say, the email summarizer with its first draft prompt) and you need to know if it's good enough to use. That means defining what “good enough” actually means, then checking whether your outputs meet that bar. In the literature, metrics for absolute evaluation are often grouped into one of the following:

大多数团队都是从这里开始工作的。你们已经构建了一个系统（比如那个包含初稿提示的邮件摘要生成工具），现在需要确定它是否足够好以投入使用。这意味着要明确“足够好”到底意味着什么，然后检查你的输出结果是否达到了这个标准。在相关文献中，用于绝对评估的指标通常被归类为以下几种：

Reference-based evaluation

基于参考标准的评估

> You compare the model's output against labeled data, often called *ground truth*. Depending on the task, ground truth might be a human-written summary, an extracted field (like a name or date), a classification label, or a structured record.
>
> 你将模型的输出结果与标记过的数据进行比较，这些标记过的数据通常被称为“真实数据”。根据任务的不同，真实数据可能是人类编写的摘要、提取出的字段信息（如名称或日期）、分类标签，或者结构化的记录。
>
> For the email summarizer, you'd prepare a dataset of emails with human-written summaries, then score each model output for exact match, overlap with the ref-

erence, or semantic similarity. This approach is appealing because it gives clear right-wrong comparisons. But it has two problems: labeled data is expensive to create, and many tasks have multiple valid outputs. One good summary might say, “Reset password; confirm email on file,” while an equally valid one says, “Help me log in; verify my registered address.”

对于电子邮件摘要生成器来说，你需要准备一个包含人工撰写的邮件摘要的数据集。然后，对各个模型的输出进行评分，根据精确匹配程度、与参考摘要的重叠程度以及语义相似性来进行判断。这种方法的优点在于能够清晰地区分正确和错误的摘要。不过，这种方法也存在两个问题：标注数据成本高昂，而且许多任务都有多种可行的输出结果。例如，一个不错的摘要可能是“重置密码；确认存档中的电子邮件地址”，而另一个同样不错的摘要可能是“帮助我登录；验证我的注册地址”。

Scoring against only one reference penalizes perfectly good outputs. We recommend reference-based methods only when there’s genuinely a single correct answer, like verifying whether a field was extracted correctly.

仅基于一个参考答案来评分，会严重贬低那些表现相当好的答案。我们建议仅在确实存在唯一正确答案的情况下才使用基于参考答案的方法，比如验证某个字段是否被正确提取了。

Reference-free evaluation

无参照的评估

Here, you check the output against criteria you define, without needing a gold answer.

在这里，你可以根据自己定义的标准来检验输出结果，而不需要达到完美的标准。

For the email summarizer, you might verify that the output includes a Sender: field, produces exactly three bullet points, and avoids quoted text from previous threads. You might also check that it doesn’t invent details that aren’t explicitly mentioned in the email. Correctness is usually a set of criteria like this (format, completeness, factuality) rather than matching a single right answer. You can automate these checks with rules or use another model as a judge.

对于邮件摘要生成工具，您需要确保输出中包含发件人信息字段，生成的摘要恰好包含三个要点，并且避免引用之前邮件中的内容。同时，要确保生成的摘要中没有出现邮件中未明确提及的细节信息。准确性通常涉及到多个标准，如格式、完整性、事实准确性等，而不只是简单地符合某个正确答案。您可以使用规则来自动化这些检查，或者借助其他模型作为评判依据。

Most teams don't start with labeled datasets for their specific application, so reference-free evaluation is one practical option. This book focuses primarily on reference-free methods for that reason. That said, investing in reference data pays off significantly when you can do it. Even a small set of human-labeled "golden" outputs (e.g., 50 to 100 examples) gives you a ground truth to validate automated judges against, catch regressions on known-good cases, and measure improvement over time. The error analysis process in Chapter 3 naturally produces these labeled examples as a byproduct of reading and annotating traces.[2]

大多数团队在特定应用上不会从标记好的数据集开始，因此无参考评估是一个实用的选择。本书主要关注无参考方法，原因就在于此。不过，当你能够这样做时，投资参考数据会带来显著回报。即使只有少量人工标记的"黄金"输出（例如 50 到 100 个例子），也能为你提供用于验证自动判定的真实情况，捕捉在已知良好案例上的退化，并衡量随时间的改进。第三章中的错误分析过程在读取和标注跟踪时，自然会产生这些标记好的例子作为副产品。[2]

Comparative Evaluation 比较评估

Once you have something working reasonably well, you might want to compare alternatives, such as different prompts, models, or retrieval methods. Now the question isn't "is this correct enough?" but "which of these is better?" Comparative evaluation typically takes one of these forms:

一旦你让某件事物基本运行起来，你可能想要比较不同的替代方案，比如不同的提示、模型或检索方法。现在的问题不再是"这个足够正确吗？"而是"这些之中哪个更好？"比较评估通常有以下几种形式：

Pairwise comparison 成对比较

For each input, you generate two outputs, one from prompt A and one from prompt B. An evaluator (human or model) picks which one better meets some criterion. For the email summarizer, the question might be: "Which output more accurately captures the sender's requests without adding extra details?"

对于每个输入，你生成两个输出，一个来自提示 A，一个来自提示 B。一个评估者（人类或模型）选择哪个输出更好地满足某个标准。对于电子邮件摘要器，问题可能是："哪个输出更准确地捕捉了发件人的请求，而没有添加额外细节？"

Ranking and leaderboards

排名和排行榜

Instead of comparing just two outputs, evaluators rank three or more options for the same input. Aggregate these rankings across many inputs and you get system-level scores displayed as a leaderboard. Public benchmarks like LMArena use this format to compare different AI models. Some companies maintain internal leaderboards to track which prompt, model, or configuration performs best.

与其只比较两个输出，评估者会对相同输入的三个或更多选项进行排名。将多个输入的这些排名汇总起来，就可以得到系统级别的分数，并以排行榜的形式展示。像 LMArena 这样的公共基准测试就使用这种格式来比较不同的 AI 模型。一些公司会维护内部排行榜，以追踪哪个提示、模型或配置表现最佳。

We do not cover comparative evaluation in depth in this book, because most teams' first bottleneck is getting a single configuration to work reliably. Comparative evaluation becomes useful once you have a working system and want to choose between alternatives (e.g., two prompts, two models, or two retrieval strategies). At that point, pairwise comparison is a low-cost way to make early development decisions quickly. For production changes with measurable user impact, statistically rigorous A/B testing is more appropriate. Chapter 9 touches on how comparative evaluation fits into the CI/CD workflow.

在本书中，我们没有深入讨论比较性评估。因为大多数团队面临的首要难题就是让某个配置能够稳定运行。一旦系统已经运行起来，当需要在不同方案之间进行选择时，比较性评估就变得很有用了（例如，两种提示方式、两种模型或两种检索策略）。此时，成对比较是一种低成本且能够快速做出开发决策的方法。对于那些需要衡量用户影响的实际变更，统计上严谨的 A/B 测试更为合适。第 9 章探讨了比较性评估如何融入 CI/CD 工作流程中。

Putting It Together 整合起来

You now have the building blocks for an LLM agent. These include single LLM calls, conversations, retrieval, tools, and agent loops. You understand the two main evaluation approaches: absolute evaluation (is this good enough?) and comparative evaluation (which is better?). You've seen how to structure prompts with templates.

您现在拥有了构建 LLM 代理的基础。这些基础包括单个 LLM 调用、对话、检索、工具和代理循环。您了解两种主要的评估方法：绝对评估（这个够好吗？）和比较评估（哪个更好？）。您已经看到了如何使用模板来构建提示。

When you run your prototype on real data, you'll find problems. The model might interpret instructions differently across inputs. Retrieval might return irrelevant docu-

ments. Output quality might vary unpredictably. These problems are your starting point for evaluation.

当你在真实数据上运行你的原型时，你会遇到各种问题。模型可能会在不同的输入情况下以不同的方式解释指令。检索功能可能会返回无关的文档。输出质量也可能出现不可预测的变化。这些问题就是你进行评估的起点。

The next chapter shows you how to analyze these problems systematically. You'll learn how to collect representative examples, categorize different types of failures, and distinguish between two root causes: specification failures (where your prompt was unclear or incomplete) and generalization failures (where the model behaves inconsistently despite clear instructions). This systematic error analysis is the foundation for everything that follows in the book.

下一章将向您展示如何系统地分析这些问题。您将学会如何收集有代表性的例子，对不同类型的错误进行分类，并区分两个根本原因：规范上的错误（即提示信息不清晰或不完整），以及泛化上的错误（即尽管指令清晰，模型仍然表现出不一致的行为）。这种系统的错误分析是本书后续内容的基础。

1 The code examples in this book pin a specific model version (e.g., gpt-4o-2024-08-06). By the time you read this, newer versions will be available. Replace the model string with whatever version is current for your provider; the API patterns stay the same. Chapter 9 discusses why pinning model versions matters for reproducibility.

本书中的代码示例固定了一个特定的模型版本（例如， gpt-4o-2024-08-06 ）。在你阅读本书时，将有更新的版本可用。将模型字符串替换为你的提供者当前的版本；API 模式保持不变。第 9 章讨论了固定模型版本对可重复性的重要性。

2 As more agents perform long-running, multi-step workloads (e.g., tool chains, RAG pipelines, multi-session workflows), reference-based evaluation becomes increasingly important because LLM judges and other reference-free methods struggle with complex, multi-step outputs, and labeled ground truth is more deterministic and reliable. As your product matures, it may be worth spending the time to curate labeled evaluation datasets for these systems. We do not cover reference-based evaluation in depth here, but Chen et al. (2021), Jain et al. (2025), and Yang et al. (2025) are good starting points for how benchmarks with labeled ground truth are constructed.

随着更多代理执行长时间运行的多步骤工作负载（例如工具链、RAG 管道、多会话工作流），基于参考的评估变得越来越重要，因为 LLM 裁判和其他无参考方法难以处理复杂的多步骤输出，而标记的真实情况更加确定性和可靠。随着您的产品成熟，花时间为您系统策划标记的评估数据集可能值得考虑。我们在这里没有深入探讨基于参考的评估，但 Chen 等人（2021 年）、Jain 等人（2025 年）和 Yang 等人（2025 年）是构建带有标记真实情况的基准的良好起点。

Part II. The Analyze Phase

第二部分： 分析阶段

Chapter 3. Error Analysis

第三章误差分析

The Analyze step of the evaluation lifecycle is where we look at what the system is actually doing and figure out where it fails. This chapter covers the practical methods for doing that.

评估生命周期中的"分析"步骤，旨在了解系统实际运行情况并找出其故障所在。本章将介绍实现这一目标的实用方法。

In this chapter, you will learn:

本章你将学习：

- How to read and inspect LLM traces systematically

  如何系统地读取和检查 LLM 轨迹
- How to identify and categorize failure modes

  如何识别和分类失效模式
- How to build a failure taxonomy by reading traces and clustering observations

  如何通过读取故障痕迹和聚类观察结果来构建故障分类体系
- How to generate synthetic data for testing

  如何生成用于测试的合成数据

The process of developing robust evaluations for LLM applications is inherently iterative. It involves creating test cases, assessing performance, and refining the system based on those observations. High-level guides, such as Anthropic's documentation on creating empirical evaluations for Claude (Anthropic 2024a), often depict the evaluation process as a cycle of developing test cases, engineering prompts, testing, and refining (Figure 3-1). This cycle emphasizes developing test cases, engineering and re-

fining prompts, and testing against both initial cases and held-out evaluations.

为 LLM 应用开发稳健的评估方法本质上是一个迭代过程。它包括创建测试用例、评估性能，并根据这些观察结果改进系统。一些高级指南，例如 Anthropic 公司关于为 Claude 创建实证评估的文档（Anthropic 2024a），通常将评估过程描述为一个循环：开发测试用例、设计提示、测试和改进（图 3-1）。该循环强调开发测试用例、设计和改进提示，以及针对初始用例和预留评估进行测试。

This chapter covers the *Analyze* step of our Analyze-Measure-Improve lifecycle (Figure 1-2). We walk through how to bootstrap an initial dataset, read traces, and build a structured understanding of where our application fails.

本章介绍"分析-测量-改进"生命周期中的 "*分析*" 步骤（图 1-2）。我们将逐步讲解如何引导初始数据集、读取跟踪信息，并构建对应用程序故障所在位置的结构化理解。

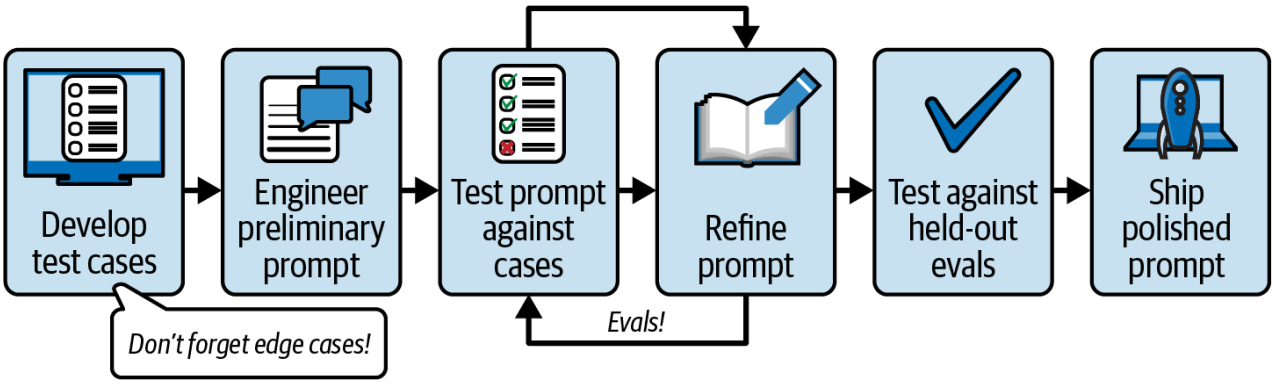


Figure 3-1. The iterative process for creating strong empirical evaluations, adapted from Anthropic (2024a)

图 3-1. 创建强有力的实证评估的迭代过程，改编自 Anthropic (2024a)

To make the discussion concrete, we use a running example: a real estate CRM assistant. This assistant is built to help real estate agents handle the everyday, often messy tasks that come up when they work with clients. A real estate professional might ask for help searching listings for a buyer, following up with warm leads, or preparing a quick market analysis. They often express these needs in conversational, ambiguous language, describing the kind of home a buyer wants, asking for "something affordable near downtown," or checking whether any open houses are available soon. The assistant must interpret these requests, translate them into the right database queries or tool calls, and sometimes chain together multiple steps without losing track of constraints. Example 3-1 describes the assistant in more detail. We return to it throughout

the rest of the book.

为了使讨论更加具体，我们使用一个实际的例子：房地产客户关系管理助手。 该助手旨在帮助房地产经纪人处理在与客户合作时经常遇到的日常琐事。 房地产专业人士可能会请求帮助，为买家寻找房源。 跟进潜在客户， 或者准备一份简短的市场分析报告。 他们通常用口语化的、含糊不清的语言表达这些需求。 描述买家想要的那种房子， 询问“市中心附近价格适中的住处”， 或者查看近期是否有开放参观的房屋。 助理必须解读这些请求。 将它们转换成正确的数据库查询或工具调用， 有时还能将多个步骤串联起来，而不会忽略限制条件。 例 3-1 对助手进行了更详细的描述。本书后续部分会多次提及该助手。

Example 3-1. Real estate agent assistant

例 3-1. 房地产经纪人助理

The assistant supports agents throughout their workflow. It can retrieve listing data, run simple market analyses, draft emails to clients, and check calendars for showings. It operates over natural-language inputs and may need to break a single request into multiple internal steps.

该助手可在经纪人的整个工作流程中提供支持。它可以检索房源数据、运行简单的市场分析、撰写发送给客户的电子邮件以及查看看房日程。它基于自然语言输入运行，并且可能需要将单个请求拆分为多个内部步骤。

A typical multi-step request might look like:

一个典型的多步骤请求可能如下所示：

- Find me 3-bedroom homes under $600k near downtown.

  请帮我找市中心附近售价低于 60 万美元的三居室住宅。

- Email the top two matches to my client.

  将结果中排名前两位的邮件发送给我的客户。

- Figure out if there are showings available for this weekend.

  查一下这周末有没有空位。

Even a short request like this requires the assistant to understand constraints, search listing data, draft a message, consult calendars, and produce a coherent final response.

即使像这样的简短请求，也需要助手了解限制条件、搜索列表数据、起草消息、查阅日历，并生成连贯的最终回复。

The system is “agentic”—the LLM decides what action to take next based on the current state of the interaction and whatever information it has already gathered. The model interprets the user’s request and produces a structured action such as querying listings, drafting an email, or checking the showings calendar. Each action triggers a downstream tool, and the tool’s output is passed back to the LLM. The model may then issue another action, refine an earlier one, or move on to the next step. This continues until the system has completed all necessary operations for the task.

该系统具有“智能体”特性——LLM（生命周期模型）会根据当前交互状态和已收集的信息来决定下一步操作。该模型会解读用户的请求，并生成结构化的操作，例如查询房源、撰写电子邮件或查看看房日程。每个操作都会触发下游工具，并将工具的输出返回给 LLM。之后，模型可能会发出另一个操作、改进之前的操作或继续执行下一步。如此循环往复，直到系统完成任务所需的所有操作。

In this chapter, we will walk through the steps of error analysis, following the overview shown in Figure 3-2. These steps operationalize the early part of error analysis, moving from raw data to a structured understanding of failure modes.

在本章中，我们将按照图 3-2 所示的概述，逐步介绍误差分析的步骤。这些步骤实现了误差分析的早期阶段，从原始数据过渡到对故障模式的结构化理解。

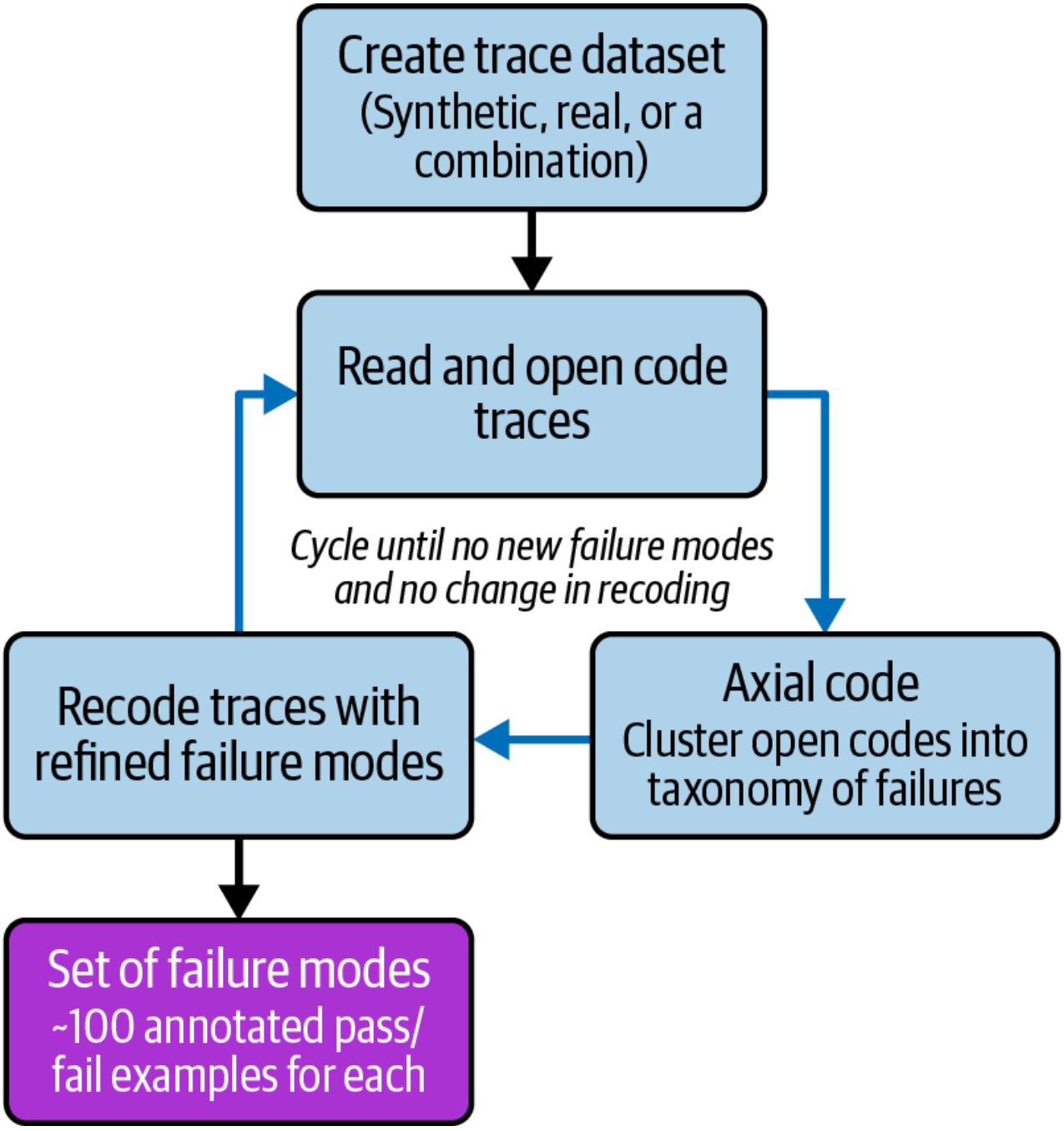


Figure 3-2. A detailed breakdown of the “Analyze” stage within our evaluation lifecycle

图 3-2. 评估生命周期中“分析”阶段的详细分解

Establishing Terminology

建立术语

Before moving forward, let's define terms that recur throughout this chapter. When a user interacts with the system, they provide a *query*, or the raw text of their request. Everything that happens afterward, from the first model response all the way to the final output, constitutes a *trace*. A trace is the complete, ordered record of all steps the system took in response to the query. This includes every LLM message, every tool

call, every value returned from a tool, and the final response shown to the user.

在继续之前，我们先来定义一下本章中反复出现的一些术语。当用户与系统交互时，他们会提供一个*查询*，也就是他们请求的原始文本。之后发生的一切都构成了一个*跟踪（trace）*。跟踪是系统响应查询所采取的所有步骤的完整有序记录。这包括每条 LLM 消息、每一次工具调用、工具返回的每一个值，以及最终显示给用户的响应。

When we evaluate LLM outputs, we evaluate traces rather than individual steps. This is because the behavior of an LLM agent only makes sense when viewed end to end. An individual message might appear reasonable in isolation, yet still lead to an incorrect final output. Reading the full trace lets us identify where problems actually originate rather than mislabeling later symptoms as independent failures.

在评估 LLM 输出时，我们评估的是整个流程，而不是单个步骤。这是因为 LLM 代理的行为只有从头到尾才能理解。单个消息单独来看可能合理，但最终的输出结果却可能不正确。读取完整的流程可以帮助我们识别问题的真正根源，而不是将后续出现的症状错误地标记为独立的故障。

In this chapter, *query* refers to the original input to the system, and *trace* refers to the entire sequence of actions the system produced in response. Queries don't always come from a human typing into a chat window; they can also come from automated pipelines, scheduled jobs, or other systems calling your agent's API.

本章中，*查询*指的是系统的原始输入，而*跟踪指*的是系统响应查询而产生的完整操作序列。查询并非总是来自用户在聊天窗口中输入的内容；它们也可能来自自动化流程、定时任务或其他调用代理 API 的系统。

Because LLM outputs are nondeterministic, running the same query twice can produce different traces. A single trace therefore shows only one possible execution path. If a failure seems intermittent, run the same query several times to see how often it occurs.

由于 LLM 输出结果不确定，运行同一查询两次可能会产生不同的跟踪信息。因此，单个跟踪信息仅显示一种可能的执行路径。如果故障似乎是间歇性的，请多次运行同一查询以查看故障发生的频率。

Creating a Starting Dataset of Traces

创建初始轨迹数据集

The first step is to assemble a dataset of traces. The purpose of this dataset is to give us concrete material to analyze, or real examples of how the system behaves in response to user requests. Without traces, we have nothing to observe, and error analysis cannot begin. How many traces do you need? There is no fixed number; the right amount depends on how complex your application is. A reasonable starting point is around 100 traces, but the real signal is *saturation*: the point at which analyzing additional traces stops surfacing new categories of errors. If you've read 60 traces and the last 20 all fall into failure modes you've already identified, you're likely saturated. If new failure types keep appearing, keep going.

第一步是收集跟踪数据。这个数据集的目的是为我们提供具体的分析素材，或者说是系统响应用户请求的真实示例。如果没有跟踪数据，我们就无从观察，错误分析也无法开始。那么，你需要多少跟踪数据呢？并没有固定的数量；合适的数量取决于你的应用程序的复杂程度。一个合理的起点是大约 100 条跟踪数据，但真正的信号是*饱和*：即分析更多跟踪数据后不再发现新的错误类型。如果你已经读取了 60 条跟踪数据，而最后 20 条都属于你已经识别过的故障模式，那么你可能已经达到饱和状态。如果新的故障类型不断出现，那就继续分析。

We frame error analysis as building a taxonomy of failure modes for our system: a structured set of categories that describe the different ways the system can go wrong. If you stop too early, you risk missing failure modes that would have surfaced with more data. If you continue long past saturation, you waste effort analyzing traces that only confirm what you already know. Simpler applications saturate earlier; more complex agentic systems with many tools and multi-step flows may require more exploration.

我们将错误分析定义为构建系统故障模式分类体系：一套结构化的类别，用于描述系统可能出现的各种故障方式。如果过早停止分析，可能会错过一些随着数据量的增加而显现的故障模式。如果分析持续过久，即使已经达到饱和状态，也会浪费精力去分析那些仅仅证实已知信息的痕迹。简单的应用程序更容易达到饱和状态；而拥有众多工具和多步骤流程的复杂智能体系统则可能需要更长时间的探索。

There are two ways to build the starting dataset of traces:

构建初始轨迹数据集有两种方法：

From production logs (preferred when available)

来自生产日志（如有则优先）

If you already have production traces, you can sample them directly. To ensure your sample covers different use cases, try to include traces that cover different features or types of user input. For small datasets, random sampling works fine. For larger datasets, you can use clustering (e.g., on query text embeddings or keyword matches) to select queries from different groups. Also sample traces where users had a bad experience: thumbs-down signals, repeated rephrasing of the same request, or sessions that ended with the user switching to another channel. If you have access to domain experts (e.g., experienced real estate agents for our CRM example), ask them to identify the kinds of queries that tend to be most challenging; they often know the hard cases before the data confirms them.

如果您已有生产环境的跟踪数据，可以直接进行采样。为了确保样本涵盖不同的使用场景，请尝试包含涵盖不同功能或用户输入类型的跟踪数据。对于小型数据集，随机采样即可。对于大型数据集，您可以使用聚类（例如，基于查询文本嵌入或关键词匹配）来选择来自不同组的查询。此外，还要对用户体验不佳的跟踪数据进行采样，例如：点踩、重复重复相同的请求，或者用户切换到其他渠道结束的会话。如果您可以联系到领域专家（例如，我们 CRM 示例中的经验丰富的房地产经纪人），请他们指出哪些类型的查询最容易出现问题；他们通常在数据证实之前就能预判到疑难案例。

From synthetic queries 来自合成查询

If you do not yet have traces available, first create a dataset of synthetic queries, then run your application on those queries to produce traces.

如果您还没有可用的跟踪数据，请先创建一个合成查询数据集，然后运行您的应用程序来生成这些查询，从而生成跟踪数据。

When generating synthetic queries, a common mistake is to simply prompt an LLM to "generate user queries for my application." This approach produces queries that are generic, repetitive, and fail to capture real usage patterns. For example, prompting an LLM with "Generate 50 queries for a real estate assistant" might yield variations like:

生成合成查询时，一个常见的错误是简单地指示 LLM"为我的应用程序生成用户查询"。这种方法生成的查询过于通用、重复，无法捕捉真实的使用模式。例如，指示 LLM"为房地产助手生成 50 个查询"可能会生成如下变体：

- Find me a house in San Francisco.

  帮我在旧金山找套房子。
- Show me homes in San Francisco.

  请带我看看旧金山的房子。
- I'm looking for a property in San Francisco.

  我正在寻找旧金山的房产。
- Can you find houses in the San Francisco area?

  你能在旧金山地区找到房子吗?

These queries all test the same basic capability. They do not vary along dimensions that matter for your application, dimensions that, if left unexplored, could hide entire categories of failure modes. For example, if all your synthetic queries are well-specified property searches, you will never discover that your assistant fails when users make ambiguous requests. Running your agent on repetitive queries will produce traces that all look similar, and you will miss failures that only surface under different conditions.

这些查询都测试相同的基本功能。它们在对您的应用程序至关重要的维度上没有变化，而这些维度如果被忽略，可能会隐藏整类故障模式。例如，如果您所有的合成查询都是明确定义的属性搜索，那么您永远不会发现当用户发出模糊请求时，您的助手会失效。对重复查询运行您的代理将产生看起来都很相似的跟踪记录，您将会错过仅在不同条件下才会出现的故障。

A better approach to generating synthetic queries is to add a layer of indirection: instead of asking an LLM to generate queries directly, first define the dimensions along which queries can vary, then generate structured combinations of those dimensions, and finally convert each combination into a natural-language query. This separation forces diversity in a controlled way and prevents the LLM from collapsing into repetitive outputs.

生成合成查询的更佳方法是增加一层间接层：与其直接让语言学习模型（LLM）生成查询，不如先定义查询可以变化的维度，然后生成这些维度的结构化组合，最后将每个组合转换为自然语言查询。这种分离方式能够以可控的方式强制生成多样性，并防止语言学习模型（LLM）输出重复内容。

Defining Dimensions 定义维度

A dimension is simply a category of variation—one axis along which requests differ from each other. For the real estate assistant, useful dimensions might include:

维度本质上是一种变异类别——请求之间差异的一个维度。对于房地产助手来说，有用的维度可能包括：

Feature 特征

What task the user wants to perform (e.g., property search, scheduling, email drafting)

用户想要执行的任务（例如，房源搜索、日程安排、电子邮件撰写）

Client persona 个人客户

The type of client being served (e.g., first-time buyer, investor, luxury buyer)

所服务的客户类型（例如，首次购房者、投资者、奢侈品买家）

Scenario type 场景类型

How clearly the user expresses their intent (e.g., well-specified, ambiguous, out-of-scope)

用户表达意图的清晰度如何（例如，明确、含糊、超出范围）

Identifying good dimensions often requires some effort to understand the product better. Talk to product managers, review user feedback, or spend time using the application yourself. That said, you do not need to enumerate every possible dimension before getting started. Once you have three sufficiently diverse dimensions, begin generating data. You can always add more dimensions later as your understanding deepens.

要确定合适的维度，通常需要花些功夫才能更好地了解产品。可以与产品经理沟通、查看用户反馈，或者亲自体验一下应用。不过，你无需在开始之前列举所有可能的维度。只要确定了三个足够多样化的维度，就可以开始生成数据了。随着理解的加深，你可以随时添加更多维度。

When choosing dimensions, focus on ones that describe where your application is likely to fail. If you do not yet know where your product might fail, build intuition by using it yourself, or recruit a few people to use it and let their failures motivate the di-

mensions.

选择维度时，重点关注那些能够描述应用程序可能出现故障的维度。如果您还不清楚产品可能出现故障的地方，可以通过自己使用来培养直觉，或者招募一些用户来使用，让他们的失败经历来启发您选择维度。

Generating Tuples and Queries

生成元组和查询

Once you have defined your dimensions, you can use an LLM to generate synthetic queries, but with a layer of indirection that ensures diversity. First, create structured combinations of your dimensions. A tuple is a specific combination of values, one from each dimension, that defines a particular use case. For example:

定义好维度后，您可以使用 LLM 生成合成查询，但需要通过间接层来确保查询的多样性。首先，创建维度的结构化组合。元组是来自每个维度的值的特定组合，用于定义特定的用例。例如：

- Feature: property search

  功能：房产搜索

- Client persona: investor

  客户画像：投资者

- Scenario type: ambiguous

  场景类型：模糊不清

This tuple describes a case where an investor is searching for properties, but the request is underspecified or vague. From this tuple, you might generate a query like: “Look up showings for good properties in San Mateo County.” A different tuple with a first-time buyer and a specific scenario might yield: “Find 3-bedroom homes under $600k near downtown that allow pets.”

这个元组描述了一个投资者寻找房产但请求不够具体或模糊的情况。基于这个元组，你可能会生成类似这样的查询：“查找圣马特奥县优质房产的看房信息。” 另一个元组描述了一个首次购房者的具体场景，可能会生成这样的查询：“查找市中心附近允许养宠物的、价格低于 60 万美元的三居室住宅。”

Writing 20 tuples by hand helps build understanding of the problem space. If you are unsure what tuples to write, consult domain experts or review real user queries for in-

spiration. Once you have a small set of example tuples and corresponding queries, use an LLM to scale up to 100 or more. Some practitioners prefer a smaller set of 10 to 20 carefully crafted queries that they invest in deeply rather than a larger synthetic set; this can work well if you continue enriching those queries over time and use them as a foundation for iterative improvement rather than a one-time test.

手动编写 20 个元组有助于加深对问题领域的理解。如果您不确定要编写哪些元组，可以咨询领域专家或查看真实用户的查询以获取灵感。一旦您拥有少量示例元组及其对应的查询，就可以使用 LLM（逻辑逻辑模型）将其扩展到 100 个或更多。一些实践者更倾向于精心编写 10 到 20 个查询，并投入大量精力进行深入研究，而不是使用更大的合成数据集；如果您随着时间的推移不断完善这些查询，并将它们作为迭代改进的基础，而不是一次性测试，那么这种方法会非常有效。

The key idea is to separate tuple generation from query generation. First, prompt the LLM to generate structured tuples. Then, for each tuple, prompt the LLM to generate a natural-language query. This two-step process works better than asking an LLM to generate queries directly, because the tuples constrain each query to cover a specific combination of dimensions:

关键在于将元组生成与查询生成分开。首先，提示语言学习模型（LLM）生成结构化元组。然后，针对每个元组，提示 LLM 生成自然语言查询。这种两步法比直接让 LLM 生成查询效果更好，因为元组将每个查询限制在特定的维度组合范围内：

```
Generate 10 random combinations of (feature, client persona, scenario)
for a real estate CRM assistant.

The dimensions are:

* Feature: what task the agent wants to perform. Possible values:
  property search, market analysis, scheduling, email drafting.
* Client persona: the type of client the agent is working with.
  Possible values: first-time homebuyer, investor, luxury buyer.
* Scenario: how well-formed or challenging the query is. Possible
  values: exact match (clearly specified and feasible), ambiguous
  request (unclear or underspecified), shouldn't be handled
  (invalid or out-of-scope).

Output each tuple in the format: (feature, client persona, scenario)

Avoid duplicates. Vary values across dimensions.
```

For each tuple, prompt the LLM to generate a corresponding natural-language query:

对于每个元组，提示 LLM 生成相应的自然语言查询：

> **We are generating synthetic user queries for a real estate CRM assistant. The assistant helps agents manage client requests by searching listings, analyzing markets, drafting emails, and reading calendars.**
>
> **Given:**
>
> **Feature: Scheduling**
> **Client Persona: First-time homebuyer**
> **Scenario: Ambiguous request**
>
> **Write a realistic query that an agent might enter into the system to fulfill this client's request. The query should reflect the client's needs and the ambiguity of the scenario.**
>
> **Example:**
> **"Find showings for affordable homes with short notice availability."**
>
> **Now generate a new query.**

Continue this process of sampling, generating, and filtering until you have around 100 high-quality, diverse examples. If the phrasing is awkward or the content misses the mark, you can throw the query out and regenerate it. Be aggressive about quality control at this stage, because future steps of error analysis depend on how representative and realistic these traces are.

重复上述采样、生成和筛选过程，直到获得大约 100 个高质量、多样化的示例。如果措辞生硬或内容不符，可以丢弃该查询并重新生成。在这个阶段要严格把控质量，因为后续的错误分析步骤取决于这些示例的代表性和真实性。

Once you have curated your queries, a useful exercise before running them is to write down your predictions: for each query, note what you expect the system to do and where you think it will fail. Comparing predictions against actual behavior surfaces assumptions you didn't know you were making.

在你精心编写好查询语句之后，运行它们之前的一个有效方法是写下你的预测：对于每个查询语句，记下你期望系统执行的操作以及你认为它会在哪些地方出错。将预测结果与实际行为进行比较，可以发现你之前没有意识到的假设。

Then run each query through your application end to end and export the full trace. The export should include all LLM steps, tool calls, inputs, and outputs. If you are using an observability platform like Braintrust, LangSmith, or Weave, you can export traces directly from their interfaces. If you have a simpler setup, you can log each step to a JSON file or a database as the agent executes. For the purposes of annotation, many teams find it easiest to export traces to a spreadsheet, with one row per trace and columns for the query, each intermediate step, and the final output. The format matters less than having a complete record that you can read and annotate.

然后，将每个查询完整地运行到应用程序的整个流程，并导出完整的跟踪记录。导出内容应包含所有 LLM 步骤、工具调用、输入和输出。如果您使用的是 Braintrust、LangSmith 或 Weave 等可观测性平台，可以直接从其界面导出跟踪记录。如果您的设置较为简单，则可以在代理执行时将每个步骤记录到 JSON 文件或数据库中。为了便于注释，许多团队发现将跟踪记录导出到电子表格是最简便的方法，每行记录一条跟踪记录，每列分别记录查询、每个中间步骤和最终输出。格式本身并不重要，重要的是拥有一个完整的、可读取和注释的记录。

With these traces in hand, you are ready to start reading them and identifying failure modes.

有了这些痕迹，你就可以开始解读它们并识别故障模式了。

Open Coding: Reading and Labeling Traces

开放式编码：读取和标记痕迹

Now, the next step is to read through your traces and annotate them with observations about what went wrong. The goal is to surface failure patterns without imposing a predefined structure.

接下来，需要通读跟踪日志，并标注出问题所在。目标是在不预设框架的前提下，发现故障模式。

Our labeling process uses a two-step approach adapted from qualitative research methods:[1] first, read traces and write free-form observations (*open coding*); then, cluster those observations into structured failure categories (*axial coding*). The idea is to let the categories emerge from what you actually see in the data, rather than starting with a predefined list. In this context, *coding* means assigning short descriptive labels to parts of the data that seem important. For each trace, you read carefully and write brief notes about what you observe. You might note where outputs are incorrect,

where actions are surprising, or where the behavior feels wrong. Each note is a potential signal of a failure mode. These annotations can be short, a phrase or a sentence or two, and do not need to follow any particular format. The goal is to capture your observations quickly. In later steps of error analysis, we will categorize and structure these observations more rigorously.

我们的标签标注过程采用两步法，该方法借鉴了定性研究方法：[1] 首先，阅读轨迹并自由记录观察结果（*开放式编码*）；然后，将这些观察结果归类到结构化的故障类别中（*轴向编码*）。其理念是让类别从你实际观察到的数据中涌现出来，而不是从预定义的列表开始。在此上下文中，*编码*是指为数据中看似重要的部分分配简短的描述性标签。对于每条轨迹，你都要仔细阅读并简要记录你的观察结果。你可以记录输出错误的地方、行为异常的地方，或者感觉不正常的地方。每条记录都可能是一个故障模式的信号。这些注释可以很简短，一个短语或一两句话即可，无需遵循任何特定格式。目标是快速记录你的观察结果。在后续的错误分析步骤中，我们将更严格地对这些观察结果进行分类和结构化。

When open coding, put yourself in the perspective of the user. Is the user actually having a good experience? Does the response address their request? At this stage, focus on noticing and recording what feels wrong rather than diagnosing causes or thinking about fixes.

在进行开放式编码时，要站在用户的角度思考。用户真的有良好的体验吗？响应是否满足了他们的需求？在这个阶段，重点在于观察和记录用户感觉不正常的地方，而不是诊断原因或思考解决方法。

A common question we get is: should you note down all the failures in a trace, or just the first one? We recommend starting with the first failure you observe, which is usually the most upstream point where something went wrong. Traces can be complex, and a single upstream error often causes a cascade of downstream issues. Focusing on the first failure is more efficient and prevents you from cataloging every symptom when many are just consequences of a single root cause. In many cases, fixing the first error resolves the entire chain.

我们经常被问到的一个问题是：在跟踪过程中，应该记录所有故障，还是只记录第一个故障？我们建议从观察到的第一个故障开始，这通常是问题发生的源头。跟踪过程可能很复杂，一个上游错误往往会引发一系列下游问题。专注于第一个故障效率更高，可以避免记录所有症状，因为很多症状可能只是同一个根本原因造成的。在很多情况下，修复第一个错误就能解决整个问题链。

You can record each trace and its corresponding annotation in a simple table or spreadsheet, with one column for the trace identifier, another for the trace itself or a sum-

mary, and a third for your free-form notes describing the first failure.

您可以将每个跟踪及其相应的注释记录在一个简单的表格或电子表格中，其中一列用于跟踪标识符，另一列用于跟踪本身或摘要，第三列用于描述第一次故障的自由格式注释。

In practice, open codes are short, lowercase notes jotted down as you read. They do not need to be polished or grammatical, but they should include enough context that a teammate reading them later can understand what went wrong without rereading the whole trace. Table 3-1 gives some realistic examples of the kinds of notes you might write while reading traces from the real estate assistant.

实际上，开放代码是你在阅读时随手记下的简短小写笔记。 它们不需要润色或符合语法，但是，它们应该包含足够的上下文，以便稍后阅读它们的队友无需重新阅读整个跟踪记录即可了解哪里出了问题。 表 3-1 给出了一些在阅读房地产助理留下的痕迹时可能会写的笔记类型的实际示例。

Table 3-1. Realistic open codes from reading traces

表 3-1. 从读取痕迹中得到的实际开放代码

Trace situation 追踪情况	Open code 开源代码
User asked for “3-bedroom homes under $600k near downtown that allow pets.” Assistant generated SQL with filters on price and bedroom count, returned 2 listings, and drafted an email to the client. 用户要求查找“市中心附近 60 万美元以下、允许养宠物的 3 居室住宅”。助手生成了包含价格和卧室数量筛选条件的 SQL 语句，返回了 2 个房源，并起草了一封发送给客户的电子邮件。	sql query dropped the pet-friendly filter even though the user explicitly asked for it; downstream listings and email all silently miss the constraint 即使用户明确要求，SQL 查询仍然忽略了宠物友好筛选条件；下游列表和电子邮件都悄无声息地遗漏了该限制。
User asked to “set up showings for these two homes this weekend.” Calendar API returned the agent as unavailable Saturday afternoon. Assistant proposed showings for both Saturday 2 p.m. and Sunday 11 a.m. 用户要求“安排本周末这两套房子的看房”。日历 API 显示经纪人周六下午无法联系。助理建议周六下午 2 点和周日上午 11 点分别安排看房。	assistant proposed a saturday slot that the calendar already showed as blocked; treated the calendar result as advisory instead of a hard constraint 助理建议了一个周六的时间段，但日历上显示该时间段已被占用；将日历结果视为建议而非硬性限制。
User asked to “send a property list to my investor client in San Mateo.” Assistant searched for high-return-on-investment (ROI) properties, but drafted a casual, emoji-heavy email pitching them as great starter homes. 用户要求“向我在圣马特奥的投资客户发送一份房产清单”。助手搜索了高投资回报率 (ROI) 的房产，但撰写了一封随意的、表情符号较多的电子邮件，将它们推销为绝佳的入门级住宅。	tone and framing don't match an investor persona; email reads like it was written for a first-time buyer even though the search itself was investor-appropriate 邮件的语气和措辞与投资者形象不符；邮件读起来像是写给首次购房者的，尽管搜索本身是符合投资者需求的。
Listing record in the database says “2 bed, 1 bath.” Assistant's draft email describes the home as a “spacious 3-bedroom with a renovated second bath.” 数据库中的房源记录显示为“2 间卧室，1 间浴室”。助理的邮件草稿将这套房子描述为“宽敞的 3 间卧室，带翻新过的第二间浴室”。	hallucinated bedroom count and bathroom upgrade; details look plausible but contradict the listing data the assistant just retrieved 卧室数量和浴室升级的幻觉；细节看似合理，但与助理刚刚检索到的房源数据相矛盾。

Trace situation 追踪情况	Open code 开源代码
User said “send the shortlist to my client.” Assistant sent the email to a contact from a different deal that came up earlier in the conversation history. 用户说“把候选名单发给我的客户”。助理把邮件发给了之前对话记录中提到的另一个交易的联系人。	wrong recipient; assistant grabbed the most recently mentioned client name instead of the one tied to the active shortlist 收件人错误；助理抓取了最近提及的客户名称，而不是与有效候选名单关联的客户名称。
User asked about “Springfield” listings. The agent’s location context is Massachusetts. Assistant returned results from Springfield, IL, without asking which one. 用户查询“Springfield”的房源信息。经纪人的所在地是马萨诸塞州。但助手返回了伊利诺伊州 Springfield 的房源，而没有询问具体是哪个城市。	ambiguous city name resolved silently to the wrong state; no clarifying question even though multiple springfields exist 含糊不清的城市名称被默默地解析到了错误的州；尽管存在多个名为斯普林菲尔德的地方，却没有人提出任何澄清问题。
User asked, “what’s a good offer price for 142 Oak St?” Assistant answered “$585k” with no comps lookup or market analysis tool call in the trace. 用户问：“橡树街 142 号的合理报价是多少？”助理回答“58.5 万美元”，但过程中没有进行任何可比房产查询或市场分析工具调用。	assistant produced a price recommendation without ever pulling comps; answer is confidently stated but not grounded in any retrieved data 助理在未进行任何市场比较的情况下给出了价格建议；答案听起来很自信，但没有任何数据依据。
During a multi-step request, the listings search tool returned an error payload. Assistant ignored the error and proceeded to draft an email summarizing “the homes I found.” 在执行多步骤请求时，房源搜索工具返回了错误信息。助手忽略了该错误，继续撰写了一封电子邮件，总结了“我找到的房源”。	tool error ignored without being surfaced or retried; assistant kept going and fabricated results instead 工具错误未被发现或重试，而是被忽略；助手继续操作并捏造了结果。
User asked for “a quick summary of this week’s leads.” Reply is six paragraphs with section head-	response is much longer than “quick summary” implied; ignored the

Trace situation 追踪情况	Open code 开源代码
ers. 用户要求“简要总结本周的线索”。回复包含六个段落，并附有章节标题。	brevity hint in the user's wording 回复内容远超“简要总结”的字面意思；忽略了用户措辞中暗示的简洁性。
User asked the assistant to “draft an email” to a client. Assistant drafted the email and immediately called send_email without showing the draft for review. 用户要求助手“起草一封电子邮件”给客户。助手起草了电子邮件，但立即调用了 send_email 而没有显示草稿供用户审核。	assistant took an irreversible action (sending the email) when the user had only asked it to draft; skipped the implicit confirmation step 用户仅要求助手起草邮件，但助手却执行了不可逆操作（发送电子邮件）；跳过了隐式确认步骤。

Notice that these notes are not categories yet. They are raw observations, written in whatever phrasing came to mind while reading. Some will turn out to be instances of the same underlying failure mode, and some will stand alone. That sorting happens later, in axial coding.

请注意，这些笔记目前还不是分类。它们是原始的观察记录，是我在阅读时想到的任何措辞。有些最终会发现是同一种潜在故障模式的实例，而有些则是独立的。这种分类将在后续的轴向编码中进行。

During open coding, do not try to group or formalize errors. Focus on collecting a rich set of raw observations. You can also assign each trace a holistic binary judgment, *acceptable* or *unacceptable*, as a fourth column. Making a clear yes-or-no decision forces sharper thinking than vague scores, and even when the judgment feels borderline, picking a side helps clarify what you consider a failure.

在开放式编码过程中，不要试图对错误进行分组或形式化。重点在于收集丰富的原始观察数据。你还可以为每条轨迹赋予一个整体性的二元判断——*可接受*或*不可接受*，作为第四列。做出明确的“是”或“否”的决定比模糊的评分更能促使你进行更深入的思考，即使判断结果感觉模棱两可，选择立场也有助于明确你对失败的定义。

Sometimes it can be difficult to articulate precisely what feels wrong about a trace. If you find yourself stuck, one strategy is to temporarily adopt a more “top-down” approach: instead of waiting for failure modes to emerge from your error analysis, consider common LLM failure categories and actively look for their manifestations. For example, knowing that *hallucination* (when the model generates information that is not

grounded in its input and presents it as if it were factual) is a common issue, you might specifically check each trace for instances where the assistant invents facts or misrepresents information. You can also draw on failure modes you have heard about that are specific to your application. If you are building a text-to-SQL system, you might look for malformed queries or incorrect joins. If you are building a customer service bot, you might look for responses that contradict company policy. The goal is not to force traces into predefined categories, but to use known failure types as "lenses" that help you identify problems that might otherwise be hard to pin down.

有时很难准确地描述某个痕迹哪里出了问题。如果你发现自己陷入了困境，一种策略是暂时采用一种更"自上而下"的方法：与其等待错误分析中出现故障模式，不如考虑常见的 LLM 故障类别，并主动寻找它们的表现形式。例如，知道*幻觉*（当模型生成的信息与其输入无关时）并将其呈现得好像是事实一样）这是一个常见问题，您可以专门检查每一条痕迹。例如，当助理捏造事实或歪曲信息时。您还可以借鉴您听说过的、与您的应用相关的特定故障模式。如果您正在构建一个文本到 SQL 的系统，您可能需要查找格式错误的查询或不正确的连接。如果你正在构建一个客户服务机器人，你可以寻找与公司政策相悖的回应。目标并非强行将痕迹归入预定义的类别，但要利用已知的故障类型作为"透镜"，帮助你识别那些原本可能难以确定的问题。

How long should open coding continue? A useful heuristic is to proceed until you have labeled at least 20 problematic traces and new failure modes have stopped appearing. At that point, you likely have enough raw material to move on to structuring your observations.

开放式编码应该持续多久？一个有用的经验法则是，持续进行直到至少标记了 20 条问题轨迹，并且不再出现新的故障模式。此时，您可能已经积累了足够的原始数据，可以开始整理您的观察结果了。

Axial Coding: Structuring Failure Modes

轴向编码：结构失效模式

Open coding produces a valuable but unstructured collection of observations. Each trace has its own raw notes about missing constraints, invalid actions, inappropriate tones, or hallucinated facts. To make sense of these observations and quantify how often different problems occur, you need to organize them into a structured set of fail-

ure modes.

开放式编码会产生大量有价值但结构松散的观察结果。每条记录都包含关于缺失约束、无效操作、不恰当语气或臆想事实的原始笔记。为了理解这些观察结果并量化不同问题发生的频率，需要将它们整理成一套结构化的故障模式。

Axial coding is the second step: identifying relationships among the notes you wrote during open coding and grouping them into higher-order categories. It moves you from isolated observations to a coherent set of distinct, recurring failure types.

轴向编码是第二步：识别你在开放式编码过程中所写笔记之间的关系，并将它们归类到更高阶的类别中。它能帮助你从孤立的观察结果过渡到一套连贯的、独特的、反复出现的故障类型。

To axial code, read through your open-coded traces and cluster similar annotations together. Some patterns are obvious. For example, traces where the assistant ignores a buyer's budget or omits a requirement like number of bedrooms cluster naturally into a broader failure mode: *violation of user constraints*. Other patterns show deeper distinctions only after reading several traces. For example, hallucinations of property features (claiming a home has solar panels when it does not) and hallucinations of client activity (scheduling a tour that the user never requested) might initially seem like the same problem. But these failures have different causes and require different fixes. Hallucinated property features likely stem from the model inventing details not present in the listing data, while hallucinated client activity suggests the model is misinterpreting or fabricating user intent. You would split them into *hallucinated listing metadata* and *hallucinated user actions* because diagnosing and addressing them requires different strategies. That said, this is just a first pass. It is fine if you end up splitting or merging categories later as your understanding of the failure modes deepens.

要进行轴向编码，请通读已公开的跟踪记录，并将相似的注释归类在一起。有些模式显而易见。例如，助手忽略买家预算或遗漏卧室数量等要求的跟踪记录自然会归类为更广泛的故障模式： *违反用户约束* 。其他模式只有在阅读多个跟踪记录后才能看出更深层次的区别。例如，房产特征的幻觉（声称房屋有太阳能电池板，而实际上没有）和客户活动的幻觉（安排用户从未要求的看房）最初可能看起来像是同一个问题。但这些故障的原因不同，需要不同的修复方法。房产特征的幻觉可能源于模型虚构了房源数据中不存在的细节，而客户活动的幻觉则表明模型误解或捏造了用户意图。您可以将它们分为*幻觉的房源元数据*和*幻觉的用户行为* 。 因为诊断和治疗这些问题需要不同的策略。 也就是说，这只是初步版本。 随着对故障模式理解的加深，以后如果需要拆分或合并类别，也是可以的。

While early clustering is often done manually, you can also use a language model to help organize your notes.

虽然早期聚类通常是手动完成的，但您也可以使用语言模型来帮助组织您的笔记。

WARNING 警告

Do not use an LLM to generate your failure taxonomy from scratch. LLMs have strong priors toward generic failure categories (e.g., "hallucination," "instruction following," "tone") and weak priors toward application-specific issues that only become visible by reading real traces. Teams that skip open coding and ask an LLM to propose failure modes often end up building months of evaluation infrastructure on top of a taxonomy that doesn't match their actual failure distribution.

不要使用 LLM 从零开始生成故障分类。LLM 对通用故障类别（例如，"幻觉"、"指令执行"、"音调"）具有很强的先验概率，而对只有通过读取实际故障跟踪才能发现的特定应用问题则缺乏先验概率。那些跳过开放式编码，直接要求 LLM 提出故障模式的团队，最终往往会花费数月时间，基于一个与其实际故障分布不符的分类体系来构建评估基础设施。

We do not recommend using an LLM for open coding itself, because that step requires your judgment and taste about what constitutes a failure. But for axial coding, an LLM can be useful: its job is not to apply your taste, but to analyze the observations you have already made and propose a way to organize them. After open coding traces, you can paste the raw open codes into an LLM, along with the traces themselves, and ask the LLM to propose preliminary groupings:

我们不建议将逻辑逻辑模型 (LLM) 用于开放式编码本身，因为这一步骤需要您自行判断何为失败。但对于轴向编码，LLM 则非常有用：它的作用并非代您评判，而是分析您已观察到的现象，并提出一种组织这些现象的方法。在完成开放式编码后，您可以将原始的开放式编码连同编码轨迹一起粘贴到 LLM 中，并让 LLM 提供初步的分组建议：

> **Below is a list of open-ended annotations describing failures in an LLM-driven real estate CRM assistant. Please group them into a small set of coherent failure categories, where each category captures similar types of mistakes. Each group should have a short descriptive title and a brief one-line definition. Do not invent new failure types; only cluster based on what is present in the notes.**

LLM-generated groupings can help organize initial ideas, but they should not be accepted blindly. The LLM does not know how your system works or how you would go about fixing different types of failures, so its proposed categories may merge issues

that should be kept separate or split issues that belong together. You will almost always need to review the taxonomy and make edits yourself: renaming categories, merging ones that are too similar, or splitting ones that lump together failures with different root causes.

LLM 生成的分类有助于整理初步想法，但不应盲目接受。LLM 不了解您的系统如何运作，也不了解您将如何解决不同类型的故障，因此它提出的类别可能会合并本应分开的问题，或者拆分本应归为一类的问题。您几乎总是需要审查分类并自行进行编辑：重命名类别、合并过于相似的类别，或拆分将不同根本原因的故障混为一谈的类别。

At the end of axial coding, you have a list of distinct failure modes, along with representative examples illustrating each one. These examples serve as reference points as you move into the next stage of error analysis: systematically labeling and quantifying failures across all traces.

轴向编码完成后，您将得到一份不同的故障模式列表，以及说明每种模式的代表性示例。这些示例将作为参考点，帮助您进入下一阶段的故障分析：系统地标记和量化所有线路上的故障。

Labeling Traces with Structured Failure Modes

使用结构化故障模式标记轨迹

At this point, you have two artifacts:

至此，你拥有两个物品：

- A collection of traces, each with its initial free-form annotation from open coding

  一系列轨迹，每条轨迹都带有其最初基于开放编码的自由形式注释。
- A defined list of structured failure modes from axial coding

  轴向编码中定义的结构化故障模式列表

The next step is to go back through your traces and label each one with the failure modes that apply. This bridges the gap from qualitative observation to quantitative data. For each trace, compare its open codes against your list of failure modes and determine which ones apply. If you derived the failure modes manually during axial coding, this is straightforward: you apply the mapping you developed. If an LLM assisted in generating the categories, you need to ensure those categories are consistently applied back to each trace. You can do this manually, or you can prompt an LLM with

each trace's annotation and your list of failure modes and ask it to assign the appropriate labels. When using an LLM for this step, make sure you review its assignments to catch mistakes and refine definitions where needed.

下一步是回顾您的走线，并为每条走线标注适用的故障模式。这有助于将定性观察转化为定量数据。对于每条走线，将其开路代码与您的故障模式列表进行比较，确定哪些故障模式适用。如果您在轴向编码过程中手动推导出了故障模式，则此步骤非常简单：只需应用您创建的映射即可。如果 LLM 协助生成了类别，则需要确保这些类别始终如一地应用于每条走线。您可以手动执行此操作，也可以使用每条走线的注释和您的故障模式列表提示 LLM，并要求其分配相应的标签。如果使用 LLM 执行此步骤，请务必检查其分配结果，以发现错误并在需要时完善定义。

In your spreadsheet, add a new column for each failure mode. For each trace, mark a 1 if that failure is present and a 0 if it is absent. Table 3-2 shows what this looks like for the real estate assistant from Example 3-1. In the table, each row is a trace, and each column is a failure mode. A 1 means that failure is present in that trace.

在电子表格中，为每种故障模式添加一列。 对于每个跟踪记录，如果存在该故障，则标记为 1；如果不存在该故障，则标记为 0。 表 3-2 展示了示例 3-1 中房地产助手的这种结构。表中，每一行代表一条跟踪记录，每一列代表一种故障模式。1 表示该跟踪记录中存在故障。

Table 3-2. Labeling traces with structured failure modes.

表 3-2. 标记具有结构化故障模式的痕迹。

Trace ID 追踪 ID	Open code 开源代码	Missing constraint 缺少约束	Inappropriate tone 语气不当	Wrong tool 工具错误	Hallucinated fact 幻觉事实
tr-041	“SQL query missed the budget cap” “SQL 查询超出预算限制”	1	0	0	0
tr-042	“Email tone too aggressive for luxury buyer” “邮件语气对奢侈品买家来说过于咄咄逼人”	0	1	0	0
tr-043	“Called send_email before checking calendar” “在检查日历之前调用了 send_email ”	0	0	1	0
tr-044	“Budget missed AND tone too casual” “预算超支，而且语气过于随意”	1	1	0	0
tr-045	“Said property has pool, it does	0	0	0	1

Trace ID 追踪 ID	Open code 开源代码	Missing constraint 缺少约束	Inappropriate tone 语气不当	Wrong tool 工具错误	Hallucinated fact 幻觉事实
	not" "该房产有游泳池，但实际上没有。"				

Once the columns are populated, you can count the occurrences of each failure mode and divide by the total number of traces to get a prevalence rate. In this small example, Missing Constraint appears in two out of five traces (40%) and Inappropriate Tone also appears in two out of five (40%).

列数据填充完毕后，您可以统计每种故障模式的出现次数，然后除以跟踪总数，即可得到发生率。在这个小例子中，“缺少约束”出现在五条跟踪记录中的两条（40%），“语气不当”也出现在五条跟踪记录中的两条（40%）。

As you work through this process, you will likely discover edge cases. Some traces may not cleanly match any defined failure mode. Others may point to nuances that suggest a category needs to be refined or split. It is fine to adjust your failure mode definitions as you go.

在执行此过程的过程中，您可能会发现一些特殊情况。某些故障轨迹可能无法完全匹配任何已定义的故障模式。其他轨迹则可能揭示一些细微差别，表明需要对某个类别进行细化或拆分。您可以根据实际情况随时调整故障模式的定义。

You can compute prevalence rates with a simple formula in a spreadsheet, or with tools like pandas or pivot tables if you want to slice the data by different dimensions. This tells you which failures are most common, which is useful for deciding where to focus improvement efforts.

您可以使用电子表格中的简单公式计算患病率，或者如果您想按不同维度对数据进行切片，则可以使用 pandas 或数据透视表等工具。这可以帮助您了解哪些故障最为常见，从而有助于确定改进工作的重点方向。

Iteration and Refinement

迭代与改进

Error analysis is not a one-time process. It is common to go through two or three rounds of reviewing traces and refining your failure mode taxonomy. Early rounds often uncover failure modes that were initially missed, or make clear that your definitions need adjustment. As you analyze more traces, whether by labeling ones you skipped earlier or by sampling new queries, your understanding of the data deepens and the taxonomy becomes more precise.

错误分析并非一蹴而就。通常需要经过两到三轮的跟踪审查，并不断完善故障模式分类。早期几轮审查往往会发现最初遗漏的故障模式，或者明确指出需要调整定义。随着分析的跟踪数据越来越多（无论是标记之前忽略的跟踪，还是抽取新的查询样本），您对数据的理解也会不断加深，分类也会变得更加精确。

Refinement takes different forms. Sometimes you merge categories that turn out to be too similar to distinguish reliably. Other times you split a category that was lumping together failures with different root causes. And sometimes you simply clarify a definition to make it easier to apply consistently.

改进的形式多种多样。有时，我们会合并过于相似、难以可靠区分的类别。有时，我们会拆分那些将不同根本原因导致的故障混为一谈的类别。有时，我们只是简单地澄清定义，使其更容易被一致地应用。

For example, a second round of analysis on the real estate CRM assistant might expose a new error that was not apparent in the first batch: the assistant occasionally misinterprets location names by defaulting to a local city without asking for clarification. A user asking about Springfield listings might receive results from the wrong state entirely. This would lead you to add *location ambiguity errors* as a distinct failure mode.

例如，对房地产客户关系管理 (CRM) 助手进行第二轮分析可能会发现第一批分析中未发现的新错误：助手有时会错误地解读地点名称，默认显示某个本地城市，而没有询问用户是否需要进一步确认。例如，查询斯普林菲尔德房源的用户可能会收到来自完全错误州的搜索结果。这就需要将*地点歧义错误*添加为一种独立的故障模式。

Iteration is not endless. In practice, two serious rounds of open coding and reannotation are often sufficient to approach theoretical saturation. After that, additional error analysis on the same dataset typically yields diminishing returns. That said, error analysis is something we always return to periodically. After fixing errors, shipping to more users, or observing new patterns of user behavior, cycle back through the process with fresh data. The value of error analysis comes from engaging with what

your system is actually doing now, not from repeatedly analyzing a static dataset.

迭代并非永无止境。实际上，两轮严谨的开码编码和重新标注通常足以接近理论饱和。之后，对同一数据集进行额外的错误分析通常会带来收益递减。即便如此，我们仍然会定期进行错误分析。在修复错误、向更多用户发布产品或观察到新的用户行为模式后，我们会使用新的数据重新进行整个流程。错误分析的价值在于关注系统当前的实际运行情况，而不是反复分析静态数据集。

A NOTE ON EXHAUSTIVE LABELING
关于详尽标签的说明

Earlier, we recommended labeling only the first failure for each trace. This keeps the initial annotation process fast and focused. But first-failure labeling has a limitation: it undercounts failure modes that tend to occur later in traces. For example, if Inappropriate Tone often appears after an earlier error like Missing Constraint, your first-failure labels will rarely capture it. This makes it hard to precisely measure the impact of fixes.

之前，我们建议只标记每个跟踪记录中的第一次故障。这样可以保证初始标注过程快速高效。但仅标记第一次故障存在一个局限性：它会低估跟踪记录中后续出现的故障模式。例如，如果“语气不当”故障经常出现在“缺少约束”等较早错误之后，那么仅标记第一次故障的标签就很难捕捉到它。这使得我们难以精确衡量修复的效果。

If you fix Inappropriate Tone but only recorded it in a handful of traces, you won't be able to tell whether your fix actually worked. If you want to measure the impact of a fix, go back and annotate *all instances* of that failure mode, not every failure mode. Just annotate the ones you're actively fixing or tracking.

如果你修复了“语气不当”问题，但只在少数几个跟踪记录中记录了它，你就无法判断你的修复是否真的有效。如果你想衡量修复的效果，请返回并标注*所有出现该故障模式的实例*，而不是所有故障模式。只需标注你正在积极修复或跟踪的那些故障即可。

As you gain more experience with error analysis, the process becomes much faster. You develop intuition for what to look for, your taxonomy stabilizes, and labeling becomes routine. You might also find that you label all failures in open coding instead of just the first one, because it is very fast for you to do open coding—which is great.

随着错误分析经验的积累，整个过程会变得非常迅速。你会逐渐培养出寻找错误点的直觉，你的分类体系也会趋于稳定，标记错误也会成为一种习惯。你可能还会发现，你会标记所有开放式编码中的错误，而不仅仅是第一个错误，因为开放式编码对你来说非常快捷——这很棒。

Common Pitfalls 常见陷阱

There is no single correct way to do error analysis. You may find that you want to spend more time on some stages than others, depending on your team's working style and how difficult your application is to evaluate. However, there are many bad ways to do error analysis. We list some common pitfalls in this section.

错误分析没有唯一正确的方法。根据团队的工作方式和应用程序的评估难度，您可能需要在某些阶段投入更多时间。然而，错误分析中存在许多错误的做法。本节列举了一些常见的陷阱。

The most common mistake in early error analysis is failing to test on representative data. If your initial query set does not reflect the diversity and difficulty of real user behavior, the traces you produce will be uninformative. Either no serious failures occur, or the failures that do appear are not the ones that would arise in production.

早期错误分析中最常见的错误是未能使用具有代表性的数据进行测试。如果初始查询集不能反映真实用户行为的多样性和复杂性，那么生成的跟踪信息将毫无价值。要么根本不会出现严重的故障，要么出现的故障并非生产环境中会遇到的故障。

A second common mistake is skipping open coding altogether. Instead of reading real traces and observing how the system actually fails, teams often default to generic categories pulled from LLM research, such as "hallucination," "instruction following," and "verbosity." These labels may sound reasonable, but without grounding in real examples, they often miss application-specific issues. In the real estate CRM assistant, for instance, errors like proposing unavailable showings or misidentifying client personas are far more damaging than generic verbosity. Many evaluation vendors promote off-the-shelf metrics, which can lead engineers toward metrics that sound useful but do not capture what actually matters for their application.

第二个常见的错误是完全忽略开放式编码。团队通常不会去阅读真实的代码跟踪信息，观察系统实际的故障原因，而是默认采用从 LLM 研究中提取的通用分类，例如"幻觉"、"指令执行"和"冗长"。这些标签听起来似乎合理，但如果没有实际案例的支持，它们往往会忽略特定应用的问题。例如，在房地产 CRM 助手系统中，推荐不可用的看房信息或错误识别客户画像等错误，远比泛泛的冗长描述更具破坏性。许多评估供应商推广现成的指标，这可能会引导工程师使用一些看似有用但实际上无法反映其应用真正重要问题的指标。

Another frequent pitfall is the inappropriate use of Likert scales during early annotation. A Likert scale asks annotators to rate qualities like relevance or helpfulness on a numeric scale, typically 1 to 5. While widely used, Likert scales introduce substantial

noise when applied without a detailed rubric. When rubrics are underspecified, rating scales lead to lower agreement between annotators and higher subjective variance. In contrast, forcing binary decisions about specific failure modes (whether a problem occurred or not) produces more reproducible annotations.

另一个常见的误区是在早期标注阶段不恰当地使用李克特量表。李克特量表要求标注者对相关性或有用性等特质进行评分，评分范围通常为 1 到 5。虽然李克特量表应用广泛，但如果没有详细的评分标准，就会引入大量的噪声。当评分标准不够明确时，评分量表会导致标注者之间的一致性降低，主观差异增大。相比之下，强制对特定故障模式做出二元判断（例如，问题是否发生）则能产生更具可重复性的标注结果。

Finally, treating initial annotations and failure modes as fixed is a mistake. It is normal for annotation schemas to evolve after reviewing more data. New examples may expose edge cases that require refining definitions. Freezing the schema too early locks your evaluation infrastructure around an incomplete understanding of system behavior. Relatedly, excluding domain experts from the labeling process is a misstep, especially for applications that require deep subject matter knowledge. Outsourcing annotation to people without domain expertise often leads to superficial or incorrect labels. If domain experts find your annotation tools cumbersome, invest in simpler interfaces that make their participation easier.

最后，将初始标注和故障模式视为固定不变是错误的。标注模式在审查更多数据后进行演进是正常的。新的示例可能会暴露出需要完善定义的极端情况。过早地冻结模式会将评估基础架构限制在对系统行为理解不完整的框架内。同样，将领域专家排除在标注过程之外也是一个错误，尤其对于需要深厚领域知识的应用而言。将标注工作外包给缺乏领域专业知识的人员通常会导致标注肤浅或错误。如果领域专家觉得你的标注工具使用起来很麻烦，那就应该开发更简洁的界面，让他们更容易参与其中。

NOTE 笔记

Doing error analysis well is a skill that takes practice. If you want structured reps, we run a live course where you work through error analysis on your own systems (or guided problems) and get concrete feedback on your setup and findings.

做好故障分析是一项需要练习的技能。如果您需要系统化的练习，我们提供在线课程，您可以在自己的系统（或我们指导的问题）上进行故障分析，并获得关于您的设置和分析结果的具体反馈。

Putting It Together 整合

At the end of the Analyze phase, you should have three concrete outputs:

分析阶段结束时，您应该得到三个具体的输出结果：

- A dataset of traces, each annotated with free-form observations about what went wrong

  一个包含多条轨迹的数据集，每条轨迹都附有关于出错原因的自由形式的注释。
- A taxonomy of failure modes, derived from clustering those observations

  基于对观察结果进行聚类分析而得出的失效模式分类法
- Initial counts of how often each failure mode appears in your dataset

  数据集中每种故障模式出现频率的初始统计

The taxonomy is the most important output. It gives you and your team a shared vocabulary for describing how the system fails, which drives every decision in the Measure and Improve phases. If your taxonomy doesn't match the failures your users actually experience, the evaluators you build on top of it will measure the wrong things.

分类体系是最重要的成果。它为你和你的团队提供了一套共同的词汇，用于描述系统故障，并指导"衡量与改进"阶段的每一个决策。如果你的分类体系与用户实际遇到的故障不符，那么你基于此构建的评估工具就会衡量错误的东西。

In Chapter 4, we cover how to get your team aligned on annotation criteria and build the collaborative processes that keep the taxonomy accurate as the system evolves.

在第 4 章中，我们将介绍如何让您的团队在注释标准上达成一致，并构建协作流程，以随着系统的演进保持分类法的准确性。

1 The method draws on grounded theory (Glaser and Strauss 2017; Strauss and Corbin 1990), though we use it pragmatically rather than following the full methodology.

该方法借鉴了扎根理论（ Glaser 和 Strauss 2017 ； Strauss 和 Corbin 1990 ），尽管我们务实地使用它，而不是遵循完整的方法论。

Chapter 4. Collaborative Evaluation Practices

第四章 协作评估实践

Error analysis gives us a list of failure modes, as discussed in Chapter 3. The next question is whether different people on the team agree on what qualifies as a failure. This chapter covers how to align human judgment. You will learn:

错误分析为我们提供了各种故障模式的信息，详见第 3 章的内容。下一个问题是，团队中的不同成员是否对什么才算是故障有共识。本章将探讨如何协调人类判断的标准。你将了解到：

- When a single "benevolent dictator" expert is enough for evaluation, and when you need multiple annotators

  当只需要一位"仁慈的独裁者"专家来进行评估时，以及当需要多名注释者参与工作时，就需要这种情况了。

- A nine-step collaborative annotation workflow for producing consistent rubrics

  一种由九个步骤组成的协作注释工作流程，用于生成统一的教学标准评分细则。

- How to measure inter-annotator agreement with Cohen's Kappa (and when to use alternatives like Fleiss's Kappa or Krippendorff's Alpha)

  如何衡量不同注释者之间的一致性，可以使用 Cohen 的 Kappa 系数；而何时使用 Fleiss 的 Kappa 系数或 Krippendorff 的 Alpha 系数则取决于具体情况。

- How to run alignment sessions that resolve disagreements and improve rubrics

  如何执行能够解决分歧并改进评分标准的对齐操作过程

- Common pitfalls in collaborative evaluation and how to avoid them

  协作评估中常见的错误及如何避免这些错误

Error analysis depends on human judgment at every step. You decide what qualifies as a failure, and how to categorize errors. But evaluation criteria are often subjective (what one person considers "helpful," another might find verbose), and individual judgment is inconsistent (the same person might rate the same trace differently on different days). If the labels are unreliable, everything built on top of them (failure mode

counts, automated evaluators, improvement decisions) inherits that noise.

错误分析在每一步都依赖于人类的判断。由你来决定什么才算是一种失败，以及如何对错误进行分类。但评估标准往往具有主观性——一个人认为“有帮助”的内容，另一个人却可能觉得它过于冗长。而且，不同人的判断也会不一致——同一个人在不同的日子里对同一个代码段的评价可能会有所不同。如果标签的可靠性不可靠，那么基于这些标签所做的所有决策（如故障模式的统计、自动化评估工具、改进建议等）都会受到这种不确定性因素的影响。

This chapter covers *collaborative evaluation practices*: structured approaches for getting multiple people to agree on what qualifies as a pass or a fail. As a motivating example, Notion runs evaluations across roughly 70 engineers and built a dedicated team whose job is to define what “good” looks like for each AI feature and write it down as a rubric. Their framework accounts for *criteria drift*, the fact that what qualifies as “good” shifts over time as the team’s understanding of users and edge cases deepens. Shankar et al. (2024d) studied the same phenomenon in a controlled setting and found that developers routinely revised past labels as they saw more examples. Evaluation criteria are not fixed. They evolve as the team sees more data, and the rubrics have to evolve with them.

本章探讨了协作式评估实践：如何制定结构化方法，让多人达成一致，确定什么才算合格。作为一个实例，Notion 团队在大约 70 名工程师的参与下进行了评估，并组建了一个专门团队，负责定义每个 AI 功能的“优秀”标准，并将这些标准整理成评估准则。他们的框架考虑到了标准的变化——随着团队对用户需求和边缘情况的理解加深，什么才算是“优秀”，也会随着时间而发生变化。Shankar 等人（2024d）在受控环境下研究了这一现象，发现开发者们会在看到更多案例后定期修改之前的评估标准。评估标准并不是固定的，而是随着团队对数据的了解加深而不断演变，相应的评估准则也需要随之调整。

When a Single Expert Is Enough

当一位专家就足够了的时候

Before diving into collaborative workflows, consider: *do you actually need multiple annotators?* In many cases, the answer is no. In small- to medium-sized companies, appointing a single principal domain expert (someone whose judgment serves as the definitive standard for evaluation) is often more effective.[1]

在讨论协作工作流程之前，先考虑一下：真的需要多个注释者吗？在很多情况下，答案是否定的。在小型到中型公司中，任命一位领域专家作为主导航，由其进行判断作为评估的标准，往往更为有效。[1]

We call this the “benevolent dictator” model, borrowing a term from open source software governance. Just as some successful open source projects have a single maintainer whose judgment is final, some evaluation efforts benefit from having one person whose word is the “ground truth.”

我们将这种模式称为“仁慈的独裁者”模式，这个术语来源于开源软件治理领域。就像一些成功开源项目由一位维护者负责所有决策一样，一些评估工作也受益于有一位人作为“权威依据”来指导整个过程。

The benevolent dictator model works well when a few conditions hold:

当满足某些条件时，仁慈的独裁者模式效果很好：

- There is a clear authority figure whose judgment the team trusts. This person defines what is acceptable technically and helps the team understand whether the product matches what users actually want. When one person’s judgment is the standard, there are no annotation conflicts to resolve.

  有一个明确的权威人物，团队信任他的判断。这个人能够界定在技术层面上什么是可行的，并帮助团队了解产品是否符合用户的实际需求。当一个人的判断成为标准时，就无需解决任何关于注释的冲突问题了。

- The evaluation criteria are within that person’s expertise. By involving them throughout the evaluation process, you uncover preferences and expectations that they might not be able to articulate upfront. The process of reviewing traces helps them clarify what a “passable” AI interaction looks like.

  评估标准属于该人的专业领域之内。通过让他们在整个评估过程中参与进来，你可以发现他们的一些偏好和期望，这些可能是他们无法在初期阶段明确表达的。回顾这些过程有助于他们明确什么是“尚可接受的”人工智能交互。

- The volume of labeling work is manageable for one person. This keeps the process fast and simple, and gives the expert a stake in the AI’s development. They feel invested because they have had a hand in shaping it, which makes them more likely to champion the product.

  对于一个人来说，标记工作的量是完全可以处理的。这样既能保持流程的高效和简单，又能让专家参与到人工智能的开发过程中。他们因为参与了产品的设计过程而感到有成就感，因此更有可能积极推广这款产品。

NOTE 注意/备注

If you are an independent developer, you are the domain expert by default. Be honest with yourself about your expertise, and regularly validate your assumptions against real user feedback. If you must rely on leadership as your domain expert, make sure their judgments reflect actual user needs, not personal preferences.

如果你是一个独立的开发者，那么你自然而然就是该领域的专家。要诚实地了解自己的专业知识，并定期通过真实的用户反馈来验证你的假设是否正确。如果你必须依赖领导层来担任领域专家的角色，那么请确保他们的决策能够反映用户的实际需求，而不是个人偏好。

Who might serve as a benevolent dictator? Here are some examples:

谁可能适合担任仁慈的独裁者呢？以下是一些例子：

- A psychologist for a mental health AI assistant

  一位负责心理健康相关人工智能助手的心理学家
- A lawyer for an AI that analyzes legal documents

  一位负责处理人工智能生成的法律文件的律师
- A customer service director for a support chatbot

  一位负责支持聊天机器人的客户服务主管
- A lead teacher or curriculum developer for an educational AI tool

  一位负责教育人工智能工具的教学主管或课程开发者
- The CEO or founder in a small startup

  在小型初创企业中，CEO 或创始人通常承担整个企业的责任。

Many developers attempt to act as the domain expert themselves, or find a convenient proxy (such as their manager). This can be a recipe for disaster. People throughout an organization will have varying opinions about what is acceptable, and it is impossible to make everyone happy. The benevolent dictator model works precisely because it cuts through this ambiguity by designating a single source of truth.

许多开发者试图亲自担任领域的专家角色，或者找一个方便的代理来代表他们行事（比如他们的经理）。但这实际上可能是导致问题的原因。因为整个组织中的人们对于什么是可以接受的行为有着不同的看法，所以不可能让每个人都满意。而"仁慈的独裁者"模式之所以有效，正是因为它通过指定一个单一的权威来源，从而消除了这种不确定性。

The benevolent dictator model has limits, however. If the domain is too complex for one person to evaluate comprehensively, if the criteria are contested among stakeholders, or if you need to demonstrate that your evaluation process is rigorous and unbiased, you will need to involve multiple annotators. The rest of this chapter covers how to do that effectively.

不过，这种仁慈的独裁者模式也有其局限性。如果某个领域过于复杂，以至于一个人难以全面评估，或者相关标准在各方利益相关者之间存在争议，或者你需要证明自己的评估过程是严谨且公正的，那么就需要让多个评估者参与进来。本章的其余部分将介绍如何有效地实现这一点。

A Collaborative Annotation Workflow

协作式注释工作流程

When a single expert is not sufficient, you can use a structured process for involving multiple people in evaluation. This section outlines a workflow for collaboratively defining and refining evaluation criteria. The goal is to produce clear, consistent rubrics that multiple people (and eventually automated evaluators) can apply reliably.

当单个专家不足以完成评估任务时，可以采用结构化流程来让多人参与评估过程。本节概述了协作定义和完善评估标准的工作流程。我们的目标是制定出清晰、统一的评分标准，以便多人（最终还包括自动化评估工具）都能可靠地应用这些标准进行评估。

Should your team tackle one criterion at a time or several at once? In practice, most teams annotate multiple criteria in a single pass through the data. If you are already reviewing a trace, it is efficient to label that trace for factual correctness, tone, completeness, and any other criteria that matter, all at once. However, when you measure agreement and run alignment sessions, treat each criterion separately. Two annotators might both recognize when a response contains factual errors, but have completely different intuitions about what qualifies as “too formal” or “too casual” in tone. If you compute a single agreement score across all criteria, you will not know which rubrics are working and which need refinement. Measure agreement per criterion, and run

alignment sessions focused on one criterion at a time.

你的团队应该一次只处理一个标准，还是可以同时处理多个标准呢？实际上，大多数团队在一次性处理数据时会标注多个标准。如果你正在审核一份记录，那么一次性标注该记录的事实准确性、语气、完整性以及其他重要的标准是非常有效的做法。然而，在衡量一致性并进行一致性评估时，应该分别对待每个标准。两位注释者可能会都识别出回答中存在的事实错误，但他们对“过于正式”或“过于随意”这种语气的理解却截然不同。如果你只计算所有标准的一致性得分，就无法知道哪些评分标准有效，哪些需要改进。因此，应该分别衡量每个标准的一致性，并针对每个标准进行单独的一致性评估。

Our collaborative evaluation workflow has nine steps:

我们的协作评估工作流程包含九个步骤：

Step 1: Assemble the annotation team.

步骤 1：组建注释团队。

Select two or more annotators with relevant domain expertise or stakeholder perspective. Clarify their roles and the scope of the annotation task from the outset. In larger organizations, you might involve representatives from engineering, product, legal, and customer support.

选择两名或两名以上的注释者，他们应具备相关的领域专业知识或能够从利益相关者的角度提供意见。需要明确他们的职责以及注释任务的范围。在规模较大的组织中，可以考虑邀请来自工程、产品、法律和客户支持等方面的代表参与这项工作。

Step 2: Draft initial rubrics.

步骤 2：起草初步的评分标准。

Create first versions of your rubrics, one per evaluation criterion. Each rubric should include a working definition (e.g., “Is the response factually accurate with respect to the source document?”) and a few illustrative Pass/Fail examples. These drafts often emerge from earlier error analysis. You might be formalizing failure modes you identified while reading traces yourself.

首先为每一个评估标准创建相应的评分标准草案。每个评分标准都应包含明确的定义（例如：“该回答在事实准确性方面是否符合原始文档的描述？”），并给出一些典型的通过/失败示例。这些草案通常源自早期的错误分析过程。你可能在编写这些草案时，已经确定了某些可能导致失败的环节。

Step 3: Select a shared annotation set.

步骤 3：选择一组共享的注释集。

Curate a common set of 20–50 representative traces that all annotators will label. Choose examples that are directly relevant to the criteria you are evaluating, and deliberately include borderline or ambiguous cases. These edge cases are where the rubrics get tested. If everyone agrees on the easy cases but disagrees on the hard ones, you know the rubrics need work.

请挑选 20 到 50 个具有代表性的案例进行评审，这些案例需要由所有评估者进行标注。选择那些与你所评估的标准直接相关的案例，并特意包含一些边缘或模糊的案例。这些边缘案例正是评估标准发挥作用的地方。如果大家都对简单的案例持有相同意见，但在复杂的案例上存在分歧，那么说明我们的评估标准还需要改进。

Step 4: Label independently.

步骤 4：独立地进行标记。

Each annotator labels all examples on their own, using the draft rubrics. No discussion is allowed during this phase. The goal is to surface differences in how annotators interpret the rubrics, not to reach consensus yet. If annotators talk during this phase, their judgments become correlated, and you lose the ability to detect ambiguities.

每个注释者都会独立地对所有示例进行标注，使用预先制定的评分标准。在这个阶段，不允许进行任何讨论。我们的目标是揭示注释者在理解评分标准方面的差异，而不是达成一致意见。如果注释者在这个阶段进行交流，他们的判断就会相互关联，这样就无法发现其中的模糊之处了。

Step 5: Measure inter-annotator agreement.

步骤 5：测量不同注释者之间的一致性。

After labeling, compute inter-annotator agreement (IAA) scores for each criterion separately, using metrics like Cohen's Kappa (discussed in "Measuring Inter-Annotator Agreement"). Low agreement on a particular criterion flags that its rubric is vague or that annotators are interpreting it differently. A criterion with high agreement is ready to move forward. One with low agreement

needs more work.

在标注完成后，需要分别计算每个标准下的注释者一致性得分。可以使用 Cohen 的 Kappa 指标来计算一致性得分（详见“衡量注释者一致性”部分）。某个标准的一致性较低，说明该评分标准不够明确，或者注释者对该标准的理解存在差异。而一致性较高的标准则表明可以继续进行下一步操作。相反，一致性较低的指标则需要进一步改进。

Step 6: Facilitate alignment sessions.

步骤 6：组织协调会议。

Bring annotators together to discuss cases where they disagreed. Focus on one criterion at a time. Mixing discussions of factual accuracy and tone in the same conversation creates confusion. The goal is to understand the source of disagreement and improve the rubric, not to argue about past labels. “Facilitating Alignment Sessions” covers this in more detail.

让注释者们聚在一起，讨论那些让他们产生分歧的案例。每次只关注一个标准。在同一场讨论中同时涉及事实准确性与语气问题会造成混乱。我们的目标是理解分歧的根源，并改进评分标准，而不是为了争论过去的标签。“促进共识会议”对此有更详细的说明。

Step 7: Revise the rubrics.

步骤 7：修改评分标准。

Update each rubric based on insights from the alignment discussion. Clarify definitions, refine examples, and adjust labeling criteria as needed. The rubrics should become more precise with each iteration.

根据讨论结果更新每个评分标准。明确各项定义，完善示例说明，并根据需要进行调整标签说明。随着每次迭代的进行，这些评分标准将会变得更加精确。

Step 8: Iterate. 步骤 8：迭代。

Repeat the process: select new examples, label independently, measure IAA per criterion, and revise. Continue until agreement reaches acceptable levels, typically a Cohen's Kappa of 0.6 or higher for each criterion you care about.

重复这个过程：选择新的样本，独立地对它们进行标注，根据各项标准来衡量 IAA 的值，然后进行调整。持续这个过程，直到各项标准的一致性达到可接受的水平——通常每个标准对应的 Cohen's Kappa 值应达到 0.6 或更高。

Step 9: Finalize rubrics and labels.

步骤 9：确定评分标准及各项标签的命名方式。

Once agreement is consistently high, document the final rubrics and generate a consensus-labeled dataset. This "gold standard" set serves two purposes: it validates that humans can apply the rubrics consistently, and it provides ground truth for evaluating automated LLM-as-judge evaluators (which we cover in Chapter 5).

一旦一致性评分一直很高，就需记录最终的评分标准，并生成一个由多人共同认可的评估数据集。这一"黄金标准"数据集具有双重作用：它验证了人类能够一致地使用这些评分标准；同时，它也为评估基于 LLM 的自动化评估器提供了基准数据（我们将在第五章中详细讨论这一点）。

The workflow (Figure 4-1) requires effort, but it pays off.

该工作流程（见图 4-1）需要付出一定的努力，但所带来的回报是值得的。

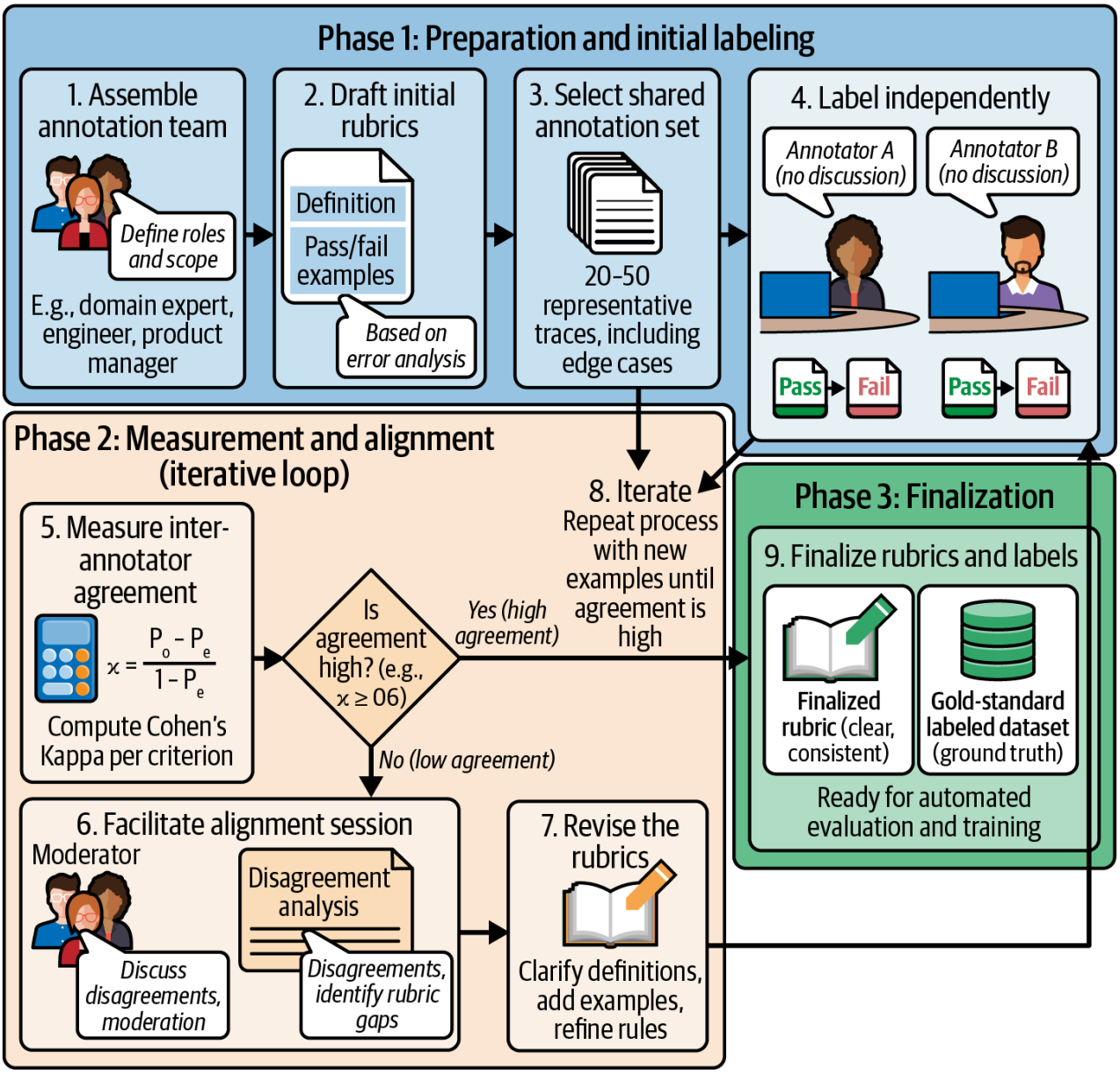


Figure 4-1. The phases of the collaborative evaluation workflow

图 4-1. 协作评估工作流程的各个阶段

The rubric you produce will be clearer and more consistently applicable than anything one person could create alone. And the gold-standard labels you generate will provide a reliable foundation for training automated evaluators, measuring system performance, and tracking improvement over time.

您生成的评分标准将会比任何单一个人单独制定的标准都要清晰明了，且具有更强的可重复性。而您所创建的黄金标准标签则可以为训练自动化评估工具、衡量系统性能以及跟踪长期改进效果提供可靠的依据。

Measuring Inter-Annotator Agreement

测量注释者之间的一致性

How do you know whether your annotators are applying the rubric consistently? You measure their agreement.

如何知道你的注释者是否始终遵循相同的评分标准？可以通过测量他们的评分一致性来实现这一目标。

Percent Agreement: Simple but Misleading

百分比一致性：简单但具有误导性

The simplest measure is *percent agreement*: the proportion of items where both annotators assigned the same label. If two annotators each labeled 100 traces and agreed on 85 of them, the percent agreement is 85%.

最简单的衡量指标是一致性百分比：即两位注释者都给出相同标签的项目所占的比例。例如，如果两位注释者各自标注了 100 个轨迹，其中有 85 个轨迹的意见一致，那么一致性百分比就是 85%。

Mathematically, percent agreement (also called observed agreement, P_o) is:

从数学上讲，一致性百分比（也称为观测到的一致性，P_o）定义为：

$$P_o = \frac{\text{number of items where both annotators agree}}{\text{total number of items}}$$

Percent agreement is easy to compute and easy to interpret. But it has a serious flaw: it does not account for agreement that could happen by chance.

百分比一致性很容易计算，也很容易解释。不过，这种方法有一个严重的缺陷：它无法考虑到那些可能是偶然发生的一致性情况。

Consider a scenario where 95% of your traces are Pass and only 5% are Fail. If both annotators simply labeled everything as Pass, they would agree on 95% of traces, even though neither is actually reading the traces carefully. The high agreement would be

an artifact of the skewed label distribution, not evidence that the rubric is clear.

假设有 95%的测试案例通过，只有 5%的测试案例失败。如果两位评估者都将所有测试案例标记为“通过”，那么他们就会对 95%的测试案例达成一致意见，尽管实际上他们并没有仔细审查这些测试案例。这种高度的一致性只是由于标签分布不平衡造成的，并不能证明评估标准是清晰明确的。

The label skew becomes more severe as your system improves. Early in development, you might have a roughly even mix of Pass and Fail traces, so chance agreement is around 50%. But as you fix bugs and the system gets better, more traces become Pass. If 90% of traces are Pass, chance agreement rises to about 82% (0.9 × 0.9 + 0.1 × 0.1 = 0.82). A percent agreement of 85% would sound good, but it barely exceeds what you would expect from random labeling.

随着系统性能的提升，标签分配的不对称性会逐渐加剧。在开发初期，通过率和失败率的比例可能大致相等，因此一致性约为 50%。但是，随着你不断修复漏洞并改进系统，更多的记录会被标记为通过。如果 90%的记录都是通过，那么一致性将上升到约 82%（0.9×0.9 + 0.1×0.1 = 0.82）。85%的一致性水平已经相当不错了，但这几乎还低于从随机标签分配中预期的结果。

Cohen's Kappa: Adjusting for Chance

科恩系数 Kappa：考虑偶然因素的影响

Cohen's Kappa (κ) addresses this problem by comparing observed agreement to expected chance agreement (Cohen 1960). The formula is:

科恩系数 Kappa (κ) 通过比较观察到的符合程度与预期的偶然符合程度来解决这一问题（Cohen 1960）。公式如下：

$$\kappa = \frac{P_o - P_e}{1 - P_e}$$

Here, P_o is the observed percent agreement, and P_e is the expected agreement if both annotators were labeling randomly (but with their actual label distributions).

在这里， P_o 表示观测到的百分比一致性，而 P_e 则表示如果两位注释者都是随机标注的话，预期会达到的一致性水平（当然，这里使用的是实际的标签分布）。

To compute P_e, you calculate how often each annotator uses each label, then compute the probability that they would agree by chance. For binary labels (Pass/Fail):

要计算 P_e 的值，需要先计算每个注释者使用每个标签的频率，然后计算出他们偶然达成一致的概率。对于二元标签（通过/未通过）的情况如下：

$$P_e = (P_{\text{Pass},1} \times P_{\text{Pass},2}) + (P_{\text{Fail},1} \times P_{\text{Fail},2})$$

A Kappa of 1 means perfect agreement. The annotators agreed on everything, and none of that agreement is attributable to chance. A Kappa of 0 means the annotators agreed exactly as often as chance would predict, so their agreement is meaningless. A negative Kappa means they agreed less often than chance, which suggests systematic disagreement (perhaps they are using opposite interpretations of the rubric).

当 Kappa 值为 1 时，意味着完全一致。所有注释者之间的意见完全一致，这种一致性并非偶然产生。当 Kappa 值为 0 时，意味着注释者们的意见出现的概率与随机情况相同，因此他们的共识毫无意义。如果 Kappa 值为负数，则说明注释者们的意见出现的概率低于随机情况，这表明他们之间存在系统性分歧（也许他们对评分标准的理解方式恰恰相反）。

Let's try working through the calculation on a concrete set of labels. Two annotators labeled 100 email summaries for "informativeness." Here are the results:

让我们尝试用具体的标签来进行分析。有两位注释者对 100 份电子邮件摘要进行了"信息量"的标注。以下是标注结果：

	Annotator B: Pass 注释员 B：通过	**Annotator B: Fail 注释员 B：失败**	**Total 总计**
Annotator A: Pass 注释员 A：通过	75	10	85
Annotator A: Fail 注释员 A：失败	5	10	15
Total 总计	80	20	100

Now we take the following steps:

现在我们执行以下步骤：

Step 1: Compute observed agreement (P_o)

步骤 1：计算观测到的一致性程度（P_o）

The annotators agreed on 75 + 10 = 85 items:

这些注释者们在 75 + 10 = 85 项上达成了共识：

$$P_o = \frac{85}{100} = 0.85$$

Step 2: Compute expected agreement (P_e)

步骤 2：计算预期的一致性程度（P_e）

> Annotator A labeled 85% as Pass, 15% as Fail. Annotator B labeled 80% as Pass, 20% as Fail:
>
> 注释员 A 将 85%的条目标记为“通过”，15%的条目标记为“失败”。注释员 B 将 80%的条目标记为“通过”，20%的条目标记为“失败”。

$$P_e = (0.85 \times 0.80) + (0.15 \times 0.20) = 0.68 + 0.03 = 0.71$$

Step 3: Compute Kappa

步骤 3：计算 Kappa 值

$$\kappa = (0.85 - 0.71)/(1 - 0.71) = 0.14/0.29 \approx 0.48$$

Despite 85% raw agreement, the Kappa is only 0.48 (moderate agreement). Much of the apparent agreement was due to both annotators favoring Pass.

尽管两个评分者之间的评分一致性达到了 85%，但总体一致性仅为 0.48（属于中等程度的一致性）。这种看似一致的结果，很大程度上是因为两位评分者都倾向于选择“通过”这一选项。

Interpreting Kappa Scores

解读卡帕评分结果

A standard interpretation of Kappa values, following Landis and Koch (1977), is:

根据 Landis 和 Koch 在 1977 年提出的方法，Kappa 值的标准解释方式如下：

- <0: Poor (worse than chance)

  <0: 很差（比正常情况下还要糟糕）>
- 0.00–0.20: Slight 0.00–0.20：轻微
- 0.21–0.40: Fair 0.21–0.40：公平
- 0.41–0.60: Moderate 0.41–0.60：中等程度
- 0.61–0.80: Substantial 0.61–0.80：相当不错
- 0.81–1.00: Almost perfect

  0.81–1.00：几乎完美

In practice, aim for $\kappa \geq 0.6$ before treating your labels as reliable. Below that threshold, the rubric likely needs refinement, because annotators are interpreting it too differently to produce consistent labels.

在实践中，在认为标签具有可靠性之前，应先将目标设定在 $\kappa \geq 0.6$ 以下。如果低于这个阈值，那么评分标准可能需要改进，因为评分者对数据的解读各不相同，从而导致不一致的标签结果。

How strict you need to be depends on what you're using the labels for. For rubric iteration (that is, deciding whether your failure categories need splitting or merging), κ around 0.5 is workable. For validating an LLM judge against human labels, aim for at least 0.6. For labels that feed into model training or regulatory reporting, aim for 0.7 or higher. Note that $\kappa \geq 0.6$ is very challenging for subjective tasks; in many real-world annotation projects, κ around 0.4 already represents meaningful progress.

你需要的准确率范围取决于你使用这些标签来做什么。在评估评分标准时（即判断各个失败类别是否需要合并或拆分），0.5 左右是一个可行的目标。而当需要验证 LLM 的判罚结果是否与人类标注的结果一致时，目标应达到 0.6 以上。对于那些用于模型训练或法规报告的标签来说，准确率应达到 0.7 或更高。需要注意的是，对于主观性较强的任务来说，达到 1.0 是非常具有挑战性的；而在许多实际的标注项目中，达到 0.4 左右就已经算是取得了显著的进展了。

IAA can be viewed, somewhat, as a ceiling on your automated evaluators. If human annotators agree at $\kappa = 0.7$, we cannot expect an LLM judge to reliably exceed that level of accuracy, because the labels it's being compared against are themselves noisy. Keep the agreement level in mind when evaluating judge performance in Chapter 5.

IAA 可以视为自动评估器的上限。如果人类注释者能够达成一致，那么我们就不能期望 LLM 的评判结果能够可靠地超过这个精度水平，因为与其对比的标签本身就有噪声。在评估评判者的表现时，请记住这个一致性标准，详见第 5 章。

If your Kappa is significantly below 0.6, do not continue blindly labeling more traces. You will just be producing inconsistent data. Look at the specific traces where annotators disagreed. In "Preparing for the Session", Annotators B and C both read the same response about a duplex being "popular with families," but B saw it as a persona mismatch while C saw it as relevant tenant-demand context. That disagreement highlighted that the rubric never specified whether family-oriented language qualifies as investor-appropriate when it describes rental appeal. A quick discussion of this one trace led to an explicit decision rule and improvement to the rubric. Without that cali-

bration, the team would have continued labeling inconsistently.

如果您的 Kappa 值明显低于 0.6，请不要继续盲目地标记更多数据。这样做会导致数据的不一致性。请重点关注注释者之间存在分歧的那些数据。在“准备会议”环节，注释者 B 和 C 对关于“双户住宅受家庭欢迎”这一描述的看法相同，但 B 认为这是一种人物角色不匹配的情况，而 C 则认为这是符合租户需求的背景信息。这种分歧表明，评分标准并未明确说明，当描述出租房产的吸引力时，是否使用以家庭为导向的语言才是合适的。针对这一数据进行的简短讨论，最终促成了一个明确的决策规则，并改进了评分标准。如果没有这种校准，团队可能会继续采用不一致的标记方式。

Some failure modes are also inherently harder to agree on. Factual errors tend to produce high agreement, because either the response says the property has three bedrooms or it does not. But judgments about tone, persona fit, or “appropriateness” often do not. In that example, “client persona match” hit a Kappa of only 0.3 or 0.4 on the first pass. The team improved alignment by splitting “client persona match” into two separate criteria: one for whether the response includes information relevant to the client’s goals (like cap rates for investors), and another for whether it avoids language that sounds like it is addressing a different audience. Annotators had been agreeing on the first question but disagreeing on the second, so separating them showed where the rubric actually needed work.

有些失败模式本身就难以达成一致的意见。关于事实方面的错误往往能带来较高的一致性，因为回答中要么明确说明该属性包含三个卧室，要么就不包含。但关于语气、角色匹配或“适用性”等方面的判断则往往难以达成一致。在那个例子中，“客户角色匹配”这一指标在初次评估时的一致性仅为 0.3 或 0.4。团队通过将“客户角色匹配”分解为两个独立的标准来改进一致性：第一个标准是回答中是否包含与客户目标相关的信息（比如投资者关心的资本回报率），第二个标准是回答中是否避免使用那些听起来像是在针对其他受众的语言。注释者们在第一个问题上达成了一致，但在第二个问题上存在分歧，因此将这两个标准分开处理有助于明确该指标实际存在的缺陷。

Python Implementation Python 实现

Here is a Python implementation of Cohen’s Kappa:

以下是 Cohen 的 Kappa 系数的 Python 实现：

```
from collections import Counter

def cohens_kappa(rater1, rater2):
    """
    Compute Cohen's Kappa for two lists of categorical ratings.
```

```
    Args:
        rater1: List of labels from first annotator
        rater2: List of labels from second annotator

    Returns:
        float: Cohen's Kappa score
    """
    assert len(rater1) == len(rater2), "Rating lists must have same length"
    n_items = len(rater1)

    if n_items == 0:
        return 1.0  # Convention: perfect agreement if nothing to rate

    # Observed agreement
    agreements = sum(1 for i in range(n_items) if rater1[i] == rater2[i])
    p_observed = agreements / n_items

    # Expected agreement by chance
    labels = set(rater1) | set(rater2)
    count1 = Counter(rater1)
    count2 = Counter(rater2)

    p_expected = sum(
        (count1.get(label, 0) / n_items) * (count2.get(label, 0) / n_items)
        for label in labels
    )

    # Handle edge case where expected agreement is 1
    # (all items have the same label from both raters)
    if p_expected == 1.0:
        if p_observed == 1.0:
            return 1.0
        raise ValueError(
            "Expected agreement is 1.0 but observed agreement is not; "
            "check that the input labels are correct."
        )

    return (p_observed - p_expected) / (1 - p_expected)


# Example usage
annotations_a = ['Pass', 'Fail', 'Pass', 'Pass', 'Fail', 'Pass', 'Fail']
annotations_b = ['Pass', 'Fail', 'Fail', 'Pass', 'Fail', 'Pass', 'Pass']
print(f"Cohen's Kappa: {cohens_kappa(annotations_a, annotations_b):.3f}")
```

When to Use Other Metrics

何时使用其他指标

Cohen's Kappa is designed for a specific situation: two annotators rating the same items with categorical labels. When your setup differs, other metrics may be more appropriate.

科恩的卡帕系数适用于一种特定情况：两名评测者对同一组带有分类标签的项目进行评分。当实际情况与这种假设不同时，可能需要使用其他指标来进行评估。

More than two annotators

超过两位注释者

If three or more people are labeling the same traces, use Fleiss's Kappa, which generalizes Cohen's Kappa to multiple raters. Where Cohen's Kappa compares one pair of annotators, Fleiss's Kappa measures the average agreement across all items, adjusted for chance. Like Cohen's Kappa, the formula is:

如果有三个人或以上对同一组痕迹进行标注，那么可以使用 Fleiss 的 Kappa 公式。该公式将 Cohen 的 Kappa 公式扩展到了多个评估者的情况。虽然 Cohen 的 Kappa 公式是比较了一对标注者之间的一致性，但 Fleiss 的 Kappa 公式则衡量的是所有标注者之间的平均一致性，同时考虑了随机性的影响。其计算公式与 Cohen 的 Kappa 公式相同：

$$\kappa = \frac{\overline{P}_o - \overline{P}_e}{1 - \overline{P}_e}$$

Here, $\overline{P}_o$ is the average observed agreement across all items (computed by looking at how often annotators assigned the same label to each item), and $\overline{P}_e$ is the expected agreement by chance (computed from the overall distribution of labels across all annotators). The interpretation is the same as Cohen's Kappa (values above 0.6 indicate substantial agreement), but the calculation accounts for comparing across more than one pair.

这里，$\overline{P}_o$ 表示所有项目之间的平均一致性水平（通过计算注释者对每个项目分配相同标签的频率来得出）；$\overline{P}_e$ 则表示偶然情况下预期的一致性水平（根据所有注释者分配标签的总体分布来计算）。这种计算方法与 Cohen 的 Kappa 检验类似（数值大于 0.6 表示存在显著的一致性），但计算过程中考虑了多对之间的比较。

Ordinal ratings 顺序评分

If your labels have a natural order (like a 1-5 quality scale), standard Kappa treats all disagreements as equally bad: rating something a 4 versus a 5 counts the same as rating it a 1 versus a 5. That is often not what you want, because being one point off is less serious than being four points off. Weighted Kappa addresses this by assigning different weights to different levels of disagreement. The formula is:

如果标签之间存在自然的排序关系（例如 1 到 5 的评分体系），那么标准的 Kappa 方法会将所有差异视为同等严重的问题：将某项的评分定为 4，与将其评分定为 5 一样严重。但这往往不符合我们的期望，因为相差 1 点比相差 4 点要糟糕得多。加权 Kappa 方法通过为不同程度的差异分配不同的权重来解决这个问题。其计算公式如下：

$$\kappa_w = \frac{P_o(w) - P_e(w)}{1 - P_e(w)}$$

The weighted observed agreement $P_o(w)$ and weighted expected agreement $P_e(w)$ are computed using a weight matrix that specifies how "bad" each type of disagreement is. A common choice is quadratic weights, where the penalty for disagreement grows with the square of the distance between labels: a 1 versus 5 disagreement is penalized 16 times more than a 1 versus 2 disagreement.

加权观测一致性 $P_o(w)$ 和加权预期一致性 $P_e(w)$ 是通过使用权重矩阵来计算的，该矩阵决定了每种类型的不一致性的"严重程度"。常见的选择是使用二次权重，即不一致性的惩罚随着标签之间距离的平方而增加：与 1 对 5 的不一致相比，1 对 2 的不一致性受到的惩罚要高出 16 倍。

That said, we generally recommend binary Pass/Fail labels over ordinal scales. With a 5-point scale, you have to define what distinguishes a 3 from a 4, and annotators often have different intuitions about these boundaries. Binary labels force you to make one clear decision: does this trace meet the bar or not? If your application is mature enough that you want finer-grained quality distinctions (say, you are prioritizing *which* acceptable responses to improve further), ordinal scales can be useful. But start with binary labels until you have a specific reason to need more granularity.

不过，我们通常建议使用二元标签来表示通过/失败的情况，而不是序数标签。在五点量表的情况下，需要明确界定 3 分和 4 分之间的区别，而评分者往往对这些界限有不同的理解。二元标签则迫使你做出明确的判断：这条记录是否通过了标准？如果你的应用程序已经足够成熟，需要更细致的质量区分（比如，你需要优先处理哪些可接受的反馈以进行改进），那么序数标签就很有用。但在没有具体理由需要更高粒度的情况下，还是继续使用二元标签吧。

Missing data

In practice, not every annotator labels every trace. Someone gets sick, or you deliberately split the work so that only a subset of annotators sees each item. Cohen's Kappa assumes complete overlap. Both annotators must have labeled all the same items. Krippendorff's Alpha, named after communication scholar Klaus Krippendorff, handles missing data gracefully. The formula is:

$$\alpha = 1 - \frac{D_o}{D_e}$$

Here, D_o is the observed disagreement (computed from the actual labels) and D_e is the expected disagreement by chance. Unlike Kappa, which computes agreement, Alpha computes disagreement and subtracts it from 1. This formulation naturally handles missing data because it only considers pairs of labels that actually exist. Alpha also works with any number of annotators and any measurement scale (binary, ordinal, interval, or ratio), making it a flexible default when your annotation setup is complex.

Alpha values are interpreted similarly to Kappa: 1 means perfect agreement, 0 means agreement no better than chance, and negative values mean systematic disagreement. Krippendorff recommends $\alpha \geq 0.8$ for reliable data and suggests treating α between 0.67 and 0.8 as acceptable for tentative conclusions. These thresholds are stricter than the typical Kappa guidance, so do not be alarmed if data that clears $\kappa \geq 0.6$ still falls short of $\alpha \geq 0.8$.

For most binary or categorical labeling tasks with two raters and complete coverage, Cohen's Kappa is the right choice. It is simple to compute and widely understood. When your situation is more complex, Krippendorff's Alpha is a flexible alternative that handles multiple raters, ordinal scales, and missing data in a single framework.

COHEN'S KAPPA AND LLM-AS-JUDGE EVALUATORS

LLM-as-judge evaluators are the focus of Chapter 5, but since we are discussing agreement metrics, note this common mistake to avoid: *using Cohen's Kappa to measure how well an LLM judge matches human-generated "ground truth" labels.* Cohen's Kappa measures agreement between two peers. Neither is assumed to be "correct." This is appropriate when two humans are developing a rubric together. You want to know whether they interpret the rubric the same way, not which one is right.

Once you have established gold-standard human labels, you have a source of truth. If you want to know whether an LLM judge matches those labels, you are no longer asking "do these two raters agree?" You are asking "does this judge get the true answer?" That is a classification problem, and you need to know which way the judge is wrong. Did the judge say Pass when the human said Fail (a false positive)? Or Fail when the human said Pass (a false negative)? A judge that is too lenient will miss bad outputs. A judge that is too strict will flag good outputs for unnecessary review. With classification metrics like precision, recall, and true/false positive rates, you can diagnose these failure modes separately. From Cohen's Kappa, you only learn that humans and LLM judges disagree, so do not make the mistake of using it for judge evaluation.

Facilitating Alignment Sessions

When annotators disagree, you need to understand why. Alignment sessions are structured discussions where annotators review their disagreements and work together to improve the rubric.

当注释者之间存在分歧时，我们需要了解原因。对齐会议是一种结构化讨论形式，注释者们会回顾彼此的分歧之处，并共同努力改进评分标准。

Preparing for the Session
为这次会议做准备

Before bringing annotators together, you must:

在将注释者聚集在一起之前，你必须做到以下几点：

- Calculate and share IAA scores.

  计算并分享 IAA 评分结果。

- Identify the specific disagreements.

  明确具体的分歧点。

- Appoint a neutral moderator.

  请任命一位中立的主持人。

To calculate and share IAA scores, let everyone see the overall agreement level so they understand the scope of the problem. For example, you might report: "On the *factual accuracy* criterion, we achieved a Kappa of 0.74 (substantial agreement). On *appropriate tone for client persona*, we achieved a Kappa of 0.38 (fair agreement). We will focus the alignment session on tone."

在计算并分享 IAA 评分时，让每个人都能看到整体的一致性水平，这样他们就能了解问题的严重程度。例如，你可以这样报告："在事实准确性方面，我们的 Kappa 值为 0.74，表明一致性相当高。在符合客户角色所需的适当语气方面，我们的 Kappa 值为 0.38，表明一致性一般。我们将把注意力集中在语气上的调整上。"

When you identify the specific disagreements, pull out the traces where annotators gave different labels. Prepare a summary document listing each disputed trace, the labels each annotator assigned, and any notes they provided.

当你确定了具体的分歧点后，请找出那些被不同注释者赋予不同标签的痕迹。准备一份摘要文档，其中列出每一个有争议的痕迹、每个注释者所分配的标签，以及他们提供的任何相关注释。

To make this concrete, consider the real estate CRM assistant from the previous chapter, which helps agents search listings, schedule showings, and draft client emails. Suppose your team is evaluating whether the assistant's responses are "appropriate for the client persona." The initial rubric might look like this:

具体来说，可以参考上一章中提到的房地产 CRM 助手。该助手可以帮助经纪人搜索房源、安排看房时间，以及起草客户邮件。假设你的团队正在评估这个助手的回复是否"符合目标客户的需求"。初步的评估标准可能如下：

> **Criterion: Client Persona Match**
>
> **Pass: The response is tailored to the client's background and goals.**
> **An investor should receive information relevant to investment decisions. A first-time homebuyer should receive information that helps them understand the buying process.**
>
> **Fail: The response ignores or mismatches the client's background and goals.**

Examples:
[Pass] Investor client asks about a property. Response includes rental income potential and cap rate.
[Fail] Investor client asks about a property. Response focuses entirely on school districts and family amenities.

This rubric seems reasonable. It defines the criterion, explains the intent, and provides examples. But it leaves important questions unanswered. What if a response includes *both* investment metrics and family-oriented language? What counts as "tailored"? Is mentioning one relevant detail enough, or does the entire response need to match? How should annotators handle responses that are neutral and generic rather than actively mismatched?

这个评分标准看起来挺合理的。它明确了评判标准，解释了评分意图，并给出了具体的示例。不过，还是有一些重要的问题没有得到解答。如果一份回答同时包含了投资相关的指标和符合家庭需求的表述，该怎么办呢？什么才算是"量身定制的"回答呢？仅仅提及一个相关细节是否足够，还是整个回答都必须与要求相符？对于那些中性、泛泛的回答，注释者应该如何处理呢？

After independent labeling, your disagreement summary might look like this:

在独立标注完成后，您的不一致意见摘要可能如下所示：

Trace #23: Investor client asked about a duplex. Assistant responded: "This duplex is in a growing neighborhood with a strong rental market. The 3-bed/2-bath layout is popular with families, and the backyard is a nice bonus. Current tenant pays $2,400/month."

- Annotator A: Pass (mentions rental income and market strength)
- Annotator B: Fail (the "popular with families" and "backyard" language is inappropriate for an investor)
- Annotator C: Pass (the family language describes tenant appeal, which is relevant to rental investment)

All three annotators applied the rubric in good faith, but they reached different conclusions. Annotator A focused on whether investment-relevant information was present. Annotator B flagged the family-oriented language as a mismatch. Annotator C interpreted that same language as relevant investment context (explaining why the property attracts tenants). The rubric did not specify how to weigh these competing

considerations, so each annotator made their own judgment call.

三位注释者都本着诚实的态度应用了评分标准，但他们的结论却有所不同。注释者 A 关注的是是否存在与投资相关的信息。注释者 B 认为那些以家庭为导向的表述方式并不合适。注释者 C 则将同样的描述视为与投资相关的信息，从而解释了为什么该房产能够吸引租户。由于评分标准并未明确说明如何权衡这些相互冲突的考虑因素，因此每位注释者都自行做出了判断。

Someone needs to keep the discussion focused on improving the rubric rather than winning arguments about specific labels. Ideally, you would appoint a neutral moderator who is not one of the annotators. A project lead or someone from an adjacent team who understands the product but did not participate in labeling can work well.

Running the Session

The goal of an alignment session is not to force agreement on past labels. The goal is to understand why annotators interpreted the rubric differently and to fix the rubric so future annotations will be consistent.

For each disagreement, work through these questions:

What did each annotator think?

Have each person explain their reasoning without interruption. In the Trace #23 example, Annotator C might say: "I labeled it Pass because the family-oriented language was explaining tenant appeal, which is useful information for an investor deciding whether the property will stay rented." Annotator B might counter: "But the rubric says the response should be tailored to the client. *Popular with families* and *nice backyard* sound like you are selling to a homebuyer, not an investor." Both interpretations are defensible, which means the rubric needs to address this ambiguity.

Where does the rubric fail?

Identify the specific word, phrase, or gap in the rubric that allowed different interpretations. In Trace #23, the rubric said the response should be "tailored to the client's background and goals" but did not specify whether language that serves a secondary purpose (describing tenant demographics) qualifies as tailored or mismatched. The rubric also lacked guidance on how to handle mixed responses that contain both investor-relevant and family-oriented content.

What would make future annotators agree?

Focus on forward-looking changes, not retrospective blame. For Trace #23, the team might decide to add: "Family-oriented language is acceptable if it explicitly connects to investment value (e.g., *popular with families, which keeps vacancy rates low*). Family-oriented language without investment framing should be marked Fail."

Timebox each disagreement to prevent the session from dragging on. Five minutes per case is usually enough. If you cannot resolve it quickly, flag it for follow-up and move on.

Techniques for Improving the Rubric
改进评分标准的技巧

Based on the disagreements you uncover, you can improve the rubric in several ways:
根据您发现的这些分歧，您可以以几种方式改进评分标准：

Clarify wording. 请明确一下措辞。

Vague or overloaded terms cause most disagreements. In the real estate assistant example, "tailored to the client's background and goals" turned out to be ambiguous. Annotators disagreed about whether describing tenant demographics qualifies as investor-tailored content. The revised rubric should make the boundaries explicit:

模糊或过于复杂的术语往往是引发分歧的原因。在房地产代理这个例子中，"根据客户的历史背景和目标进行定制"这一表述就存在歧义。相关人员在是否将租客的人口统计信息也视为为投资者量身定制的内容这个问题上产生了分歧。修改后的评分标准应该明确这些界限。

Pass: The response focuses on information relevant to the client's goals:
- Investors: Financial metrics (cap rate, rental income, appreciation), market conditions, tenant stability, and property condition. Descriptions of tenant demographics are acceptable only if explicitly connected to investment value.
- First-time buyers: Affordability, financing options, neighborhood livability, and process guidance.

Fail: The response includes content mismatched to the client's goals without connecting it to their actual needs.

Add examples. 添加示例。

Concrete Pass/Fail examples are often clearer than abstract definitions. When a disagreement surfaces an edge case, add it to the rubric as an illustrative example. After the Trace #23 discussion, you might add:

具体的通过/失败示例通常比抽象定义更清晰。当出现分歧时，将边缘案例添加到评分标准中作为说明示例。在 Trace #23 的讨论后，您可能会添加：

Examples:
[Pass] Client is an investor. Response: "This duplex offers a 6.2% cap rate with stable rental history. The 3-bed layout is popular with long-term family tenants, which keeps turnover low."
[Fail] Client is an investor. Response: "This duplex is in a great neighborhood. The 3-bed layout is popular with families, and the backyard is a nice bonus." (No connection to investment value)
[Pass] Client is a first-time buyer. Response: "This home qualifies for FHA loans, which means you could put down as little as 3.5%."
[Fail] Client is a first-time buyer. Response: "The cap rate is suboptimal but the IRR looks promising if you refi in year three."

Add decision rules. 添加决策规则。

For tricky situations, provide explicit rules. For example: "If the response mentions family-oriented features (backyard, school district, kid-friendly) to an investor client, label as Pass only if the response explicitly connects those features to rental demand, tenant stability, or property value. Otherwise, label as Fail."

对于复杂的情况，需要提供明确的规则。例如："如果回答中提到了与家庭生活相关的特点（如后院、学区、适合儿童的活动等），那么只有当回答中明确将这些特点与租赁需求、租户稳定性或房产价值联系起来时，才标记为"通过"。否则，就标记为"失败"。"

Split the criterion. 将标准进行拆分。

Sometimes disagreements point to a single rubric trying to capture multiple distinct concepts. If annotators consistently agree on whether financial metrics are present but disagree on whether lifestyle language is acceptable, consider splitting "client persona match" into two separate criteria: "includes relevant infor-

mation" and "avoids mismatched framing." Measuring agreement separately for each criterion will tell you which one needs more work.

有时候，分歧的出现是因为试图用一个标准来涵盖多个不同的概念。如果注释者们在"财务指标是否包含在内"这一问题上达成一致，但在"是否使用与客户生活方式相关的表述"这一问题上存在分歧，那么可以将"客户形象的匹配度"分解为两个独立的标准："是否包含相关信息"和"是否避免不一致的表述"。分别衡量这两个标准的一致性，就能看出哪个方面需要更多的改进。

Acknowledge irreducible ambiguity.

承认存在无法消除的歧义。

Some cases are genuinely borderline. A response that mentions "good schools nearby" to an investor, without elaboration, might be interpreted as either a rental-demand signal or mismatched homebuyer language. If reasonable people could legitimately disagree, document the ambiguity and establish a tiebreaker rule (e.g., "if the connection to investment value is implied but not explicit, label as Pass").

有些情况确实处于模糊地带。对一个投资者提到"附近有好学校"，但没有进一步说明，可能会被解读为要么是租赁需求信号，要么是购房者的不匹配语言。如果理性的人可以合情合理地存在分歧，记录下这种模糊性，并建立一种平局决定规则（例如，"如果与投资价值的联系是暗示性的但不是明确的，标记为通过"）。

What to Avoid 应避免的事项

Do not rely on majority voting to fix the rubric. Voting can resolve individual label disagreements, but voting does not improve the rubric. If three annotators vote 2-1 on a trace, you have resolved that trace but learned nothing about why they disagreed. Use voting only as a last resort for finalizing labels after the rubric is stable.

不要依赖多数投票来决定评分标准。投票可以解决个别标签上的分歧，但并不能改善评分标准本身。如果三位评注者对某个轨迹的评分分别为 2-1，那么你们已经解决了该轨迹的问题，但并未了解到他们为何会有不同的意见。只有在评分标准确定之后，才将投票作为最终确定标签的手段。

Do not let discussions become adversarial. The goal is to improve the rubric, not to prove that one annotator was "right." Frame disagreements as information about

where the rubric needs work.

不要让讨论变得对立起来。我们的目标是改进评分标准，而不是证明某个注释者是正确的。将分歧视为关于评分标准需要改进的地方的信息。

Do not skip documentation. Write down every rubric change and the reasoning behind it. Future annotators (and future you) will need this context.

请不要跳过文档记录的步骤。请详细记录每次评分标准的变更以及背后的原因。未来的注释者（以及未来的你）会需要这些信息作为参考。

Escalation 升级

Sometimes annotators cannot reach agreement even after discussion. When this happens, escalate to a project lead or senior domain expert who can make a binding decision. Document the ruling and add it to the rubric. Persistent disagreement often indicates that the criterion itself is poorly defined or that the annotators have fundamentally different values, both of which are important to surface.

有时，即使经过讨论，注释者也无法达成一致意见。在这种情况下，应将问题上报给项目负责人或领域内的资深专家，让他们做出具有约束力的决定。将这一决定记录下来，并添加到评估标准中。持续存在的分歧往往表明该评判标准本身不够明确，或者注释者们的价值观存在根本差异，这两点都需要被充分考虑到。

Save Everything 保存所有内容

Even after you have trained or prompted an automated evaluator, keep your gold-standard dataset and documentation. You may need them later for:

即使在您已经训练或提示了自动化评估器之后，仍然保留您的黄金标准数据集和相关文档。以后您可能会需要它们。

Validating automated evaluators

正在验证自动化评估工具

When you build an LLM-as-judge or other automated evaluator (covered in Chapter 5), you need ground truth labels to measure whether the evaluator is accurate. The gold-standard dataset provides that ground truth.

当你构建一种作为裁判的 LLM 或其他自动化评估器时（详见第 5 章），你需要真实的标签来衡量该评估器的准确性。标准数据集就提供了这些真实标签。

Fine-tuning 微调

If you decide to train a specialized LLM, the labeled data becomes training data.

如果你决定训练一个专门的大语言模型，那么那些经过标注的数据就成为了训练数据。

Auditing 审计

If stakeholders question your evaluation methodology, you can point to the rigorous process that produced your rubrics and labels.

如果利益相关者对你的评估方法提出质疑，你可以指出正是这一严谨的过程才产生了你所使用的评分标准和分类方式。

Iteration 迭代

As your system evolves, you may need to update your evaluation criteria. The documentation of past decisions helps you understand why the rubric looks the way it does.

随着系统的不断发展，你可能需要更新你的评估标准。回顾过去的决策过程，有助于你理解为何当前的评分标准是这样的。

Common Pitfalls 常见陷阱

Collaborative evaluation is powerful, but it requires careful management. Here are the most common ways teams go wrong.

协作式评估非常有效，但也需要仔细的管理。以下是团队常犯的错误类型。

Skipping Independent Annotation
跳过独立注释步骤

If annotators discuss traces or the rubric extensively before labeling independently, their judgments become correlated. They converge on a shared interpretation through discussion rather than by applying the rubric as written. This makes subsequent IAA

measurements meaningless. You are measuring groupthink, not rubric clarity.

如果注释者在独立标注之前先对评分标准或相关细节进行充分讨论，那么他们的判断就会变得相互关联。他们通过讨论来达成一致的解释，而不是按照既定的评分标准来进行判断。这样一来，后续的 IAA 测量结果就毫无意义了。实际上，你们正在陷入集体思维的影响，而不是追求评分标准的清晰性。

The whole point of independent annotation is to detect ambiguity. If annotators coordinate beforehand, you lose this signal. They might achieve high agreement, but a new annotator given only the rubric might interpret it completely differently.

独立注释的本质就是为了解决歧义问题。如果注释者之间事先进行协调，那么这种解决歧义的方式就不存在了。他们或许能够达成高度一致的意见，但一个新的注释者仅凭给定的评分标准，可能会得出完全不同的解释。

Note that if you have decided to use the benevolent dictator model (a single annotator), this pitfall does not apply. There is only one annotator, so independence is not an issue. But if you have multiple annotators, their initial labels must be independent.

请注意，如果您决定采用仁慈独裁者模型（即由一名注释者负责所有任务），那么这种缺陷就不成立了。因为只有一个注释者，所以独立性并不是一个问题。但如果你有多个注释者，他们的初始标签就必须保持独立。

Starting with a Vague Rubric

从模糊的评分标准开始

If your initial rubric is overly vague or incomplete, the first annotation round will produce low agreement and frustrate your annotators. Some vagueness is unavoidable (you cannot anticipate every edge case), but you can invest effort in the initial draft. Include clear definitions and at least a few concrete examples before asking people to start labeling. The real estate assistant example earlier in this chapter illustrates how a rubric that says only “tailored to the client’s background and goals” invites three different interpretations, while splitting that criterion into “includes relevant informa-

tion" and "avoids mismatched framing" produces much more consistent labels.

如果您的初始评分标准过于模糊或不完整，那么在第一轮注释过程中，达成一致的几率就会很低，这会让注释者感到沮丧。有些模糊性是不可避免的（您无法预见到每一种情况），但您可以在初稿阶段投入更多精力来明确这些定义，并至少提供几个具体的例子。本章前面提到的"房地产助理"例子就说明了：如果评分标准只是写着"符合客户的背景和目标"，那么就会产生三种不同的解读；而将这一标准分解为"包含相关信息"和"避免不一致的表述"两个子标准，就能得到更加一致的标签。

Using Only Percent Agreement
仅使用百分比一致性指标

Percent agreement is intuitive but misleading when label distributions are skewed. Teams often celebrate 90% agreement without realizing that chance alone would produce 80% agreement given their label distribution. Always compute Cohen's Kappa (or an appropriate alternative) to understand whether agreement exceeds chance.

百分比一致性虽然直观，但在标签分布存在偏斜时可能会产生误导。团队常常庆祝达到90%的一致性，却并未意识到，根据他们的标签分布，单纯的概率水平就已经能实现80%的一致性了。因此，务必计算科恩系数（或类似指标），以判断一致性是否超出了偶然性范围。

Ignoring Temporal Drift 忽略时间偏差

Interestingly, throughout several deployments, we have found that annotator standards can "drift"; that is, they can change over time because of new business context, fatigue, or simply having seen more examples. After labeling hundreds of traces, annotators often become more lenient or more strict compared to when they started. To catch drift, it's important to periodically relabel a small anchor set of 10 to 15 traces that you labeled at the beginning of the project. Drift itself isn't bad; it is just important that you know it exists, so you can correct or recalibrate labels as needed.

有趣的是，在多次部署过程中，我们发现注释标准可能会发生变化。也就是说，由于新的业务环境、疲劳程度的变化，或者仅仅是因为观察到了更多的例子，注释者的标准会随着时间而调整。在标记了数百条轨迹之后，注释者通常会比刚开始时更加宽松或严格。为了捕捉这种变化，定期重新标记一些最初被标记的 10 到 15 条轨迹是非常重要的。这种变化本身并不坏，重要的是要意识到它的存在，这样就能根据需要进行修正或重新调整标签。

Synthetic-Annotator Collusion

合成注释器勾结行为

One other issue we have seen is that two annotators might use the same LLM to help accelerate their grading process and, as a consequence, have extremely high agreement. In this case, the annotations are converging because they share the same LLM's biases, not because the rubric is clear. High κ in this situation is misleading, because it reflects the LLM's consistency with itself rather than genuine human agreement on the criteria.

我们还注意到另一个问题：两位注释者可能会使用相同的语言模型来加速他们的评分过程，因此他们的评分结果具有极高的一致性。在这种情况下，评分结果趋同是因为他们使用了相同的语言模型的预设偏见，而不是因为评分标准本身非常明确。在这种情况下，高达 κ 的一致性其实具有误导性，因为它反映的是语言模型内部的稳定性，而非人类在评分标准上的真正一致性。

Focusing on Past Labels Instead of Future Consistency

专注于过去的标签，而不是未来的一致性。

Alignment sessions can easily devolve into arguments about whether a specific trace "should have been" Pass or Fail. This is unproductive. The goal is to improve the rubric so that future annotators will agree, not to relitigate past decisions. Keep discussions focused on: "What change to the rubric would have made us agree on this case?"

对齐环节很容易演变成关于某个具体评分"应该为通过还是失败"的争论。这种讨论毫无成效。我们的目标是改进评分标准，让未来的注释者能够达成一致意见，而不是重新讨论过去的决定。请保持讨论的焦点在于："对评分标准进行哪些修改才能让我们在这个案例上达成一致？"

Putting It Together 整合起来

At the end of collaborative evaluation, you should have three outputs:

在协作评估结束时，你应该得到三个结果：

- A *refined rubric* with clear definitions and concrete examples for each failure mode, sharp enough that a new annotator can apply it without additional coaching.

一个完善的评分标准，包含对每个故障模式清晰的定义和具体的示例说明。这个标准足够精确，以至于新的评分者无需额外指导就能使用它进行评分。

- A *gold-standard labeled dataset* with consensus labels. You'll use these labels in the next chapter to validate automated evaluators.

  这是一个遵循黄金标准的标注数据集，其中包含了共识性标签。在下一章中，我们将使用这些标签来验证自动化评估器的性能。

- *Documentation* of edge cases and the decisions you made about them, so the rubric's reasoning is preserved as the team evolves.

  记录所有极端情况以及你们针对这些情况的决策过程，这样在团队发展过程中，评分标准的相关依据就能被保留下来。

The rubric and the gold-standard dataset are the bridge between human judgment and automated measurement. With clearer rubrics, you can build more accurate LLM-as-judge evaluators; with reliable gold-standard labels, you can meaningfully validate those evaluators. The next chapter covers how to build those evaluators and how to calibrate them against the gold labels you've produced here. Together, the two outputs provide the foundation for measuring system performance at scale and tracking improvement over time.

评分标准和黄金标准数据集是人工判断与自动化评估之间的桥梁。有了更清晰的评分标准，就能打造出更精确的基于大语言模型的评估工具；而可靠的黄金标准标签则能有效地验证这些评估工具的准确性。下一章将介绍如何构建这些评估工具，以及如何将它们与你们生成的黄金标准标签进行校准。这两种输出方式共同为大规模测量系统性能以及跟踪性能改进提供了基础。

In summary, when a single expert's judgment is sufficient (because the domain is narrow, the criteria are clear, and the expert has time), use the benevolent dictator model. It is simpler and faster. But when subjectivity, complexity, or scale demand multiple perspectives, the collaborative workflow in this chapter will help you build evaluation infrastructure that you can trust.

总结来说，当单个专家的判断就足够满足需求时（因为领域较为狭窄，标准明确，且专家有时间进行评估），可以采用仁慈独裁者模型。这种方式更简单、更快捷。但当我们面临主观性、复杂性或规模问题，需要多个角度的评估时，本章所述的协作工作流程就能帮助你构建出值得信赖的评估体系。

1 The key factor is not the number of employees but whether authority can meaningfully be delegated to a single individual. Appointing a trusted expert as the decision-maker can unblock the

labeling bottleneck that many teams face. This is not always feasible or appropriate. Generally, appointing a single principal domain expert is more feasible the smaller the company is.

关键因素并非员工的数量，而是是否能够将决策权合理地委托给单个个体。任命一位值得信赖的专家作为决策者，可以解决许多团队面临的决策瓶颈问题。不过，这种做法并不总是可行或合适。一般来说，在公司规模较小时，任命一位专业领域的专家更为合适。

Part III. The Measure Phase

第三部分： 测量阶段

Chapter 5. Implementing Automated Evaluators

第五章。自动化评估器的实施

In Chapter 3, we identified failure modes by reading traces. In Chapter 4, we aligned the team on what constitutes a failure. Now we automate that measurement so we can track failure rates at scale without manually reviewing every trace.

在第三章中，我们通过读取痕迹来识别失效模式。在第四章中，我们就什么构成失败进行了团队的一致。现在我们自动化了这项测量，这样可以在不手动审查每条痕迹的情况下，大规模跟踪故障率。

In this chapter, you will learn:

在本章中，你将学到：

- How to translate failure modes into precise, automatable metrics

  如何将故障模式转化为精确、可自动化的指标
- When to use code-based checks versus LLM-as-judge evaluators

  何时使用基于代码的检查，而何时作为法官使用 LLM 评估者
- How to build and validate an LLM-as-judge using training, dev, and test sets

  如何利用训练、开发和测试集构建并验证一个作为评判的 LLM
- How to estimate the true success rate of your agent in production

  如何估算你的代理人在生产中的真实成功率
- How to quantify uncertainty using bootstrapping and confidence intervals

  如何利用自助法和置信区间来量化不确定性

Automated evaluators can compute various types of metrics.[1] Some metrics are reference-free (as defined in Chapter 2), assessing inherent qualities of an output or its adherence to certain rules without needing a "golden" or ground truth answer. Others are reference-based, comparing the agent's output to a known correct or ideal re-

sponse. For many failure modes, we can conceptualize and implement both reference-free and reference-based checks.

自动评估器可以计算各种类型的指标。[1] 有些指标是无引用的（如第 2 章定义），评估输出的固有特性或其对某些规则的遵守情况，无需"黄金"或确凿的答案。另一些则基于引用，将代理的输出与已知的正确或理想反应进行比较。对于许多失败模式，我们可以概念化并实现无引用和基于引用的检查。

In this chapter, we first discuss *what* metrics to measure, focusing on how to translate our categorized failure modes into precise, automatable metrics and how to approach the curation of reference examples when needed. Then we explore *how to implement* automated evaluators for metrics, covering both straightforward code-based checks and LLM-as-judge techniques.

本章首先讨论应衡量哪些指标，重点是如何将我们分类的失效模式转化为精确、可自动化的指标，以及在需要时如何策划参考样本。随后，我们探讨如何实现自动化评估器来衡量指标，涵盖了基于代码的简单检查和大型语言模型作为裁判的技术。

We continue with our running example from Chapter 3, Example 3-1: a real estate chat-based CRM assistant that interprets natural language queries from real estate agents, issues SQL, drafts client emails, reads calendars, and searches the internet.

我们继续使用第三章示例 3-1 中的例：一个基于房地产聊天的 CRM 助手，能够解读房地产经纪人的自然语言查询，发布 SQL，起草客户邮件，阅读日历，并搜索互联网。

Defining the Right Metrics (What to Measure)

定义正确的指标（衡量什么）

Effective measurement begins by defining precise, quantifiable metrics for each failure mode identified during our error analysis. To illustrate, Table 5-1 lists several categories of failures we might have uncovered for our real estate CRM assistant from Example 3-1 during such an analysis.

有效测量始于为错误分析中识别的每种故障模式定义精确且可量化的指标。举例来说，表 5-1 列出了我们在例 3-1 中可能发现的几类失败案例。

Table 5-1. Example failure modes from real estate CRM assistant error analysis

5-1 表。房地产 CRM 助理错误分析中的故障模式示例

Failure mode category 失效模式类别	**Description 描述**
Missing SQL constraints 缺失的 SQL 约束	Omits user-specified filters (e.g., pets_allowed , max_price) in SQL 在 SQL 中省略用户指定的过滤器（例如 pets_allowed ， max_price ）
Incorrect SQL aggregation 错误的 SQL 聚合	Generates SQL grouping by day instead of week due to ambiguous phrasing 由于表述模糊，它按天而非周生成 SQL 分组
Invalid tool usage 无效工具使用	Fabricates nonexistent tool names or actions (e.g., book_showings) 伪造不存在的工具名称或动作（例如， book_showings ）
Incomplete email content 邮件内容不完整	Fails to include key client details (budget, location) in communications 在沟通中未能包含关键客户细节（预算、地点）
Persona-tone mismatch 人格-音调不匹配	Uses language unsuitable for the client's persona (e.g., informal with luxury client) 使用不适合客户形象的语言（例如，与奢侈客户的非正式对话）
Incorrect tool sequencing 错误的工具序列	Attempts email composition before retrieving necessary listing data 在获取必要的列表信息之前，尝试通过电子邮件撰写
Unjustified tool calls 无理的工具调用	Issues actions (scheduling, messaging) not requested or grounded in prior steps 出现未被请求或基于先前步骤的操作（排程、消息发送）
Location ambiguity 地点模糊	Interprets a common place name (e.g., “Springfield”) with the wrong region 错误地解释一个常见地名（例如“斯普林菲尔德”）的地区

After listing the failure modes, we can identify which ones are simple to fix due to ambiguity in our prompt specifications. Recall the Three Gulfs model (Figure 1-1), where developers struggle to write LLM applications due to the Gulfs of Comprehension (i.e., understanding their data, as described in Chapter 3), Specification (i.e., writing good prompts and agents), and Generalization. Gulf of Specification errors, stemming from ambiguous or incomplete instructions we provided to the LLM, can be easy to resolve if we edit the prompt. The evals that we write target mainly the Gulf of Generalization.

列出故障模式后，我们可以识别哪些因提示规范不明确而易于修复。回想三湾模型（图 1-1），开发者因理解之湾（即理解数据，如第三章所述）、规范（即编写好的提示和代理）以及泛化而难以编写大型语言模型应用。由于我们向 LLM 提供的歧义或不完整指令，Gulf of Specification 错误很容易解决，如果我们编辑提示词。我们撰写的评估主要针对泛化湾地区。

In other words, we want to *fix ambiguity first, then measure generalization.* There are two reasons for this. First, for *efficiency reasons*: many specification failures can be resolved rapidly, often by simply adding clarity or detail to an existing prompt.[2] Building an automated evaluator for a failure mode that a prompt edit can fix is wasted effort. Evals cost time and effort to create and maintain, so we save them for issues that persist after we have clarified the prompt. Second, more fundamentally, we want our evaluation efforts to accurately reflect the LLM's ability to generalize correctly from clear instructions, not its capacity to decipher our potentially ambiguous intents. Building evaluators for poorly specified tasks essentially measures how well the LLM can "read our minds," which is not a scalable or reliable indicator of robust performance.[3]

换句话说，我们首先要解决歧义，然后再测量泛化。这有两个原因。首先，出于效率考虑：许多规范失败可以通过快速解决，通常只需在现有提示中添加清晰度或细节即可。[2] 为一个可以通过提示编辑修复的失败模式构建自动评估器是浪费精力。评估的创建和维护需要时间和精力，因此我们会保留它们，以应对在澄清提示后仍存在的问题。其次，更根本的是，我们希望评估工作准确反映 LLM 从清晰指令中正确泛化的能力，而不是它解读我们潜在含糊意图的能力。为定义不明确的任务构建评估器，本质上衡量的是 LLM"读心"的能力，这既不是可扩展也不可靠的稳健性能指标。[3]

Consider the incomplete email content example from Table 5-1. The incomplete email content might be a specification failure if our email generation prompt doesn't clearly instruct the LLM on mandatory fields to include based on the context. We can improve

our prompt to fix this. In contrast:

请考虑表 5-1 中的不完整的电子邮件内容示例。如果邮件生成提示没有根据上下文明确指示 LLM 必须包含的字段，那么不完整的邮件内容可能是规范上的失败。我们可以改进提示来修正这个问题。相比之下：

Missing SQL constraints 缺失的 SQL 约束

If the prompt *clearly* stated the constraint (e.g.，"max_price: $600k") but the LLM still omitted it, it points to a generalization failure. This is a good candidate for an automated evaluator.

如果提示词明确说明了约束（例如 "max_price: $600k" ），但 LLM 仍然遗漏了约束，那就说明泛化失败。这很适合自动评估。

Persona-tone mismatch 人格-音调不匹配

Assuming the client persona was clearly provided to the LLM would also be a generalization failure.

假设客户端角色明确提供给 LLM 也是一种泛化失败。

Invalid tool usage (calling a nonexistent tool)

工具使用无效（调用不存在的工具）

This is almost always a generalization failure (or a more fundamental model issue) if the available tools are well-defined to the LLM.

如果现有工具对 LLM 定义良好，这几乎总是泛化失败（或更根本的模型问题）。

Once we have this refined list of generalization failures, we can design specific, automatable metrics for each. We try to design both a reference-based and a reference-free metric for each significant failure mode. For any chosen failure mode, we then carefully consider what an automated evaluator measures and, if pursuing a reference-based approach, curate the necessary high-quality reference examples.

一旦我们有了这份精细的泛化失败清单，就可以为每个失败设计具体的、可自动化的指标。我们尝试为每个显著失效模式设计基于参考和无参考的指标。对于任意选择的失败模式，我们会仔细考虑自动评估器所衡量的标准，如果采用基于参考的方法，则会筛选必要的高质量参考样本。

For reference-based metrics, we first curate input-output pairs where the reference output exemplifies the ideal behavior (i.e., correct output that does not exhibit the

failure mode). The automated evaluator then measures how closely the agent's actual output aligns with this "golden" reference. For example, if a failure mode is missing SQL constraints (see Table 5-1), our reference data would consist of user queries and the corresponding ideal SQL, which *does* include all necessary constraints. The evaluator checks if the agent-generated SQL matches or correctly incorporates these constraints as seen in the reference.

对于基于参考的指标，我们首先筛选出输入输出对，其中参考输出体现了理想行为（即正确且不表现出失效模式的输出）。自动评估器随后测量代理的实际输出与这个"黄金"参考的高度匹配程度。例如，如果失败模式缺少 SQL 约束（见表 5-1），我们的参考数据将由用户查询和相应的理想 SQL 组成，理想 SQL 包含所有必要的约束。评估器检查代理生成的 SQL 是否符合或正确包含了引用中所见的这些约束。

Reference-based metrics are invaluable during iterative development (prompt engineering or model fine-tuning) and are often key components of continuous integration (CI) checks, which we discuss in Chapter 9.

基于引用的指标在迭代开发（提示工程或模型微调）中极为宝贵，且常常是持续集成（CI）检查的关键组成部分，我们在第 9 章中讨论。

For reference-free metrics, we define intrinsic properties or rules related to the failure mode. The evaluator measures adherence to these defined properties. For instance, continuing with "missing SQL constraints," a reference-free check might involve parsing the generated SQL to verify the presence of keywords associated with expected constraints (e.g., if a price was mentioned in the input, does the SQL reference a price-related column?). It could also check for general SQL validity.

对于无参考度量，我们定义了与失效模式相关的内在属性或规则。评估者衡量对这些定义属性的遵循情况。例如，继续"缺失的 SQL 约束"，一个无引用检查可能涉及解析生成的 SQL 以验证与预期约束相关的关键词（例如，如果输入中提到价格，SQL 是否引用与价格相关的列？）。它还可以检查一般 SQL 的有效性。

For "persona-tone mismatch," a reference-free approach might involve using an external judgment process to classify the tone of the generated email (e.g., formal, casual, data-driven)[4] and then check if this classification aligns with the target persona's known preferences. We could also check for the presence or absence of specific jargon or phrases indicative of a mismatch.

对于"人物与语气不匹配"，一种无引用的方法可能包括通过外部判断过程对生成邮件的语气（如正式、随意、数据驱动）[4] 进行分类，然后检查该分类是否符合目标人物已知的偏好。我们还可以检查是否有特定行话或短语，以显示不匹配。

Reference-free metrics are particularly powerful because they can often be adapted to run efficiently at scale on new, unlabeled data, enabling broader monitoring and assessment, potentially even in online settings.

无参考文献的指标尤为强大，因为它们通常可以调整为在新的、未标记的数据上大规模高效运行，从而实现更广泛的监控和评估，甚至可能在在线环境中实现。

As part of both types of metrics, we might incorporate *executability checks*. These checks assess functional correctness, not just the text itself. For instance, if the LLM generates a SQL query, the evaluator might execute it against a test database to ensure it runs without error and yields plausible results. If it generates tool calls, the evaluator might simulate these calls to verify their logical consistency and potential to achieve the intended outcome. These checks confirm whether the output actually works, not just whether it looks right.

作为这两种指标的一部分，我们可能会包含可执行性检查。这些检查不仅评估文本本身，而是功能正确性。例如，如果 LLM 生成 SQL 查询，评估器可能会对测试数据库执行，以确保查询无误且结果合理。如果它生成工具调用，评估者可能会模拟这些调用，以验证其逻辑一致性及实现预期结果的潜力。这些检查确认输出是否正常，而不仅仅是外观是否正确。

Table 5-2 presents sample reference-based and reference-free metrics for some of the failure modes identified in Table 5-1.

表 5-2 展示了表 5-1 中部分失效模式的基于引用和无参考的示例指标。

Table 5-2. Reference-based and reference-free metrics for real estate CRM failures

表 5-2。基于参考和无参考的房地产 CRM 失败指标

Failure mode 故障模式	Reference-based metric 基于参考度量	Reference-free metric 无参考度量
Missing SQL constraints 缺失的 SQL 约束	Compare generated SQL abstract syntax tree (AST) against a golden AST to verify presence of all expected WHERE clauses 将生成的 SQL 抽象语法树（AST）与黄金 AST 进行比较，以验证所有预期的 WHERE 子句的存在	Regex or AST traversal to detect mandatory filter keywords (e.g., pets_allowed , max_price) 正则表达式或 AST 遍历以检测强制过滤关键词（例如： pets_allowed ， max_price ）
Invalid tool usage 无效工具使用	Match sequence of tool calls to a reference trace containing only valid tool identifiers 将工具调用序列与仅包含有效工具标识符的参考跟踪进行匹配	Static schema check: ensure each invoked tool name exists in the registered tool registry 静态模式检查：确保每个调用的工具名称都存在于注册工具注册表中
Incomplete email content 邮件内容不完整	LLM-as-judge check for whether the generated email includes all required fields from the context (budget, location, client name) 作为法官的 LLM 检查生成的邮件是否包含上下文中的所有必填字段（预算、地点、客户名称）	Keyword presence check for mandatory sections (budget, location, client name) using regex patterns 使用正则表达式模式对必修部分（预算、地点、客户名称）进行关键词存在性检查
Persona-tone mismatch 人格-音调不匹配	Compare judge's tone label on generated email against human-labeled references 将生成邮件中法官的语气标签与人工标记的引用进行比较	LLM-as-judge prompt for binary tone classification (Pass/Fail) based on persona definitions 基于人格定义的二元音调分类（通过/不通过）的 LLM 作为评判提示
Location ambiguity 地点模糊	Compare disambiguated location entity IDs against ground truth re-	Geocoding-based check: map mentioned place to coordinates and verify consistency with

Failure mode 故障模式	Reference-based metric 基于参考度量	Reference-free metric 无参考度量
	gion codes 将消除歧义的地点实体 ID 与地面真实区域代码进行比较	user-specified region 基于地理编码的检查：地图上提到的地点与坐标进行验证，并验证与用户指定区域的一致性

Implementing Metrics (How to Measure)

实施指标（如何衡量）

The goal of automated evaluators is to estimate the prevalence of each failure mode across a set of traces. Manual labeling at this scale is too slow and expensive, so we turn to automation. The challenge is ensuring each evaluator is well-grounded in the human-defined failure criteria, because otherwise the metrics are meaningless.

自动评估器的目标是估计每种失效模式在一组痕迹中的流行率。在这种规模下手动标记既慢又昂贵，因此我们转向自动化。挑战在于确保每位评估者都基于人为定义的失败标准，否则这些指标毫无意义。

Each failure mode has its own dedicated evaluator. Depending on the nature of the failure, we can implement this either with code or with an LLM. *Code-based evaluators* are ideal when the failure definition is objective and can be checked with rules. For example:

每种故障模式都有自己的专用评估器。根据失败的性质，我们可以用代码或大型语言模型实现。基于代码的评估器在失败定义客观且可用规则检查时是理想的。例如：

- Parsing structure (e.g., checking JSON validity or SQL syntax)

  解析结构（例如，检查 JSON 有效性或 SQL 语法）
- Using regex or string matching to detect required or forbidden phrases

  使用正则表达式或字符串匹配来检测必需或禁止的短语
- Counting or verifying structural constraints (e.g., a summary has three bullet points)

  计数或验证结构约束（例如，摘要包含三个要点）

- Executing the tool call generated by an LLM, and checking that the execution did not raise any errors

  执行由大型语言模型生成的工具调用，并检查执行过程中没有引发任何错误

- Logical checks (e.g., if the query asks for a dog-friendly apartment, does the output mention “pets allowed”?)

  逻辑检查（例如，如果查询要求允许携带宠物的公寓，输出是否提及“允许携带宠物”？）

Code evaluators are fast, cheap, deterministic, and interpretable. They can support both reference-free checks (e.g., length limits) and reference-based ones (e.g., value extraction compared to a gold set).

代码评估器快速、便宜、确定性强且可解释。它们既支持无引用检查（例如长度限制）和基于引用的检查（例如，与金色集合的价值提取）。

LLM-as-judge evaluators are used when the failure mode involves interpretation or nuance that code can't capture (Zheng et al. 2023). For instance, was the tone appropriate? Is the summary faithful? Is the response helpful? In these cases, we use a separate LLM (distinct from the main application) to judge outputs for a single failure mode. Each metric may require a different LLM-as-judge evaluator, tailored to its specific failure definition. This is expected, and desirable, when evaluating multiple aspects of quality. LLM judges work best for narrowly defined, binary tasks (e.g., when the answer can be Pass or Fail). Open-ended ratings or feedback are harder to supervise and align. However, using LLMs for evaluation introduces bias, can be inconsistent, and adds inference cost. Prompt design, clear criteria, and calibration against human judgments are all important.

当失败模式涉及代码无法捕捉的解释或细微差别时，LLM 作为评判的评估器被使用（Zheng 等，2023）。比如，语气合适吗？摘要忠实吗？这个回复有帮助吗？在这种情况下，我们使用独立于主应用的大型语言模型（LLM）来判断单一故障模式的输出。每个指标可能需要不同的 LLM 评审者，根据其具体的失败定义进行定制。在评估质量的多个方面时，这种做法是预期且理想的。LLM 评判最适合狭义的二元任务（例如，答案可以是通过或不通过）。开放式评分或反馈更难监督和协调。然而，使用 LLM 进行评估会引入偏见、不一致且增加推理成本。提示设计、明确标准以及与人类判断的校准都很重要。

Table 5-3 outlines how we implement different evaluators for different failure modes, choosing programmatic checks when definitions are objective and LLM-as-judge

prompts when nuance or interpretation is required.

表 5-3 概述了我们如何针对不同失败模式实施不同的评估者，当定义客观时选择程序性检查，当需要细致差别或解释时使用 LLM 作为评判的提示。

Table 5-3. Programmatic versus LLM-as-judge evaluators for real estate CRM failures

5-3 表。程序化评估师与以法学博士为评判者的房地产 CRM 失败评估者比较

Failure mode 故障模式	**Programmatic evaluator 程序评估器**	**LLM-as-judge evaluator 作为评审的 LLM 评估者**
Persona-tone mismatch 人格-音调不匹配	N/A (subjective) 无（主观）	Few-shot LLM prompt with definitions and examples to judge tone alignment 少量样本的 LLM 提示，附有定义和示例，用于判断语气对齐
Unjustified tool calls 无理的工具调用	Regex-based filter to flag any action not grounded in input context 基于正则表达式的过滤器，用于标记任何与输入上下文无关的操作	Prompt to assess if every tool call is requested or justified by prior steps 提示评估每个工具调用是否都被请求或基于先前步骤的合理性

Figure 5-1 illustrates the workflow for building an LLM-as-judge evaluator, which we subsequently expand upon.

图 5-1 展示了构建 LLM 作为评审评估者的工作流程，我们随后对此进行了扩展。

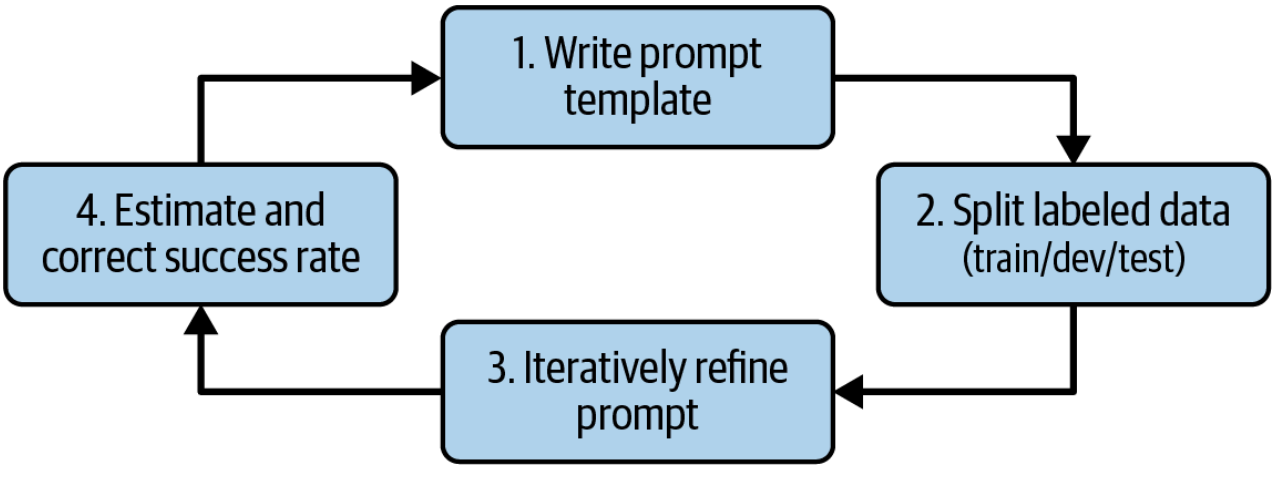


Figure 5-1. Workflow for building an LLM-as-judge evaluator

图 5-1。构建 LLM 作为评审评估者的工作流程

Writing LLM-as-Judge Prompts

作为法官写 LLM 题目

The term “LLM-as-a-judge” was introduced by Zheng et al. (2023), who trained specialized LLM judges using large amounts of human preference data. This fine-tuning process aligns the judge’s behavior with human expectations and has shown strong performance in benchmark settings.

“LLM 即评判”一词由 Zheng 等人（2023 年）提出，他们利用大量人类偏好数据培训专业 LLM 评判。这一微调过程使评委的行为符合人类预期，并在基准环境中表现出色。

In practice, such fine-tuning is rarely feasible. Most applications lack sufficient labeled preference data, especially in early stages of development. Instead, we use off-the-shelf LLMs as judges and rely on prompt engineering to align their outputs with our intended failure definitions.

但实际上，这种微调很少可行。大多数应用缺乏足够的标记偏好数据，尤其是在开发初期阶段。相反，我们使用现成的大型语言模型作为判断工具，并依赖提示工程来使其输出与预期的失败定义对齐。

The same paper also documented several reliability problems that LLM judges run into. The research was done by Lianmin Zheng and colleagues at LMSYS (the group that runs Chatbot Arena, a popular leaderboard where LLMs are compared head-to-head). They tested whether an LLM judge's ratings matched the ratings human reviewers would give, and found three systematic biases:

同一篇论文还记录了 LLM 评判者遇到的若干可靠性问题。这项研究由 LMSYS 的 Lianmin Zheng 及其同事完成（该团队运营 Chatbot Arena，一个大型语言模型进行一对一比较的热门排行榜）。他们测试了 LLM 评委的评分是否与人类审核员的评分相符，发现了三种系统性偏差：

Position bias 位置偏置

When asked to compare two responses side by side, judges tend to favor whichever response comes first (or, for some models, whichever comes second), even when the responses are of equal quality. Zheng et al. (2023) reported that GPT-

4 showed this bias on roughly 26% of its judgments in their study.

当被要求并排比较两个回答时，评委往往倾向于偏好先出现的回答（或某些模型中，是第二个答案），即使两个回答质量相当。Zheng 等人（2023）报告称，GPT-4 在其研究中约有 26% 的判断显示出这种偏差。

Verbosity bias 冗长性偏差

Longer responses get rated as higher quality even when a shorter response is equally correct. A judge that is susceptible to this bias will reward an agent for padding its outputs.

即使较短的回复同样正确，较长的回复也会被评为更高质量。容易受这种偏见影响的评委会奖励代理人填充其输出。

Self-enhancement bias 自我增强偏差

When the judge and the agent being judged are the same model (e.g., GPT-4 judging GPT-4's outputs), the judge tends to rate the agent's outputs more favorably than a neutral judge would.

当评委和被评判的代理是同一模型（例如，GPT-4 评判 GPT-4 的输出时），评委往往比中立评委更倾向于对代理的输出给予更积极的评价。

These biases do not make LLM judges useless, but they mean that raw judge scores cannot be trusted on their own. We have to design prompts and validation procedures that control for them, which is what the rest of this chapter covers.

这些偏见并不意味着 LLM 法官无用，但它们意味着单靠原始法官分数是不可信的。我们必须设计对它们进行控制的提示和验证程序，这也是本章剩余内容的内容。

We now introduce a framework for designing LLM-as-judge evaluators that are both reliable and grounded. Each evaluator corresponds to a single metric, which targets a specific failure mode identified during error analysis. We restrict the task to binary outputs (Pass/Fail or Yes/No).[5] A well-structured LLM-as-judge prompt contains four essential components:

我们现在引入一个设计既可靠又扎实的 LLM 即法官评估者的框架。每个评估器对应一个指标，针对误差分析中识别的特定失效模式。我们将任务限制为二进制输出（通过/不通过或是/否）。[5] 结构良好的 LLM 评审题目包含四个基本要素：

1. Clear task and evaluation criterion

1. 明确的任务和评估标准

Each prompt focuses on one well-scoped failure mode. Vague tasks lead to unreliable judgments. Instead of asking whether an email is "good," we ask whether "the tone is appropriate for a luxury buyer persona."

每个提示都聚焦于一个范围明确的失败模式。模糊的任务会导致不可靠的判断。我们不再问邮件是否"好"，而是问"语气是否适合奢华买家形象"。

2. Precise Pass/Fail definitions

2. 精确的通过/不通过定义

We define what constitutes a Pass (failure absent) and a Fail (failure present), directly based on the failure descriptions developed during error analysis.

我们根据错误分析中产生的故障描述，定义了 Pass （缺失故障）和 Fail （存在失效）。

3. Few-shot examples 3. 少发示例

Examples help calibrate the judge's decision boundary. We include labeled outputs that clearly Pass and clearly Fail . These are best drawn from human-labeled traces, as discussed in "Data Splits for Designing and Validating LLM-as-Judge".[6]

举例有助于校准法官的裁决界限。我们包含明显 Pass 和明显 Fail 的标记输出。这些数据最好从人工标记的痕迹中提取，详见《设计与验证 LLM 作为评判的数据分割》。[6]

4. Structured output format

4. 结构化输出格式

The judge responds in a consistent, machine-readable format, typically a JSON object with two fields: reasoning (one- to two-sentence explanation) and answer ("Pass" or "Fail"). This structure improves both accuracy and interpretability.

法官以一致且机器可读的形式回应，通常是带有两个字段的 JSON 对象：reasoning （一到两句解释）和 answer （"通过"或"不通过"）。这种结构不仅提高了准确性，也提升了可解释性。

Example 5-1 shows a prompt template for evaluating tone appropriateness in a real estate CRM assistant from Example 3-1.[7] Each LLM-as-judge evaluator follows this

structure but is tailored to its specific failure mode.

示例 5-1 展示了一个用于评估房地产 CRM 助手语气适宜性的提示模板，来自示例 3-1。[7] 每个作为法官的 LLM 评估者遵循此结构，但针对其特定的失败模式进行定制。

Example 5-1. LLM-as-judge prompt template for tone appropriateness

例子 5-1。LLM 作为裁判提示模板，用于语气适宜性

```
You are an expert evaluator assessing outputs from a real estate
assistant chatbot.

Your Task: Determine if the assistant-generated email to a client
uses a tone appropriate for the specified client persona.

Evaluation Criterion: Tone Appropriateness

Definition of Pass/Fail:
- Fail: The email's tone, language, or level of formality is
  inconsistent with or unsuitable for the described client persona.
- Pass: The email's tone, language, and formality align well with
  the client persona's expectations.

Client Personas Overview:
- Luxury Buyers: Expect polished, highly professional, and
  deferential language. Avoid slang or excessive casualness.
- First-Time Homebuyers: Benefit from a friendly, reassuring, and
  patient tone. Avoid overly complex jargon.
- Investors: Prefer concise, data-driven, and direct communication.
  Avoid effusiveness.

Output Format:
Return your evaluation as a JSON object with two keys:
1. "reasoning": A brief explanation (1-2 sentences) for your
   decision.
2. "answer": Either "Pass" or "Fail".

Examples:

---

Input 1:
Client Persona: Luxury Buyer
Generated Email: "Hey there! Got some truly awesome listings for
you in the high-end district. Super views, totally posh. Wanna
check 'em out ASAP?"
```

Evaluation 1:
{
  "reasoning": "The email uses excessive slang ('Hey there',
    'awesome', 'totally posh', 'ASAP') and an overly casual tone,
    which is unsuitable for a Luxury Buyer persona.",
  "answer": "Fail"
}

Input 2:
Client Persona: First-Time Homebuyer
Generated Email: "Good morning! I've found a few properties that seem like a great fit for getting started in the market, keeping your budget in mind. They offer good value and are in nice, welcoming neighborhoods. Would you be interested in learning more or perhaps scheduling a visit?"

Evaluation 2:
{
  "reasoning": "The email adopts a friendly, reassuring tone
    ('great fit for getting started', 'nice, welcoming
    neighborhoods') suitable for a First-Time Homebuyer, and
    clearly offers next steps.",
  "answer": "Pass"
}

Now, evaluate the following:

Client Persona: {{CLIENT_PERSONA_HERE}}
Generated Email: {{GENERATED_EMAIL_HERE}}

Your JSON Evaluation:

Data Splits for Designing and Validating LLM-as-Judge

设计与验证 LLM 作为法官的数据拆分

Designing an LLM-as-judge closely resembles training a classifier, except the "training" happens through prompt engineering, not parameter tuning. Instead of learning from data via gradient descent, we manually select examples and write instructions that guide the model's behavior in context. To ensure the resulting evaluator generalizes and doesn't overfit, we divide our labeled traces (from Chapters 3 and 4) into three disjoint sets:[8]

设计 LLM 作为评判非常类似于训练分类器，只不过"训练"是通过即时工程实现的，而非参数调优。我们不是通过梯度下降从数据中学习，而是手动选择示例并编写指导模型在上下文中行为的指令。为了确保得到的求值器能够推广且不过度拟合，我们将带标签的迹（来自第 3 章和第 4 章）分为三个不相交的集合：[8]

Training set 训练集

A pool of labeled examples we may draw from when constructing the prompt. These examples are candidates for few-shot demonstrations, typically clear-cut Pass and Fail cases that illustrate the boundaries of the failure mode.[9]

一组带标签的示例池，我们可以在构建提示时参考。这些例子适合进行少量样本演示，通常是明显的 Pass 和 Fail 案例，展示了失效模式的边界。[9]

Development (dev) set 开发（开发）集

A separate, larger set of labeled traces used to refine the prompt. After each edit (changing instructions, rewording criteria, or swapping few-shot examples), we evaluate judge outputs on the dev set by comparing them to human labels. *Dev set examples must never appear in the prompt itself.* This ensures we can measure how well the judge generalizes beyond the training examples.

一组独立且较大的标记痕迹用于细化提示。每次编辑（更改指令、重写标准或交换少数示例）后，我们通过将裁判输出与人工标签进行比较来评估开发者集的输出。开发者集的示例绝不能出现在提示词本身。这确保我们能够衡量评委在训练示例之外的概括能力。

Test set 测试集

A held-out set we use only to compute the alignment of the LLM judge, *after* finalizing the LLM judge prompt. We never look at this set during judge prompt development. It gives us an unbiased estimate of the judge's real-world accuracy (metrics like true positive rate and true negative rate), which we later use

to adjust estimated success rates in production traces ("Estimating True Success Rates with Imperfect Judges").

我们只用一个保留集合来计算 LLM 评判者的对齐，前提是 LLM 评审提示词最终确定。我们在评审提示开发时从不看这套。它为我们提供了评委真实准确率的无偏估计（如真阳性率和真阴性率等指标），我们随后用以调整制作痕迹的估算成功率（"用不完美裁判估算真实成功率"）。

Reusing examples across splits (especially from dev or test in the prompt) causes data leakage and inflates accuracy estimates.[10] Unlike traditional supervised learning, we don't need large training sets. In-context learning typically saturates after a small number of well-chosen examples.[11] As a result, we allocate more data to evaluation. *A typical split might assign 10%–20% of labeled traces to the training set, and 40%–45% each to dev and test.*

在不同分割中重复使用示例（尤其是在提示中来自开发或测试）会导致数据泄露并夸大准确率估计。[10] 与传统的监督学习不同，我们不需要大规模的训练集。情境学习通常在少数精心挑选的例子后就达到饱和状态。[11] 因此，我们分配更多数据用于评估。典型的拆分可能将 10%–20%的标记痕迹分配给训练集，40%–45%分配给开发和测试。

To reliably estimate judge performance, both dev and test sets include enough Pass and Fail examples (ideally 30–50 of each). These proportions may not reflect real-world prevalence, especially when failures are rare. This imbalance is acceptable and deliberate, as we explain in "Estimating True Success Rates with Imperfect Judges".

为了可靠估计裁判表现，开发集和测试集都包含足够的 Pass 和 Fail 示例（理想情况下各 30–50 个）。这些比例可能并不反映现实中的患病率，尤其是在故障罕见的情况下。这种不平衡是可以接受且刻意为之，正如我们在《用不完美法官估算真实成功率》中所解释的那样。

Iterative Prompt Refinement for the LLM-as-Judge

作为评判的 LLM 迭代提示精炼

After splitting labeled data into training, development, and test sets, we enter the core of judge construction: iteratively refining the prompt to align the LLM's decisions with expert labels. This process mirrors the tuning loop of a classifier, but instead of adjusting parameters, we revise prompt text and examples. The loop proceeds as fol-

lows:

将标记数据拆分为训练、开发和测试集后，我们进入评判构建的核心：迭代优化提示，使 LLM 的决策与专家标签保持一致。这个过程类似于分类器的调谐循环，但我们不是调整参数，而是修改提示文本和示例。循环过程如下：

1. Write a baseline prompt.

1. 写一个基线提示。

Start with an initial prompt using the components in "Writing LLM-as-Judge Prompts", combining a task description with clear definitions and a structured output format, along with a few-shot example set drawn from the training set.

从"写 LLM 即评判提示"中的组件开始，结合任务描述、清晰定义和结构化输出格式，并附带从训练集中抽取的少量示例。

2. Evaluate on dev set.

2. 在开发者集上进行评估。

Run the LLM-as-judge over all examples in the development set. Compare each judgment (Pass or Fail) to the human-provided ground truth.

用 LLM 作为评判者来分析开发集中的所有示例。将每个判断（ Pass 或 Fail ）与人类提供的真实情况进行比较。

3. Measure agreement. 3. 度量协议。

Let *P* be the total number of dev examples labeled Pass , of which *p* were correctly judged Pass . Let *F* be the total number labeled Fail , of which *f* were correctly judged Fail . Then compute:

设 P 为标记为 Pass 的开发实例总数，其中 p 被正确判定为 Pass 。设 F 为标记为 Fail 的总数，其中 f 被正确判定为 Fail 。然后计算：

$$\mathrm{TPR} = \frac{p}{P} \quad (\text{true positive rate}) \mathrm{TNR} = \frac{f}{F} \quad (\text{true negative rate})$$

We refer to a positive as a Pass, and a negative as a Fail.

我们把正面称为通过，负面称为不及格。

NOTE 注意/备注

Why do we use TPR and TNR instead of precision and recall? Our end goal is to estimate the *true* pass rate of the application. A judge can only misestimate that pass rate in two ways: (1) by missing real Passes, or (2) by passing real Fails. TPR and TNR capture these two error modes directly: TPR is the fraction of actual Passes the judge correctly calls Pass, and TNR is the fraction of actual Fails it correctly calls Fail. Precision and recall for each class are not wrong to compute, but they don't align as cleanly with this goal.

为什么我们用 TPR 和 TNR 而不是精确和召回？我们的最终目标是估算申请的真实通过率。评委只能通过两种方式误估通过率：（1）错过真实的及格，或（2）通过真实的不及格。TPR 和 TNR 直接捕捉了这两种错误模式：TPR 是裁判正确判定“通过”的实际通过率比例，TNR 是正确判定失败的实际失败率。每个类别的精度和回忆率计算起来没错，但它们与目标的一致性不够清晰。

---

4. Inspect disagreements.

4. 审查分歧。

Review false passes (judge said Pass but human said Fail) and false fails (judge said Fail but human said Pass) to identify ambiguous criteria or missing edge cases.

复核错误通过（法官说通过，但人类说不通过）和假错误不合格（法官说不通过，但人类说通过），以识别模糊的标准或缺失的边缘案例。

5. Refine the prompt.

Based on dev set errors:

- Clarify task wording for Pass/Fail criteria.
- Swap in more illustrative few-shot examples from the training set.
- If the dev set has some failure modes that are not in the training set, add representative traces to the training set and consider using them in the prompt for the next round.

6. Repeat.

Reevaluate the revised prompt on the same dev set and recalculate metrics. Continue until performance stabilizes or improves acceptably.

When to Stop Refining

We stop when TPR and TNR reach satisfactory levels (e.g., >90%). Thresholds depend on application needs, since missing a real failure may be costlier than flagging a false one.[12]

While tools like DSPy (Khattab et al. 2024) can automate this loop by optimizing prompts over a dev set, we recommend manual iteration first. It builds intuition about both the failure mode and the judge's behavior.[13]

If Alignment Stalls

If the judge continues to perform poorly (e.g., low TPR and TNR), consider one of the following strategies. We can use a more capable LLM: a larger or newer model may resolve subtle or context-sensitive errors. We can also decompose the criterion, breaking a complex failure mode into smaller, more atomic checks, each with its own judge. Or you can improve your labeled data by adding more diverse, high-quality examples to the training set, especially for edge cases.[14]

NOTE 注意/备注

The tools in this chapter are enough to calibrate a judge against human labels on your own. The hard parts are stack-specific: which traces to surface, what to do when distributions drift, and how to keep the judge stable through model upgrades. In our live course, teams bring their own logs and work through these issues with us and with peers shipping similar systems.

Estimating True Success Rates with Imperfect Judges

After achieving consistent, high TPR and TNR on the dev set, we freeze the LLM-as-judge prompt and run it on the test set. This gives us two things. First, an estimated pass or success rate on the test set: the fraction of examples the judge labels as Pass. Second, estimates of the judge's TPR and TNR, computed by comparing its predictions to ground truth human labels.

NOTE 注意/备注

This section is more mathematical than earlier ones. We walk through it step by step and provide Python code to make it easy to apply, even without a background in statistics. Product managers can skip to the end of this section, or use the code directly.

But this only tells us how the judge behaves on the test split. In practice, we want to estimate how often a failure mode appears in a much larger (often unlabeled) dataset, such as new traces from a production agent. The problem is that our judge is imperfect. If we run it over thousands of new outputs and simply count the Pass predictions, we get a biased estimate of the true pass rate, because the judge occasionally misses failures or flags passes incorrectly.

但这只是告诉我们法官在测试分段时的表现。实际上，我们希望估算失效模式在更大（通常未标记）数据集中出现的频率，比如生产代理的新痕迹。问题是我们的法官并不完美。如果我们对成千上万个新输出进行计算，并简单统计通过预测，我们得到的真实通过率估计有偏差，因为评委偶尔会漏掉失败或错误标记通过。

This section shows how to:

本节展示了如何：

1. Use the judge's TPR and TNR to correct its raw predictions and estimate the true pass rate, that is, the fraction of examples a human would consider passing the evaluation criterion.

   利用评委的 TPR 和 TNR 来修正其原始预测，并估算真实通过率，即人类认为通过该评估标准的例子比例。

2. Quantify the uncertainty around this estimate by constructing a 95% confidence interval.[15] If the lower bound is high, we can trust that the LLM agent is performing acceptably. If the interval is wide, we may need to improve the TPR and TNR of the judge.

   通过构建 95%置信区间来量化该估计值的不确定性。[15] 如果下界很高，我们可以相信 LLM 代理的表现还算可接受。如果间隔较长，我们可能需要改进法官的 TPR 和 TNR。

We now present a procedure (Figure 5-2) to estimate the true pass or success rate of an evaluation metric over new, unlabeled traces.[16]

我们现在提出一种方法（图 5-2），用于估算评估指标在新无标记痕迹上的真实通过率或成功率。[16]

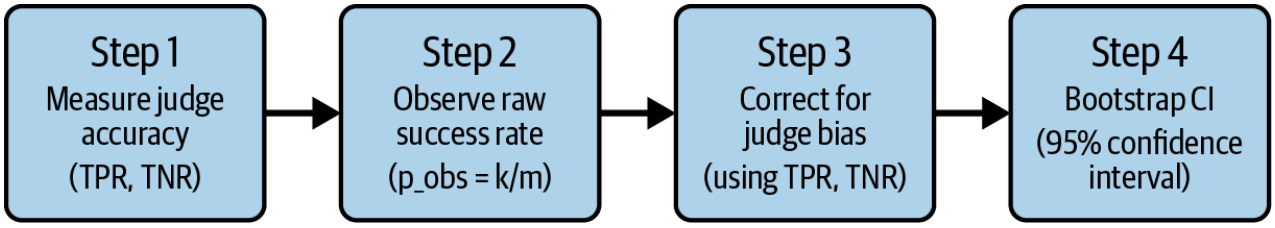


Figure 5-2. Workflow for estimating the true success rate with bias correction and confidence intervals

图 5-2。用于估算真实成功率的工作流程，包含偏差修正和置信区间

Step 1: Measure Judge Accuracy

第一步：衡量法官的准确性

On our held-out test set, we compare judge predictions to human labels and compute TPR and TNR as defined in "Iterative Prompt Refinement for the LLM-as-Judge".

在我们保留的测试集中，我们将评判预测与人工标签进行比较，并计算《作为 LLM 作为评判的迭代提示细化》中定义的 TPR 和 TNR。

Step 2: Observe Raw Success Rate

第二步：观察原始成功率

We run the judge on *m* new, unlabeled traces and let *k* be the number it labels Pass. The raw success rate is:

我们对 m 条新的、未标记的痕迹运行法官，设 k 为它标记的数字，Pass。原始成功率为：

$$p_{\text{obs}} = \frac{k}{m}$$

Step 3: Correct the Observed Success Rate

步骤 3：纠正观察到的成功率

Because the judge is imperfect, p_{obs} is biased. We adjust it to estimate the true success rate (Rogan and Gladen 1978):

因为法官不完美，p_{obs} 有偏见。我们调整以估算真实成功率（Rogan 和 Gladen 1978）：

$$\hat{\theta} = \frac{p_{\text{obs}} + \text{TNR} - 1}{\text{TPR} + \text{TNR} - 1} \quad \text{(clipped to } [0, 1]\text{)}$$

Two edge cases can arise here, and they require different handling. First, if TPR + TNR – 1 is near zero (e.g., both around 50%), the correction blows up and the judge is no

better than random chance; in this case, the correction is unreliable and should not be used. Second, even with a good judge, numerical noise can push $\hat{\theta}$ slightly below 0 or above 1; clipping the result to [0, 1] handles this case. The bootstrap code in "Python Code for Estimating Success Rates" uses a threshold of 0.05 for the denominator and clips the final estimate to [0, 1].

这里可能出现两种极端情况，它们需要不同的处理方式。首先，如果 TPR + TNR – 1 接近零（例如两者都约为 50%），修正结果会被推翻，裁判不过是随机随机;在这种情况下，校正不可靠，不应使用。其次，即使是有好判断力，数值噪声也可能将 $\hat{\theta}$ 略低于 0 或超过 1;将结果裁剪为[0， 1]即可处理此情况。"Python Code for Estimating Success Rates"中的自助代码使用分母阈值 0.05，并将最终估计值剪短为[0， 1]。

Table 5-4 summarizes how different TPR/TNR combinations affect the corrected estimate.

表 5-4 总结了不同 TPR/TNR 组合如何影响修正后的估计。

Table 5-4. Effect of TPR and TNR on the corrected estimate

表 5-4。TPR 和 TNR 对修正估算的影响

TPR	TNR	Effect on corrected estimate 对修正估计的影响
High (>0.85) 高（>0.85）	High (>0.85) 高（>0.85）	Accurate estimate with a tight confidence interval. 准确估计，置信区间较窄。
High (>0.85) 高（>0.85）	Low (<0.7) 低血糖（<0.7）	Estimate shifts downward (false positives inflate observed success count); confidence interval widens. 估计值下降（假阳性会夸大观察到的成功计数）；置信区间扩大。
Low (<0.7) 低血糖（<0.7）	High (>0.85) 高（>0.85）	Estimate shifts upward (false negatives inflate observed failure count); confidence interval widens. 估计值向上移动（假阴性使观察到的失效计数膨胀）;置信区间扩大。
Low (<0.7) 低血糖（<0.7）	Low (<0.7) 低血糖（<0.7）	Estimate is unreliable; confidence interval is very wide. Improve the judge before drawing conclusions. 估算不可靠;置信区间非常宽。在下结论前先提升法官的意见。

Step 4: Quantify Uncertainty with a Bootstrap

第四步：用自助工具量化不确定性

Our corrected estimate $\hat{\theta}$ comes from a single test set, and if we happened to draw a different test set we would get a slightly different number. To measure how much this number could vary, we use a statistical technique called a *bootstrap*. The idea of a bootstrap is to repeatedly resample our existing test set (drawing examples with replacement, so the same example can appear multiple times in a resample) and recompute the estimate on each resampled set. The spread of those estimates tells us how

confident we can be in the one we computed from the original data.

我们修正后的估计值 $\widehat{\theta}$ 来自单一测试集，如果我们抽取了不同的测试集，得到的数值会略有不同。为了衡量这个数字可能的变化幅度，我们使用一种称为自助法的统计技术。自助法的理念是反复对现有测试集进行重采样（用替换画出样本，这样同一样本可以多次出现在重采样中），并对每个重采样集重新计算估计值。这些估计的分布告诉我们对原始数据计算出的估计值的信心。

Concretely, each bootstrap iteration samples (with replacement) the full set of (human label, judge prediction) pairs, recomputes TPR and TNR on that sample, applies the correction to obtain a new $\widehat{\theta}^*$, and records it. After *B* iterations, the 2.5th and 97.5th percentiles of these $\{\widehat{\theta}^*\}$ values form our 95% confidence interval.

All together, the following sidebar gives the full procedure for estimating the true success rate for an evaluation metric, along with a 95% confidence interval.

ESTIMATING SUCCESS RATES WITH LLM-AS-JUDGE

Here we use an LLM-as-judge to estimate the success rate for some evaluation metric:

Inputs 输入内容

- Test set of *N* examples $\{y_i, \hat{y}_i\}$ where $y_i \in \{\text{Pass}, \text{Fail}\}$ is the human label and $\hat{y}_i$ is the judge prediction
- Unlabeled batch: *m* traces with *k* judged Pass
- Number of bootstraps *B*

Outputs 输出结果

Point estimate $\widehat{\theta}$ and 95% confidence interval $[L, U]$ for true success rate

Procedure

1. Judge accuracy on test set

   TPR = (Number of actual Pass traces correctly predicted as Pass) / (Total number of actual Pass traces)

   TNR = (Number of actual Fail traces correctly predicted as Fail) / (Total number of actual Fail traces)

2. Raw observed success rate

   $$p_{\text{obs}} = \frac{k}{m}$$

3. Correct judge bias

   $$\widehat{\theta} = \frac{(p_{\text{obs}} + \text{TNR} - 1)}{(\text{TPR} + \text{TNR} - 1)}$$

   $$\widehat{\theta} = \text{clip}(\widehat{\theta}, 0, 1)$$

4. Bootstrap confidence interval

   自助置信区间

   - For $b = 1$ to B:

     对于 $b = 1$ 到 B：

     - Draw a bootstrap sample of size N from the test set (with replacement)

       从测试集中抽取大小为 N 的引导样本（带替换）

     - Recompute TPR^* and TNR^* on that sample

       对该样本重新计算 TPR^* 和 TNR^*

- $\widehat{\theta}^* = \frac{(p_{obs} + TNR^* - 1)}{(TPR^* + TNR^* - 1)}$

- $\widehat{\theta}^* = \text{clip}(\theta^*, 0, 1)$

- Sort all $\widehat{\theta}^*$ values 排序所有 $\widehat{\theta}^*$ 值

- L = 2.5th percentile L = 第 2.5 百分位

- U = 97.5th percentile U = 第 97.5 百分位

- Return $\widehat{\theta}, L, U$ 返回 $\widehat{\theta}, L, U$

Python Code for Estimating Success Rates

用于估算成功率的 Python 代码

Here we provide a Python code snippet that computes a point estimate of the LLM agent's success rate $\widehat{\theta}$, as well as a 95% confidence interval for the true success rate:

这里我们提供了一个 Python 代码片段，计算 LLM 代理成功率 $\widehat{\theta}$ 的点估计，以及真实成功率的 95%置信区间：

```
import numpy as np

def estimate_success_rate(
    test_labels,
    test_preds,
    unlabeled_preds,
    B=20000
):
    """
    Args:
      test_labels: array-like of 0/1, human labels on test set
                   (1 = Pass).
      test_preds: array-like of 0/1, judge predictions on test set
                  (1 = Pass).
      unlabeled_preds: array-like of 0/1, judge predictions on
                       unlabeled data (1 = Pass).
      B: number of bootstrap iterations.

    Returns:
      theta_hat: point estimate of true success rate.
      L, U: lower and upper bounds of a 95% bootstrap CI.
```

```
    """
    test_labels     = np.asarray(test_labels, dtype=int)
    test_preds      = np.asarray(test_preds, dtype=int)
    unlabeled_preds = np.asarray(unlabeled_preds, dtype=int)

    # Step 1: Judge accuracy on test set
    P = test_labels.sum()
    F = len(test_labels) - P
    TPR = ((test_labels == 1) & (test_preds == 1)).sum() / P
    TNR = ((test_labels == 0) & (test_preds == 0)).sum() / F

    # Step 2: Raw observed success rate
    p_obs = unlabeled_preds.sum() / len(unlabeled_preds)

    # Step 3: Correct estimate
    denom = TPR + TNR - 1
    if denom < 0.05:
        raise ValueError("Judge accuracy too low for correction")
    theta_hat = (p_obs + TNR - 1) / denom
    theta_hat = np.clip(theta_hat, 0, 1)

    # Step 4: Bootstrap CI
    N = len(test_labels)
    idx = np.arange(N)
    samples = []
    for _ in range(B):
        boot_idx = np.random.choice(idx, size=N, replace=True)
        lbl_boot = test_labels[boot_idx]
        pred_boot = test_preds[boot_idx]
        P_boot = lbl_boot.sum()
        F_boot = N - P_boot
        if P_boot == 0 or F_boot == 0:
            continue
        TPR_star = (
            (lbl_boot == 1) & (pred_boot == 1)
        ).sum() / P_boot
        TNR_star = (
            (lbl_boot == 0) & (pred_boot == 0)
        ).sum() / F_boot
        denom_star = TPR_star + TNR_star - 1  # Rogan-Gladen denominator
        if denom_star < 0.05:
            continue  # denominator near zero makes the estimate unreliable
        theta_star = (p_obs + TNR_star - 1) / denom_star
        samples.append(np.clip(theta_star, 0, 1))

    if not samples:
```

```
        raise RuntimeError(
            "No valid bootstrap samples; check inputs"
        )

    L, U = np.percentile(samples, [2.5, 97.5])
    return theta_hat, L, U
```

NOTE 注意/备注

We have open sourced judgy, a small Python library (it only depends on numpy), that contains the preceding Python function.

我们有开源的 judgy ，一个小型 Python 库（它只依赖 numpy），包含了前面提到的 Python 函数。

To build intuition for how the success rate estimate varies with the LLM-as-judge's error rates, we present a small, illustrative thought experiment. *These plots are not meant to model a real-world data collection process exactly, but to help readers reason about how success rate estimates are corrected and how judge imperfections impact uncertainty.* For teaching purposes, we assume a fixed number of human-labeled examples (50 successes and 50 failures) and a "ground truth" success rate of 80% on a larger unlabeled set (note that in practice, this 80% number is unknown).

为了直观地理解成功率估计如何随作为评判的 LLM 的错误率变化，我们提出一个小型的说明性思维实验。这些图表并非旨在精确模拟真实世界的数据收集过程，而是帮助读者理解成功率估计如何被修正，以及判断缺陷如何影响不确定性。出于教学目的，我们假设有固定数量的人工标记示例（50 个成功和 50 个失败），以及在更大未标记数据集上的"真实"成功率为 80%（注意，实际中这个 80%的数字未知）。

In practice, our estimates of TPR and TNR are themselves empirical and become more reliable as we add more labeled data. To isolate the effects, we hold one error rate constant while varying the other.

实际上，我们对 TPR 和 TNR 的估计本身是实证的，随着添加更多标注数据，这些估算会变得更加可靠。为了区分这些效应，我们保持一个错误率恒定，同时改变另一个。

When the judge misses many real successes (left of Figure 5-3), the raw success rate (the lower series of dots) starts far below the true 80% and the uncertainty (shaded) is huge. As it learns to catch more successes (moving right), the raw rate rises toward

80% and the interval narrows, while the corrected line stays fixed on 80%.

当评委错过许多实际成功案例（见图 5-3 左侧）时，原始成功率（下方的点数）远低于真实 80%，不确定性（阴影）非常大。随着它学会捕捉更多成功（向右移动），原始率会上升到 80%，区间缩小，而修正后的线仍固定在 80%。

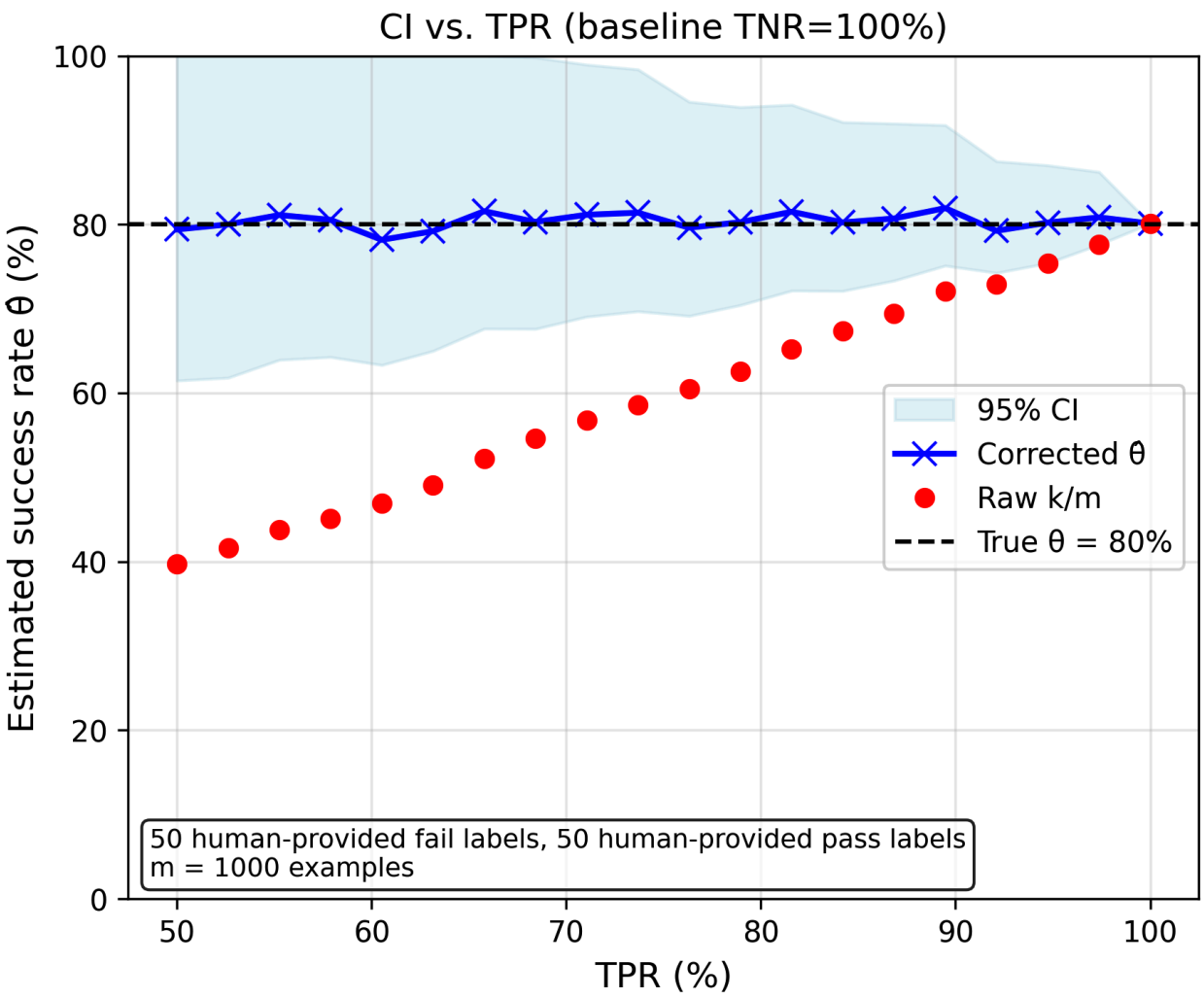


Figure 5-3. How the judge's true positive rate biases the raw pass rate, and how bias correction recovers the true 80%

NOTE

These plots assume fixed TPR/TNR for illustration. In practice, these rates are estimated from our data and their reliability improves as we label more examples.

When the judge classifies many real failures as successes (left of Figure 5-4), the raw success rate sits above the true 80% with a bit of uncertainty. As it improves at rejecting failures (moving right), the raw rate falls back to 80% and the interval shrinks.

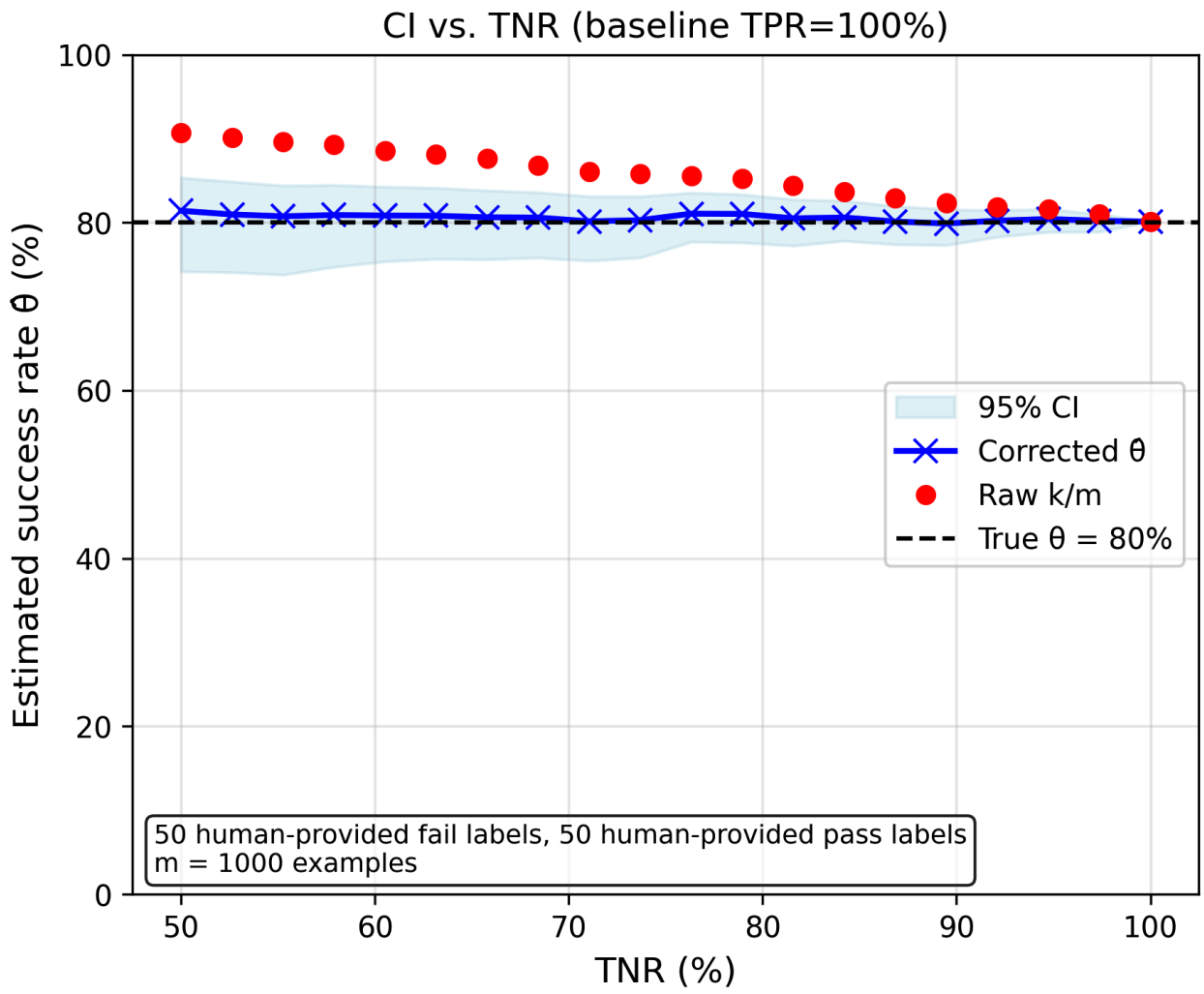


Figure 5-4. How the judge's true negative rate biases the raw pass rate: the raw estimate sits above the true 80% while the judge still accepts real failures, and falls back toward it as the judge improves

---

NOTE 注意/备注

LLM-as-judge alignment priorities include the following:

- The bias correction reliably removes bias in both extreme error modes.
- Judge errors mainly inflate uncertainty (wider confidence intervals), rather than shifting the corrected estimate.
- Improving TPR (the judge's ability to identify true successes) narrows the confidence interval the most.

Improving the TPR (the judge's ability to correctly identify true successes) tends to narrow the confidence interval for our estimated true success rate ($\hat{\theta}$) the most. When TNR = 1, the correction formula simplifies to $\hat{\theta} = \frac{p_{\text{obs}}}{\text{TPR}}$. If TPR is small (i.e., judge misses many successes), this division "boosts" the underestimated p_{obs} by a correspondingly large factor. When TPR is small, even minor absolute variations in its bootstrapped estimate can lead to large relative changes in $\frac{1}{\text{TPR}^*}$, and thus to large fluctuations in the corrected $\hat{\theta}^*$. This increased variance naturally results in a wider confidence interval, reflecting greater uncertainty.

USING CONFIDENCE INTERVALS TO MAKE SHIPPING DECISIONS

Suppose your corrected success rate estimate is $\widehat{\theta} = 0.87$, with a 95% confidence interval of [0.81, 0.93]. Your team has set a minimum acceptable success rate of 0.85 for this feature.

The point estimate (0.87) exceeds the threshold, but the lower bound of the interval (0.81) falls below it. In practice, you'd decide based on the cost of being wrong. If shipping a feature with an 81% success rate would cause serious user harm, the interval is too wide to ship confidently; you'd collect more test data or improve the judge to tighten the interval. If the feature is low-stakes and 81% is still acceptable, you might ship and monitor.

A narrow interval like [0.85, 0.89] gives you much more confidence in the decision, because even the worst case still meets the threshold. The width of the interval, not just the point estimate, determines whether you have enough evidence to act.

Optional: Group-wise Metrics for Evaluating Multiple Outputs

LLM agents often generate multiple candidates per input, whether it's multiple completions, retrieved documents, or answer options. In these cases, we can use metrics that evaluate the quality of the *group* of outputs, not just each one in isolation.

LLM 代理通常每个输入生成多个候选人，无论是多个补全、检索的文档还是答案选项。在这些情况下，我们可以使用评估产出组质量的指标，而不仅仅是单独评估每个输出。

One popular metric is *Success@k* (also called *Pass@k* in code generation).[17] Formally, Pass@*k* is the fraction of evaluation problems for which at least one of the top *k* outputs is correct (e.g., passes all unit tests) (Jain et al. 2025; Chen et al. 2021; Kulal et al. 2019). For example, if a code generator produces five candidate completions per problem, Pass@5 is the fraction of problems for which at least one candidate passes all the tests.

一个流行的指标是 Success@k（在代码生成中也称为 Pass@k）。[17] 形式上，Pass@k 是评估问题中，至少有前 k 个输出正确（例如通过所有单元测试）的比例（Jain 等，2025;Chen 等，2021;Kulal 等，2019）。例如，如果一个代码生成器每个问题产生五个候选补全，Pass@5 是至少有一个候选问题通过所有测试的比例。

Precision@k measures how many of the top *k* outputs are correct. It's the fraction of relevant items in the top *k*. For example, if an LLM proposes 5 API calls and 3 are valid, Precision@5 = 0.6. This tells us how good the outputs near the top are.

Precision@k 衡量前 k 个输出中有多少是正确的。它是前千项相关物品的比例。例如，如果一个 LLM 提出 5 个 API 调用且 3 个有效，Precision@5 = 0.6。这告诉我们顶部输出的质量如何。

Recall@k measures coverage: how many of the total relevant items are found in the top *k*. If 3 out of 10 correct answers appear in the top 3 suggestions, Recall@3 = 0.3. High precision with low recall means the system is right when it guesses, but misses a lot.

Recall@k 衡量覆盖范围：前 K 中总相关项目中有多少。如果前三个选项中有 10 个答案中有 3 个正确，Recall@3 = 0.3。高精度低召回率意味着系统在猜测时是对的，但会错过很多。

We can also move past binary judgments and measure how *semantically close* the outputs are to the desired response. To do this, we use *embeddings* (vector representations of text). We can compute cosine similarity between the embedding of each candidate output and a reference (like the input or an ideal answer). This gives a score that reflects how well the meaning matches, even if the wording differs.

我们也可以超越二元判断，测量输出与期望反应的语义接近程度。为此，我们使用嵌入（文本的向量表示）。我们可以计算每个候选输出嵌入与参考（如输入或理想答案）之间的余弦相似度。这给出的分数反映了含义的匹配程度，即使措辞不同。

We may also care about *diversity*. Especially in generative settings, similar outputs are undesirable. For example, a fashion recommender avoids suggesting nearly identical outfits every day. One common diversity metric is *average pairwise semantic similarity*.

我们也可能关心多样性。尤其是在生成式环境中，类似的输出是不理想的。例如，时尚推荐者避免每天推荐几乎相同的穿搭。一个常见的多样性指标是平均的两对语义相似度。

Let $O = \{o_1, \ldots, o_N\}$ be the outputs, and e_i the embedding for o_i. Define a set *P* of index pairs to compare. Then, where $\cos_{sim}(u, v) = (u \cdot v)/(\|u\|\|v\|)$:

设 $O = \{o_1, \ldots, o_N\}$ 为输出， e_i 为 o_i 的嵌入。定义一组索引对 P，用于比较。那么，其中 $\cos_{sim}(u, v) = (u \cdot v)/(\|u\|\|v\|)$：

$$\text{AvgSimilarity} = \frac{1}{|P|} \sum_{(i,j) \in P} \cos_{sim}(\mathbf{e}_i, \mathbf{e}_j)$$

A lower average similarity means higher diversity. But the choice of which pairs (*i*, *j*) to include in *P* matters. Here's a simple implementation in code:

较低的平均相似度意味着多样性更高。但选择包含哪些对（，）在 P 中很重要。这里有一个简单的代码实现：

```
from sklearn.metrics.pairwise import cosine_similarity
import numpy as np

# Example: 5 embeddings, each 384-dim
embeddings = np.random.rand(5, 384)

# Pairwise similarity matrix
sim_matrix = cosine_similarity(embeddings)

# Average of unique pairs (exclude diagonal)
triu_indices = np.triu_indices_from(sim_matrix, k=1)
avg_similarity = sim_matrix[triu_indices].mean()

print(f"Average pairwise similarity: {avg_similarity:.4f}")
```

For example, in a fashion stylist application, we might only care about avoiding similarity between today's suggestion (o_i) and those from the past week (o_j where *j* indexes recent days). In that case, *P* would include pairs (*i*, *j*) where *j* ranges over the past seven outputs. This results in roughly seven comparisons per day, targeting short-term repetition. Averaging over all pairs would dilute this focus and miss near-duplicate suggestions.

例如，在时尚造型师应用中，我们可能只关心今天的建议（o_i）与上周的建议（o_j，索引最近几天）之间的相似性。在这种情况下，P 会包含对（，），其中 范围在过去七个输出中。这意味着每天大约有七次比较，目标是短期重复。对所有配对的平均会稀释这种关注点，并错过近似重复的建议。

In summary, when evaluating groups of outputs, use metrics like Success@*k*, Precision@*k*, and Recall@*k* for correctness. Use semantic similarity to capture relevance, and average pairwise similarity to capture diversity. Define what matters for your task and choose metrics accordingly.

总之，在评估输出组时，使用 Success@k、Precision@k 和 Recall@k 等指标来判断正确性。使用语义相似度捕捉相关性，使用平均两两相似度捕捉多样性。定义对你的任务重要的事情，并据此选择指标。

Common Pitfalls 常见陷阱

The most common mistake when implementing LLM-as-judge evaluators is *omitting examples from the prompt entirely.* Without concrete examples, the model lacks grounding in what constitutes a failure for the task at hand. This often leads to vague or inconsistent behavior, even when the task definition is otherwise clear.

在实施 LLM 作为评审评估者时，最常见的错误是完全省略了提示中的示例。没有具体例子，模型缺乏对何谓任务失败的基础。这常常导致行为模糊或不一致，即使任务定义本身很明确。

A second pitfall is attempting to do too much in a single prompt. Some teams try to evaluate multiple criteria at once (tone, content correctness, next steps) in a single Pass/Fail decision. This introduces ambiguity and makes it harder to diagnose errors. Breaking complex metrics into narrower, more specific metrics and prompts yields better alignment and more reliable judges.

第二个陷阱是试图在一个提示中做太多事情。有些团队试图在单一通过/不通过决策中同时评估多个标准（语气、内容正确性、下一步）。这会带来歧义，也使得诊断错误变得更加困难。将复杂的指标拆分为更狭窄、更具体的指标和提示，可以带来更好的对齐和更可靠的评判。

Another common issue is skipping the alignment step altogether. Teams often assume the LLM-as-judge will “just work” out of the box. Sometimes that is true, especially for simple or broadly familiar metrics like sentiment polarity (i.e., positive or negative). But many evaluation metrics are domain-specific or tied to the unique “vibe” of a product. Examples include tone alignment for luxury clients, completeness of a listing summary, and justification of a tool call. These require effort to align. Judges don’t come pretrained on a product’s values, so we have to teach them. Prompt refinement and human-labeled validation are essential to ensure our evaluators actually reflect what matters in our agents. Aligning a judge (by refining prompts and validating its agreement with human labels) is an investment that pays off enormously. An aligned judge can replace expensive human evaluation across thousands of traces, making agent-

wide monitoring feasible.

另一个常见问题是完全跳过校准步骤。团队常常认为 LLM 作为评委会“随便”发挥作用。有时候确实如此，尤其是对于简单或广泛熟悉的指标，比如情感极性（即正面或负面）。但许多评估指标是领域特定的，或者与产品独特的“氛围”相关。例如，为豪华客户调整语气对齐、房源摘要的完整性，以及工具调用的合理性。这些需要努力才能对齐。评委们不会预先接受产品价值观的培训，所以我们必须教他们。及时的改进和人工标识的验证对于确保评估者真正反映代理中重要的品质至关重要。通过完善提示并验证其与人类标签的一致性来协调评委，是一项回报丰厚的投资。一个对齐的法官可以替代数千条痕迹中昂贵的人工评估，使整个代理的监控成为可能。

A related pitfall is *data leakage* between the judge prompt and the evaluation set. Data leakage happens when teams include traces as few-shot examples in the judge prompt and also use those same traces in the evaluation set to measure alignment. The metrics become contaminated: TPR and TNR may appear high, not because the judge is generalizing well, but because it has seen the specific examples before. Any trace used in the prompt must be excluded from the evaluation set.

相关的陷阱是裁判提示与评估集之间的数据泄露。当团队在评判提示中包含少量样本的痕迹，并在评估集中使用这些痕迹来衡量对齐时，就会发生数据泄漏。这些指标被污染了：TPR 和 TNR 看起来很高，不是因为法官在概括得当，而是因为他们以前见过具体例子。提示中使用的任何痕迹必须从评估集中排除。

Moreover, even after alignment, many teams fail to revisit the process. Production data can drift. New failure modes may emerge, LLM updates may shift behavior (L. Chen et al. 2024), and evaluation metrics may evolve (Shankar et al. 2024d). We recommend rerunning the alignment process regularly (e.g., weekly): continue labeling a handful of traces, recomputing TPR and TNR, and checking whether confidence intervals remain acceptably tight. If not, realign the judge. This is not a one-off task. It's part of the ongoing lifecycle of LLM-powered applications, much like monitoring in MLOps (Sculley 2015; Shankar et al. 2024c). When observing new failure modes in production, we can add examples of them to all three splits (training, dev, and test).

此外，即使实现了对齐，许多团队仍未能重新审视这一过程。生产数据可能会漂移。新的失效模式可能出现，LLM 更新可能改变行为（L. Chen 等，2024），评估指标也可能演变（Shankar 等，2024d）。我们建议定期（例如每周）重新运行比对过程：继续标记少量痕迹，重新计算 TPR 和 TNR，并检查置信区间是否保持可接受的紧密。如果没有，就重新调整法官的意见。这不是一次性的任务。它是基于 LLM 应用的持续生命周期的一部分，类似于 MLOps 中的监控（Sculley 2015;Shankar 等，2024c）。在生产环境中观察新的失败模式时，我们可以在三种分段（训练、开发和测试）中添加这些示例。

One more subtlety: LLM-as-judge evaluators are themselves stochastic. Running the same judge on the same trace twice can produce different labels, especially at higher temperature settings. To check for this, run your judge three to five times on the same trace with temperature > 0 and see whether the reasoning varies while the final label stays stable. If the label flips across runs, the judge is uncertain about the trace, and the criterion's definition in the rubric may need more detail or more examples to remove the ambiguity.

还有一点微妙之处：作为评判的 LLM 评估者本身也是随机的。在同一条线路上用同一个评判机两次，尤其是在高温设置下，可能会产生不同的标签。为了检查这一点，可以在温度 >0 的同一条线上用评判机运行三到五次，看看推理是否会变化，而最终标签保持稳定。如果标签在跑数间反转，评委对该线路的定义不确定，评分标准在评分标准中可能需要更多细节或更多例子来消除歧义。

Putting It Together 整合起来

At the end of this chapter, you should have automated evaluators that can score traces without human review. The practical workflow is:

本章结束时，你应该会有自动化评估员，可以在没有人工审核的情况下评分痕迹。实际工作流程如下：

1. For each failure mode from your taxonomy, define a metric that can be computed automatically.

   对于分类法中的每个失效模式，定义一个可自动计算的指标。

2. Use code-based checks where the criterion is precise enough (e.g., SQL AST matching, regex for required fields).

   使用基于代码的检查，条件足够精确（例如，SQL AST 匹配，要求字段的正则表达式）。

3. Use LLM-as-judge evaluators for criteria that require judgment (e.g., tone, completeness, relevance).

   对于需要判断的标准（如语气、完整性、相关性）使用 LLM 作为评审者的评估者。

4. Validate every judge against human-labeled ground truth using TPR/TNR on a held-out test set.

   在保留的测试集中使用 TPR/TNR 验证每位法官是否符合人类标记的真实信息。

5. Use the Rogan-Gladen correction to estimate the true success rate, and bootstrap confidence intervals to quantify your uncertainty.

使用 Rogan-Gladen 修正来估算真实成功率，并用自助置信区间来量化你的不确定性。

Automated evaluators need ongoing maintenance.[18] Production data drifts, failure modes evolve, and model updates can shift judge behavior. Rerun the alignment process regularly by labeling fresh traces and recomputing TPR/TNR.

自动评估器需要持续维护。[18] 生产数据漂移、故障模式演变，模型更新可能改变评判行为。定期通过标记新鲜痕迹并重新计算 TPR/TNR 来重新运行比对过程。

The next three chapters apply these evaluators to specific application types: multi-turn conversations (Chapter 6), retrieval (Chapter 7), and tool use (Chapter 8).

接下来的三章将这些评估器应用于具体应用类型：多回合对话（第 6 章）、检索（第 7 章）和工具使用（第 8 章）。

1 *The LLM Evaluation Guidebook* is a nice online resource for automated evaluators and corresponding Jupyter notebooks (Fourrier 2024).

1 LLM 评估指南是一个很好的在线资源，专门针对自动评估器和对应的 Jupyter 笔记本（Fourrier 2024）。

2 Teams can rush to build evals for preferences that they never specified in the prompt (like concise responses or a specific structure). A better approach is to first include such instructions and only create an evaluator if the LLM still fails to follow them.

2 团队可以急于构建那些他们从未在提示中指定的偏好（如简明回答或特定结构）的评估。更好的方法是先包含此类指令，只有在 LLM 仍然无法执行时才创建评估器。

3 Another large class of things to "just fix" when doing evals is tool specification errors, where the fix is glaringly obvious. Suppose the customer wants to reschedule an appointment but no proper tool is exposed to the LLM for rescheduling, only for scheduling. The fix might be as simple as adding a rescheduling tool.

3 在做评估时，另一类需要"直接修正"的问题是工具规格错误，修正问题非常明显。假设客户想重新安排预约，但没有合适的工具暴露给 LLM 进行重新安排，只有用于安排。解决办法可能就是添加一个重新调度工具。

4 We detail methods for automating such nuanced judgments, including the use of other LLMs as evaluators, in "Implementing Metrics (How to Measure)".

4 我们在《实现指标（如何衡量）》中详细介绍了自动化此类细致判断的方法，包括使用其他大型语言模型作为评估者。

[5] Restricting evaluation tasks to be binary helps us make alignment more tractable, especially because we are not fine-tuning the LLM judges.

[5] 将评估任务限制为二元对立有助于我们更易于对齐，尤其是因为我们没有对 LLM 评判进行微调。

[6] While we primarily focus on binary (Pass/Fail) judgments for clarity and alignment, if using finer-grained scales (e.g., 1–3 severity), include examples for *every* point on the scale.

[6] 虽然我们主要关注二元（通过/不通过）判断以保证清晰和对齐，但如果使用更细致的量表（例如 1–3 严重程度），应为量表上的每个点提供示例。

[7] We pick a failure mode and examples that are easy to interpret for readers, but in practice, your criteria might be more complex and your few-shot examples might be "borderline" or difficult cases to evaluate.

[7] 我们选择一个失败模式和易于读者解读的例子，但实际上，你的标准可能更复杂，少数样本的例子可能是"边缘"或难以评估的案例。

[8] A common question arises here: How do we get from the initial traces used for error analysis (Chapter 3) or collaborative rubric design (Chapter 4) to the larger dataset needed for the train, dev, and test splits? We recommend that a single, trusted domain expert define the failure modes, finalize the rubric, and label the full set of traces (e.g., 100–200) required for robust splits. This is the most efficient path, as IAA does not have to be measured. But if a single expert is not feasible (e.g., large teams, highly subjective criteria), start with a smaller collaborative set (e.g., 20–50) to refine the rubric until agreement is high, then have one person apply it to label the additional traces needed for the full dataset. The following sections assume you now have a "gold-standard" labeled dataset ready to be split for building and validating your LLM-as-judge.

[8] 这里出现了一个常见问题：我们如何从用于误差分析（第 3 章）或协作评分标准设计（第 4 章）的初始痕迹，到进行 train、开发和测试分拆所需的更大数据集？我们建议由单一可信的领域专家定义失效模式，确定评分标准，并标注鲁棒拆分所需的全部痕迹（例如 100–200）。这是最高效的路径，因为 IAA 无需被测量。但如果单一专家不可行（例如大团队、高度主观标准），则从较小的协作组（例如 20–50 人）开始细化评分标准直到高度共识，然后由一人应用该指标来标记完整数据集所需的额外痕迹。以下部分假设你现在已经拥有一个"金标准"标记的数据集，准备拆分用于构建和验证你的 LLM 作为评判。

[9] Few-shot demonstrations are concrete examples embedded in the prompt to guide the model's reasoning.

[9] 少数样本演示是嵌入提示中的具体示例，用以指导模型的推理。

[10] A common mistake is inadvertently including evaluation cases as few-shot examples in the prompt. This leaks information and undermines the validity of reported metrics. Many people incorrectly describe their evaluation: they either don't use a held-out set at all, or they report accuracy on the development set as if it reflects the judge's true performance. This leads to highly

inflated and misleading estimates of success rates.

10 一个常见错误是无意中将评估案例作为少数样本示例包含在提示中。这会泄露信息，削弱报告指标的有效性。许多人错误地描述他们的评估：要么根本不使用预估的评估集，要么将发展集的准确性报告为准确，仿佛这反映了评委的真实表现。这导致成功率估计被严重夸大且误导性。

11 Studies suggest that LLM performance often plateaus after one to eight examples (Min et al. 2022). More examples can even degrade accuracy or exceed context length limits.

11 研究表明，LLM 的表现往往在一到八个示例后趋于稳定（Min 等，2022）。更多的例子甚至可能降低准确性或超过上下文长度限制。

12 This pragmatic approach of setting TPR/TNR targets is often more actionable than optimizing a single combined metric like an F1 score, though the latter can be considered if relative costs are well defined.

12 这种设定 TPR/TNR 目标的务实方法，通常比优化单一综合指标（如 F1 分数）更具可行性，尽管如果相对成本明确，后者也可以考虑。

13 Another reason to manually iterate on LLM-as-judge prompts is to be able to change labels. Sometimes we might rethink our failure definition and decide a failing trace should actually pass, and vice versa.

13 另一个手动迭代 LLM 作为评判提示的原因是能够更改标签。有时我们会重新思考失败的定义，认为失败的痕迹应该通过，反之亦然。

14 Make sure you trust your labels. A common mistake companies make is to outsource labeling to people (or even LLMs) who do not have *any* context on the application.

14 确保你信任你的唱片公司。公司常犯的一个错误是把标签工作外包给对应用没有任何背景信息的人（甚至是大型语言模型）。

15 A *confidence interval* is a range that expresses our uncertainty about a number. In this case, the true success rate of our LLM agent means that if we repeated this whole evaluation process many times, 95% of the resulting intervals would contain the actual success rate.

15 置信区间是一个表达我们对某个数字不确定性的范围。在这种情况下，我们 LLM 代理的真实成功率意味着如果我们多次重复整个评估过程，95%的区间将包含实际成功率。

16 This is the foundation for building trust in the judge's outputs. Without this trust, others will struggle to trust your product or your work. Many LLM-as-judge guides imply that prompt tuning alone is sufficient. Don't fall into that trap.

16 这是建立对评委成果信任的基础。没有这种信任，别人将难以信任你的产品或你的作品。许多 LLM 作为裁判的指南暗示仅凭及时调整就足够了。别掉进那个陷阱。

17 Success@*k* is pronounced “success at k.”

17 Success@k 发音为“成功在 K”。

18 To further optimize the human annotation effort involved in validating and aligning evaluators, some recent research explores methods for intelligently selecting which data points to label (Gligorić et al. 2024).

18 为了进一步优化验证和对齐评估者所需的人工注释工作，一些近期研究探讨了智能选择标记数据点的方法（Gligorić 等，2024）。

Chapter 6. Evaluating Multi-Turn Conversations

第六章 评估多轮对话的效果

Until now, we have focused on evaluating interactions where the user sends one message and the agent sends one response back. We call that a *single-turn* interaction, meaning one exchange between the user and the agent. Many applications involve *multi-turn* conversations, where the user and agent go back and forth multiple times. Each exchange (one user message and the agent's response) is a *turn*. A full conversation from start to finish is a *session*.

到目前为止，我们一直专注于评估那种用户发送一条消息，而代理仅返回一条回复的互动情况。我们将其称为“单轮次互动”，即用户与代理之间的一次交流。然而，许多应用程序涉及的却是多轮次对话，用户和代理会多次来回交流。每一轮交流都算作一个“轮次”，而完整的对话则从开始到结束构成一个“会话”。

In multi-turn conversations, the agent has to maintain context across turns, follow instructions over time, and respond coherently as the conversation develops. This creates new evaluation challenges that single-turn methods do not cover.

在多轮对话中，智能体需要保持每轮对话之间的上下文连贯性，随着对话的进展不断遵循指令，并做出连贯的回应。这就带来了新的评估挑战，而单轮对话的方法无法应对这些挑战。

In this chapter, you will learn:

在本章中，你将学到：

- How to evaluate at the session, turn, and coherence levels

  如何评估会话、转向以及连贯性方面的表现？
- When to isolate failures as single-turn problems versus genuinely multi-turn issues

  何时将故障视为单次发生的事件，何时将其视为涉及多次循环的问题。

- How to use perturbation testing to probe robustness

  如何使用扰动测试来检测系统的鲁棒性

- How to build session-level evaluators before investing in turn-level analysis

  在投入轮级分析之前，如何构建会话级评估工具？

The core evaluation lifecycle, Analyze-Measure-Improve (Figure 1-2), still applies, and the key principles from previous chapters still hold. We begin with qualitative error analysis. We favor simple, often binary evaluation criteria. But multi-turn interactions add complexity in what we measure and how we collect meaningful evaluation data.

核心评估生命周期，即分析-评估-改进的过程（见图 1-2），仍然适用。前一章中提到的关键原则依然有效。我们首先进行定性错误分析。我们倾向于使用简单、通常是二元的评估标准。但是，多轮交互会使得我们评估的内容以及收集有意义评估数据的方式变得更加复杂。

Multi-turn is also where otherwise strong models tend to fall apart. Laban et al. (2025) ran over 200,000 simulated conversations against several of the leading LLMs (both commercial ones like GPT-4 and open source ones like Llama) on six different tasks, including coding, math word problems, and summarization. They compared two setups for each task: the single-turn setup, where the user gives all the information at once, and the multi-turn setup, where the user drips out the same information over several messages. Accuracy dropped by an average of 39% in the multi-turn setup. The interesting finding is *why*. The authors split the drop into two parts: a small decline in the model's raw ability (the model is still roughly as smart as it was in single-turn) and a much larger increase in unreliability. What they mean by unreliability is that the model tends to make an early guess after the first user message, commit to it, and then fail to revise that guess when the user provides new information in later messages. Once the model is on the wrong track, it stays there. This is worth keeping in mind: when multi-turn conversations in our own agents break down, it is usually not that the underlying model got worse. It is that the model committed to an answer too

early and never came back to fix it.

多轮对话也是那些原本强大的模型容易出问题的地方。Laban 等人（2025 年）对超过 20 万次对话进行了模拟，使用了几款领先的大语言模型（包括像 GPT-4 这样的商业模型以及像 Llama 这样的开源模型），并针对六项不同的任务进行了测试，包括编程、数学应用题和摘要生成等任务。他们针对每项任务比较了两种对话方式：单轮对话，即用户一次性提供所有信息；多轮对话，即用户分多次提供相同的信息。结果显示，在多轮对话中，模型的准确率平均下降了 39%。有趣的是，模型准确率下降的原因是什么？作者将这一下降分为两部分：模型原始能力的轻微下降（模型仍然和单轮对话时一样智能），以及不可靠性的显著增加。他们所说的不可靠性是指模型在收到用户的第一条信息后，往往会做出初步判断并坚持这一判断，而不再考虑其他信息。然后，当用户在后续的消息中提供新信息时，模型就无法修正之前的猜测。一旦模型走上了错误的方向，它就会一直保持那种错误状态。这一点值得记住：当我们自己的智能代理进行多轮对话时，出现问题时，通常不是模型本身变坏了。而是模型过早地给出了一个答案，之后却再也没有回来修正这个错误。

Evaluating at Different Levels

在不同层面上进行评估

Multi-turn evaluation operates at three distinct levels. These levels are not mutually exclusive. Rather, they give us different lenses on the same conversation. In practice, they form a drill-down workflow: we start broad at the session level, then narrow into specific turns and coherence issues when we find failures. Figure 6-1 shows how the three levels fit together.

多轮评估在三个不同的层面上进行操作。这些层面并不是相互排斥的，相反，它们为我们提供了看待同一话题的不同角度。实际上，这三个层面构成了一个逐步深入的工作流程：我们首先从整个会话层面开始分析，然后逐步聚焦具体的对话轮次以及其中的连贯性问题。图 6-1 展示了这三个层面之间的相互关系。

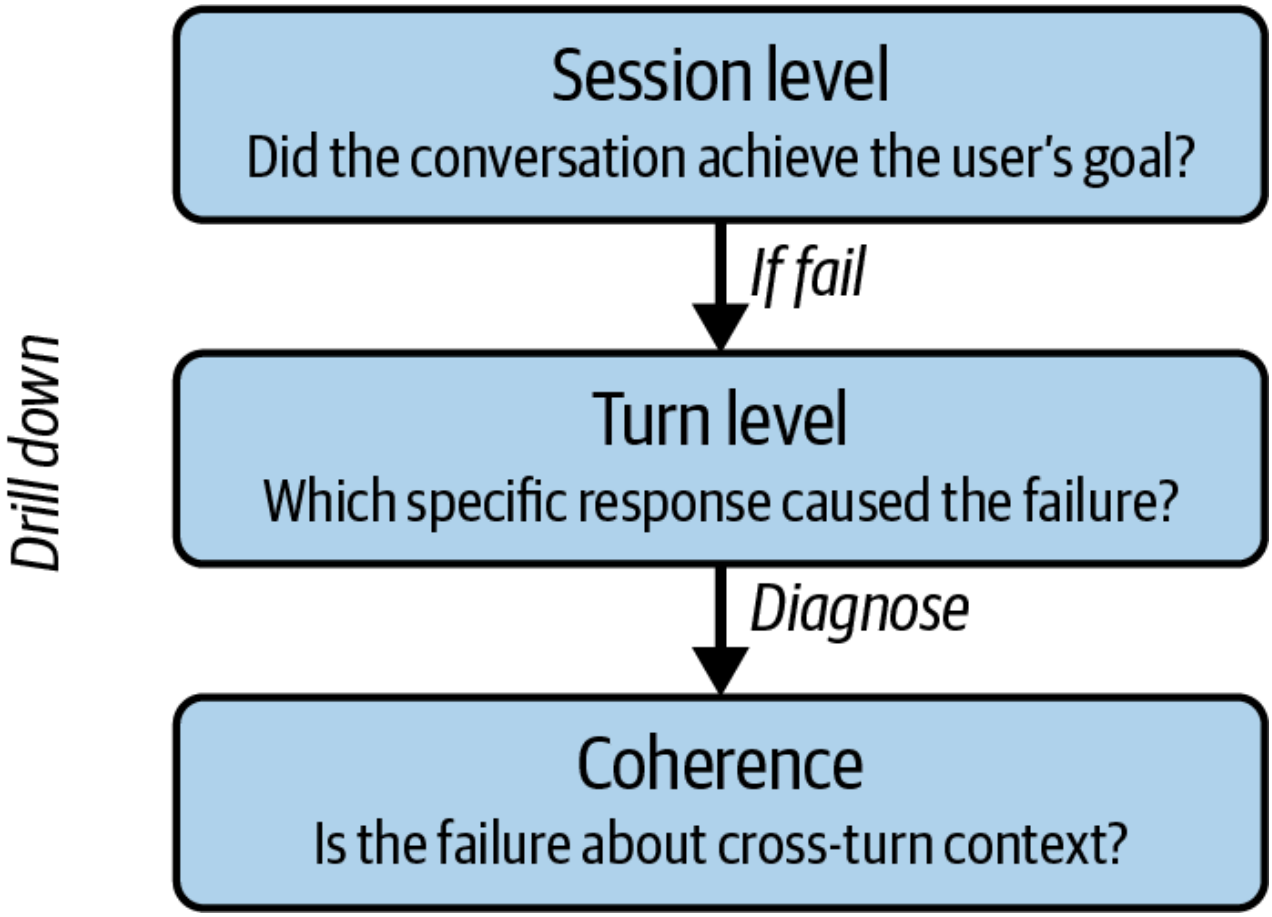


Figure 6-1. The three levels of multi-turn evaluation, starting at the session level, then drilling into turns and coherence when failures are found

图 6-1 展示了多轮评估的三个层次：从会话层面开始，然后逐步深入到具体的轮次和连贯性评估。在发现问题时，还需要进行进一步的调整。

Session Level 会话级别

At the session level, we ask whether the full conversation achieves the user's intended goal. This assessment is often binary (pass or fail) and gives us a high-level view of system effectiveness.[1] For example, if a peer feedback assistant ends a session without eliciting any useful feedback from the user, we might mark the entire session as a failure, even if individual turns were well formed. The overall task was not accomplished.

在会话层面，我们会判断整个对话是否实现了用户的预期目标。这种评估通常只有两种结果——成功或失败，它能让我们大致了解系统的运行效率。例如，如果某个同伴反馈助手在结束会话时未能从用户那里获得任何有用的反馈，那么即使每个对话轮次都处理得很好，我们也可能会将整个会话视为失败。也就是说，整个任务并没有完成。

Session-level evaluation is where we recommend starting. It provides the broadest performance signal with the least overhead, and it directly answers the question that matters most: did the system do what the user needed?

我们建议从会话级评估开始。这种评估方式能够提供更全面的性能信息，同时带来的负担也是最少的。它直接回答了最重要的问题：系统是否实现了用户的需求？

TIP 提示/窍门

Start with session-level evaluation and resist the urge to score every turn. A single Pass/Fail label per conversation tells you how often the system works. Turn-level analysis comes later, only for sessions that have already been flagged as failures.

从会话级别的评估开始，不要急于为每一轮对话都打分。每轮对话只有一个通过/失败的标志，这个标志可以显示系统运行的正确频率。而针对具体轮次的分析则需要在那些已经被标记为失败的任务上进行。

Turn Level 切换等级

Turn-level evaluation is a diagnostic tool, not a primary evaluation strategy. We use it to pinpoint *where* things went wrong within a session that has already been flagged as a failure.

Turn-level 评估是一种诊断工具，而非主要的评估方法。我们将其用于确定在已被标记为失败的某个会话中，问题具体出在何处。

Once we identify a failing session, we walk through its individual turns and ask the same questions we would in single-turn evaluation. Is this response relevant? Is it correct? Is the tone appropriate? The goal is to find the specific turn where the conversation went off track, which tells us what to fix.

一旦我们发现某个对话环节存在问题，我们就会逐一分析其中的各个回合，并提出与单轮评估时相同的问题：这个回应是否恰当？是否正确？语气是否合适？我们的目标是找到对话出偏差的具体环节，从而知道需要改进的地方。

Conversational Coherence and Memory
对话式连贯性与记忆

The third level focuses on properties that only emerge across multiple turns. Within a session, we check whether the assistant remembers what the user said earlier. Across sessions (for applications that persist state), we examine whether the assistant appropriately retains knowledge through stored summaries or memory tools.

第三个层次关注的是那些只有在多次互动中才会显现的特性。在单次会话中，我们会检查助手是否还记得用户之前所说的话。而在多个会话之间（对于需要持久保存状态的应用来说），我们会进一步考察助手是否能够通过存储的摘要或记忆工具来妥善保留知识。

Coherence failures, where the assistant loses track of something the user said earlier, are subtle. They often only surface after several turns and can be difficult to catch in standard single-turn evaluations. Common examples include:

coherence 失效，即助手无法记住用户之前说过的话，这种情况很微妙。通常只有在多次对话之后才会显现出来，而且在标准的单次对话评估中很难被发现。常见的例子包括：

Forgetting stated preferences

忘记原有的偏好设定

The user specifies a constraint early in the conversation, and the assistant ignores it in a later turn.

用户在对话的早期提出了一个限制条件，而助手在后续的对话中却忽略了这个条件。

Contradicting earlier responses

与之前的回答相矛盾

The assistant provides conflicting information across turns.

这个助手在不同的对话中提供了相互矛盾的信息。

Losing track of the task

无法记住这项任务了

In a multi-step workflow, the assistant loses context about which steps have been completed.

在多步骤的工作流程中，助手会失去对已完成步骤的上下文了解。

When you find a coherence failure, it's useful to distinguish among three different causes, because each one calls for a different fix:

当你发现连贯性出现问题时，区分三种不同的原因是非常有用的，因为每种原因都需要不同的解决方案来修复：

Anchoring failure *锚定失效*

The model committed to an early answer and refused to update it, even when the user provided new information in later turns. The Laban et al. (2025) study

found this is the most common cause of multi-turn degradation.

该模型承诺会尽快给出答案，但拒绝进行更新，即使用户在后续对话中提供了新的信息。Laban 等人（2025 年）的研究发现，这是多轮对话中信息失效最常见的原因。

Context truncation 上下文截断

The conversation exceeded the model's context window, or the application truncated earlier turns to save tokens, so the model literally cannot see the earlier constraint.

这次对话超出了模型的上下文范围，或者应用程序为了节省代币而截断了部分对话内容，因此模型实际上无法看到之前的限制条件。

Instruction drift 教学方法的落后

The model followed the system prompt faithfully in early turns but gradually stopped adhering to it as the conversation grew longer; the instructions are still in the context, but the model's attention to them weakened.

在对话的初始阶段，模型严格遵循系统的提示进行交互，但随着对话的深入，模型逐渐不再那么遵守这些提示了；虽然这些指示仍然存在于上下文之中，但模型对它们的关注程度却减弱了。

To fix these errors, for anchoring failures, prompting strategies can help (e.g., "Before answering, reread all user constraints stated so far"); for context truncation, architectural changes are needed (summarization of earlier turns, or a memory system); for instruction drift, repeating key instructions periodically within the conversation can keep the model on track.

为了解决这些错误，对于锚定失败的情况，可以使用提示策略来辅助用户理解（例如："在回答之前，请重新阅读所有迄今为止提到的用户约束条件"）；对于上下文截断的问题，则需要进行架构上的调整（比如总结之前的交互内容，或者引入记忆系统）；而对于指令偏离的问题，则可以在对话过程中定期重复关键指令，从而让模型保持正确的理解方向。

Some applications maintain state across sessions, not just within a single conversation. For example, a personal assistant might remember that a user prefers morning meetings, dislikes seafood, or has an upcoming deadline. Memory systems typically store facts as summaries or key-value (KV) pairs and inject them into the context at the start

of each new session.

一些应用程序会在多个会话之间保持状态，而不仅仅是在一次对话中。例如，个人助手可以记住用户更喜欢早晨时间进行会议、不喜欢海鲜，或者有一个即将到来的截止日期。记忆系统通常将信息以摘要或键值对的形式存储起来，并在每个新会话开始时将这些信息融入上下文之中。

We will not cover how to implement memory systems here, but here is how to think about evaluating them. Memory evaluation breaks down into four questions:

我们不会在这里讨论如何实现内存系统，但以下是关于如何评估内存系统的思考方式。内存评估可以分为四个问题：

Storage 存储

When the user states a new fact (e.g., "I prefer morning meetings"), does the system save it? You can test this by stating a preference in one session and checking whether the memory store contains it afterward.

当用户提出新的信息（例如："我更喜欢在早晨开会"），系统是否会将其保存下来？你可以通过在一次会话中表达某个偏好，然后查看记忆存储中是否保存了该信息来测试这一点。

Retrieval 检索/获取

When a new session starts, does the system retrieve the right facts for the current context? For example, if the user asks about scheduling, the system should retrieve the morning-meeting preference but not unrelated facts about food preferences.

当新的会话开始时，系统是否能够获取与当前上下文相关的事实信息呢？例如，如果用户询问关于日程安排的问题，系统应该只获取与早晨会议安排相关的信息，而不需要获取与食物偏好无关的其他信息。

Update 更新信息

When the user contradicts a stored fact (e.g., "Actually, afternoons work better now"), does the system update the old fact or keep both? For example, a user who said "I'm vegetarian" six months ago and then said "I've started eating fish" should not still be getting vegetarian recommendations.

当用户与存储的信息相矛盾时（例如："实际上，现在下午工作更好"），系统应该更新旧的信息，还是保留两者都有效呢？比如，一个用户在六个月前说过自己是素食者，之后又说开始吃鱼了，那么他们就不应该继续收到关于素食的推荐了。

Staleness 陈旧感

> Does the system apply facts that are no longer relevant? A stored fact about a user's address may become stale after they move, and the system should have a way to expire or reconfirm old facts.
>
> 该系统是否采用了已经不再相关的信息？例如，关于用户地址的存储信息，当用户搬家后，这些信息可能会变得过时。因此，系统应该有一种机制来使这些旧信息失效或重新确认其准确性。

For each question, you can write targeted test cases: set a fact in one session, modify or contradict it in a second session, and check the system's behavior in a third session. The evaluation structure is similar to retrieval evaluation (covered in Chapter 7), where you check whether the right information is surfaced at the right time.

对于每个问题，你可以编写有针对性的测试案例：在第一个测试案例中设定一个事实；在第二个测试案例中修改或反驳该事实；在第三个测试案例中检查系统的行为表现。这种评估方式类似于检索评估方式（详见第 7 章），即检查在正确的时间是否呈现了正确的信息。

Practical Strategies for Multi-Turn Evaluation

多轮评估的实用策略

The rest of this section covers collecting traces, isolating failures, defining session-level success, user simulators, stratifying by conversation length, and perturbation testing.

本节的其余部分涵盖了收集数据、识别故障、定义会话级成功标准、用户模拟器、根据对话长度进行分层处理以及干扰测试等内容。

Collecting Traces 收集痕迹

We begin by collecting an initial dataset of multi-turn traces. A lightweight chat interface is often enough. Team members simulate user behavior and engage in 10 to 15 tasks each, trying out different intents and personas. With just a few participants, we can quickly gather 100 or more diverse traces.

我们首先从收集一组多轮对话的跟踪数据开始。通常只需要一个简单的聊天界面即可。团队成员模拟用户行为，每人执行 10 到 15 个任务，尝试不同的意图和角色设定。只需几名参与者，我们就能迅速收集到 100 条或更多的多样化跟踪数据。

We then manually review these conversations, collecting examples of both successes and failures, and asking the basic questions you would ask of any trace: whether the assistant accomplished the user's goal, whether it followed instructions throughout, and where things went wrong if they did.

我们会手动审查这些对话记录，收集关于成功与失败的案例。同时，我们会提出一些基本的问题，这些问题适用于任何对话记录：该助手是否实现了用户的意图？它是否始终遵循了用户提供的指示？如果出了问题，问题出在何处呢？

Isolating Failures 隔离故障

When we find failures in multi-turn traces, a critical first step is determining whether the failure is *actually* multi-turn in nature. Many failures that surface during a conversation are, at their core, single-turn problems. Drawing this distinction saves significant debugging effort.

当我们发现多圈轨迹中的故障时，至关重要的第一步就是确定该故障是否确实属于多圈结构的问题。在调试过程中出现的许多故障，实际上本质上都是单圈结构的问题。明确这种区别能够大大节省调试时间。

The technique is straightforward: take the failing turn and try to reproduce the issue with a single-turn prompt that includes all the relevant context. Example 6-1 shows a case where a shopping assistant gives incorrect information on the second turn of a conversation.

这种技巧很简单：选择出现问题的回合，然后尝试用包含所有相关信息的单一回合提示来重现该问题。示例 6-1 展示了一个案例，其中购物助手在对话的第二个回合给出了错误的信息。

Example 6-1. A multi-turn failure that reduces to a single-turn problem
示例 6-1：一种多圈故障情况，最终演化为单圈故障问题。

User: Hi, I want to know about returns.

用户：您好，我想了解一下关于退货的相关信息。

Bot: Sure, what product are you asking about?

机器人：当然，您想了解哪种产品的信息呢？

User: The X1000. 用户：X1000。

Bot: The return window for X1000 is 15 days. *(Incorrect; it should be 30.)*

机器人：X1000 的退货期限是 15 天。（不正确；应该是 30 天。）

We can test whether this is truly a multi-turn issue by condensing it into a single prompt: "What is the return window for product X1000?" If the LLM still responds with "15 days," the failure is not caused by the conversational context. It is a retrieval or grounding issue, and we can debug it using the single-turn techniques from Chapters 3 and 5.

我们可以通过将问题简化为一个单独的提示来测试这是否确实是一个多轮对话的问题：例如"产品 X1000 的退货期限是多久？"如果大语言模型仍然回答"15 天"，那么问题就不在于对话的上下文，而是检索或定位方面的问题。我们可以使用第三章和第五章中提到的单轮对话技术来解决这个问题。

But not all failures simplify this way. Some are inherently tied to the conversational setup, shown in Example 6-2.

但并非所有的失败都能以这种方式简化。有些失败本质上与对话的上下文紧密相关，如示例 6-2 所示。

Example 6-2. A failure that is inherently multi-turn: the assistant forgets a stated preference

示例 6-2：一种本质上涉及多轮选择的故障情况：助手忘记了某个指定的优先顺序。

User: I'm looking for a movie.

用户：我在寻找一部电影。

Bot: Great! Any preferred genre?

机器人：太好了！有没有喜欢的类型呢?

User: I love sci-fi. 用户：我喜欢科幻小说。

Bot: Okay, sci-fi. Any actors you like?

机器人：好吧，科幻题材吧。有没有你喜欢的演员呢?

User: Not particularly. 用户：并没有特别的感觉。

Bot: How about *The Love Llama*, a heartwarming romantic comedy?

机器人：那部名为《爱的羊驼》的浪漫喜剧怎么样？它是一部非常温馨的浪漫喜剧哦。

Here, the assistant clearly forgot the "sci-fi" constraint.[2] There is no way to reproduce this as a single-turn problem because the failure is precisely about maintaining context across turns.

在这里，助手显然忘记了"科幻"这一限制条件。[2] 无法将这个问题设计成只需要一次对话就能解决的问题，因为问题的核心就在于如何保持对话中的上下文连贯性。

To study it systematically, we can truncate the trace before the failing turn and sample the LLM multiple times to observe how often it retains versus drops the user's stated preference. We can construct a dataset of these "N-1 turn" prompts, either by extracting prefixes from real conversations or generating them synthetically, to test short-term memory and consistency at scale.

为了系统地研究这个问题，我们可以在故障发生前的那一轮对话中截断对话记录，同时多次采样 LLM 的输出，以观察它保留用户所述偏好信息的频率以及改变这些偏好信息的频率。我们可以通过从实际对话中提取前缀信息，或者人工生成这些前缀信息，来构建包含"N-1 轮对话"提示的数据集，从而大规模地测试短期记忆能力以及一致性问题。

Operationalizing Session-Level Success

将会话级成功指标转化为可操作的成果

Session-level Pass/Fail needs a concrete definition, and the right definition depends on the application. For a customer support bot, success might mean "the user's issue was resolved and the user did not contact support again within 48 hours." For a coding assistant, success might mean "the generated code passes all tests on the first attempt." For a peer feedback assistant, success might mean "the session produced at least two specific, actionable feedback points."

在会话级别的通过/失败标准中，需要有一个明确的定义。而具体的定义则取决于具体的应用场景。对于客户支持机器人来说，成功可能意味着"用户的问题得到了解决，并且用户在 48 小时内没有再次联系支持部门"。对于代码辅助工具来说，成功则可能意味着"生成的代码能够一次性通过所有的测试"。而对于同伴反馈辅助工具而言，成功则可能意味着"该会话至少提出了两个具体的、可操作的反馈点"。

Whatever definition you choose, write it down as a rubric and validate it with the collaborative annotation process from Chapter 4. If your team can't agree on whether a

session passed, the definition needs more detail.

无论你选择何种定义，都将其写下来作为评判标准，并通过第 4 章中的协作注释流程来确认其准确性。如果团队无法就某个会议是否通过达成一致意见，那么就需要对定义进行更详细的说明。

User Simulators for Generating Test Conversations
用于生成测试对话的用户模拟器

Manually simulating multi-turn conversations is slow, so many teams use an LLM as a *user simulator* to generate the user side of a conversation automatically.

手动模拟多轮对话的过程非常缓慢，因此许多团队使用大语言模型作为用户模拟器，从而自动生成对话中的用户行为部分。

The simplest setup works as follows: you write a system prompt for a second LLM that describes a user persona and a goal, then you run a loop where the simulator LLM generates a user message, your agent responds, and the simulator generates the next user message, continuing for a fixed number of turns or until the simulator decides the goal is met. For example, the simulator's system prompt might say:

最简单的设置如下：你为第二个大型语言模型编写一个系统提示，其中描述了用户角色和目标。然后，通过循环机制来运行这个过程——模拟器模型生成用户消息，你的智能体做出回应，接着模拟器再生成下一个用户消息。这个过程会持续固定数量的轮次，或者直到模拟器认为目标已经达成为止。例如，模拟器的系统提示可以是这样的：

> **You are playing the role of a first-time home buyer.**
> **Your goal: find a 3-bedroom house under $500k near downtown.**
> **You are impatient and tend to change your mind.**
> **On each turn, respond as this user would.**
> **If the assistant has fully addressed your request, respond with [DONE].**

You can generate many different conversations by varying the persona (e.g., budget, location, personality, level of domain knowledge) and the goal (e.g., simple lookup, multi-step request, complaint, ambiguous question). To get diverse behavior, vary the user's communication style across runs: some personas should be terse, some verbose,

some confrontational, and some should change their mind mid-conversation.

通过不同的角色设定（如预算、位置、性格、领域知识水平）和不同的目标（如简单查询、多步骤请求、投诉、模糊问题等），你可以创造出多种不同的对话场景。为了获得多样的行为表现，需要在不同的运行环境中改变用户的沟通风格：有些角色应该表达简洁明了，有些则应该言辞激烈，还有些角色则应该在不确定的对话过程中改变主意。

More sophisticated approaches automate the persona generation itself. For example, AgentInstruct (Mitra et al. 2024) uses a pipeline where one LLM generates persona-goal pairs, a second LLM plays the user role with that persona, and a third LLM evaluates the resulting conversation, producing thousands of diverse multi-turn interactions without manual prompt writing. LMSYS's Chatbot Arena (Chiang et al. 2024) takes a different approach: it collects real human conversations at scale and uses them to calibrate what realistic user behavior looks like, which can inform how you design your synthetic personas.

更先进的方法能够自动化角色构建过程。例如，AgentInstruct（Mitra 等人，2024 年）采用一种流程：首先由 LLM 生成角色与目标之间的关联；然后由另一个 LLM 扮演该角色；最后由第三个 LLM 评估最终的对话效果。这样，无需人工编写提示即可生成数千种多样化的多回合互动场景。LMSYS 的 Chatbot Arena（Chiang 等人，2024 年）则采用不同的方法：它收集大量真实的人类对话，并利用这些对话来校准用户行为的真实表现，从而帮助设计更真实的合成角色。

User simulators are useful for generating a large volume of multi-turn traces quickly, but they have an important limitation: LLM-generated user behavior tends to be more consistent and cooperative than real user behavior. Real users make typos, ignore the bot's questions, change topics abruptly, and express frustration in ways that are hard to simulate. Simulators also tend to produce conversations that are shorter and more goal-directed than real sessions, which means they can miss the long-tail failures that only show up in extended, meandering interactions. Use simulators to build initial coverage, but always validate against real user traces when they're available.

用户模拟器在快速生成大量多轮对话记录方面非常有用，但它们有一个重要的局限性：由大语言模型生成的用户行为往往比真实用户的行为更加一致且合作性更强。真实用户会犯拼写错误、忽略机器人的问题、突然改变话题，还会以难以模拟的方式表达沮丧情绪。此外，模拟器生成的对话通常较短且目标导向性更强，这意味着它们可能会遗漏那些只在长时间、复杂的互动中才会出现的异常情况。虽然可以使用模拟器来初步测试功能，但在使用模拟器生成的结果时，务必结合真实的用户行为数据进行验证。

Stratifying by Conversation Length

按对话时长进行分层处理

We have found that failures often correlate with conversation length; that is, an agent that works well in short conversations may stop working as context accumulates (e.g., 8 or 10 turns in). When building your test set, include conversations of varying lengths, stratify into buckets (e.g., 2 to 4 turns, 5 to 9 turns, 10 or more turns), and report success rates separately for each bucket. If success rates drop sharply at a specific length, that tells you where to focus debugging, and whether the issue is context truncation, instruction drift, or something else.

我们发现，失败情况往往与对话的长度有关；也就是说，那些在短对话中表现良好的代理，在对话内容逐渐复杂时可能会失效（例如，对话长度达到 8 到 10 轮时）。在构建测试数据集时，应包含不同长度的对话，并将它们分为不同的类别（例如，2 到 4 轮、5 到 9 轮、10 轮以上），然后分别报告每个类别的成功率。如果某个特定长度的对话成功率显著下降，那么就可以据此确定调试的重点，并判断问题是由于上下文不足、指令模糊还是其他原因造成的。

Perturbation Testing 微扰测试

Once we have a set of multi-turn traces that pass (the agent handles them correctly), we can test how robust the agent is by making small, deliberate changes to those traces and checking whether the agent still handles them. This is *perturbation testing*: we take a passing conversation, modify one thing, and see if the agent breaks.

一旦我们有一组能够通过测试的多轮对话记录（且代理能够正确处理这些记录），我们就可以通过对这些记录进行微小的、有意的修改，来测试代理的鲁棒性。看看代理是否仍然能够正确处理这些修改后的记录。这就是扰动测试：我们从一个能够通过测试的对话开始，对其中某个元素进行修改，然后观察代理是否会因此出现问题。

The kinds of modifications we can try include:

我们可以尝试的修改方式包括：

Changing the user's goal midconversation

在对话过程中改变用户的目标

Starting from Example 6-2, we could insert, "Actually, my partner hates sci-fi. Can we do a thriller instead?" after the user originally asked for sci-fi. If the

agent still recommends sci-fi, it has failed to update its understanding.

从示例 6-2 开始，我们可以在用户最初请求科幻作品之后添加如下建议：“实际上，我的伴侣不喜欢科幻作品。我们能不能尝试推荐一些惊悚小说呢？”如果代理仍然推荐科幻作品，那就说明它未能更新自己的理解范围。

Adding ambiguity 增加了模糊性

We can rephrase a clear user message to be vague or underspecified, and see whether the agent asks for clarification or guesses incorrectly.

我们可以把明确的用户指令重新表述为模糊或不够具体的语句，然后观察智能体是否会请求澄清，或者是否做出了错误的猜测。

Contradicting the agent 与代理人所述相矛盾

If the agent suggests something, we can have the user say “no, that's wrong” and check whether the agent corrects course or doubles down.

如果代理提出了一些建议，我们可以让用户说“不，那样做是错的”，然后看看代理是否会纠正错误或者坚持原来的做法。

Perturbation testing is useful because it generates failing traces from known-good ones without waiting for production failures to occur. It is especially good at catching brittleness: an agent that works on the exact conversations in its test set but fails when anything changes slightly.

扰动测试非常有用，因为它能够从已知正常的代码中生成故障现象，而无需等待实际系统出现故障。这种测试方法特别擅长发现那些具有脆弱性的问题——即那些在测试集中的特定对话环境下能够正常运行，但在出现细微变化时就会失败的代理程序。

NOTE 注意/备注

Perturbation testing is philosophically similar to adversarial robustness testing in the ML literature. While the two overlap, there are differences: adversarial testing tries to break the system with worst-case inputs, while perturbation testing applies naturalistic variations (the kinds of changes real users actually make) to passing conversations. Both are valuable, but perturbation testing is easier to get started with because the perturbations come from realistic user behavior rather than adversarial optimization.

扰动测试在哲学上类似于机器学习领域中的对抗性鲁棒性测试。虽然这两种方法有相似之处，但仍然存在差异：对抗性测试试图用最坏情况下的输入来破坏系统，而扰动测试则采用真实用户实际会做出的自然变化来测试系统的表现。这两种方法都有价值，但扰动测试更容易实施，因为扰动来源于真实的用户行为，而非对抗性优化过程。

One useful metric to track over time is a *brittleness score*: for each passing trace, count how many perturbations it takes to make the agent fail, then report the median across traces. An agent with a median brittleness score of 1 (one small change breaks it) is much more fragile than one with a score of 5. You can track the brittleness score across releases to see whether the agent is getting more or less robust.

一个有用的指标是跟踪时间变化的脆性分数：对于每个通过的跟踪，计算使其失效需要多少扰动，然后在所有跟踪中报告中位数。一个中位数脆性分数为 1（一个小变化就破坏它）的代理比分数为 5 的代理脆弱得多。你可以跟踪脆性分数跨版本，以查看代理是否变得更稳健或更不稳健。

Automated Evaluation of Multi-Turn Traces

多轮跟踪的自动化评估

Once we have identified clear failure types through manual review, we can build automated evaluators using the same techniques from Chapter 5. These may be LLM-as-judge models, programmatic filters, or a hybrid of both.

一旦我们通过人工审查确定了明确的失败类型，我们就可以使用第 5 章中的相同技术来构建自动化评估工具。这些工具可以是基于大语言模型的裁判模型、程序化的过滤器，或者两者的结合体。

The key question is *what level to evaluate at.* For most systems, we recommend starting with session-level evaluators. A session-level LLM-as-judge receives the full conversation and issues a Pass/Fail verdict on whether the user's goal was met. This is the highest-leverage evaluator to build first because it directly measures what matters.

关键问题在于应该评估哪个层次。对于大多数系统来说，我们建议从会话级别的评估器开始使用。会话级别的 LLM 作为评判器可以接收完整的对话内容，并判断用户的任务是否完成。这是最具有影响力的评估方式，因为它直接衡量了关键指标。

Turn-level evaluators are useful for specific, well-defined failure modes. For example:

针对特定、定义明确的故障模式，级别评估工具非常有用。例如：

- A programmatic check that verifies the assistant never contradicts information it provided in a previous turn

  一个程序性检查，用于确保助手在之前的对话中从未出现过与所提供的信息相矛盾的情况。

- An LLM-as-judge that evaluates whether each tool call in the conversation was justified by the preceding user message

  一个以裁判身份工作的 LLM，负责判断对话中每个工具的使用是否得到了之前用户消息的支撑，是合理的。

- A coherence check that extracts user-stated constraints and verifies they are respected in the assistant's final recommendation

  一种一致性检查机制，它会提取用户指定的约束条件，并验证这些约束条件在助手的最终推荐中被正确遵循。

Not every evaluator needs to assess every turn. In fact, for many systems, we only need turn-level analysis when debugging specific paths. Session-level evaluation provides a broader performance signal with less overhead.

并非每个评估者都需要对每一个操作环节进行评估。实际上，对于许多系统来说，只有在调试特定路径时，才需要用到操作级分析。会话级评估能够提供更全面的性能信息，同时带来的额外负担也较小。

Common Pitfalls 常见陷阱

There are several pitfalls specific to multi-turn evaluation to watch for:

在多轮评估过程中，需要注意几个潜在的陷阱：

Failures that correlate with conversation length

与对话时长相关的失败情况

> Many systems behave well in the first two or three turns but then degrade. They may forget context, ignore instructions, or contradict earlier responses. Our test sets should include conversations of varying lengths to detect these position-dependent failures. If we only test short conversations, we will miss problems that real users encounter in longer sessions.
>
> 许多系统在前两三个回合中表现良好，但随后会退化。它们可能会忘记上下文、忽略指令或与之前的回应相矛盾。我们的测试集应该包含不同长度的对话，以检测这些与位置相关的失败。如果我们只测试简短的对话，我们将错过真实用户在较长会话中遇到的问题。

Overinvesting in turn-level evaluation

过度投资于回合级评估

> It is tempting to build evaluators for every turn in a conversation, but this quickly becomes expensive and noisy. Most of the signal comes from session-level evaluation. We reserve granular analysis for focused debugging: identify failing sessions first, then drill into a handful of representative traces to find root causes.
>
> 构建对话中每个回合的评估器很有诱惑力，但这很快就会变得昂贵且嘈杂。大部分信号来自会话级评估。我们保留粒度分析用于专注调试：首先识别出失败的会话，然后钻入少量有代表性的跟踪记录以找到根本原因。

Confusing multi-turn failures with single-turn failures

将多轮失败与单轮失败混淆在一起

> As illustrated in Example 6-1, many failures that appear during multi-turn conversations are actually single-turn problems in disguise. Always attempt to isolate the failure before building multi-turn-specific evaluators. This avoids overengineering solutions for problems that have simpler fixes.
>
> 如示例 6-1 所示，在多轮对话中出现的许多故障，实际上都是经过伪装后的单轮对话中的问题。因此，在构建针对多轮对话的评估工具之前，应尽力将故障隔离出来。这样就能避免为那些可以简单解决的问题而设计复杂的解决方案，从而避免不必要的过度工程化处理。

Putting It Together 整合起来

Here is a workflow for evaluating multi-turn conversations:

以下是一种用于评估多轮对话的工作流程：

1. Define session-level success concretely (write it as a rubric).

   明确界定会话级别的成功标准（将其写成一个评分细则）。

2. Collect traces, either from production logs, manual simulation, or user simulators.

   收集轨迹，可以来自生产日志、手动模拟或用户模拟器。

3. Stratify by conversation length and report success rates per bucket.

   按对话长度分层，并报告每个分层的成功率。

4. For failing sessions, diagnose whether the failure is anchoring, context truncation, or instruction drift.

   对于失败的会话，诊断失败是否是由于锚定、上下文截断或指令漂移。

5. Isolate each failure to determine whether it's truly multi-turn or a single-turn problem in disguise.

   隔离每个失败，以确定它是否真的是多轮问题，还是伪装成单轮问题的假象。

6. Use perturbation testing to check the robustness of passing conversations.

   使用扰动测试来检查通过对话的鲁棒性。

7. Build session-level automated evaluators first; add turn-level evaluators only for specific, well-defined failure modes.

   首先构建会话级别的自动评估器；仅针对特定、明确定义的故障模式添加回合级别的评估器。

In the next chapter, we turn to evaluating a specific architecture that introduces its own set of challenges: RAG.

在下一章中，我们将转向评估一个特定的架构，该架构引入了自己的一套挑战：RAG。

[1] As discussed in Chapters 3 and 4, binary labels force a clear definition of success. Over time, a single Fail category can be split into more specific failure modes.

[1] 如第 3 章和第 4 章所述，二元标签强制明确定义成功。随着时间的推移，单个 Fail 类别可以被拆分为更具体的故障模式。

[2] In practice, it can take 10 or more turns to run into such an egregious error. These errors are also more likely with nonfrontier or smaller LLMs.

[2] 在实际操作中，可能需要 10 轮或更多轮才能遇到这种严重的错误。这些错误也更有可能出现在非前沿或较小的 LLMs 上。

Chapter 7. Evaluating Retrieval-Augmented Generation

第七章 评估检索增强生成技术

The previous chapters covered evaluation techniques that apply broadly across LLM applications. This chapter narrows the focus to one architecture, namely RAG, where the retrieval stage brings evaluation questions of its own.

在前面的章节中，我们介绍了适用于大语言模型应用的评估技术。在本章中，我们将重点聚焦于一种特定的架构，即 RAG 架构。在 RAG 架构中，检索阶段本身就需要进行相关的评估。

In this chapter, you will learn:

在本章中，你将学到：

- How to build evaluation datasets for retrieval using synthetic query generation

  如何使用合成查询生成方法来构建用于检索的评估数据集
- How to measure retrieval quality with Precision@*k*, Recall@*k*, MRR, and NDCG@k

  如何使用 Precision@k、Recall@k、MRR 和 NDCG@k 来衡量检索质量
- How to tune chunking strategies with grid search

  如何使用网格搜索方法来调整分块策略
- How to evaluate generation quality using faithfulness and relevance

  如何通过忠实度和相关性来评估生成质量

RAG architectures are pervasive across customer support, enterprise search, scientific QA, and coding assistance. Evaluating RAG requires more than measuring final answer correctness. Each stage of the pipeline (query construction, retrieval, reranking, and generation) can independently introduce failure modes.[1] At the same time, failures across these stages are interdependent. If retrieval returns the wrong documents, the generator has no way to produce a correct answer. If the user's query is ambiguous, re-

trieval may pull back something only loosely related, and reranking and generation will be working from a bad starting point.

RAG 架构广泛应用于客户支持、企业搜索、科学问答和代码辅助等领域。评估 RAG 性能不仅仅取决于最终答案的正确性。流程的每一个阶段（查询构建、检索、重新排序、生成）都可能引发问题。同时，这些阶段之间的故障是相互关联的：如果检索阶段返回了错误的文档，那么生成器就无法生成正确的答案；如果用户的问题不明确，检索可能会遗漏一些与问题关系不大的重要信息，从而导致重新排序和生成过程无法顺利进行。

In this chapter, we focus on two parts of the evaluation loop: *Analyze* and *Measure*. We show how to:

在本章中，我们重点讨论评估流程中的两个部分：分析与测量。我们将介绍如何执行这两个步骤。

- Analyze whether failures arise in retrieval, in generation, or in the interaction between them.

  分析在检索过程中、生成过程中，或者它们之间的交互过程中，是否存在失败的情况。
- Measure retrieval quality using metrics like Recall@*k*, MRR, and NDCG@*k*.

  通过诸如 Recall@k、MRR 和 NDCG@k 等指标来评估检索质量。
- Evaluate generation using context-sensitive metrics such as faithfulness and relevance.

  使用与上下文相关的评估指标来评估生成结果，例如忠实度和相关性等指标。

We begin by constructing evaluation datasets for retrieval, then introduce diagnostic metrics and methods for attributing failures across the pipeline.

我们首先构建用于检索的评估数据集，然后引入诊断性指标和方法，以分析整个流程中的故障原因。

Overview 概述

A basic RAG pipeline retrieves context from a knowledge base (e.g., a document corpus, structured table, or the web) based on a query, and feeds it into an LLM for generation. Unlike traditional search, the consumer of this context is the LLM itself, which can process many passages simultaneously. This shifts the retrieval objective: high *re-*

call becomes critical to ensure we include all potentially relevant content.

基本的 RAG 流程以查询为基础，从知识库中检索上下文信息（例如文档集、结构化表格或网络上的数据），然后将该信息传递给 LLM 进行生成。与传统搜索方式不同，这种上下文的接收者就是 LLM 本身，它能够同时处理大量上下文信息。这改变了检索的目标：高召回率变得至关重要，以确保包含所有可能相关的内容。

However, simply retrieving as many documents as possible is not ideal. LLMs have limited context windows, and irrelevant text can degrade performance. LLM APIs also charge per token, so excess tokens increase cost. To balance coverage and quality, many RAG systems use a two-stage retrieval approach (retrieve broadly, then rerank). More recent systems go further with *agentic RAG*, where the model writes its own retrieval queries, examines the results, and reformulates the query if the initial results are insufficient. In agentic RAG, the model decides when and what to retrieve at each step, rather than following a fixed retrieve-then-generate pipeline. We focus on the two-stage pattern here (Figure 7-1) because the evaluation principles are the same: you still need to measure whether the right documents were retrieved and whether the generated output is faithful to them. Agentic RAG adds evaluation questions (e.g., Did the model choose to retrieve at the right time? Did it write a good query?), which we cover in Chapter 8.

不过，仅仅获取尽可能多的文档并不是理想的解决方案。大语言模型的保护范围有限，而无关紧要的文本可能会降低模型的性能。此外，大语言模型 API 会对每个标记进行收费，因此使用过多的标记会增加成本。为了在覆盖范围和质量之间取得平衡，许多 RAG 系统采用两阶段检索方法：首先进行广泛检索，然后再对结果进行重新排序。更先进的系统则采用了“代理式 RAG”技术，即模型自行编写检索查询，检查查询结果，如果初始结果不够充分，则重新调整查询。在代理式 RAG 中，模型能够在每个步骤中决定何时以及获取哪些文档，而不是遵循固定的检索-生成流程。我们在这里重点讨论两阶段模式（见图 7-1），因为评估原则保持不变：仍然需要衡量是否检索到了正确的文档，以及生成的输出是否忠实于这些文档。代理式 RAG 还增加了一些评估问题，例如……这个模型是否选择了在正确的时间进行检索？它的查询语句是否得当？这些问题将在第 8 章中讨论。

Several other developments have reshaped retrieval since 2023. Hybrid retrieval, combining keyword search (BM25) with dense embeddings using RRF (Cormack et al. 2009), has become the default in many production systems because it covers both exact-match and semantic-similarity cases. Late-interaction models like ColBERT (Khattab and Zaharia 2020) offer a middle ground between fast embedding-based retrieval (where queries and documents are encoded independently) and expensive reranking (where a model scores each query-document pair together, which is more accurate but slower). Long-context models (128k+ tokens) have also raised the ques-

tion of whether RAG is still necessary for some use cases; in practice, RAG remains useful because it provides attribution, reduces cost, and keeps the context focused on relevant information.

自 2023 年以来，还有几项发展改变了信息检索的方式。混合式信息检索结合了关键词搜索（BM25）和基于 RRF 的密集嵌入技术（Cormack 等人，2009 年），已成为许多生产系统中的默认方案，因为它能够同时处理精确匹配和语义相似性的查询。像 ColBERT 这样的晚期交互模型（Khattab 和 Zaharia，2020 年）则提供了介于基于嵌入的快速检索和复杂的重新排序之间的解决方案。长上下文模型（128k 以上令牌）也引发了关于某些应用场景是否仍需 RAG 的问题；实际上，RAG 仍然非常有用，因为它能够明确信息来源、降低成本，并专注于相关上下文信息。

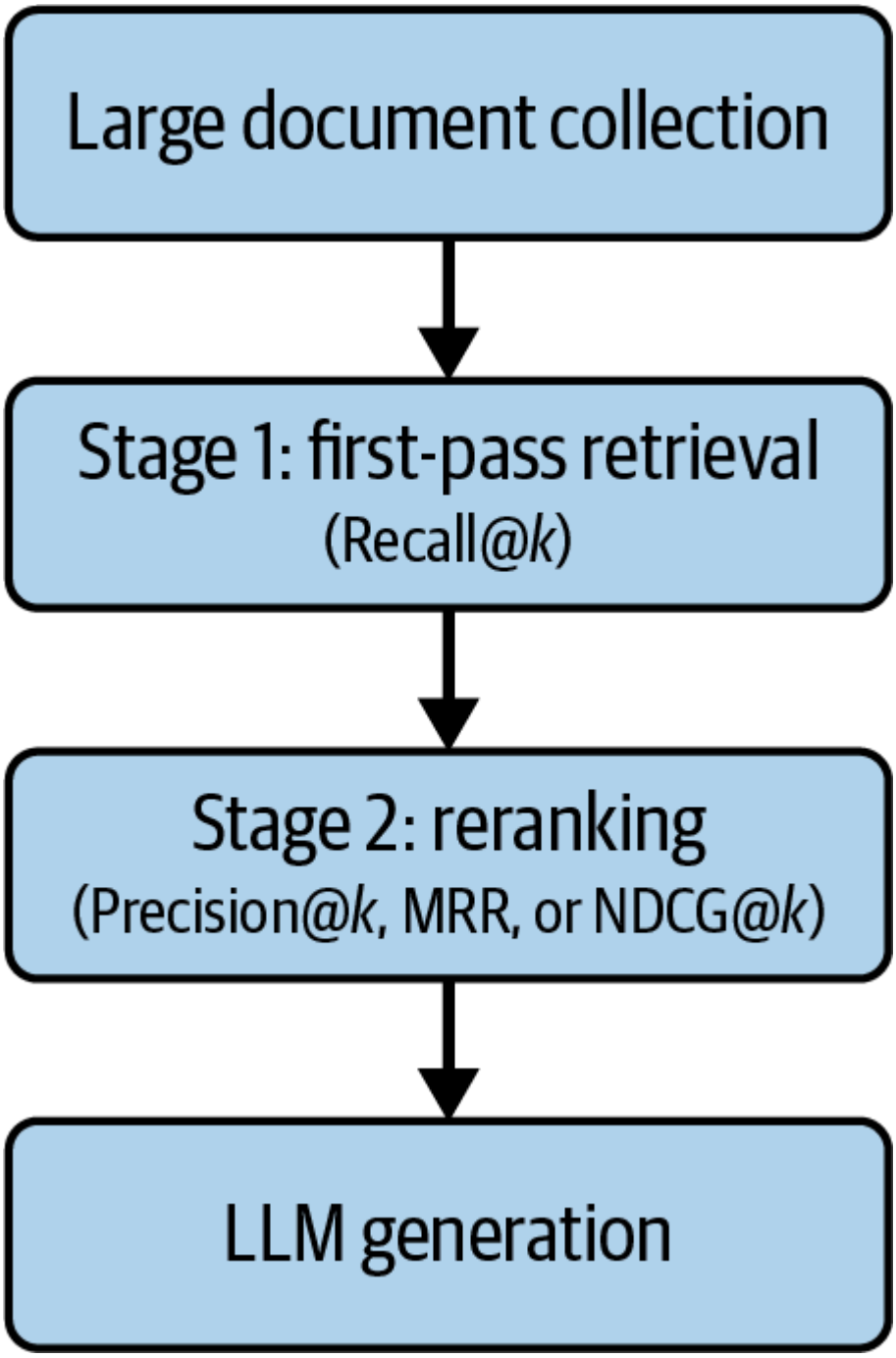


Figure 7-1. The two-stage retrieval in RAG

图 7-1. RAG 中的两阶段检索过程

The two-stage pattern remains a solid starting point for evaluation:

这种两阶段模式仍然是一个可靠的评估起点：

Stage 1: first-pass retrieval

阶段 1：初步检索

The first stage scans the entire corpus and returns a broad set of candidates. Here, we can use fast, lightweight methods that are cheap to run over millions of documents. Common choices are embedding-based search with approximate nearest neighbor indexes (e.g., FAISS, HNSW, or hosted vector stores like Pinecone or Weaviate on top of embeddings from models like OpenAI's `text-embedding-3` or Cohere's `embed-v3`), or sparse retrieval methods like BM25 over an inverted index. The goal at this stage is high *recall*, even at the cost of precision: we want to make sure the right document is somewhere in the candidate set, even if it is not yet at the top.

第一阶段会扫描整个文档集，并列出一批候选文档。在这个阶段，我们可以使用快速且轻量级的方法来处理数百万份文档。常见的搜索方式包括基于嵌入式的搜索，使用近似最近邻索引（如 FAISS、HNSW，或者利用 Pinecone 或 Weaviate 等托管向量存储系统，这些系统基于 OpenAI 的 `text-embedding-3` 或 Cohere 的 `embed-v3` 模型生成的嵌入数据）。另一种方法是使用稀疏检索方法，比如基于倒索引的 BM25 算法。在这个阶段，我们的目标是实现高召回率，即使是以牺牲精度为代价：我们需要确保正确的文档一定存在于候选列表中，即使它目前并不排在最顶端。

Stage 2: reranking 第二阶段：重新排序

The second stage takes the broad set of candidates from Stage 1 and reorders them more carefully, putting the most relevant ones at the top. A reranker is, intuitively, a more careful "reader." Where Stage 1 compared a single embedding of the query against a single embedding of each document, a reranker looks at the query and a candidate document together and produces a relevance score that takes both into account. Because it considers the pair jointly, it can catch nuances that a one-shot embedding similarity would miss, but it is also more expensive per call, which is why we only run it on the small candidate set instead of the whole corpus. The two common implementations of a reranker are cross-encoder models (such as Cohere Rerank, BGE reranker, or a fine-tuned MiniLM cross-encoder), which are trained specifically to score (query, document) pairs, and LLM-as-reranker setups, where we prompt a general LLM with the retrieved documents and ask it to return a new ordering or per-document

scores. At this stage we prioritize precision and ranking quality rather than coverage.

第二阶段会将第一阶段产生的众多候选者进行重新排序，将最相关的候选者放在最顶端。直观来说，重新排序器就像是一个更加细致的“分析者”。在第一阶段中，只是将查询嵌入与每个文档的嵌入进行了比较，而在第二阶段中，重新排序器会同时考虑查询和候选文档，从而计算出一个综合考虑两者相关性的评分。由于它同时考虑了这两组信息，因此能够捕捉到一次性嵌入相似度所忽略的细微差别。不过，这种方法的计算成本更高，因此我们只在较小的候选者集合上进行实验，而不是整个语料库。目前常见的两种重新排序器实现方式分别是：交叉编码器模型（如 Cohere Rerank、BGE reranker 或经过微调的 MiniLM 交叉编码器），这些模型专门用于评估查询和文档之间的相关性；以及使用 LLM 作为重新排序器的方案。在这个阶段，我们会使用已获取的文档来提示通用大语言模型，要求其生成新的排序结果或针对每个文档的评分。此时，我们更注重模型的精确度和排序质量，而非覆盖面的广度。

We start by evaluating the retrieval component on its own. Before we assess the quality of generated answers, we verify that the system consistently finds the right context. If the retriever is not surfacing the relevant documents, no amount of prompt engineering or model tuning on the generation side will fix the output.

我们首先单独评估检索组件的功能。在判断生成的答案质量之前，我们需要确保系统能够始终找到正确的上下文信息。如果检索器无法找到相关的文档，那么无论如何调整提示方式或优化模型，都无法改善最终的输出结果。

Synthetically Generating Query-Answer Pairs

人工生成的问答对

To evaluate retrieval, we need a dataset of queries paired with the specific document *chunks* that contain the correct answers. The most reliable option is to manually curate a set of realistic questions and map each to the exact chunk(s) in which the answer appears. While this provides high-quality supervision, it is often expensive and time-consuming.

为了评估检索效果，我们需要一个包含各种查询的数据集，以及那些包含正确答案的具体文档片段。最可靠的方法是手动整理出一系列真实的问题，并将这些问题与包含正确答案的文档片段进行对应。虽然这种方法可以提供高质量的监督信息，但过程通常既耗时又费力。

To scale up this dataset more easily, we can generate synthetic question-answer-context triples using an LLM (Figure 7-2).

为了更轻松地扩展这个数据集，我们可以使用大语言模型生成合成的问题-答案-上下文三元组（如图 7-2 所示）。

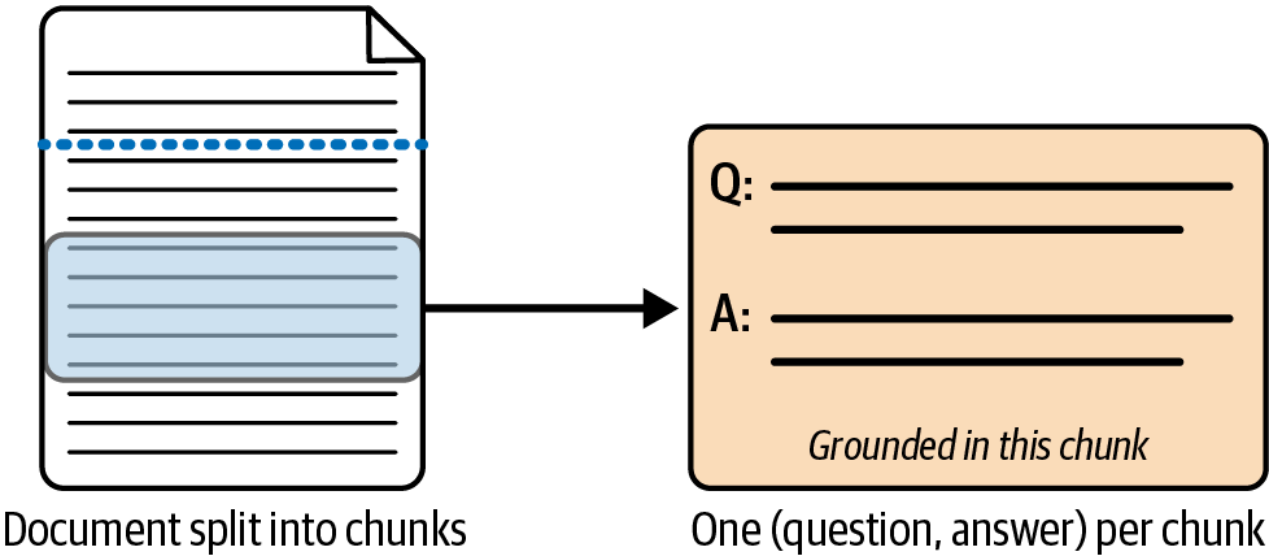


Figure 7-2. Generating synthetic QA pairs for retrieval evaluation

图 7-2. 生成用于检索评估的合成 QA 对

Documents are split into chunks (the same units the retriever indexes, which serve as retrieval "targets"). For each chunk, an LLM extracts a salient fact and writes a question that the fact answers. The chunk, fact, and question together form a (question, answer, context) triple where the correct retrieval target is known by construction. Here is an example:

这些文档被分割成若干块（与检索器索引所使用的单位相同，这些单位被用作检索的"目标"）。对于每一块数据，大型语言模型会提取出一个关键事实，并据此提出一个问题。这块数据、该事实以及所提出的问题共同构成了一个三元组（问题、答案、上下文），而正确的检索目标可以通过这种结构直接确定。以下是一个例子：

```
You are a helpful assistant generating synthetic QA pairs for retrieval
evaluation.

Given a chunk of text, extract a specific, self-contained fact from it.
Then write a question that is directly and unambiguously answered by that
fact alone. Return your output in the following JSON format:

{
  "fact": "...",
  "question": "..."
}
```

Chunk:
"{text_chunk}"

Creating Harder Synthetic Queries

创建更复杂的合成查询语句

The basic synthetic generation approach from "Synthetically Generating Query-Answer Pairs" tends to produce questions that are easy for a retriever to answer, because each question is built from a single chunk and uses that chunk's exact wording. A retriever that matches on keywords will find the right chunk most of the time. In production, user queries are messier. They use different phrasing, reference topics that appear in multiple documents, and sometimes ask for comparisons across chunks. To stress-test the retriever, we can generate synthetic questions that deliberately resemble content in multiple chunks but are only correctly answered by one of them:

在"合成化生成问答对"的基本方法下，所生成的问题通常很容易被检索器回答，因为每个问题都是基于单个片段构建的，并且使用了与那个片段完全相同的表述。如果检索器能够根据关键词进行匹配，那么它大多数时候都能找到正确的片段。但在实际使用中，用户提出的问题往往更加复杂。它们会使用不同的措辞，提及多个文档中出现的主题，有时还会要求对多个片段进行比较。为了对检索器进行压力测试，我们可以生成一些合成问题，这些问题刻意模仿了多个片段的内容，但只有其中一个片段能够正确地回答这些问题。

1. *Select a target chunk.*
1. *选择目标数据块。*

   Choose a chunk A from the corpus that contains a clear, fact-based answer.

   从语料库中选择一个包含明确、基于事实的回答的段落。

2. *Identify similar chunks.*
2. *找出类似的模块或部分。*

   Use an embedding model or keyword search to find chunks B, C, etc., that are similar to A but do *not* contain the answer.

   使用嵌入模型或关键词搜索，找到与 A 相似但不包含答案的片段 B、C 等。

3. *Generate a discriminative question.*
3. *生成一个具有区分性的问题。*

Ask the LLM to write a question that is *only* answered by Chunk A, but which reuses terminology or themes from B or C.

请这个大语言模型编写一个问题，该问题的答案只有“Chunk A”提供，但其中使用了“Chunk B”或“Chunk C”中的术语或主题。

Let's see what this looks like in practice for a retrieval system built over a financial news corpus:

让我们看看在基于金融新闻数据构建的检索系统中，实际情况会是什么样吧：

Step 1 (target chunk)

步骤 1（目标区块）

Chunk A: “In April 2020, the company reported a 17% drop in quarterly revenue, its largest decline since 2008.”

部分 A：“2020 年 4 月，该公司公布的季度收入出现了 17%的下降，这是自 2008 年以来最严重的下滑。”

Step 2 (similar chunk)

步骤 2（类似的步骤）

Chunk B: “The company experienced significant losses in 2008 during the financial crisis but rebounded by the end of 2009.” This chunk mentions the same company, the same year (2008), and the phrase “financial crisis,” so a keyword-based retriever would score it highly.

第二部分：“公司在 2008 年的金融危机期间遭受了重大损失，但到 2009 年底又恢复了元气。”这部分提到了同一家公司、同一年份（2008 年），以及“金融危机”这一关键词。因此，基于关键词检索的工具会给出较高的评分。

Step 3 (generated question)

步骤 3（生成问题）

“When did the company experience its largest revenue decline since the 2008 financial crisis?” Only Chunk A answers this, but the overlapping language in Chunk B makes it a plausible distractor that the retriever has to distinguish

from.

“这家公司在什么时候经历了自 2008 年金融危机以来最严重的收入下降？"只有第一部分回答了这个问题，但第二部分中的类似表述使得它有可能是一个干扰项，需要被区分出来。

Filtering Synthetic Questions

过滤掉那些人造的问题

Synthetic questions can sometimes be unrealistic or off-domain. We can filter them using a process similar to how we filter unrealistic synthetic queries in Chapter 3. We manually review a subset and rate each question for realism and relevance, then use those labeled examples in a few-shot prompt to another LLM to score the rest. We keep only those deemed realistic.

这些合成问题有时可能不切实际，或者与领域无关。我们可以采用与第三章中过滤不切实际合成问题类似的方法来处理这些问题。我们会手动筛选一部分问题，评估其真实性和相关性，然后利用这些带有标签的例子来引导另一个大语言模型对剩余问题进行评估。最终，我们只保留那些被认定为真实的问题。

Here is an example prompt for filtering:

以下是用于过滤的示例提示：

```
You are an AI assistant helping us curate a high-quality dataset of
questions for evaluating a real estate information retrieval system.

Here are examples of realistic and unrealistic user queries we have
manually rated:

Realistic Queries (Good Examples):
- "What are the property taxes for 123 Main St?"
  Rating: 5 — Typical, concise, property-specific.

- "Show me 3-bedroom houses in the North Berkeley area with a backyard."
  Rating: 5 — Natural filter-based query.

Unrealistic Queries (Bad Examples):
- "Tell me about the color of the third brick from the left on the
  chimney of the house that was sold yesterday."
  Rating: 1 — Overly specific and unnatural.
```

- "If I combine the square footage of all houses listed in zip code 94704, what is the total?"
  Rating: 2 — Odd aggregation, not representative of real search behavior.

Your Task:
Rate the following generated question for realism on a scale of 1 to 5.
Provide a brief explanation.

Generated Question: "{question_to_evaluate}"

Output Format:
Explanation: [your brief explanation]
Rating: [your 1-5 rating]

Note that we use a 1–5 rating here for filtering purposes only: the goal is to roughly rank queries by realism and domain relevance so we can discard the weakest ones, not to measure failure rates. In Chapter 4, we recommended binary labels for evaluation because they produce higher IAA; the 1–5 scale here is appropriate because we are filtering synthetic data for quality, not evaluating system outputs. This task supports the construction of a "good enough" dataset to begin testing our retrieval pipeline.

请注意，我们在这里使用 1 到 5 的评分只是为了筛选目的而已：我们的目标是根据真实性和领域相关性对查询进行大致排序，从而剔除最弱的查询，而不是用来衡量失败率。在第四章中，我们建议使用二进制标签进行评估，因为这样的标签能带来更高的综合评估指标；而 1 到 5 的评分则更为合适，因为我们是在筛选合成数据以追求质量，而不是评估系统的输出结果。这项任务有助于构建出一个"足够好的"数据集，从而开始测试我们的检索流程。

Metrics for Retrieval Quality

检索质量的评估指标

Once we have a dataset of queries and their corresponding relevant document chunks, we can evaluate our retriever using standard ranking metrics. The choice of metric depends on what matters for our downstream application. We present four commonly used metrics in the following sections.

一旦我们有了查询数据集以及与之对应的相关文档片段，我们就可以使用标准的排名指标来评估我们的检索器性能。选择何种指标取决于下游应用的需求。在接下来的章节中，我们将介绍四种常用的排名指标。

Precision@k and Recall@k

Precision@k 和 Recall@k

Precision@*k* and Recall@*k* are the most basic retrieval metrics. They evaluate the quality of the top *k* documents retrieved in response to a query. The value of *k* can vary depending on the query. A factoid question like "What is the property tax for 123 Main St?" might only need *k* = 1 or 2, while a synthesis question like "Summarize recent market trends for 3-bedroom houses downtown" might need *k* = 5 to 10.

Precision@*k* measures what fraction of the top *k* retrieved documents are actually relevant:

$$\text{Precision@}k = \frac{\text{Number of relevant documents in top } k \text{ results}}{k}$$

Recall@*k* measures what fraction of *all* relevant documents for a query appear within the top *k* results:

$$\text{Recall@}k = \frac{\text{Number of relevant documents in top } k \text{ results}}{\text{Total number of relevant documents for the query}}$$

Both metrics treat relevance as binary: a document is either relevant or not. If our evaluation dataset uses graded relevance scores (e.g., 0 to 3), we first binarize by applying a threshold (e.g., treat score >1 as relevant).

In practice, we often care more about recall than precision for the first stage of RAG. If relevant information is missing entirely from the retrieved set (low recall), the generator has no way to produce a correct answer. Including some irrelevant content alongside relevant content is less harmful than missing the relevant content entirely, though as we discuss in "Mean Reciprocal Rank", position within the context still matters (see the "lost in the middle" effect). In early retrieval stages, ensuring that relevant content is included is far more important than excluding noise.

TIP

When in doubt about which retrieval metric to start with, use Recall@*k* for the first retrieval stage and NDCG@*k* for the reranking stage. Use Recall to check whether the right documents made it into the candidate set; use NDCG to check whether they ended up at the top.

如果不确定从哪个检索指标开始，可以在第一次检索阶段使用 Recall@k 指标，而在重新排序阶段则使用 NDCG@k 指标。使用 Recall 指标来确认哪些文档确实进入了候选集；使用 NDCG 指标则来判断这些文档是否真的排在了最顶端。

Mean Reciprocal Rank 倒数排名

Mean reciprocal rank (MRR) measures how early the first relevant document appears in the ranking. For a single query, the reciprocal rank is $1/\mathbf{rank}$, where rank is the position of the first relevant document. If no relevant document appears in the top *k* results, the reciprocal rank is 0. MRR is the average across all queries *Q* in the evaluation set, where $\mathbf{rank}_q$ is the position of the first relevant document for query *q* (or infinity, yielding 0, if no relevant document appears in the top *k*):

平均相关文档的排名指标（Mean Reciprocal Rank，MRR）用于衡量在排名中第一个相关文档出现的时间。对于单个查询，相关排名的值为 $1/\mathbf{rank}$ ，其中 rank 表示第一个相关文档在排名中的位置。如果前 k 个结果中没有任何相关文档，那么相关排名的值为 0。MRR 是评估集中所有查询的平均值，其中 $\mathbf{rank}_q$ 表示查询 q 中第一个相关文档在排名中的位置（如果前 k 个结果中没有任何相关文档，则取值为无穷大，即 0）。

$$\mathbf{MRR@}k = \frac{1}{|Q|} \sum_{q \in Q} \frac{1}{\mathbf{rank}_q} \quad \mathbf{where}\ \mathbf{rank}_q \leq k$$

A simpler alternative is *hit rate@k* (also called Success@*k* in some contexts): the fraction of queries for which at least one relevant document appears in the top *k* results. Hit rate is often the most practical starting metric for retrieval, because in many applications one relevant document is enough to produce a correct answer.

一个更简单的替代方法是命中率@k（在某些情况下也被称为成功率@k）：指那些至少有一个相关文档出现在前 k 个结果中的查询比例。命中率通常是最实用的检索指标，因为在许多应用中，一个相关文档就足以得到正确的答案。

MRR is especially useful when only one key fact is required to answer a query. Even though the LLM may process multiple retrieved chunks, ranking the correct one first can reduce noise and simplify reasoning. The reason is that LLMs do not attend to their context uniformly. N. F. Liu et al. (2024) ran a multi-document question-answering ex-

periment where the answer was hidden among 20 retrieved documents, and measured accuracy as a function of the answer's position. They found a U-shaped curve: accuracy was highest when the answer was in position 1 or position 20, and dropped by more than 30 percentage points when the answer was in position 10 (the middle). They called this the *lost in the middle* effect. The implication for RAG is that where a relevant document lands in the retrieval order matters as much as whether it is retrieved at all, which is why MRR (rewarding the top position) is still useful even when we have multiple relevant chunks. For *multi-hop* questions, meaning questions whose answer requires combining facts from two or more retrieved chunks (e.g., "Which of our West Berkeley listings has the highest property tax?" requires both the list of West Berkeley listings and the tax information for each), Recall@*k* or NDCG@*k* is more informative than MRR.

当只需要一个关键信息来回答某个问题时，MRR 特别有用。尽管大语言模型可能会处理多个检索到的信息块，但先对它们进行排序可以消除冗余，简化推理过程。原因是大语言模型并不一定会均匀考虑上下文中的信息。N. F. Liu 等人于 2024 年进行了一项多文档问答实验，结果显示答案隐藏在 20 个检索到的文档中，他们测量了准确性与答案位置之间的关系。结果发现，当答案位于第 1 位或第 20 位时，准确性最高；而当答案位于第 10 位时，准确性则下降了 30 个百分点以上。他们将这种现象称为"中间位置的缺失效应"。这对于 RAG 来说意味着，在检索顺序中，相关文档的位置与是否能被检索到同样重要。因此，即使有多个相关文档，MRR 仍然很有用，因为它能够奖励那些位于顶部位置的文档。块状处理。所谓多跳问题，指的是那些需要结合来自两个或多个数据块的信息来回答的问题（例如："在我们的西伯克利德列表中，哪一家房产的房产税最高？"这个问题就需要同时考虑西伯克利德列表以及每栋房产的税务信息）。Recall@k 或 NDCG@k 比 MRR 更具备信息价值。

Normalized Discounted Cumulative Gain (NDCG@k)
标准化折现累积收益（NDCG@k）

Precision, Recall, and MRR all treat relevance as binary: a document is either relevant or not. But in practice, some documents are more useful than others. For a query like "Summarize market trends for luxury condos downtown," a recent market analysis report is highly relevant, a general city economic overview is somewhat relevant, and a restaurant guide is irrelevant. *Graded relevance* captures these distinctions by assigning each document a score on a scale, such as 0 (irrelevant), 1 (marginally relevant), 2 (rel-

evant), or 3 (highly relevant).

在精确性、召回率以及 MRR 模型中，所有系统都将相关性视为二元判断：文档要么是相关的，要么就无关。但实际上，有些文档比其他文档更有用。例如，对于“总结市中心豪华公寓的市场趋势”这一查询，最新的市场分析报告显然非常相关；一般的城市经济概况则有一定相关性，而餐厅指南则毫无用处。分级相关性机制通过给每个文档分配一个评分来体现这种差异，评分范围可以是 0（无关）、1（略微相关）、2（相关）或 3（非常相关）。

NDCG@*k* is designed for this setting. It answers the question: *how well does our ranking place the most relevant documents first?* The metric is built on two intuitions:

NDCG@k 正是为这种场景而设计的。它的作用在于回答这样一个问题：我们的排序方式究竟能将哪些最相关的文档排在最前面呢？这个评估指标基于两个直觉原理来构建：

> *More relevant documents should contribute more to the score.*
>
> *更相关的文档应该能更好地提升评分。*
>
> > A document with relevance 3 matters more than one with relevance 1.
> >
> > 与相关性为 3 的文档相比，与相关性为 1 的文档更为重要。

> *Position matters: earlier is better.*
>
> *时间很重要：越早开始越好。*
>
> > A highly relevant document at rank 1 is more valuable than the same document at rank 5, because users (and LLMs) tend to focus on what comes first.
> >
> > 等级为 1 的非常相关的文档，其价值远高于等级为 5 的相同文档。因为用户（以及大型语言模型）往往更关注最突出的信息。

The formula captures both ideas. For each of the top *k* documents, we take its relevance score and divide by the logarithm of one plus its position. The logarithmic discount means the first few positions matter a lot, while later positions contribute less and less:

这个公式同时包含了这两个概念。对于排名前 k 的文档中的每一个，我们取其相关性得分，然后将其除以 1 加上其排名对数的结果。这种对数折扣机制意味着，排名靠前的几个位置非常重要，而后面位置的贡献则越来越小。

$$\mathrm{DCG@}k = \sum_{i=1}^{k} \frac{\mathrm{rel}_i}{\log_2(i+1)}$$

Here, rel_i is the graded relevance score of the document at rank *i*. The $\log_2$ in the denominator is what creates the position discount: rank 1 divides by $\log_2(2) = 1$ (no discount), rank 2 divides by $\log_2(3) \approx 1.58$, rank 4 divides by $\log_2(5) \approx 2.32$, and so on. Figure 7-3 shows how this plays out across rank positions. A document at rank 1 contributes its full relevance score to DCG. A document at rank 2 contributes about 63% as much, and a document at rank 9 contributes only about 30%. Highly relevant documents add a lot more to the total score when they sit near the top.

在这里，rel_i 表示文档的相关性评分，其排名为第 . 分母中的 $\log_2$ 代表了位置折扣的计算依据：排名为第 1 位的文档，其评分直接计入总得分；排名为第 2 位的文档，其贡献约为总得分的 63%；而排名为第 9 位的文档，其贡献则仅为总得分的 30%左右。显然，相关性极高的文档在排名靠前的位置时，对总得分的贡献要大得多。图 7-3 展示了这一机制在各个排名位置上的体现。

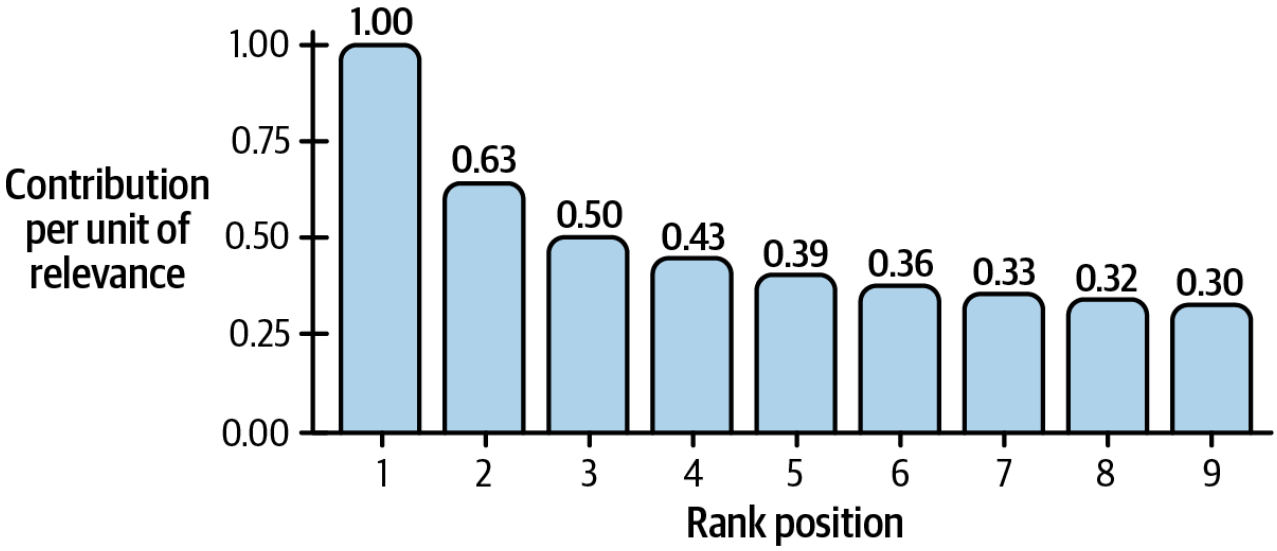


Figure 7-3. The DCG position discount

图 7-3. DCG 位置的折扣情况

DCG alone is hard to interpret because its value depends on how many relevant documents exist and how relevant they are. To normalize, we compute the *Ideal* DCG (IDCG@k): we take the same retrieved documents, re-sort them in the best possible order (highest relevance first), and compute DCG on that ideal ranking. NDCG@*k* is then the ratio:

仅 DCG 这一指标就难以解释，因为其数值取决于有多少相关文档存在，以及这些文档的相关性如何。为了使其具有规范性，我们计算理想 DCG（IDCG@k）：选取相同的文档集合，然后按照最佳排序顺序对其进行重新排序（从相关性最高的文档开始），再计算该理想排序下的 DCG 值。NDCG@k 则是一个比率值，用于表示相对重要性。

$$\mathrm{NDCG@}k = \frac{\mathrm{DCG@}k}{\mathrm{IDCG@}k}$$

A score of 1.0 means our system ranked its documents perfectly. A score of 0.5 means it captured about half the value it could have achieved with better ordering. This normalization makes scores comparable across queries, even when different queries have different numbers of relevant documents.

得分为 1.0 意味着我们的系统完美地排序了文档。而得分为 0.5 则意味着系统只处理了大约一半的潜在价值。这种标准化处理使得不同查询的得分具有可比性，即使不同查询相关的文档数量各不相同。

Why does NDCG@*k* matter for RAG? Two reasons:

为什么 NDCG@k 对 RAG 来说很重要呢？有两个原因：

Not all chunks are equally useful.

并非所有的部分都同样有用。

For a query like “Summarize market trends for luxury condos downtown,” a recent market analysis report is highly relevant, while a general city economic overview is only marginally relevant. NDCG@*k* rewards systems that rank the market report first.

对于像“总结市中心豪华公寓的市场趋势”这样的查询，最近发布的市场分析报告非常相关；而一般的城市经济概况则相关性较弱。NDCG@k 的评分系统将市场报告排在首位。

Context windows are finite.

上下文窗口的数量是有限的。

If the total size of retrieved documents exceeds the LLM’s context limit, only the first few will be used. NDCG@*k* ensures the most relevant documents are prioritized.

如果检索到的文档总大小超过大语言模型的上下文限制，那么只有前几个文档会被使用。NDCG@k 算法能够确保优先处理最相关的文档。

One caveat to keep in mind is that NDCG@*k* can produce counterintuitive scores. A system that retrieves only marginally relevant documents but ranks them in perfect order may score highly, while a system that retrieves a highly relevant document but ranks it lower may score worse. Always interpret NDCG@*k* alongside recall to get the full pic-

ture.

需要注意的是，NDCG@k 可能会产生一些违反直觉的评分结果。一个只检索到稍微相关的文档，但将这些文档按完美顺序排序的系统，可能会得到较高的评分；而一个虽然能检索到非常相关的文档，但却将其排名较低的系统，则可能得到较低的评分。因此，在评估 NDCG@k 时，应该结合召回率来整体判断其表现。

Evaluating and Optimizing Chunking Strategies

评估和优化分块策略

Before our retriever can search for relevant content, we divide our documents into smaller pieces called *chunks*. A chunk is the unit of text the retriever can return. If a chunk is too small, a single relevant idea might be split across two chunks, and the retriever may only find one half. If a chunk is too large, it may contain mostly irrelevant text that dilutes the useful content and wastes tokens in the LLM's context window.

在检索器能够搜索到相关内容之前，我们会将文档分割成更小的单元，这些单元被称为“块”。一个块是检索器能够返回的文本单位。如果块太小，那么一个相关的想法可能会被分配到两个不同的块中，导致检索器只能找到其中的一半内容。而如果块太大，那么其中可能包含大量无关内容，这会稀释有用的信息，同时也会浪费 LLM 上下文窗口中的资源。

The simplest approach is *fixed-size chunking*: we split each document into consecutive segments of a fixed number of tokens (e.g., 256 tokens each). To avoid cutting sentences or ideas in half at chunk boundaries, we can add *overlap*, where consecutive chunks share some tokens. For example, with a chunk size of 256 and an overlap of 128, the first chunk covers tokens 1–256, the second covers tokens 129–384, and so on. The overlap ensures that content near a boundary appears in at least two chunks, giving the retriever a better chance of finding it. Figure 7-4 illustrates this layout.

最简单的方法就是采用固定大小的块化策略：我们将每个文档分割成若干个固定数量的标记段（例如，每个段包含 256 个标记）。为了避免在块边界处将句子或想法分割开来，我们可以引入重叠机制，即连续的几个段会共享一些标记。例如，当块大小为 256 个标记，重叠量为 128 个标记时，第一个段包含标记 1 到 256，第二个段包含标记 129 到 384，依此类推。这种重叠机制确保靠近边界的内容至少出现在两个段中，从而增加了检索器找到这些内容的机会。图 7-4 展示了这种布局方式。

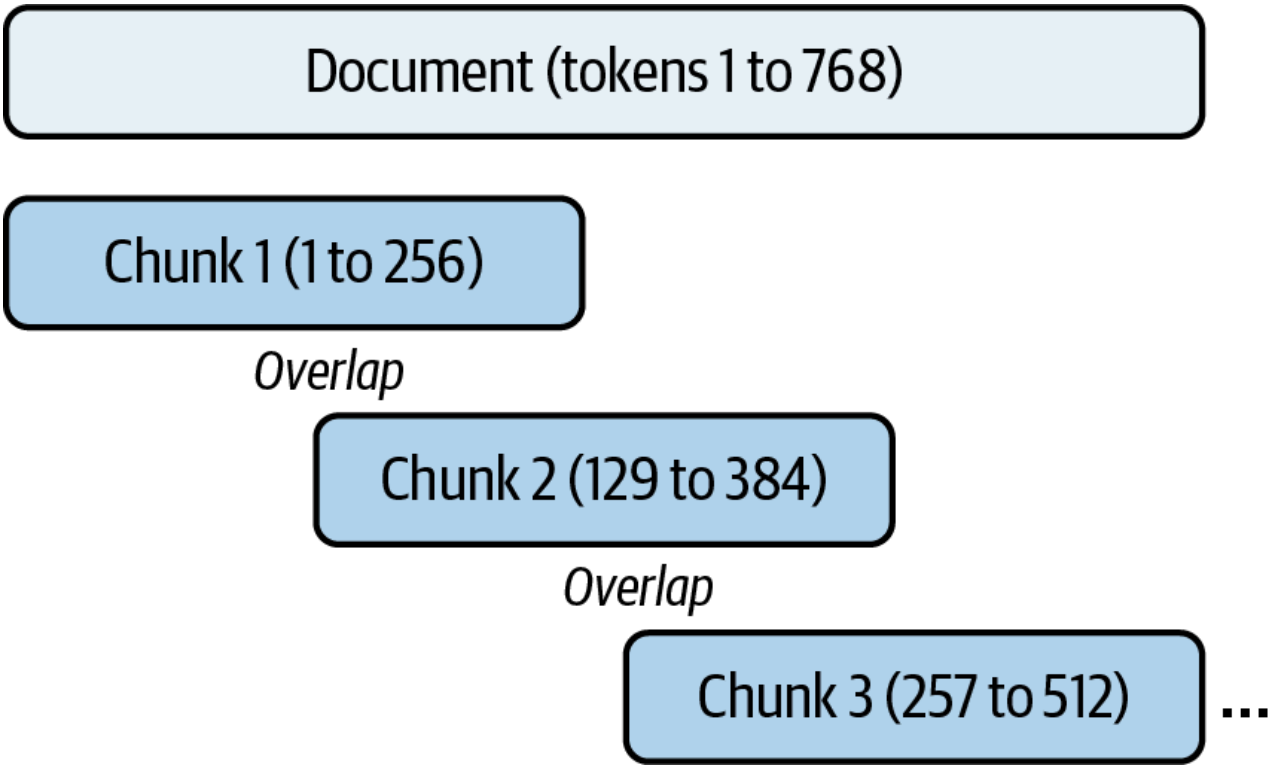


Figure 7-4. Fixed-size chunking with overlap

图 7-4. 带重叠的固定大小数据块分割方式

Tuning Chunk Size and Overlap

调整块大小和重叠程度

The right chunk size and overlap depend on the structure of our documents and the kinds of questions users ask. There is no single best setting, so we try different combinations and measure which one gives the best retrieval results.

合适的 chunk 大小和重叠程度取决于我们的文档结构以及用户提出的问题类型。并没有一种最佳的设置方案，因此我们会尝试不同的组合，然后衡量哪种组合能带来最佳的检索效果。

A practical approach is a *grid search*: we pick a few candidate values for chunk size and overlap, reindex the document corpus for each combination, and measure retrieval metrics on our evaluation dataset. Table 7-1 shows a hypothetical example of what these results might look like. Each row is a different configuration, evaluated on the same set of queries. The best-performing configuration is bolded.

一种实用的方法就是网格搜索：我们选取几个不同的块大小候选值，针对每个组合情况重新索引文档集，然后在评估数据集上测量检索性能指标。表 7-1 展示了一个假设的例子，展示了这些结果的样子。每一行代表一种不同的配置，这些配置都在相同的查询集上进行评估。表现最好的配置被加粗显示。

Table 7-1. Hypothetical grid search results for chunking parameters

表 7-1. 分块参数下的虚拟网格搜索结果

Chunk size (tokens) 块大小（令牌数）	Overlap (tokens) 重叠（代币）	Recall@5 召回@5	NDCG@5
128	0	0.82	0.69
128	64	0.88	0.75
256	0	0.86	0.74
256	**128**	**0.89**	**0.77**
512	0	0.80	0.72
512	256	0.83	0.74

In this example, 256 tokens with 50% overlap (128 tokens) yields the best retrieval performance. Notice that adding overlap consistently improves results across all chunk sizes, because it reduces the chance of splitting relevant content across chunk boundaries. Also notice that very large chunks (512 tokens) actually perform worse than medium-sized ones (256 tokens), likely because they include too much irrelevant content alongside the relevant passages.

在这个示例中，当使用 256 个标记时，如果它们之间的重叠率达到 50%，那么检索性能将会达到最佳。需要注意的是，增加重叠程度能够持续改善所有块大小下的检索效果，因为这可以减少相关内容在不同块边界处被分割的风险。此外，还需要注意到，非常大的块（512 个标记）实际上比中等大小的块（256 个标记）表现更差，这可能是因为较大的块包含了太多无关内容，而相关内容却相对较少。

The specific values in this table are illustrative. In practice, the best configuration depends on your documents and queries. The key takeaway is that chunking is a tunable parameter that can significantly affect retrieval quality, and a systematic search can make a large difference.

表中的具体数值仅供参考。实际上，最佳配置取决于你的文档和查询类型。关键点是，分块处理是一个可调节的参数，它会对检索质量产生显著影响；而采用系统化的搜索方式则能带来巨大的差异。

Other Chunking Approaches

其他分组方法

Fixed-size chunking is the simplest strategy and a reasonable starting point, but it is one of many options. The right approach depends on the structure of your documents, and it is common to try several and compare them using the retrieval metrics from the previous section:

固定大小的块化是最简单的策略，也是一个合理的起点。不过，这只是一个众多选择中的一种。正确的方法取决于你的文档结构。通常，需要尝试几种方法，并通过上一节中提到的检索指标来比较它们的效果。

Semantic or structural chunking

语义或结构性的分组处理

Instead of splitting at a fixed token count, we can split at natural boundaries in the document, such as section headings, paragraph breaks, or topic shifts. This keeps semantically related content in the same chunk. For example, if a report has sections on “Market Trends” and “Pricing,” a structural chunking strategy puts each section in its own chunk, regardless of how many tokens it contains. The chunks will vary in size, but each one will be about a single coherent topic.

我们不必在固定的标记数时进行分割，而是可以在文档中的自然边界处进行分割，比如章节标题、段落划分或主题转换点。这样就能将语义相关的内容放在同一个区块中。例如，如果一份报告包含“市场趋势”和“定价”两个章节，那么无论这些章节包含多少标记，它们都会被放在不同的区块中。这些区块的大小可以不同，但每个区块都只涵盖一个相关的主题。

Contextual augmentation 情境增强

We can enrich each chunk with surrounding context before embedding it. For example, we might prepend the document's title and section headings to a chunk containing “Table 2.” This makes the chunk more self-contained and gives the retriever stronger signals about what it contains.

我们可以在嵌入每个数据块之前，为其添加周围的上下文信息。例如，我们可以在包含“表 2”的数据块前加上文档的标题和章节标题。这样就能让数据块更加独立，同时也能为检索器提供更明确的指示，使其能够更准确地识别数据块中包含的内容。

Evaluating Generation Quality

评估生成质量

With a well-performing retrieval system in place, we turn to evaluating what the LLM does with the retrieved context. Even if retrieval is perfect, the generation step can still fail. We evaluate generation along two dimensions.

在拥有性能良好的检索系统之后，接下来就是评估大语言模型如何利用所检索到的上下文信息了。即使检索过程非常完美，生成步骤仍然可能出现问题。我们是从两个维度来评估生成的效果的。

The first is *faithfulness*: does the output accurately reflect the information in the retrieved context? In RAG applications, we expect the answer to be grounded in what was retrieved, not in the LLM's own knowledge. Even if the LLM "knows" the right answer from its pretraining data (sometimes called its *parametric knowledge*, meaning the information baked into the model's weights), we want it to base its response on the evidence we provided. Faithfulness failures come in three forms:

首先是准确性问题：输出结果是否能够准确反映所检索到的上下文中的信息呢？在 RAG 应用中，我们期望答案基于所检索到的信息，而不是基于大语言模型自身的知识。即使大语言模型能够从其预训练数据中"知道"正确的答案（有时被称为参数化知识，即模型权重中蕴含的信息），我们也希望它能够根据我们提供的证据来做出回应。准确性方面的缺陷有三种形式：

Hallucinations 幻觉/妄想

The output includes information that is not in the retrieved documents.

输出结果中包含了一些在检索到的文档中不存在的信息。

Omissions 遗漏/省略

The output ignores relevant information that *is* in the retrieved documents.

该输出结果忽略了从文档中提取出的相关信息。

Misinterpretations 误解

The output distorts or inaccurately represents information from the retrieved documents.

这种输出结果会歪曲或错误地呈现从检索到的文档中获取的信息。

The second dimension is *relevance*: does the answer address what the user actually asked? An answer can be perfectly faithful to the context but still miss the point. For example, if the user asks "What are the HOA fees for this property?" and the retrieved context contains both HOA fees and tax information, a response that discusses taxes instead of HOA fees is faithful but not relevant.

第二个维度是相关性：回答是否能够解决用户实际提出的问题？一个回答虽然能够完全符合上下文信息，但仍然可能忽略重点。例如，如果用户问："这栋房子的业主协会费用是多少？"而提供的信息中同时包含了业主协会费用和其他税务信息，那么一个只讨论税务而不提及业主协会费用的回答虽然信息准确，但却是无关紧要的。

We build evaluators for both dimensions using the techniques from Chapter 5. For faithfulness, an LLM-as-judge works well: the judge receives both the retrieved context and the generated answer, then checks whether every claim in the answer is supported by the context. For relevance, simpler checks can sometimes suffice. A programmatic evaluator might verify that the answer mentions the specific entity or topic from the query, though for open-ended queries an LLM-as-judge is usually needed.

我们使用第 5 章的技术来构建两个维度的评估器。对于忠实度，一个作为法官的大型语言模型效果很好：法官会收到检索到的上下文和生成的答案，然后检查答案中的每个声明是否都得到了上下文的支持。对于相关性，有时简单的检查就足够了。一个程序化评估器可能会验证答案是否提到了查询中的特定实体或主题，但对于开放式查询，通常需要一个作为法官的大型语言模型。

A useful technique for debugging faithfulness issues is *context swapping*. We replace the retrieved documents with unrelated ones from a different topic, then run the same query through the pipeline. If the answer stays roughly the same, the LLM is ignoring the retrieved context and relying on its parametric knowledge instead. This is a grounding problem, and it can be subtle because the answer may still happen to be correct. The risk is that when the LLM *does not* already know the answer, it will hallucinate rather than use the retrieved evidence. Context swapping is not a metric to run at scale, but it is a valuable debugging tool for understanding whether our pipeline is actually using its retrieval results.

用于调试忠实度问题的一种有效方法是上下文替换。我们用与查询无关的其他文档替换掉检索到的文档，然后再次执行相同的查询。如果查询结果大致保持不变，那就说明 LLM 正在忽略检索到的上下文信息，而依赖其参数化知识来做出判断。这其实是一个基础性问题，而且可能很微妙，因为答案有时可能仍然是正确的。风险在于，当 LLM 还不知道答案时，它可能会产生幻觉，而不是使用检索到的信息。虽然上下文替换不是一种可以大规模应用的工具，但它是一种有价值的调试手段，可以帮助我们了解我们的处理流程是否真的使用了检索到的信息。

NOTE 注意/备注

If a context-swap test shows the LLM ignoring retrieved documents, the fix is usually in the prompt, not the retriever. Adding an explicit instruction like “Base your answer only on the provided documents” or “If the documents do not contain the answer, say so” often resolves grounding problems.

如果上下文交换测试表明大语言模型忽略了某些文档，那么问题通常出在提示语上，而不是检索器本身。添加一些明确的指示，比如“请仅基于提供的文档来回答”或者“如果文档中不包含答案，请说明这一点”，通常可以解决这个问题。

Common Pitfalls 常见陷阱

Several pitfalls recur when evaluating RAG pipelines:

在评估 RAG 管道时，会出现一些常见的问题：

Relying only on end-to-end metrics

仅依赖于端到端指标进行评估

A single correctness score does not tell us whether retrieval or generation is to blame. To isolate failures, we measure each component independently: Recall@*k* for retrieval, faithfulness for generation.

单一的准确率并不能告诉我们是检索过程出了问题，还是生成过程的问题。为了准确找出问题所在，我们分别对每个组件进行衡量：检索方面的召回率@k，以及生成方面的准确性。

Overfitting to synthetic evaluation data

过度拟合于合成评估数据

While synthetic data is useful for bootstrapping, it often fails to capture the ambiguity and diversity of real user queries. Many generated questions are too easy or too tightly coupled to specific wording in source documents. Worse, the designated “gold” chunk may not be unique or even optimal. We regularly validate on real queries from logs or human-curated examples. Filtering synthetic data helps, but it does not fully substitute for real-world test sets. Synthetic queries tend to skew toward extractive answers and miss multi-hop or judgment-based

questions.

虽然合成数据在自举法测试中很有用，但它往往无法捕捉到真实用户查询的复杂性和多样性。许多生成的题目过于简单，或者与源文档中的特定措辞过于紧密地关联。更糟糕的是，那些所谓的“优质”题目可能并不具有独特性，甚至可能不是最优选择。我们通常会对来自日志记录或人工筛选的示例数据进行验证。过滤掉合成数据确实有帮助，但它无法完全替代真实的测试集。合成题目往往倾向于给出简单的答案，而忽略了那些需要多层次推理或基于判断的题目。

WARNING 警告

Synthetic QA pairs often mirror the exact vocabulary of the source chunk, making retrieval artificially easy. If your Recall@*k* looks great on synthetic data but users still get bad answers, supplement with 20 to 30 real user queries from logs before drawing conclusions.

合成 QA 对通常会完全复制原始文本中的词汇内容，从而让检索变得极其容易。如果某种召回率指标在合成数据上表现良好，但用户的实际表现却很差，那么请先参考 20 到 30 个真实用户的查询记录，然后再做出判断吧。

Choosing the wrong metric

选择错误的衡量标准

If the LLM needs a single fact, MRR may suffice. If it needs to synthesize multiple chunks, Recall@*k* or NDCG@*k* is a better fit. Choosing the wrong metric leads us to optimize for behaviors that do not actually improve output quality.

如果大语言模型只需要一个事实信息，那么 MRR 指标就足够了。但如果需要综合多个事实来生成完整内容，那么 Recall@k 或 NDCG@k 这样的指标更为合适。如果选择错误的评估指标，就会导致我们优化那些实际上并不能提升输出质量的行为。

Ignoring chunking as a variable

将"chunking"作为一个变量进行忽略处理

Chunks that are too small scatter relevant information. Chunks that are too large contain irrelevant noise. Retrieval metrics like Recall@*k* can vary significantly based on chunking alone, even with the same retriever model.

过于小的数据块会分散相关信息；而过于大的数据块则包含大量无关噪声。仅根据数据块的划分方式，检索指标如 Recall@k 的值就会有很大差异，即使使用相同的检索模型也是如此。

Evaluating generation without checking grounding

在不进行接地检查的情况下进行发电评估

> Fluency and overlap with a reference answer are not enough. A response that sounds correct but contradicts the retrieved evidence undermines the entire purpose of RAG. We need context-sensitive metrics that verify whether the generation aligns with what was retrieved.
>
> 仅仅具备流畅性和与参考答案的重叠是不够的。一个听起来正确的回答，但如果与检索到的信息相矛盾，那么这种回答就违背了 RAG 系统的初衷。我们需要一些基于上下文的评估指标，以验证生成的答案是否与检索到的信息一致。

Not separating retrieval correctness from answer correctness

不要将检索的正确性与答案的正确性分开考虑。

> A common and valuable diagnostic is to check whether retrieval succeeded but the answer was still wrong, or whether retrieval failed but the answer was coincidentally right. Separating these two cases tells you whether your bottleneck is in retrieval or in generation, which determines where to invest improvement effort.
>
> 一种常见且宝贵的诊断方法是检查检索是否成功但答案仍然错误，或者检索是否失败但答案恰好正确。区分这两种情况可以告诉你瓶颈是在检索还是生成，这决定了你应该在哪里投入改进的努力。

Putting It Together 整合起来

Here is a workflow for evaluating RAG pipelines:

这里有一个评估 RAG 管道的流程：

1. Build a dataset of queries with labeled gold chunks (the documents that should be retrieved for each query).

   构建一个包含标记黄金块的查询数据集（每个查询应该检索的文档）。

2. Measure retrieval quality using hit rate@*k* or Recall@*k* for coverage, and MRR or NDCG@*k* for ranking quality.

   使用 hit rate@k 或 Recall@k 来衡量检索质量（覆盖范围），以及使用 MRR 或 NDCG@k 来衡量排名质量。

3. Separately evaluate generation for faithfulness (does the answer follow from the retrieved content?) and relevance (does it address the user's question?).

   分别评估生成内容的可信度（答案是否能够基于检索到的内容得出？）以及相关性（它是否能够回答用户的疑问？）。

4. Diagnose failures by separating retrieval errors from generation errors: did the system retrieve the right documents but produce a wrong answer, or did it retrieve the wrong documents?

   通过区分检索错误和生成错误来诊断故障：系统是检索到了正确的文档，但给出了错误的答案吗？还是说系统检索到了错误的文档？

5. Treat chunking strategy as a variable: experiment with different chunk sizes, overlap, and structural boundaries, and measure how they affect retrieval metrics.

   将分块策略视为一个变量：尝试不同的分块大小、重叠和结构边界，并衡量它们对检索指标的影响。

In the next chapter, we shift to more flexible architectures where LLMs operate in loops, construct intermediate reasoning steps, or issue tool calls dynamically. These agentic systems introduce new challenges for evaluation, especially around attribution, planning accuracy, and tool use correctness.

在下一章中，我们转向更灵活的架构，其中大型语言模型在循环中运行，构建中间推理步骤或动态发出工具调用。这些自主系统为评估带来了新的挑战，特别是在归因、规划准确性和工具使用正确性方面。

[1] In some systems, these stages are executed agentically: an LLM agent proposes queries or reranking strategies, observes retrieval results, and iteratively refines its actions in a loop.

在某些系统中，这些阶段是自主执行的：一个大型语言模型代理提出查询或重新排序策略，观察检索结果，并在循环中迭代地完善其行为。

Part IV. Production and Improvement

第四部分：生产与发展

Chapter 9. Continuous Integration and Deployment for LLM Agents

第 9 章 大型语言模型代理的持续集成与部署

In July 2023, researchers at Stanford and Berkeley published a study tracking GPT-4's behavior over time (L. Chen et al. 2024). They ran the same math tasks on the GPT-4 API in March 2023 and again in June 2023, and found that accuracy on identifying prime numbers dropped from 84% to 51%. OpenAI had updated the model behind the `gpt-4` API endpoint without any announcement. Teams that had been calling `gpt-4` in production woke up one morning with a model that behaved differently on their tasks, and they had no way of knowing until users complained.

在 2023 年 7 月，斯坦福大学和伯克利大学的研究人员发表了一项研究，旨在追踪 GPT-4 随时间的性能变化（L. Chen 等人，2024 年）。他们在 2023 年 3 月和 6 月分别使用 GPT-4 接口进行了同样的数学任务测试，结果发现，识别质数正确的准确率从 84%下降到了 51%。OpenAI 在没有任何公告的情况下更新了该 `gpt-4` 接口背后的模型。那些一直在使用 `gpt-4` 接口的团队，某天早上发现模型在任务执行时的表现发生了改变，但他们直到用户提出投诉后才意识到这一点。

This is the kind of problem that continuous evaluation is designed to catch. Getting an LLM agent to work well once is hard enough. Keeping it working over time is a different challenge entirely. User needs change, data distributions shift, and model providers push updates without warning. A nightly regression suite that reevaluated our prompts against a fixed set of test cases would have caught that accuracy drop before it reached production.

这就是持续评估旨在解决的问题类型。让大型语言模型能够稳定运行已经相当困难了，而长时间保持其运行则是一个完全不同的挑战。用户需求会不断变化，数据分布也会发生变化，而且模型提供商也会随时推出新的更新版本。如果每周都重新评估我们的提示参数，并在固定的测试案例下进行验证，那么就能在准确率下降之前及时发现问题，从而避免实际应用中出现问题。

In this chapter, we cover how to build that kind of safety net. We introduce two practices borrowed from software engineering: *continuous integration* (CI) and *continuous deployment* (CD). By the end, you will understand:

在本章中，我们将介绍如何构建这样的安全机制。我们会从软件工程中借鉴两种做法：持续集成和持续部署。通过本章的学习，你将理解如何实施这些做法。

- How to build a *golden dataset* of curated test cases that catches regressions before they ship

  如何构建一份包含精心挑选的测试案例的黄金数据集，以便在软件发布之前发现潜在的回归问题。

- How to monitor real-world performance in production using the evaluators from Chapter 5

  如何使用第 5 章中的评估工具来监控生产环境中的实际性能

- How CI and CD connect into a continuous improvement loop that keeps your agent healthy over time

  如何使客户体验管理和客户满意度评估成为持续改进循环的一部分，从而长期保持代理人的良好状态？

- When and how to revisit your prompts as foundation models improve

  当基础模型不断改进时，何时以及如何重新使用这些提示作为模型的基础。

Adapting CI/CD for LLM systems is not a straightforward translation from traditional software or even from earlier MLOps. LLM agents have unique properties that change how we test and monitor them. Table 9-1 outlines two key concepts that organize the rest of this chapter.

将 CI/CD 流程适配到 LLM 系统中并非易事，这远远超出了传统软件或早期机器学习运维系统的能力范围。LLM 代理具有独特的特性，这改变了我们测试和监控这些系统的方式。表 9-1 列出了两个关键概念，它们有助于理解本章后续内容的组织方式。

Table 9-1. Two roles of continuous evaluation for LLM agents

表 9-1. 连续评估在 LLM 代理中的两种作用

Role 角色	What it does 它做什么	Example 示例
CI: Unit tests for known unknowns CI：对已知未知项进行的单元测试	Guards against regressions on failure modes we have already identified. Runs before deployment. 我们已经针对那些已识别出的故障模式制定了防衰退措施。在部署之前进行测试吧。	After editing the prompt for our real estate assistant (Example 3-1), CI checks confirm that the "Location Ambiguity" bug we fixed last month has not reappeared. 在修改了房地产助理的提示后（参见示例 3-1），CI 检查确认我们上个月修复的"位置歧义"问题已不再出现。
CD: Online monitoring for unknown unknowns CD：对未知之未知进行在线监控	Tracks system behavior on live traffic. Surfaces new failure modes we did not anticipate during development. 在实时流量环境下测试系统行为。发现了在开发过程中未曾预料到的新故障模式。	A new neighborhood-targeting feature causes the LLM to confuse "West Berkeley" with "Berkeley West" (a nonexistent area). Online monitoring detects the spike in location errors before too many users are affected. 一个新的针对特定社区的功能导致大语言模型将"West Berkeley"误认为是"Berkeley West"（一个不存在的地区）。在线监控系统在用户受到过多影响之前，及时发现了这一位置识别错误的情况。

These two roles are complementary. We use CI to catch what we know about, and CD to catch what we do not yet know about. Neither is sufficient alone. Together they form a loop (Figure 9-1). Each code change runs through golden dataset tests and LLM judge evaluations in parallel, and it must pass both at a deploy gate before it ships. In production, monitors continuously sample and evaluate traffic, and new failure modes feed back into the golden set for the next cycle.

这两种角色是互补的。我们使用 CI 来捕获我们已经了解的信息，而使用 CD 来捕获我们还不知道的信息。单独来看，任何一种角色都不足以完成全部任务。它们共同构成了一个循环（见图 9-1）。每次代码变更都会经过黄金数据集测试以及 LLM 评估，并且必须在部署阶段通过这两项测试后才能投入使用。在实际生产环境中，监控器会持续地抽样并评估流量情况，而新的故障模式会被反馈到黄金数据集中，以便下一轮测试时使用。

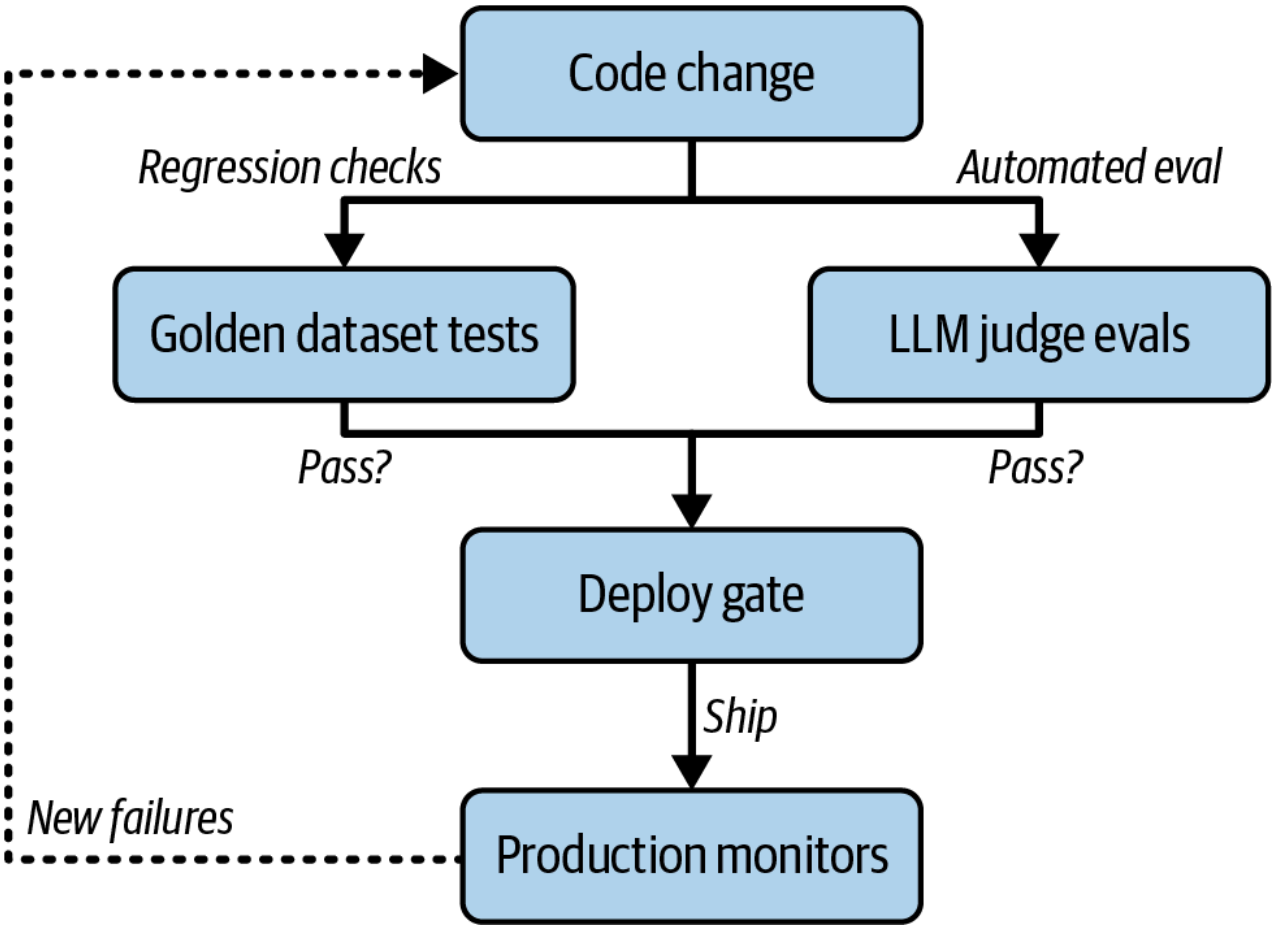


Figure 9-1. The CI/CD pipeline for LLM agents

图 9-1. LLM 代理的 CI/CD 流水线

LLM agents also differ from traditional machine learning (ML) systems in ways that affect how we implement CI/CD. Table 9-2 outlines the key differences.

机器学习智能体与传统机器学习系统有所不同，这些差异影响了我们实施持续集成与持续交付流程的方式。表 9-2 列出了主要差异。

Table 9-2. How CI/CD differs between traditional ML and LLM-based agents

表 9-2. 传统机器学习代理与基于大语言模型的代理在 CI/CD 方面的差异

Aspect 方面/视角	Traditional ML 传统机器学习	LLM agents LLM 代理程序
Training data 训练数据	Large labeled datasets. CI validates against substantial test sets after retraining. 大型标记化数据集。在重新训练之后，CI 能够针对大量的测试集进行验证。	Few labeled examples, curated around known failure modes. Core logic lives in prompts and the foundation model, not in trained weights. 少量的标注样本，这些样本围绕已知的故障模式进行设计。核心逻辑存在于提示和基础模型中，而不是在训练出的权重中。
Failure modes 故障模式	Data drift, concept drift, feature engineering bugs. 数据漂移、概念漂移、特征工程中的错误。	Prompt sensitivity, silent model updates by providers, hallucinations, LLM-as-judge drift, multi-step reasoning errors. 快速响应能力、供应商无需人工干预的模型更新机制、幻觉现象、大语言模型作为裁判时的偏差问题、多步推理过程中的错误。
Measuring quality 衡量质量	Standard metrics (accuracy, F1, AUC) on well-defined labels. 在定义明确的标签上使用的标准指标（准确率、F1 值、AUC 值）。	Quality is subjective and multidimensional (tone, faithfulness, relevance). Requires custom evaluators, often LLM-as-judge, on sampled production traffic. 质量是个主观且多维的概念（包括风格、忠实度、相关性等方面）。因此需要使用专门的评估工具来进行评估，通常可以利用大型语言模型作为评估者，对样本化的生产成果进行评判。
Determinism 决定论	Same input produces same output for a fixed model version. 对于固定的模型版本来说，相同的输入会产生相同的输出。	Same input can produce different outputs (temperature > 0). CI tests must tolerate variation or run multiple times. 相同的输入可能会产生不同的输出（温度值可能大于 0）。置信度测试必须能够容忍这种差异，或者进行多次测试以获得准确结果。

CI: Building a Safety Net Against Regressions

CI：构建一道抵御回归问题的安全网

CI in software engineering aims to verify that code changes integrate correctly and do not introduce regressions. This goal applies equally to LLM agents. Traditional ML also has CI practices. However, while ML CI might measure overall accuracy against large test sets, LLM CI focuses narrowly on preventing regressions for known failure modes using curated golden datasets (Table 9-2). CI does not reliably predict overall production accuracy. Its purpose is to ensure stability as the agent evolves.

在软件工程中，CI 的目的是确保代码变更能够正确集成，不会引入新的问题或回归问题。这一目标同样适用于 LLM 代理。传统的机器学习方法也具备 CI 机制。不过，虽然机器学习中的 CI 可以通过大型测试集来衡量整体准确性，但 LLM 的 CI 则专注于利用精心挑选的基准数据集来预防已知的故障模式导致的回归问题（参见表 9-2）。CI 并不能可靠地预测生产环境中的整体准确性。它的作用在于确保随着代理的不断发展，系统能够保持稳定性。

The foundation of LLM CI is a *golden dataset*: a hand-curated set of input examples with corresponding reference outputs. We create this dataset through human review and labeling (Chapter 3), often using collaborative evaluation methods (Chapter 4) to agree on what "correct" looks like.

LLM CI 的基础是一个珍贵的数据集：这是一组经过人工筛选的输入示例，以及对应的参考输出。我们通过人工审核和标注来创建这个数据集（详见第 3 章），通常会采用协作式评估方法来确定什么是"正确的"答案（详见第 4 章）。

On each proposed change (a prompt tweak, a model swap, a new tool), we run all golden inputs through the agent and evaluate the outputs with automated evaluators (Chapter 5). This is similar to what the Applied LLMs guide calls "assertion-based unit tests": we take real inputs from production, define at least a few evaluation criteria for each, and check that the agent still meets them. When using LLM-as-judge evaluators in CI, pin the exact judge model version (e.g., gpt-4o-2024-05-13) to prevent CI re-

sults from fluctuating due to unannounced provider updates.

对于每一个拟议的修改（无论是简单的提示调整、模型替换，还是新工具的引入），我们都会将所有可能的输入参数通过智能体进行验证，并使用自动化评估工具来评价结果（详见第 5 章）。这类似于《应用型大语言模型指南》中所提到的“基于断言的单元测试”：我们采用实际生产中的输入数据，为每一个输入参数定义至少几个评估标准，然后检查智能体是否仍然满足这些标准。在持续集成过程中使用大语言模型作为评估工具时，需要固定特定的评估模型版本（例如 gpt-4o-2024-05-13 ），以避免由于供应商的未知更新而导致持续集成结果出现波动。

A well-constructed golden dataset should include:

一个构建良好的黄金数据集应该包含以下内容：

- Examples covering the agent's core features and capabilities

  涵盖代理核心功能和能力的示例

- Representative instances of critical failures observed in the past, acting directly as regression tests

  过去观察到的典型关键失败案例，它们实际上起到了回归测试的作用。

- Challenging edge cases uncovered during error analysis

  在错误分析过程中发现的具有挑战性的边缘情况

- Examples designed to exercise different components or paths within the agent (e.g., specific tool calls, RAG scenarios, different prompt branches)

  这些示例旨在锻炼智能体内部的不同组件或路径功能（例如，特定的工具调用、RAG 场景、不同的提示分支等）。

In terms of size, a mature golden dataset typically includes 100 to 200 examples representing core functionality, previously observed bugs, and known edge cases. It is not a random sample of production data. It is purpose-built to stress-test behaviors we want to preserve.

在规模方面，一个成熟的黄金数据集通常包含 100 到 200 个示例，这些示例代表了核心功能、之前发现的漏洞以及已知的边缘情况。这并非对生产数据的随机采样，而是专门为测试我们希望保持的行为而构建的。

CI evaluations on this dataset are reference-based: for each golden input, we check whether the new agent output aligns with the gold reference. As we discover new fail-

ure modes (e.g., via monitoring or error analysis), we expand the dataset accordingly.

在这个数据集上的 CI 评估是基于参考标准的：对于每一个有效的输入，我们会检查新代理的输出是否与参考结果一致。当我们发现新的故障模式时（例如，通过监控或错误分析），我们会相应地扩展数据集。

We illustrate these principles with CI checks from Example 3-1 in Table 9-3. If any of these checks fail on a corresponding golden example, the CI build should fail, preventing the potentially regressive code change from being merged.[1]

我们通过表 9-3 中的示例 3-1 中的 CI 检查来说明这些原则。如果相应的黄金示例中有任何一项检查失败，那么 CI 构建就应该失败，这样就能防止可能具有负面影响的代码修改被合并到代码中。[1]

Table 9-3. Examples of CI checks for a real estate assistant agent

表 9-3. 房地产助理代理人的置信度检验示例

Check 检查	Input 输入	Reference 参考文献/出处	CI evals CI 评估
Tone match (LLM judge) 语气匹配（语言模型评判）	Agent prompt and client persona 代理提示与客户角色描述	Human-verified, persona-aligned output 经过人工验证的、与真实身份一致的输出结果	LLM judge verifies tone matches expected persona behavior 法学硕士 judge 确认语调与预期的人物行为相符
Tool argument schema validation 工具参数模式验证	Query to fetch calendar availability 查询获取日历上的可用时间	Tool schema definition 工具模式定义	Code-based validator ensures arguments conform to schema 基于代码的验证器能够确保参数符合相关规范
Email address existence check 电子邮件地址的有效性检查	Query instructing an email to a client 查询以电子邮件方式通知客户	Known client email database 已知的客户电子邮件数据库	Code check confirms recipient address exists 代码检查确认接收地址是存在的

Maintaining the golden dataset is an ongoing process. As new failure modes or critical edge cases are discovered (often through online monitoring, discussed in the next section), they must be added to the golden set. Part of maintenance also includes remov-

ing or updating stale examples: if the application's behavior or data has changed, old golden examples may no longer exercise live code paths. Periodically audit the golden set to flag examples that no longer match the current system, and retire or replace them so the CI checks remain meaningful.

维护"黄金数据集"是一个持续的过程。当发现新的故障模式或关键情况时（这些情况通常是通过在线监控发现的，具体细节将在下一节中讨论），这些新的案例必须被添加到"黄金数据集"中。维护工作还包括删除或更新那些已经过时的示例：如果应用程序的行为或数据发生了变化，那么旧的黄金示例可能就无法再正确模拟实际代码路径了。定期审查"黄金数据集"，找出那些不再符合当前系统要求的示例，并对其进行更新或替换，以确保持续性的集成测试仍然具有意义。

Common Mistakes with CI

使用 CI 时常见的错误

We have seen teams commonly make a few mistakes when they start using golden datasets.

我们已经看到，当团队开始使用"黄金数据集"时，他们通常会犯一些错误。

The most common is confusing CI pass rate with production quality. A golden dataset is small and curated. Passing CI means the agent still handles the specific cases we have tested. It says nothing about how the agent performs on the thousands of queries it will see in production. CI serves a similar purpose to a smoke test in traditional software: if it fails, something is definitely wrong. If it passes, we have not broken anything we know about, but we have not proven anything about general quality either.

最常见的问题是将集成测试通过率与系统的实际性能混为一谈。一个优质的黄金数据集应该是规模较小且经过精心挑选的。通过集成测试意味着该代理仍然能够处理我们测试过的特定案例。但这并不能反映该代理在实际生产环境中面对成千上万种查询时的表现如何。集成测试的作用类似于传统软件中的烟雾测试：如果测试失败，那就说明肯定存在问题；如果测试通过，那说明我们已知的缺陷都没有被暴露出来，但也不能证明系统的整体质量一定很好。

Another common mistake is *data leakage*: golden set examples accidentally ending up inside the agent's prompts as few-shot examples, or inside the LLM judge's prompts. When the model has already seen the test cases as part of its instructions, it will perform well on those cases regardless of whether it has actually improved. The CI results become meaningless. To avoid this, keep the golden set in a separate file or repository

from any prompt content, and audit regularly for overlap.

另一个常见的错误是数据泄露：黄金集示例意外地出现在代理的提示中作为少样本示例，或者出现在 LLM 裁判的提示中。当模型已经将其作为指令的一部分看到测试用例时，无论它是否真正改进了，它都会在这些用例上表现良好。CI 结果变得毫无意义。为了避免这种情况，请将黄金集保存在与任何提示内容分开的文件或存储库中，并定期检查重叠情况。

WARNING 警告

Data leakage between the golden set and prompts is the most common way teams unknowingly invalidate their CI. If anyone on the team copies a golden example into a few-shot prompt, that example no longer tests generalization. Automate a check that diffs golden inputs against prompt content on every commit.

在"黄金样本"与提示信息之间发生数据泄露的情况，是团队在不知不觉中导致集成测试失效的最常见原因。如果团队中的某个人将黄金样本复制到了一次性提示中，那么这个样本就不再能代表整体情况了。因此，应该自动化一个检查机制，每次提交代码时都对比黄金样本与提示内容是否一致。

Teams also ask how large the golden set should be and how often to expand it. A good starting point is 20 to 50 examples covering core features, known failure modes, and edge cases. As the system matures and new failure modes are discovered, the set grows. Mature systems often have 100 to 200 examples. We add examples when we confirm a new failure mode through error analysis (Chapter 3) or online monitoring, not just because more data is available. A bloated test suite is expensive to maintain and slow to debug when tests fail. Every golden example should represent a specific behavior you care about preserving.

团队还会讨论"黄金案例集"的规模应该有多大，以及多久扩展一次。一个合适的起点是包含 20 到 50 个示例，涵盖核心功能、已知的故障模式以及边缘情况。随着系统的成熟，当发现新的故障模式时，案例集就会不断扩展。成熟的系统通常会有 100 到 200 个示例。当我们通过错误分析或在线监控确认出现新的故障模式时，就会添加相应的示例。这样做并不是因为有了更多的数据，而是因为我们需要保留那些重要的行为示例。过于庞大的测试集不仅维护成本高，而且在测试出现错误时调试起来也非常困难。每个黄金案例都应该能够体现你希望保留的特定行为。

Revisiting Prompts When Models Improve

当模型性能提升时，重新审视提示策略

Foundation models improve over time. When a new model version is released, reevaluate your prompts rather than running them through CI unchanged.

基础模型会随着时间而不断改进。当新的模型版本发布时，应该重新评估你的提示条件，而不是保持不变地使用旧的模型进行训练。

Many prompts accumulate detailed instructions that were added to work around earlier model limitations. For example, a prompt for a product recommendation summarizer might include the instruction: “The summary should be 4-5 sentences. Start with the product name. End with a recommendation.” If the new model already produces well-structured summaries by default, these instructions may be unnecessary. Removing them can make the prompt cheaper (fewer tokens), faster, and sometimes more accurate, because the model can apply its own judgment rather than rigidly following outdated constraints.

许多提示积累了为解决早期模型限制而添加的详细指令。例如，一个用于产品推荐摘要生成器的提示可能包含这样的指令：“摘要应为 4-5 句话。以产品名称开头。以推荐结尾。”如果新模型默认就能生成结构良好的摘要，这些指令可能就不再必要了。移除它们可以使提示更经济（减少 token 数量）、更快，有时甚至更准确，因为模型可以运用自己的判断，而不是死板地遵循过时的限制。

When upgrading to a new model version, we recommend the following process:

在升级到新模型版本时，我们建议遵循以下流程：

1. Run your existing prompts through CI to see what breaks and what still passes.

   将现有的提示通过 CI 进行验证，看看哪些部分存在问题，哪些还是可以正常通过验证。

2. For prompts that pass, try removing the most prescriptive instructions and rerunning. If the output quality stays the same or improves, keep the simpler version.

   对于通过测试的提示，可以尝试删除那些过于具体的说明，然后重新运行测试。如果输出质量保持不变或有所改善，那么就继续使用这个更简单的版本。

3. For prompts that fail, investigate whether the failure is a genuine regression or just a change in formatting that your evaluators are sensitive to.

对于出现问题的提示，需要检查这种问题究竟是真正的回归问题，还是仅仅是格式上的变化而已——因为评估者可能会对这些格式上的变化很敏感。

4. Rerun error analysis (Chapter 3) on a sample of outputs from the new model. New models can introduce new failure modes as well as fix old ones.

   对新模型生成的样本数据进行重新运行错误分析（第 3 章）。新模型可能会引入新的故障模式，同时也会修复旧的故障问题。

Voiceflow, a company that builds voice and chatbot agents for customers, documented one such migration in late 2023. One of its customers was using an agent to figure out what a user was asking about (e.g., "is this a billing question or a shipping question?") so the company could route the request to the right next step. This kind of agent is usually called an *intent classifier*. When OpenAI released a newer version of GPT-3.5, Voiceflow swapped it in, expecting a drop-in improvement. Instead, accuracy on the customer's dataset dropped by about 10 percentage points on the exact same prompt. Voiceflow's team had to go back and rewrite parts of the prompt (changing a phrase from "descriptions and intents" to "descriptions and actions," adding one worked-out example at the top of the prompt, and leading with a sentence telling the model the task was important) before the team could recover the original accuracy. Without a regression test that ran the new model against the customer's data, the team would have shipped the degraded agent to production and found out about the problem only when users complained.

Voiceflow 是一家为客户构建语音和聊天机器人代理的公司。在 2023 年底，该公司记录了一例这样的迁移案例。其一位客户使用机器人来解析用户的问题内容（例如："这是关于账单的问题还是运输问题？"），这样公司就可以将请求分配到正确的下一步操作。这种机器人通常被称为"意图分类器"。当 OpenAI 发布新版 GPT-3.5 时，Voiceflow 将其替换进来，期望能带来显著的改进。然而，在相同的提示下，客户数据集上的准确率下降了约 10 个百分点。Voiceflow 的团队不得不重新编写部分提示内容（将"描述与意图"改为"描述与行动"，在提示顶部添加了一个有效的示例，并增加了一句说明模型任务的重要性的句子），才使得准确率恢复到原来的水平。如果没有针对新模型与客户数据进行回归测试，团队就会将存在缺陷的代理程序投入生产使用。而那时问题出现时，用户才会开始抱怨。

In October 2024, OpenAI quietly swapped the model behind the `gpt-4o` alias. Developers discovered that function calls stopped triggering correctly: the model returned tool responses as plain text instead of structured function calls. Teams that had pinned their model version (e.g., `gpt-4o-2024-05-13`) were unaffected. Teams using the unversioned alias had to scramble to diagnose the issue from customer complaints. The takeaway is to always pin model versions in production and treat a model upgrade

as a code change that goes through CI before deployment.

在 2024 年 10 月，OpenAI 悄然更换了 gpt-4o 这一别名的模型。开发人员发现，函数调用不再能正常触发：模型现在以纯文本形式返回结果，而不是结构化的函数调用。那些固定了模型版本的团队（例如 gpt-4o-2024-05-13 ）并未受到影响。而使用未版本化的别名的团队则不得不紧急诊断问题，以应对客户的不满。因此，建议在生产环境中始终固定模型版本，并将模型升级视为一种代码变更，需要在部署前通过持续集成流程进行验证。

Beyond pinning, it's good to have a model version lifecycle policy: set a maximum version age (e.g., six months after a newer version is available), and when it's time to upgrade, run the new version through the same CI-gated process as any other change. Providers also sometimes change behavior behind the same version string through undocumented backend configuration changes (L. Chen et al. [2024] documented significant behavioral shifts in GPT-4 over a three-month period); if your CI metrics shift without any change on your side, an API behavioral drift is a likely explanation, and rerunning your golden set against the pinned version can confirm it.

除了设定版本限制之外，还有一个不错的做法就是为模型版本制定生命周期政策：设定最大版本保留时间（例如，在更新版本发布后的六个月内）。当到了需要升级的时候，让新版本通过与以往相同的集成测试流程进行验证。有时候，提供者也会通过未公开的后端配置更改来修改同一版本的行为（L. Chen 等人[2024 年]记录了 GPT-4 在三个月内行为发生的显著变化）。如果您的集成测试指标发生了变化，而没有任何人为干预，那么很可能是 API 行为发生了改变。此时，重新使用黄金数据集与固定版本进行对比，就可以确认这一点了。

TIP 提示/窍门

Pin exact model versions (e.g., gpt-4o-2024-05-13) in both your agent and your LLM-as-judge evaluators. Unversioned aliases like gpt-4o can change without notice, causing both your agent and your evaluators to shift behavior simultaneously, which makes regressions nearly impossible to diagnose.

在你的代理程序和智能评估器中，所有版本都必须是完全相同的模型版本（例如， gpt-4o-2024-05-13 ）。像 gpt-4o 这样的未版本化的别名可能会随时发生变化，这会导致你的代理程序和评估器同时改变行为，从而使得回归分析几乎无法进行诊断。

CD and Online Monitoring

光盘播放与在线监控

We rely on CI to catch regressions on known issues. With CD, we track what happens in production, where users send queries we did not anticipate and the agent encounters situations our golden dataset does not cover.

我们依赖 CI 来捕捉已知问题中的回归现象。而通过 CD，我们可以实时监测生产环境中的情况，用户可能会提出我们未预料到的查询请求，而代理则可能会面临我们的黄金数据集无法涵盖的情况。

Observability 可观测性

Online evaluation starts with logging. For each production request (or a representative sample), we capture:

在线评估从日志记录开始。对于每一个生产请求（或代表性样本），我们都会进行记录。

- The initial input and relevant metadata (e.g., user/session IDs, flags)

  初始输入以及相关的元数据（例如，用户/会话 ID、标志等）
- All intermediate LLM calls: prompts, responses, and intermediate reasoning

  所有中级大语言模型调用操作：提示语、响应以及中间阶段的推理过程
- All tool calls: names, arguments, responses, and errors

  所有工具的调用情况：包括名称、参数、响应结果以及错误信息等。
- Retrieved documents in RAG pipelines

  在 RAG 处理流程中检索到的文档
- The final output shown to the user

  最终的输出结果已呈现给用户了。
- Any user feedback (thumbs-up/down, edits, follow-ups)

  任何用户的反馈（点赞/踩、修改建议、后续意见）
- Evaluator outputs (if applicable)

  评估结果输出（如适用）

These logs let us reconstruct full user interactions. Tools like LangSmith, Arize Phoenix, and W&B Weave, or integrations with OpenTelemetry, help manage and explore this data. We recommend reviewing a sample of production traces daily.

Developers build a sharper sense of output quality as they see more real data, and we catch emerging patterns earlier when we review daily. The data is organized into *traces* (defined in "Establishing Terminology"), *spans*, and *sessions* (Chapter 6). A trace is the full record of one user request. A span is one step within that trace (an LLM call, a tool call, a database query). A session groups related traces across a multi-turn conversation.

这些日志帮助我们还原了用户的完整操作过程。诸如 LangSmith、Arize Phoenix 和 W&B Weave 之类的工具，以及与 OpenTelemetry 的集成，都能帮助我们管理和分析这些数据。我们建议每天查看一些实际产生的跟踪数据样本。开发者通过接触更多的真实数据，能够更清晰地了解系统的输出质量；而我们则可以在每天分析这些数据时更早地发现其中的规律。这些数据被组织成了各种形式：跟踪、跨度以及会话（详见第六章的说明）。跟踪指的是一个用户的完整操作记录；跨度则是跟踪中的某个步骤（例如 LLM 的调用、工具的调用、数据库查询等）；而会话则是将多个相关跟踪数据组合在一起，用于描述多轮对话的过程。

Running Automated Evaluators in Production
在生产环境中运行自动评估器

With logging in place, we run automated evaluators (Chapter 5) on sampled production traces. A typical setup uses five to seven evaluators, a mix of code-based and LLM-as-judge, each tracking a known failure mode.

在登录完成后，我们对采样到的生产跟踪数据运行自动化评估器（参见第 5 章）。通常，系统会使用五到七个评估器，这些评估器包括基于代码的评估方式以及使用 LLM 进行评判的方式，每个评估器都会针对一种已知的故障模式进行监控。

LLM-as-judge evaluators are slow and expensive, so we usually evaluate a small sample (1% to 5% of traffic) asynchronously after the request completes. For each failure mode, we compute a corrected success rate and a 95% confidence interval using bootstrapping ("Estimating True Success Rates with Imperfect Judges"). This adjusts for the evaluator's known TPR and TNR, giving us a grounded estimate of real-world success rates.

作为评判者的 LLM 模型效率较低且成本较高，因此我们通常会在请求完成之后，对一小部分样本（占流量的 1%到 5%）进行异步评估。对于每一种故障模式，我们会通过自举法计算出修正后的成功率以及 95%的置信区间（"用不完美的评判者来估算真实的成功率"）。这种方式能够考虑到评判者已知的 TPR 和 TNR，从而得出更可靠的现实世界成功率估计。

For the real estate assistant from Example 3-1, this might look like:

对于示例 3-1 中的房地产助理来说，情况可能会是这样的：

1. Sample 1% of production traces daily.

   每天对生产批次的 1%进行采样检测。

2. Run “Hallucinated Tool Usage” and “Location Ambiguity” LLM judges on those traces.

   请对这些样本数据让大型语言模型进行“幻觉性工具使用”和“位置模糊性”相关的评估。

3. Compute the 95% confidence interval for each failure mode’s true success rate.

   计算每种故障模式真实发生率的 95%置信区间。

4. Dashboard the results and alert if any confidence interval crosses a threshold.

   在仪表板上显示结果，如果任何置信区间超过了阈值，则发出警报。

Guardrails 护栏/防护栏

Most evaluators run asynchronously after the fact. *Guardrails* are different. They run *during* execution, before the output reaches the user. Because they sit in the critical path, they need to be fast and have a very low false positive rate. In practice, this rules out LLM-as-judge for most guardrails: a two- to three-second judge call on every output adds latency users will feel. What does work well is cheap, deterministic checks that run in milliseconds: regex filters, JSON schema validation against the expected output structure, blocklist or allowlist string matches, length/format sanity checks, and lightweight parsers (e.g., sqlparse to confirm SQL is syntactically valid, ast.parse to confirm Python is parseable).

大多数评估器是在事情发生之后才进行运行的。不过，对于某些情况下，评估工作需要在执行过程中就进行。因为它们处于关键路径上，所以必须非常快速，并且要具备极低的误报率。实际上，对于大多数评估任务来说，使用 LLM 作为评估工具并不合适：每次对输出结果进行评估需要两到三秒的时间，这会增加用户的等待负担。因此，更有效的做法是采用成本低廉、执行速度快的确定性检查方式，比如正则表达式过滤、根据预期输出结构进行的 JSON 格式验证、基于黑名单或白名单的字符串匹配、长度/格式检查，以及简单的解析工具（例如，使用 sqlparse 来验证 SQL 语句的语法正确性，使用 ast.parse 来验证 Python 代码的可解析性）。

Concretely, a guardrail is just a function the application calls on every model output before forwarding the response. Here is a minimal Python example wrapping an LLM

call for the real estate assistant's email-drafting feature:

具体来说，护栏其实只是一个应用程序在转发响应之前对每个模型输出的结果进行处理的函数。以下是一个简单的 Python 示例，它封装了 LLM 的调用，用于实现房地产助手撰写邮件的功能：

```
import re

# US SSN pattern: 3 digits - 2 digits - 4 digits
SSN_RE = re.compile(r"\b\d{3}-\d{2}-\d{4}\b")
# Simple credit-card-ish pattern: 13–16 digits, optionally separated
CC_RE  = re.compile(r"\b(?:\d[ -]*?){13,16}\b")

def pii_guardrail(text: str) -> tuple[bool, str | None]:
    """Return (is_safe, reason). is_safe=False means block the output."""
    if SSN_RE.search(text):
        return False, "looks_like_ssn"
    if CC_RE.search(text):
        return False, "looks_like_credit_card"
    return True, None

def draft_email(client_request: str) -> str:
    output = llm.complete(prompt_for(client_request))
    is_safe, reason = pii_guardrail(output)
    if not is_safe:
        log_guardrail_trip(reason, output)
        return "I drafted something but it contained sensitive info, " \
               "so I'm asking you to review the source data first."
    return output
```

This pattern—run a fast check on every output, log when it fires, fall back to a safe response when it does—is all a guardrail really is. The check itself can be a regex (as as in the preceding code), a JSON schema validator (e.g., jsonschema.validate(output, response_schema) for tool call arguments), or a parse-and-catch wrapper around a SQL or code parser.

这种模式——对每个输出进行快速检查，在触发时记录，并在触发时回退到安全响应——实际上就是一个护栏。检查本身可以是一个正则表达式（如前一个代码示例中的），一个 JSON 模式验证器（例如 jsonschema.validate(output, response_schema) 用于工具调用参数），或者是一个围绕 SQL 或代码解析器的解析并捕获包装器。

NOTE 注意/备注

A guardrail that fires too often on valid outputs is worse than no guardrail at all. Users experience false positives as broken functionality. Test guardrails against a sample of passing production traces and verify that the false positive rate stays below 1% before deploying.

一个对有效输出频繁触发护栏比完全没有护栏更糟糕。用户将误报视为功能损坏。在部署前，使用通过的生产跟踪样本对护栏进行测试，并验证误报率保持在 1%以下。

Most production systems benefit from a baseline set of guardrails. Common categories include personally identifiable information (PII) detection in generated text, output schema validation (e.g., ensuring generated SQL is syntactically valid before it runs against the database), toxicity or safety classifiers on the final response, and prompt injection detection on incoming user inputs (see Chapter 8 for more on adversarial evaluation). You can think of guardrails in two layers: inter-step checks that validate tool arguments and intermediate context as the agent works, and output checks that validate the final response before it reaches the user.

大多数生产系统受益于一套基本的护栏。常见类别包括在生成文本中检测个人身份信息（PII）、输出模式验证（例如，在生成 SQL 运行 against the database 之前确保其语法有效）、最终响应的毒性或安全分类器，以及检测传入用户输入中的提示注入（有关对抗性评估的更多内容，请参见第 8 章）。你可以将护栏分为两层：在代理工作时验证工具参数和中间上下文的步骤间检查，以及在最终响应到达用户之前验证最终响应的输出检查。

When setting up guardrails, it helps to define nonnegotiable pass thresholds for safety and correctness first, before optimizing for other qualities like tone or conciseness. For example, a guardrail that blocks PII in outgoing emails is nonnegotiable and should never be relaxed; a guardrail that flags overly verbose responses is a quality preference and can be tuned over time.

在设置防护机制时，首先应确定一些不可协商的阈值，以确保安全性和正确性。例如，防止敏感信息被包含在发送的电子邮件中的防护机制就是不可协商的准则，绝不能放松；而能够识别过于冗长的回复的防护机制则属于可调整的指标，可以通过时间来优化。

When a guardrail triggers, the agent needs a predefined response:

当栏杆触发时，代理需要有一个预定义的响应。

1. Reject the output and return a safe fallback message.

   拒绝该输出结果，并返回一个安全的备用消息。

2. Retry the LLM call. Since LLMs are nondeterministic, a retry often produces a clean output.

   请重新调用大语言模型。由于大语言模型的行为具有不确定性，因此多次重试通常能够得到更清晰的输出结果。

3. Fall back to a simpler model or logic path.

   退回到更简单的模型或逻辑路径。

For the real estate assistant from Example 3-1, a guardrail runs a regex check on generated emails before sending. If it detects something that looks like a Social Security number, it rejects the output. Another guardrail parses generated SQL for syntax errors. If the SQL is invalid, it retries once. If the retry also fails, it asks the user to rephrase.

对于来自示例 3-1 的房地产助手，一个防护栏在发送前对生成的电子邮件运行正则表达式检查。如果它检测到看起来像社会安全号码的内容，它会拒绝输出。另一个防护栏解析生成的 SQL 以检查语法错误。如果 SQL 无效，它会重试一次。如果重试仍然失败，它会要求用户重新表述。

Judge Drift 判断漂移

When we first build an LLM-as-judge evaluator (Chapter 5), we validate it against human labels and measure its TPR and TNR. But those numbers do not stay constant. Two things can change under us. First, the model provider can update the LLM that the judge runs on. If we are using an alias like `gpt-4o` instead of a pinned version like `gpt-4o-2024-05-13`, the judge can start producing different scores overnight without any change on our side. Second, the team's understanding of what "correct" means can shift as they see more production data (the criteria drift we discussed in Chapter 4). The judge was calibrated against labels that reflected the team's understanding at the time. If the team's standards have moved, the judge's scores no longer reflect what the team considers correct.

当我们首次构建用于评判的 LLM 评估器时（见第 5 章），我们会将其与人类给出的标签进行比对，并测量其正确率（TPR）和错误率（TNR）。但这些数值并非一成不变。有两种因素可能会发生变化：首先，提供 LLM 模型的方可能会更新评判所使用的模型。如果我们使用的是像 `gpt-4o` 这样的别名，而不是像 `gpt-4o-2024-05-13` 这样的固定版本，那么评判者就可以在一夜之间产生不同的评分，而我们没有办法对此做出任何调整。其次，随着团队不断看到更多的实际数据，他们对"正确"的理解也会发生变化（正如我们在第 4 章中提到的标准漂移问题）。因此，评判器的评分已经不再能反映团队所认为的正确标准了。

Either way, the judge's accuracy degrades without anyone noticing unless we check for it. This is *judge drift*.

无论如何，除非我们检查，否则法官的准确性会逐渐下降。这就是所谓的法官漂移。

We handle this with three practices:

我们通过三种做法来处理这个问题：

Pin model versions in both CI and production.

包括开发版和量产版在内的 Pin 模型版本。

Use exact identifiers like `claude-3-opus-20240229`, not aliases like `claude-3-opus-latest`.

使用精确的标识符，例如 `claude-3-opus-20240229`，而不是像 `claude-3-opus-latest` 这样的别名。

Revalidate regularly. 定期重新评估。

Every few weeks, we sample recent production traces, get fresh human labels, and recompute the judge's TPR and TNR. If either drops below our threshold, we update the judge's prompt or few-shot examples and validate the new version before redeploying.

每隔几周，我们会重新采样最新的生产数据，为模型赋予新的标签，然后重新计算裁判的 TPR 和 TNR 指标。如果其中任何一个指标低于我们的阈值，我们就会更新裁判的提示或少量样本数据，并在重新应用新版本之前对更新后的结果进行验证。

Reevaluate after any model change.

在模型发生任何变更后重新评估。

If we swap the judge's underlying LLM, we rerun the full validation process.

如果我们更换判别者的底层 LLM，我们将重新运行完整的验证过程。

The Continuous Improvement Flywheel

持续改进飞轮

CI and CD are not isolated stages. They connect into a loop (Figure 9-2). We deploy with monitoring and detect issues in production. Then we diagnose them through error

analysis, improve the agent, and update our evaluators. Finally, we verify the fix passes CI and redeploy. Each cycle through this loop makes the system more robust and the evaluation suite more comprehensive.

CI 和 CD 不是孤立阶段。它们连接成一个循环（图 9-2）。我们部署时附带监控，并在生产环境中检测问题。然后通过错误分析诊断它们，改进代理，并更新我们的评估器。最后，我们验证修复是否通过 CI 并重新部署。每次通过这个循环，系统都会变得更加健壮，评估套件也会更加全面。

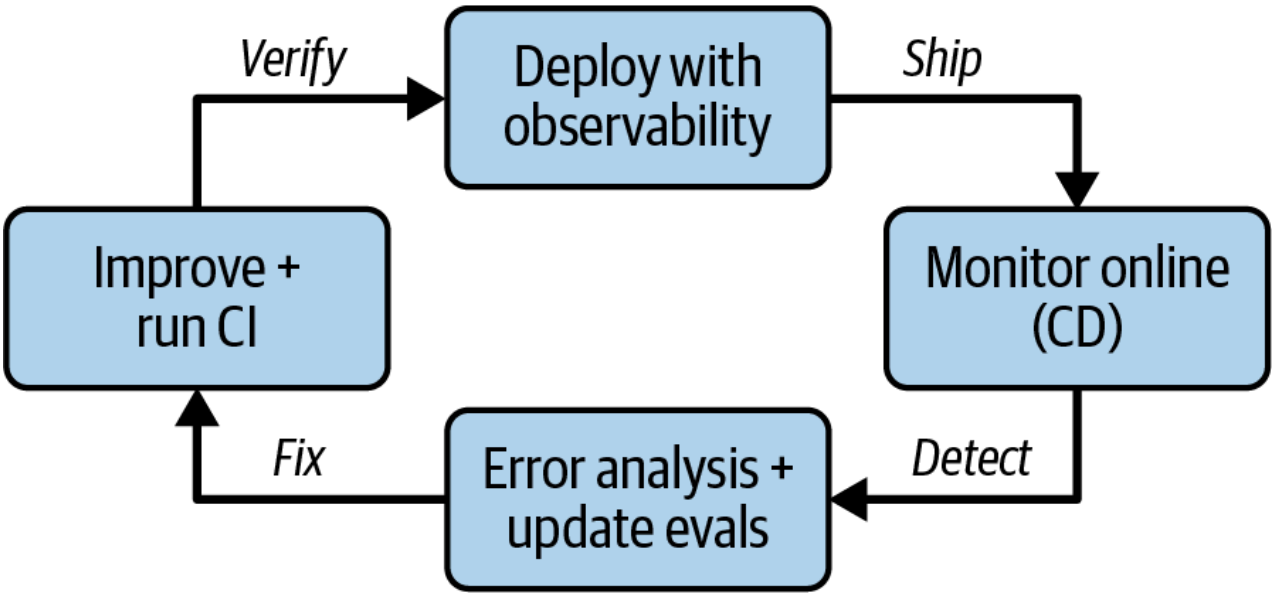


Figure 9-2. The continuous improvement flywheel

图 9-2. 持续改进的循环机制

To make this concrete, let's walk through one full cycle of the flywheel for the real estate assistant from Example 3-1.

为了具体说明这一点，让我们以示例 3-1 中的房地产助理为例，来演示整个飞轮的运行过程。

The team deploys a new version with a neighborhood-targeting feature. They have CI checks for known failure modes (tone mismatch, hallucinated tool usage, missing SQL constraints) and the new version passes all of them.

该团队采用了新版本的产品，该版本具有针对特定邻域的目标定位功能。他们已经对已知的错误模式进行了集成测试（如语气不匹配、错误的工具使用、缺少 SQL 约束等），而新版本完全通过了所有这些测试。

A week after deployment, the online monitoring dashboard shows a spike in the "Location Ambiguity" failure rate. Product metrics confirm the problem: users in cer-

tain neighborhoods have shorter sessions and lower task completion rates.

在部署一周后，在线监控面板显示“位置不确定性”故障率有所上升。产品指标也证实了这一问题的存在：某些社区的用户会话时长较短，任务完成率较低。

The team pulls the flagged traces and does error analysis (Chapter 3). They find that the LLM frequently confuses “West Berkeley” with “Berkeley West” (a nonexistent area), producing bad search results. They define a new failure mode: “Directional Location Qualifier Errors.”

团队提取了那些被标记为错误的痕迹，并进行了错误分析（参见第 3 章）。他们发现，LLM 经常将“West Berkeley”误认为是“Berkeley West”（一个不存在的地区），从而导致搜索结果出错。于是他们提出了一种新的故障模式：“方向定位修饰符错误”。

Now they update their evaluation artifacts. They add 5 to 10 examples of this failure to the golden dataset, so CI will catch it if it reappears. They build a new code-based evaluator that checks whether generated location strings match known neighborhood names.

现在，他们正在更新他们的评估工具。他们在黄金数据集中增加了 5 到 10 个此类错误的示例，这样如果这种错误再次出现，持续集成系统就能及时发现。他们还开发了一种新的基于代码的评估工具，用于检查生成的位置字符串是否与已知的邻近名称相匹配。

They fix the prompt (adding a list of valid neighborhood names to the system message), verify the fix passes CI, and redeploy. The monitoring dashboard shows the failure rate drop. The flywheel has completed one cycle, and the evaluation suite is now more comprehensive than it was before the issue surfaced.

他们修复了提示信息（在系统消息中添加了有效的社区名称列表），确认修复方案通过集成测试，然后重新部署了系统。监控仪表板显示故障率有所下降。飞轮已经完成了一个周期的运行，现在评估工具的功能比问题出现之前更加完善。

Over time, each cycle through this loop adds new test cases, refines existing evaluators, and catches problems earlier. Chapter 10 describes how to build interfaces that make the human review steps in this loop practical and efficient.

随着时间的推移，每个循环都会引入新的测试案例，改进现有的评估方法，并能够更早地发现问题。第 10 章介绍了如何构建一些接口，使得这一循环中的人工审核步骤更加实用和高效。

Common Pitfalls 常见陷阱

Here are a few recurring mistakes that we have seen, which can weaken this loop over time:

以下是一些我们观察到的一些常见错误，这些错误可能会随着时间的推移而削弱这个循环的效果：

Stale golden datasets 仍然保存着那些金色的、经过长时间使用的数据集

If the golden set stays frozen while production traffic evolves, CI becomes less useful over time. The flywheel example in the preceding section shows how to fix this: each cycle adds new examples from real failures.

如果黄金集合在生产流量发生变化的情况下仍然保持冻结状态，那么随着时间的推移，这种工具就会变得不再有用。前一节中的飞轮例子展示了如何解决这个问题：每个周期都新增一些来自实际故障的示例。

Conflicting metrics 相互冲突的度量标准

Improving one quality dimension (e.g., conciseness) can degrade another (e.g., factual accuracy). We prioritize our metrics explicitly, especially safety guardrails, and decide how to handle conflicts before they arise.

改善某个质量维度（例如简洁性），可能会影响到另一个维度（例如事实准确性）。我们会明确优先处理某些指标，尤其是那些关乎安全性的指标。同时，我们会提前解决可能出现的问题，避免产生冲突。

Disconnected AI and product metrics

脱节的 AI 与产品指标

A drop in user satisfaction or task completion rates might be an AI quality problem or a UX problem. We investigate both. If the issue is AI quality (e.g., excessive verbosity causing user drop-off), we define a new failure mode and evaluator for it.

用户满意度下降或任务完成率降低可能是人工智能质量问题的表现，也可能是用户体验方面的问题。我们同时研究这两种情况。如果问题是人工智能质量方面的问题（例如，过多的信息导致用户失去兴趣），那么我们就需要定义一种新的故障模式以及相应的评估方法。

Only looking at automated monitoring

仅关注自动化监控方面

LLM-as-judge evaluators miss subtle failures. Teams that regularly review random production traces (not only flagged ones) catch problems earlier. If user engagement drops but evaluator metrics stay flat, there is likely an undetected issue.

以评判者身份进行评估的 LLM 模型难以发现那些细微的缺陷。那些经常审查随机产生的测试数据（而不仅仅是被标记为问题的数据）的团队能够更早地发现问题。如果用户的参与度下降，而评估者的指标却保持不变，那么很可能存在一些未被发现的问题。

Putting It Together 整合起来

In CI, we run golden dataset checks before deployment to catch regressions on known failure modes. In CD, we monitor production traffic to detect new issues we did not anticipate. Together they form a loop in which we deploy, monitor, diagnose, fix, and redeploy.

在 CI 阶段，我们在部署之前会进行黄金数据集的检查，以发现已知故障模式上的回归问题。在 CD 阶段，我们会监控生产环境中的流量，以便发现那些我们尚未预料到的新问题。这两个阶段共同构成了一个循环流程：首先进行部署，然后监控情况，进行诊断，最后进行修复，再次进行部署。

Automated evaluators handle most of the monitoring, but human review remains necessary for catching subtle or novel failures. The next chapter covers how to build interfaces that make that human review practical and efficient.

自动化评估工具能够处理大部分监控工作，但对于那些细微或新颖的缺陷来说，人工审核仍然是必要的。下一章将介绍如何构建能够使得人工审核更加实用和高效的界面。

1 In some codebases and applications, LLM tests can be marked as flaky and rerun a few more times.

在一些代码环境和应用中，LLM 测试可能会被标记为不稳定，因此需要再运行几次以确认结果。

Chapter 10. Interfaces for Human Review

第 10 章 供人工审核的接口

Automated evaluators (Chapter 5) handle most of the monitoring, but humans still review a sample of traces regularly to catch issues the evaluators miss and to keep evaluators calibrated as the team's understanding of quality evolves (the criteria drift we discussed in Chapter 4). The question is how to do that review efficiently. LLM traces are complex. A single trace can include the user query along with multiple tool calls with structured arguments, retrieved documents, intermediate reasoning, and the final output. Reviewing these in a spreadsheet is painful, so we have found it very useful to build our own review interfaces. Not only is it easier to get through more traces, we end up actually *wanting* to look at more of our data, for as long as we are able to keep discovering new failure modes.

自动化评估器（第 5 章）负责处理大部分监控工作，但人类仍然需要定期审查一些样本数据，以便发现评估器可能忽略的问题，并随着团队对质量理解的深入而调整评估器的算法（正如第 4 章所讨论的，评估标准会发生变化）。问题在于如何高效地完成这些审查工作。LLM 生成的日志数据非常复杂：一个日志中可能包含用户查询、多个工具调用、结构化参数、检索到的文档、中间推理结果以及最终输出。使用电子表格来审查这些数据非常麻烦，因此我们开发了自己的审查界面，这不仅使得处理更多数据变得更加容易，而且我们还希望尽可能多地查看数据，因为只要还能发现新的故障模式，我们就会继续探索下去。

In this chapter, you will learn:

在本章中，你将学到：

- Why custom review interfaces work better than spreadsheets and generic tools

  为什么定制化的审核界面比电子表格和通用工具更适用呢？

- Key interface principles from human-computer interaction (HCI) that apply to reviewing LLM agent outputs

  适用于审核大语言模型输出结果的计算机人机交互关键原则

- Features that make reviewing faster and more reliable, organized by category

  这些功能使得审核过程更加快速且可靠，所有内容都按类别进行了整理。
- Strategies for selecting the most informative traces to review

  选择最具信息价值的记录进行审查的策略
- How to embed review into your team's engineering workflow

  如何将评审流程嵌入到团队的工程工作流程中

The Case for Custom Review Interfaces

支持自定义审核界面的理由

When teams first start reviewing LLM outputs, they usually start with a spreadsheet. This is natural: most logging systems can export traces as CSV, and spreadsheets are familiar to everyone on the team. The problem is that LLM traces do not fit well into rows and columns.

当团队开始审查大语言模型的输出时，他们通常会先使用电子表格来整理数据。这是很自然的做法：大多数日志系统都能将日志数据导出为 CSV 格式，而电子表格则对团队中的每个人来说都很熟悉。不过问题在于，大语言模型的日志数据并不适合用行和列的方式来组织。

Consider what a single trace from our real estate assistant looks like in a spreadsheet row. The user asked: "Find 3-bedroom homes under $600k near downtown and email the top two to my client." The agent made three tool calls (a listing search, a filter, and a calendar check), retrieved six listing documents, produced intermediate reasoning about which listings best match, and generated a two-paragraph email. In a spreadsheet, all of that lands in a single row. The email text overflows its cell. The tool calls appear as a JSON blob in one column. The retrieved listings sit in another JSON blob next door. To judge whether the agent picked the right listings, you have to parse the JSON by eye, cross-reference it with the query constraints, and then read the email to check if it accurately reflects the results. That is three separate acts of mental re-

construction for one trace.

想象一下，我们的房地产助理提供的查询结果在电子表格中表现为哪一行数据。用户请求：“请在市中心附近、面积在 3 间卧室以上、价格不超过 60 万美元的房源中筛选出一些房源，并将最好的两个房源信息通过电子邮件发送给我的客户。”房产经纪人执行了三个操作：房源搜索、筛选条件设置以及日历检查。他获取了六份房源信息，对哪些房源最合适进行了推理分析，然后生成了两段内容的电子邮件。在电子表格中，所有这些信息都集中在一行数据中。电子邮件的内容则占据了单元格的整个空间。这些操作的结果以 JSON 格式存储在一个列中，而检索到的房源信息则存储在另一个 JSON 对象中。要判断经纪人选择的房源是否合适，就需要通过视觉来解析 JSON 数据，将其与查询条件进行对照，然后阅读电子邮件内容以确认其是否准确反映了查询结果。这相当于需要完成三个不同的思维过程才能得出最终结论。

In a custom interface, we can lay out that same trace so it reads naturally. The user query goes at the top. Each tool call gets a collapsible panel with arguments and return values formatted clearly. The generated email looks like an actual email, with the client name, subject line, and body. A sidebar shows the client persona and retrieved listings. A trace that took five minutes in a spreadsheet takes 30 seconds here.

在自定义界面中，我们可以重新布局这个流程，使其看起来更加自然。用户查询部分位于界面顶部。每个工具调用都会有一个可折叠的面板，其中参数和返回值会被清晰格式化显示。生成的电子邮件看起来就像一封真实的邮件，包含客户姓名、主题行以及邮件正文。侧边栏中展示了客户的信息以及检索到的列表。原本在电子表格中需要五分钟才能完成的流程，在这里只需要 30 秒即可完成。

You can build these tools with frameworks like Streamlit or Gradio, or you can adopt a purpose-built trace review platform. As of 2026, several platforms provide trace visualization and annotation out of the box, including Langfuse, Braintrust, Arize Phoenix, Helicone, and W&B Weave, among others. If your team is already using one of these for observability, it may already support the review workflows described in this chapter, which can save you the effort of building a custom interface. LLMs can also help generate basic custom interfaces through rapid prototyping. For example, asking an LLM coding assistant to “generate an HTML dashboard served via FastAPI to review text generation outputs with Pass/Fail buttons and a feedback textbox” can produce a

functional starting point in minutes.

你可以使用像 Streamlit 或 GRadio 这样的框架来构建这些工具，或者选择采用专门设计的跟踪审查平台。到了 2026 年，已经有多个平台提供了即时的跟踪可视化和注释功能，比如 Langfuse、Braintrust、Arize Phoenix、Helicone 以及 W&B Weave 等。如果你的团队已经在使用这些工具进行可观测性分析，那么它们很可能已经支持本章所描述的审查工作流程，这样你就无需再花费时间去开发自定义界面了。大型语言模型还可以帮助快速创建基本的自定义界面。例如，可以让大型语言模型助手生成一个通过 FastAPI 提供的 HTML 仪表盘，该仪表盘能够显示文本生成结果，并包含“通过/失败”按钮和反馈文本框，这样就能在几分钟内得到一个功能性的起点。

If you already use an observability platform (such as LangSmith, Braintrust, or Arize) that lets you browse traces, start there. Use it for a week and write down every moment where you feel friction: a missing filter, an unreadable JSON blob, a slow page load. Those friction points tell you what to build first.

如果你已经在使用某个可观测性平台（如 LangSmith、Braintrust 或 Arize），该平台允许你浏览日志数据，那么就从那里开始吧。使用这个平台一周后，记录下所有让你感到麻烦的时刻：比如某个过滤器缺失、JSON 数据难以阅读、页面加载速度慢等问题。这些麻烦点能帮助你确定首先需要改进的地方。

Once you have a sense of what the default tools get wrong, the next question is what a good review interface looks like. Jakob Nielsen’s usability heuristics give us a practical starting framework. Table 10-1 maps the most relevant principles to concrete design choices for LLM agent review tools.

一旦你了解了默认工具存在哪些问题，下一个问题就是期待一个怎样的优秀评论界面。Jakob Nielsen 的可用性启发式规则为我们提供了一个实用的指导框架。表 10-1 将最相关的原则与 LLM 代理评论工具的具体设计选择进行了关联。

Table 10-1. A subset of Jakob Nielsen's usability heuristics (Nielsen 1994a), applied to LLM agent review interfaces
表 10-1. 雅各布·尼尔森提出的可用性启发式规则的一个子集（尼尔森，1994a），应用于 LLM 代理的界面设计评估

Principle 原则	Design implication 设计意义
Visibility of system status 系统状态的可见性	Show which trace is being reviewed, how many remain, and feedback status. Provide timely progress indicators. 显示正在审核的跟踪记录，剩余的数量，以及反馈状态。提供及时的进度指示器。
Match between system and real world 系统与现实世界之间的较量	Use familiar terms from rubrics and evaluation criteria. Make mappings intuitive. 使用与评分标准相关的熟悉术语。让这些术语的对应关系更加直观明了。
User control and freedom 用户的控制与自由	Allow navigation (Next/Previous), undo, and defer actions. Prevent lock-in of any notes. 允许导航到上一页/下一页，撤销操作，以及推迟执行某些操作。避免任何笔记被锁定。
Consistency and standards 一致性与标准	Reuse layouts, terminology, and behaviors across traces. 在多个流程中重复使用相同的布局、术语以及行为规则。
Error prevention 错误预防	Use distinct labeling controls. Enforce requirements like rationale for Fail to reduce mistakes. 使用独特的标签标识方式。严格执行相关规范，例如明确说明“未能通过”的原因，以减少错误发生的可能性。
Recognition rather than recall 识别而非回忆	Show predefined failure modes and explanations visually to reduce cognitive load. 以可视化方式展示预定义的故障模式及其解释，从而降低认知负担。
Flexibility and efficiency of use 灵活性与使用效率	Offer shortcuts and accelerators for expert users. Keep the interface intuitive for novices. 为专家用户提供快捷方式与加速工具。同时让界面对初学者来说也易于理解。

Principle 原则	Design implication 设计意义
Aesthetic and minimalist design	Emphasize signal over noise. Highlight inputs and outputs clearly.
美学与极简设计	强调信号而非噪声。清晰地突出输入和输出部分。

The following sections show how these principles translate into specific design choices for LLM review interfaces.

以下部分将说明这些原则如何转化为 LLM 审查界面的具体设计决策。

Essential Interface Elements
必要的界面元素

A review interface can show the full LLM trace (input, output, and intermediate steps like tool calls, retrievals, and internal reasoning) while keeping things manageable. One way to do this is to collapse less important sections by default and let reviewers expand them with a click when they need more detail.

评审界面可以展示完整的 LLM 执行过程，包括输入、输出以及中间步骤，比如工具调用、数据检索和内部推理等。同时，为了便于管理，一些不太重要的部分可以在默认情况下被隐藏起来；而当用户需要更多细节时，可以点击展开这些部分。

For feedback collection, we can tag known failure modes using binary checkboxes or buttons based on issues identified during error analysis (Chapter 3) or collaborative evaluation (Chapter 4). Quick overall ratings (thumbs-up/down or numeric scores) speed up the process. Open-ended feedback fields let reviewers explain their reasoning, which helps surface unexpected problems and refine evaluation criteria over time.

在收集反馈时，我们可以使用二进制复选框或按钮来标记已知的故障模式。这些工具可以根据错误分析的结果（第三章）或协作评估的结果（第四章）来选择合适的选项。简单的整体评分方式（赞成/反对或数字分数）可以加快反馈收集的过程。开放式的反馈字段让评估者能够详细说明他们的判断依据，这有助于发现意想不到的问题，并随着时间推移不断改进评估标准。

Features That Improve Review Speed and Quality

能够提升审核速度和质量的特性

The following enhancements fall into three categories. Some make individual reviews faster, others collect higher-quality feedback, and the rest help reviewers navigate the dataset.

以下这些改进措施可以分为三类。有些措施能加快个人评审的速度，有些则能收集到更高质量的反馈，还有另一些则有助于评审人员更好地处理庞大的数据集。

The following features make individual reviews faster:

以下功能可以使得单独查看评论的过程更加快速：

Keyboard shortcuts 键盘快捷键

Provide hotkeys for submitting feedback, tagging failure modes, and moving between traces. A reviewer who can press N for Next, F for Fail, and T for "Tone Mismatch" moves through traces two to three times faster than one who clicks buttons.

提供快捷键来提交反馈、标记故障模式以及切换不同的测试流程。拥有这些快捷键的评审员可以通过按 N 键进行下一步操作，按 F 键表示"失败"，按 T 键表示"音调不匹配"。与需要点击按钮的评审员相比，使用这些快捷键可以更快地完成测试流程。

Visual hierarchy 视觉层次结构

Emphasize the most important trace components (such as the final output) using layout, color, or segmentation. In Example 3-1, highlighting price and location mentions in generated emails makes factual errors jump out immediately.

通过布局、颜色或分割等方式，突出显示最重要的痕迹组件（如最终输出结果）。在示例 3-1 中，通过在生成的电子邮件中突出显示价格和位置信息，可以让事实错误一目了然。

Progressive disclosure 逐步披露

Show only basic options (Pass, Fail, Defer) at first. Show detailed failure tagging only when the reviewer marks a trace as failed.

首先仅显示基本选项（通过、失败、延期）。只有当审核员将某个任务标记为失败时，才会显示详细的失败标记信息。

These features enable collection of higher-quality feedback:

这些功能能够收集更高质量的反馈：

Inline annotation 内嵌注释

Allow span-level feedback, such as highlighting a problematic sentence or marking a faulty dialogue turn, so reviewers can point to exactly what went wrong.

允许在跨度级别上进行反馈，比如突出显示有问题的句子或标记有错误的对话片段，这样评审人员就能明确指出具体哪里出了问题。

Feedback templates 反馈模板

Offer frequently used comments (such as “Tone too casual” or “Missing key fact”) as quick-select options, optionally adapted to the selected failure tag.

提供常用的反馈意见（如“语气过于随意”或“缺失关键事实”）作为快速选择选项，可以根据所选的故障标签进行相应调整。

Defer option 推迟选择选项

Provide a clear, prominent Defer button for uncertain cases. This preserves label accuracy and prevents low-confidence guesses from polluting the dataset.

为不确定的案例提供一个清晰、显眼的“延迟处理”按钮。这样能够保持标签的准确性，避免低置信度的预测结果污染数据集。

These enhancements help reviewers better navigate the dataset:

这些改进使得评估者能够更轻松地浏览数据集：

Contextual metadata 上下文元数据

Show relevant metadata (user segment, agent version, automated evaluation results) directly in the interface so reviewers can spot version-specific regressions.

在界面中直接展示相关的元数据（用户细分信息、代理版本、自动化评估结果），这样评审人员就能及时发现与版本相关的缺陷。

Filters 过滤器

Allow filtering on dimensions relevant to the product. If users interact with your agent through text, a dedicated app, and a website, you want channel as a filter.

允许根据与产品相关的维度进行过滤。如果用户通过文本、专用应用程序或网站与您的客服人员互动，您需要使用过滤器来筛选相关用户。

Batch review and bulk actions

批量审核和批量操作

Let reviewers examine trace clusters (e.g., from the same client persona) and apply the same failure label to multiple examples. This speeds up handling of repetitive issues but should be used carefully.

让评估者检查那些相似的案例组（例如，来自同一个客户群体的案例），并对多个案例应用相同的故障标签。这样能够更快地处理重复出现的问题，但使用时应非常谨慎。

Progress indicators 进度指示器

Show counts of traces reviewed, grouped by key dimensions, so reviewers can track their progress and know when they are done.

显示已审核的跟踪记录数量，这些记录按关键维度进行分组，这样审核人员可以跟踪自己的进度，并知道何时完成工作。

TIP 提示/窍门

Keyboard shortcuts are the single highest-leverage feature for review speed. Even a minimal set (Next, Pass, Fail, Defer) can double throughput. Add them before investing in any other interface enhancement.

键盘快捷键是提升审查效率的最有效的工具。即使只配置最简单的快捷键组合（如“下一步”、“通过”、“失败”、“推迟”），也能使审查效率提高一倍。在投资其他界面优化措施之前，先配置这些快捷键吧。

Case Study: EvalGen 案例研究：EvalGen

To see these principles in action, consider EvalGen, which applies many of them in a minimal design. *EvalGen* is a lightweight interface for human evaluation of LLM outputs that demonstrates several of these ideas in practice (Shankar et al. 2024d). A publicly available version (EvalGen v2) is integrated into ChainForge (Arawjo et al.

2024).

要看到这些原则的实际应用，可以看看 EvalGen 项目。该项目将许多这些原则应用于最小化的设计中。EvalGen 是一个轻量级接口，用于人类对大型语言模型输出的评估，它切实展示了这些理念的应用（Shankar 等人，2024 年）。该项目的公开版本（EvalGen v2）已集成到 ChainForge 系统中（Arawjo 等人，2024 年）。

EvalGen displays one output at a time, allows binary feedback (thumbs-up/down), supports open-ended comments, and lets reviewers edit past labels. It also includes a rubric builder that automatically extracts candidate evaluation criteria from the original prompt and lets users select which ones to apply during review. Figure 10-1 shows the interface.

EvalGen 一次只显示一项评估结果。它支持二进制反馈（赞成/反对），允许用户输入开放式评论，并让评审人员可以修改之前的标签。该工具还包含一个评分标准构建工具，能够自动从原始提示中提取出评估标准，用户可以在评审过程中选择适用的标准。图 10-1 展示了该工具的界面。

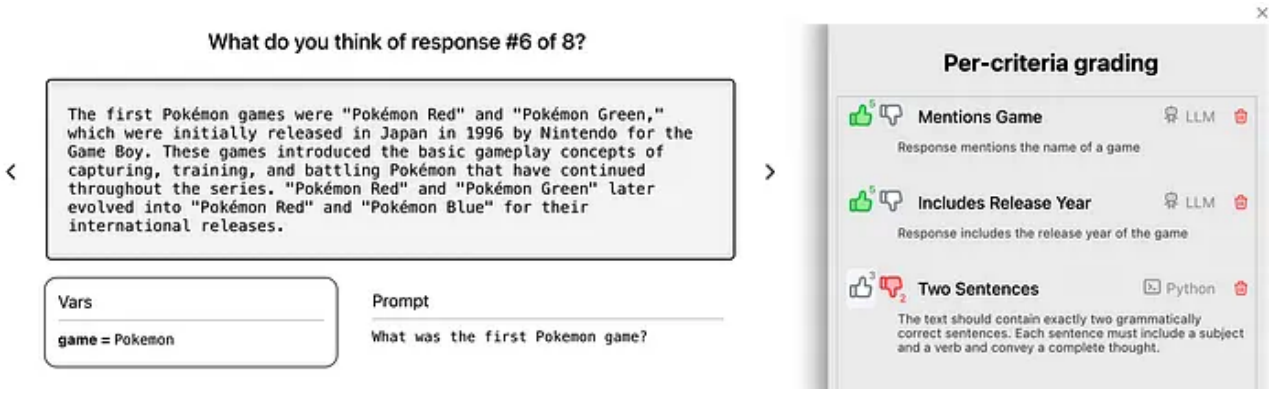


Figure 10-1. The EvalGen v2 interface (full-sized version)

图 10-1. EvalGen v2 界面（全尺寸版本）

EvalGen prioritizes *which* traces to show based on LLM judge disagreement: when multiple automated evaluators (such as separate LLMs or prompt variants) disagree on a verdict, the system surfaces that example for human review. This is a form of *uncertainty sampling*, which we discuss further in the next section.

EvalGen 根据 LLM 的判断结果来优先展示哪些测试案例。当多个自动化评估器（如不同的 LLM 或不同的提示变体）对某个结果存在分歧时，系统会将这些案例呈现给人类进行审核。这是一种不确定性采样的方法，我们将在下一节进一步讨论相关内容。

EvalGen's user studies surfaced something deeper: even human reviewers experience *criteria drift*. Over time, annotators refined what "correctness" or "polish" meant, revised prior judgments, and added new failure categories that nobody anticipated up

front. This means a review interface can support evaluation as an evolving process, not a static labeling task. We can support going back and revising earlier labels, adding new failure categories midstream, and updating rubrics as understanding deepens.

EvalGen 的用户研究揭示了一个更深层的现象：即使是人类评估者也会随着时间而改变评判标准。随着时间的推移，评估者会逐渐明确"正确"或"完善"的含义，修正之前的判断，并添加一些事先未曾预料到的失败类别。这意味着评估界面能够支持评估过程的发展，而不仅仅是一个静态的标签标注任务。我们可以支持用户回顾并修改之前的标签，在评估过程中添加新的失败类别，同时随着理解的加深而更新评估标准。

WARNING 警告

If your review interface does not let reviewers revise earlier labels, criteria drift will silently corrupt your dataset. Reviewers refine their understanding of "correct" as they see more examples, and the labels from the first 20 traces may no longer match the labels from the last 20.

如果您的评审界面不允许评审者修改之前的标签，那么随着评审者看到更多样本，评估标准会逐渐发生变化，从而导致数据集的完整性被破坏。评审者随着了解的增加，对"正确"的定义也会发生变化，最初 20 个样本的标签可能会与最后 20 个样本的标签不再匹配。

This connects directly to the collaborative evaluation practices discussed in Chapter 4, where teams work to align on what quality means. A review interface can make that alignment process easier to do regularly.

这与第 4 章中讨论的协作评估实践直接相关。在那些实践中，团队共同努力来确定所谓"质量"的含义。而统一的评审界面则使得这种一致性评估过程能够更频繁地实施。

Selecting Traces for Human Review

选择需要人工审核的痕迹数据

Even with a well-designed interface, reviewing every production trace is impossible. LLM agents can produce thousands or millions of outputs, and only a fraction can get human review. The question is which traces to show.

即使拥有设计良好的界面，也不可能逐一审查所有的生产记录。LLM 代理能够生成数千甚至数百万条输出结果，而只有其中一小部分需要人工审核。问题在于，应该展示哪些记录。

Random Sampling 随机抽样

The simplest approach is to randomly sample traces from the production stream. Random sampling is unbiased and gives a general picture of system performance when the sample size is large enough. However, it misses rare failure modes. If most outputs are already good, reviewers spend their time confirming quality rather than finding problems.

最简单的方法是从生产流中随机抽取样本。随机抽样是无偏见的，当样本量足够大时，它能够提供系统性能的整体情况。不过，这种方法无法发现罕见的故障模式。如果大多数输出结果都符合预期，那么审核人员就会把时间花在确认质量上，而不是寻找问题。

Uncertainty Sampling 不确定性采样

Uncertainty sampling focuses review on traces where automated evaluators disagree or appear unsure. The idea is borrowed from *active learning* in machine learning, but the application to LLM trace review is slightly different. In traditional active learning, a model identifies the training examples it is least sure about and asks a human to label only those, so human effort goes where it is most needed. In LLM trace review, we don't have a single model's uncertainty scores; instead, we measure disagreement between multiple automated evaluators (e.g., two different LLM judges, or an LLM judge and a rule-based check). When evaluators disagree on a trace, that trace is likely ambiguous or borderline, and reviewing it by hand gives us the most information about where our evaluation criteria need tightening.

不确定性采样技术主要关注那些被自动化评估器产生分歧或显得不确定的样本。这一思路源自机器学习中的主动学习概念，但其应用在大型语言模型样本评估中略有不同。在传统的主动学习中，模型会识别出自己最不确定的训练样本，然后由人类来对这些样本进行标注，这样就能让人类的努力发挥最大的作用。而在大型语言模型样本评估中，我们不再依赖单一模型的评估分数，而是通过多个自动化评估器之间的分歧来衡量样本的不确定性。当多个评估器对某个样本存在分歧时，这个样本很可能具有模糊性或边界性，因此手动审查这类样本能够为我们提供关于哪些评估标准需要改进的最有价值的信息。

We can identify uncertainty in several ways:

我们可以通过多种方式来识别不确定性：

- A trace fails some evaluation criteria but passes others, suggesting borderline quality.

一个产品虽然通过了某些评估标准，但未能满足其他标准，这表明其质量处于边缘状态。

- An LLM judge produces inconsistent results when run multiple times with temperature greater than zero.

  当温度值大于零时，使用 LLM 进行多次判断会得到不一致的结果。

- Different evaluators (e.g., a code-based check versus an LLM judge) give conflicting judgments.

  不同的评估者（例如，基于代码的评估与由大型语言模型进行的评估）会得出相互矛盾的判断。

This last case is what EvalGen adopted: it surfaced traces for human review specifically when automated judges disagreed.

最后一个案例就是 EvalGen 所采用的方案：当自动化判罚者出现分歧时，系统会生成人类可以审核的追踪记录。

Failure-Driven Sampling 基于失败的抽样方法

Failure-driven sampling pulls traces that some automated signal has already flagged as likely problems. For our real estate assistant, one such signal is a guardrail catching an unsolicited financial recommendation in a generated email; another is a user clicking thumbs-down. Any trace carrying a failure marker goes into the review queue.

基于故障驱动的采样方式会选取那些已被自动化系统标记为可能存在问题的痕迹。以我们的房地产助手为例，其中一个触发此机制的信号是：在生成的电子邮件中发现了未经用户请求的金融推荐内容；另一个则是用户点击了“不赞成”按钮。任何带有故障标记的痕迹都会被放入审核队列中。

Pulling those traces is the fastest way to understand *why* the system fails on cases we already know are bad. A trace shows the whole arc of what happened, which usually points to where in the pipeline things went wrong.

追踪这些线索是了解为什么在那些我们已经知道存在问题的案例中系统会出现故障的最快方法。一条线索能够揭示事件的完整经过，这通常能帮助我们找到问题出在流程的哪个环节。

This approach catches most of the obvious problems, but it only catches what our existing signals are already configured to look for. A subtle failure mode that slips past every guardrail—say, the model confusing a neighborhood's school district—will never

show up here, no matter how many traces we review. In ML terms, precision is high but recall is low. That is why we cannot rely on failure-driven sampling alone: to catch these unknown unknowns, we also need to review traces the system rates as fine.

这种方法能够捕捉到大多数明显的问题，但它只能发现那些我们现有的系统已经能够识别的异常情况。而那些被忽视的细微缺陷——比如模型将某个邻域的学区误判为其他学区的情况——无论我们审查多少条记录，都绝不会被发现。用机器学习的话来说，这种方法的精确性很高，但召回率却很低。因此，我们不能仅仅依赖基于异常情况的采样方法来检测这些未知的异常情况：为了发现这些被忽视的异常情况，我们还需要审查那些系统认为正常的记录。

In practice, we can blend all three. We can use random sampling as a baseline, uncertainty sampling to catch the ambiguous cases, and failure-driven sampling to handle the obvious problems.

在实践中，我们可以将这三种方法结合起来使用。我们可以以随机抽样作为基础，同时使用不确定性抽样来捕捉那些模糊不清的情况，而通过基于故障驱动的抽样来处理那些明显的问题。

A reasonable starting split is 50% failure-driven, 30% uncertainty-sampled, and 20% random, but the right ratio depends on your known failure rate. If failures are rare (e.g., under 5%), weight more toward random and uncertainty sampling, because failure-driven sampling will quickly exhaust the available failures. If failures are common (e.g., over 20%), weight more toward failure-driven sampling, because there are enough failures to learn from and random sampling would mostly show you things that already work. The random portion is easy to skip but provides the only unbiased estimate of overall system quality; without it, review sessions overrepresent edge cases and give a misleadingly pessimistic picture.

一个合理的分配比例应该是：50%用于基于故障的抽样，30%用于基于不确定性的抽样，另外 20%用于随机抽样。不过，具体的比例取决于你所知道的故障发生率。如果故障非常罕见（例如，低于 5%），那么应该更多地使用随机抽样和基于不确定性的抽样方法，因为基于故障的抽样可以迅速耗尽可用的故障样本。如果故障相当常见（例如，超过 20%），那么就应该更多地使用基于故障的抽样方法，因为这样的样本数量足够多，可以从中学习到有用的信息。随机抽样虽然可以跳过，但它能给出对系统整体质量的公正评估；如果没有随机抽样，那么分析结果可能会过于悲观，从而误导用户。

Navigating Trace Groups and Discovering Patterns

遍历细分组并发现模式

Reviewing individual traces catches specific errors, but many issues only become visible when we step back and look at groups of examples. Good review interfaces include tools for exploring patterns across similar traces, especially through clustering and search (see Chapter 11 for more).

审查单个痕迹可以发现特定的错误，但许多问题只有在我们退一步，从整体上观察这些例子时才会显现出来。优秀的审查界面会提供工具，帮助我们发现相似痕迹之间的模式，尤其是通过聚类和分析来实现这一点（更多内容请参见第 11 章）。

Clustering 聚类分析

Clustering groups traces that share common features. We can cluster on simple metadata or on semantic similarity.

将具有共同特征的群组进行聚类。我们可以根据简单的元数据或语义相似性来进行聚类。

For our real estate assistant from Example 3-1, we might cluster traces by client persona (“investor,” “first-time buyer”) or by feature used (email generation versus property search). Suppose we pull up the “investor” cluster and review five traces. Four out of five emails open with “Dear [name], I hope this message finds you well” and spend two paragraphs on pleasantries before mentioning any properties. We have found a systematic tone problem: the agent writes casual, relationship-building emails when investor clients expect concise, data-driven summaries. Without clustering, these five traces would be scattered among thousands, and the pattern would be invisible.

在 Example 3-1 中的房地产助理案例中，我们可以根据客户类型（如“投资者”或“首次购房者”）或使用的功能来将记录进行分组。假设我们查看“投资者”这一分组中的五条记录。在这五封邮件中，有四封以“亲爱的[姓名]，希望这封信能找到你时好”开头，并先写了两段问候语，之后才提及任何房产信息。我们发现了一个明显的风格问题：代理人撰写的邮件风格较为随意，适合建立关系，而投资者则期望收到简洁、基于数据的摘要信息。如果没有进行分组处理，这五条记录会分散在成千上万的记录中，从而无法看出其中的规律。

We can also cluster semantically. Instead of using metadata, we can embed the inputs (such as user queries) or outputs (such as generated emails) and group them based on vector similarity using algorithms like k-means. For example, semantic clustering might uncover a latent category: all queries about “affordable condos near good schools.” Reviewing that cluster shows that the agent consistently ignores HOA fees and misunderstands school-quality indicators, quoting distance to the nearest school

instead of actual ratings. This is a systematic factual gap that we can fix with a focused prompt change or tool addition.

我们还可以基于语义进行聚类。无需使用元数据，我们可以将输入数据（如用户查询）或输出结果（如生成的电子邮件）嵌入到模型中，然后利用 k-means 等算法根据向量相似性对它们进行分组。例如，语义聚类可能会发现一个潜在的类别：所有关于“价格合理的公寓，位于优质学校附近”的查询。仔细分析这个类别后，会发现客服人员总是忽略物业费用的问题，并且对学校质量的评估存在误解，他们总是引用最近学校的距离，而不是实际的评分。这是一个系统性的事实缺陷，我们可以通过针对性的提示改进或添加工具来修复这个问题。

Interfaces can surface these clusters visually so reviewers quickly find common patterns, investigate root causes, and prioritize fixes.

这些界面可以直观地展示这些集群，从而让审核人员能够快速发现常见模式，探究根本原因，并确定优先解决的方案。

Search and Similarity Tools

搜索与相似性工具

Clustering helps with broad exploration. But sometimes, after spotting one problematic trace, we want to find more like it. Review tools should support searching by keyword, metadata, or semantic content.

聚类分析有助于进行广泛的探索。不过，有时候，在发现了一个问题线索后，我们还会希望找到更多类似的情况。因此，搜索工具应该支持按关键词、元数据或语义内容进行搜索的功能。

We can search by keyword to locate traces with specific terms, filter on metadata (agent_version ， user_segment , failure tags) to slice the data precisely, or use semantic search to find conceptually similar traces even when wording varies. For example, if one email from our real estate assistant gives unsolicited financial advice, semantic search can surface other emails with the same problem, even if they use different phrasing.

我们可以通过关键词搜索来查找包含特定术语的记录；也可以根据元数据进行过滤（如 agent_version 、 user_segment 、故障标签等），以精确筛选数据；此外，还可以使用语义搜索，即使措辞有所不同，也能找到概念上相似的记录。例如，如果我们收到房地产助手发送的邮件中包含了未经请求的财务建议内容，那么语义搜索就能找到其他包含类似问题的邮件，即使这些邮件的措辞不同。

Together, clustering and search let reviewers move past one-off examples and uncover systematic failures.

通过结合聚类分析和搜索功能，审稿人能够超越个别案例，发现系统中的系统性问题。

Integrating Human Review into Engineering Workflows

将人工审核功能整合到工程工作流程中

A review interface only matters if people use it regularly and if the feedback actually leads to changes in the system. We make human review part of the team's normal engineering workflow. If nobody is using the interface, we probably have not reduced the friction enough.

只有当人们经常使用这个界面，并且反馈能够真正促使系统发生变革时，那么评审机制才具有意义。我们将用户评审作为团队日常工程工作流程的一部分。如果没人使用这个界面，那说明我们可能还没有减少使用过程中的摩擦。

To support regular use, we set up automated systems that surface the right traces at the right time. For example, we might run a daily or weekly job that selects a batch of traces using the sampling strategies from "Selecting Traces for Human Review" and notifies reviewers via Slack or email when the batch is ready. For high-priority issues (safety violations, broken tool calls, major regressions), we trigger alerts immediately and route them to on-call reviewers.

为了支持日常的使用，我们建立了自动化系统，能够在适当的时间提取出相关的记录。例如，我们会定期或每周运行一次任务，根据"选择用于人工审核的记录"中的抽样策略，选取一批记录进行审核。当这些记录准备好后，我们会通过 Slack 或电子邮件通知审核人员。对于高优先级的问题（如安全漏洞、工具调用错误、严重的问题修复等），我们会立即触发警报，并将这些问题分配给在场的审核人员处理。

Review feedback can flow back into the agent in two ways:

反馈可以通过两种途径返回到代理程序中：

Golden datasets 黄金数据集

Traces labeled by reviewers (especially failed ones) go into the CI test set for regression testing. This connects human review to the automated evaluation in-

frastructure discussed in Chapter 9.

被评审人员标记为不合格的条目会被纳入集成测试集，用于回归测试。这将人类评审与第 9 章中提到的自动化评估体系联系起来。

LLM-as-judge calibration

法学硕士作为法官的校准机制

Human labels let us monitor whether automated judges are still accurate. If humans repeatedly flag outputs as incorrect that the judge rated as Pass, we revise the judge's prompt, retrain the model, or swap it out entirely (Chapter 5).

人类的判断可以帮助我们监控自动化评委的判罚是否仍然准确。如果人类反复将某些输出结果标记为错误，而评委却将其评为“通过”，那么我们就需要修改评委的评判标准，重新训练模型，或者完全更换评委。（详见第 5 章）

To keep this loop tight, we can add simple actions to the interface. A reviewer can click a button to add a trace directly to the CI golden set, or file a bug report with trace details already prefilled. A dashboard can show which failure types appear most often and where reviewers disagree with LLM judges, so the team can prioritize fixes.

为了维持这个循环的运行，我们可以在界面中添加一些简单的操作。评审员可以点击按钮将日志直接添加到 CI 黄金集中，或者提交包含已填充详细信息的错误报告。仪表板可以显示哪些类型的故障出现频率最高，以及评审员与 LLM 评委在哪些方面存在分歧，这样团队就可以优先处理这些问题了。

Example Walkthrough: Reviewing Real Estate Assistant Emails

示例操作指南：查看房地产助理的电子邮件

To make these ideas concrete, we walk through how a review interface might evolve for our real estate assistant that drafts client emails in Example 3-1. Each stage shows how interface choices affect usability, speed, and the quality of feedback.

为了让这些想法变得具体可行，让我们来看看如何设计一款评审界面，以便我们的房地产助手能够更高效地起草客户邮件，如示例 3-1 所示。每个阶段都展示了不同的界面设计选择对可用性、处理速度以及反馈质量的影响。

Starting with a Spreadsheet

从电子表格开始吧

The most basic interface is a spreadsheet. We export logs (the user's query, client persona, and the generated email) into Google Sheets.

最基本的接口形式是一个电子表格。我们会将日志数据（用户的查询内容、客户角色信息以及生成的电子邮件内容）导出到 Google 表格中。

Here is what a single row looks like. Column A holds the trace ID: `tr-8a3f` . Column B has the timestamp. Column C contains the user query, but it is truncated to "Find me 3-bedroom homes under $600k near downto..." Column D has the client persona: "first-time buyer, young couple, budget-conscious." Column E holds the generated email, but the cell shows only the first line: "Dear Sarah and James, I hope this email finds you well. I wanted to..." To read the rest, we double-click and get a wall of unformatted text with no line breaks. Column F is the "Notes" column, where three different reviewers have typed comments separated by semicolons: "ok; seems long; tone issue maybe?"

这是单行数据的样子。A 列包含了跟踪 ID： `tr-8a3f` 。B 列是时间戳。C 列显示了用户查询内容，但被截短为"查找价格在 60,000 美元以下、位于.....地区的三居室住宅"。D 列记录了客户画像："首次购房者，年轻夫妇，注重预算"。E 列包含了生成的电子邮件内容，但单元格中只显示了第一行文字："亲爱的莎拉和詹姆斯，希望你们一切都好。我想....."。要阅读其余内容，需要双击鼠标，就会看到一堆未格式化的文本，没有换行符。F 列是"备注"列，这里有三位评审员留下的评论，用分号分隔："不错；内容有点长；语气方面可能有问题？"

After 10 traces, we realize we cannot tell whether the listed properties actually match the budget constraint because the tool call results are in Column G as a raw JSON string: `[{"address": "123 Main St", "price": 580000, ...}, {"address": ...}]` . We copy-paste it into a JSON formatter in a separate browser tab to read it.

在检查了 10 个候选方案之后，我们发现无法判断所列出的属性是否真的符合预算限制条件。因为工具调用的结果以原始 JSON 字符串的形式显示在 G 列中。我们将这些结果复制粘贴到另一个浏览器标签页中的 JSON 格式化工具中，以便进行进一步分析。

This is slow, unstructured, and error-prone.

这种方式的效率很低，缺乏结构化的处理，而且容易出错。

A Simple Custom UI

一个简单的自定义用户界面

A clear step up is building a lightweight custom UI using Streamlit, Gradio, or an AI coding assistant, along the lines of Figure 10-2. The UI shows one trace at a time with the user input, persona context, and generated email clearly separated. Reviewers click Good Email or Bad Email buttons and leave optional comments. Navigation buttons (Next/Previous, not pictured) keep the workflow moving. Even this minimal interface makes a big difference: the layout is readable, decisions are faster, and feedback has structure.

一个明显的改进是使用 Streamlit、GRadio 或 AI 编程助手来构建轻量级的自定式用户界面，就像图 10-2 所示。该界面能够同时显示一条追踪数据，其中用户输入、用户角色信息以及生成的电子邮件内容都清晰明了。评审人员可以点击“优秀电子邮件”或“不良电子邮件”按钮，并留下自己的评论。导航按钮（下一页/上一页，图中未显示）则有助于保持工作流程的流畅性。即便只是这样一个简单的界面，也能带来巨大的提升：布局更加清晰易读，决策过程更加迅速，而反馈也更有条理。

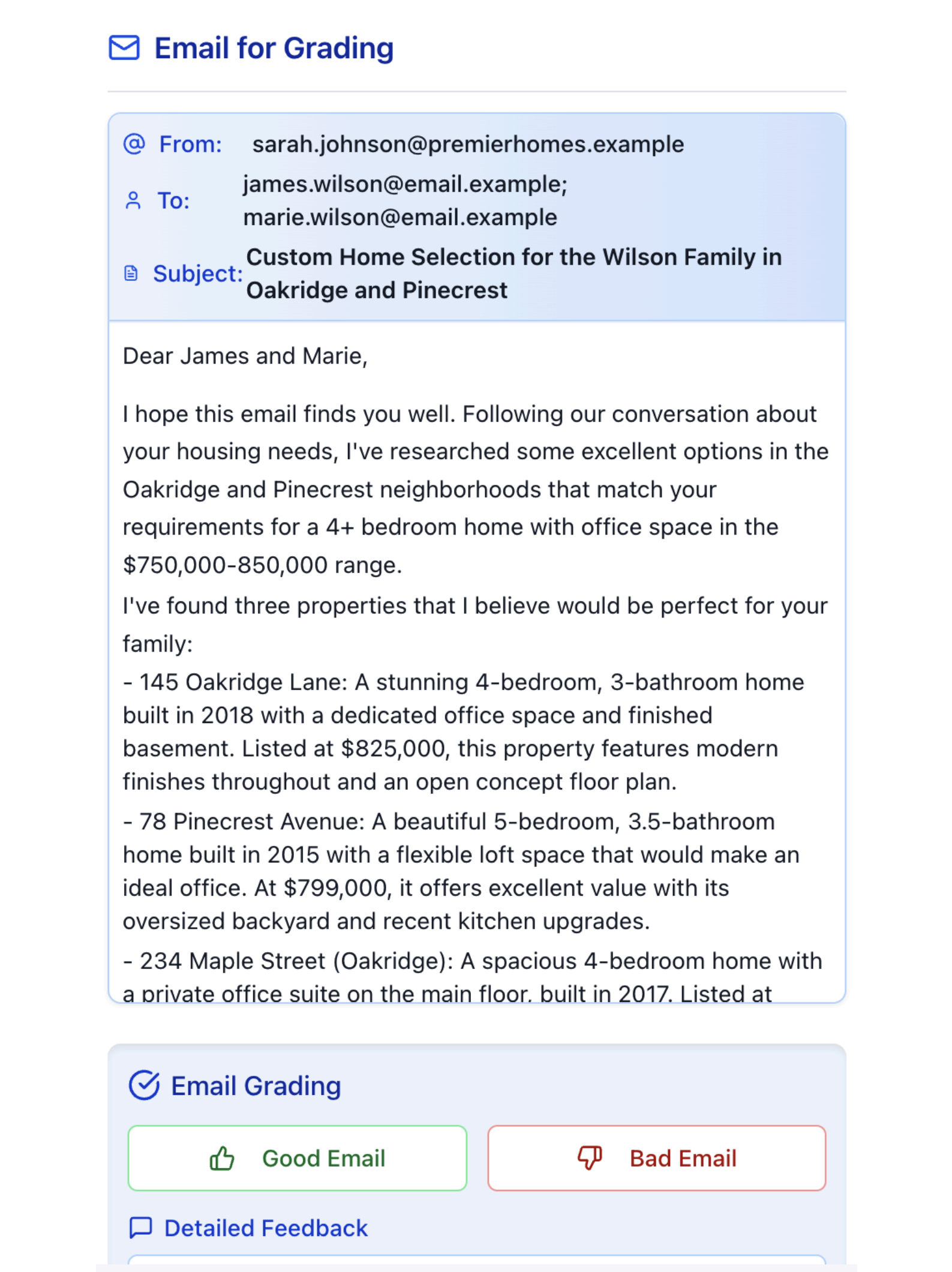


Figure 10-2. A basic custom interface for reviewing real estate assistant emails

图 10-2. 用于查看房地产助手邮件的基本自定义界面

An Advanced Review Interface

高级审核界面

A basic UI still misses features that matter. It does not show intermediate steps like tool calls or external context retrieved during RAG steps. There are no structured failure tags (the axial codes from Chapter 3), no way to highlight specific problems in the output, and no support for reviewing related traces together.

基本的用户界面仍然缺少一些重要的功能。它不会显示中间步骤，比如工具调用或在 RAG 过程中获取的外部上下文信息。也没有结构化的错误标签（第三章中的轴向代码），无法突出显示输出中的特定问题，同时也不支持一起审查相关跟踪数据。

Figures 10-3 and 10-4 show how an advanced interface handles groups of related traces, and Figure 10-5 shows the view for a single trace. In the cluster view, an email can appear in more than one cluster, so the cluster counts add up to more than the total.

图 10-3 和图 10-4 展示了高级界面如何处理相关轨迹的组。而图 10-5 则展示了单个轨迹的视图。在集群视图中，一个电子邮件可能会出现在多个集群中，因此集群数量的总和会超过实际的总数。

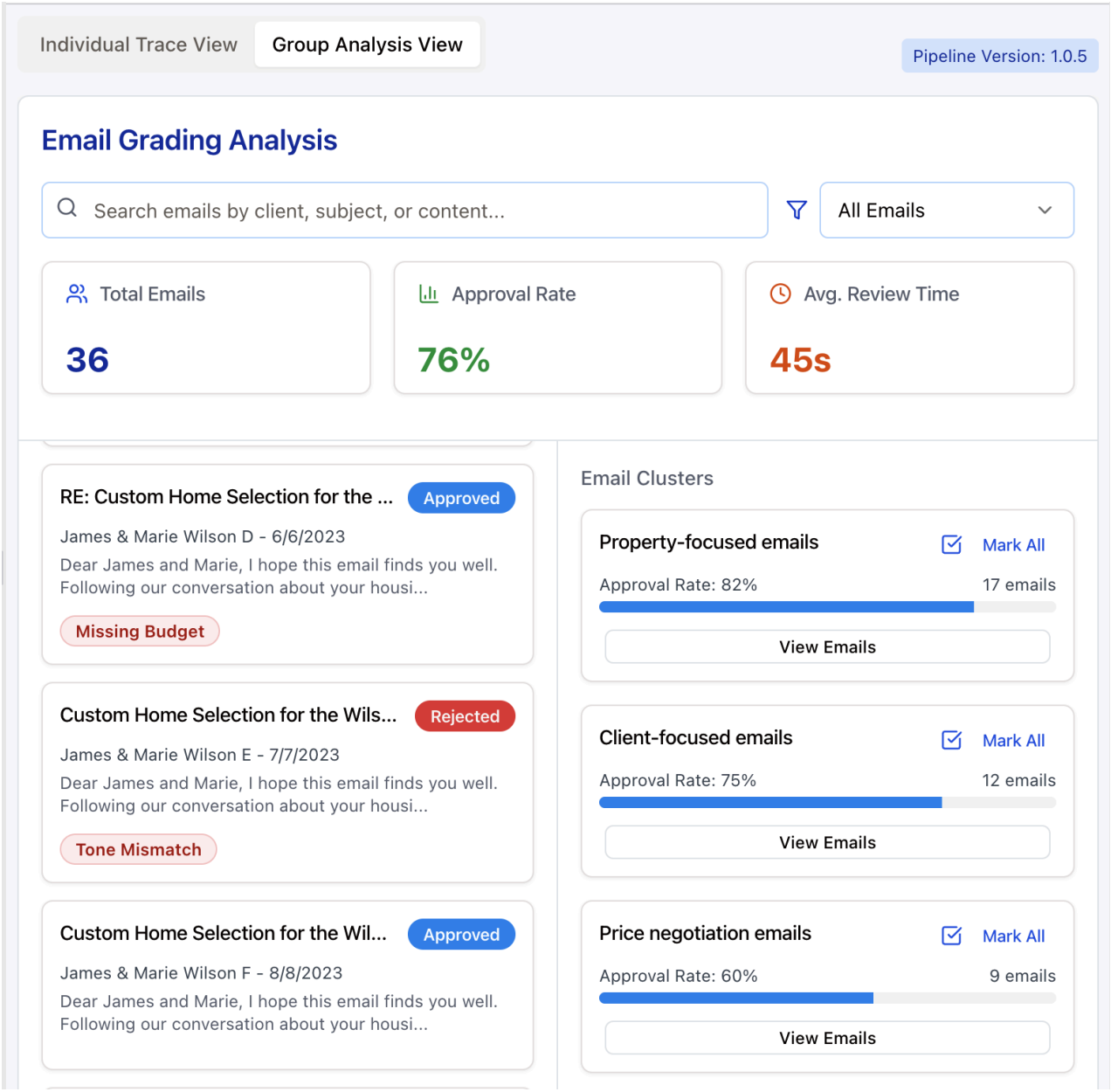


Figure 10-3. Cluster view showing groups of emails (such as property-focused or client-focused examples) that reviewers can drill into (full-sized version)

图 10-3：集群视图，展示了评论者可以深入查看的电子邮件分组（例如以房产或客户为中心的分组）。（全尺寸版本）

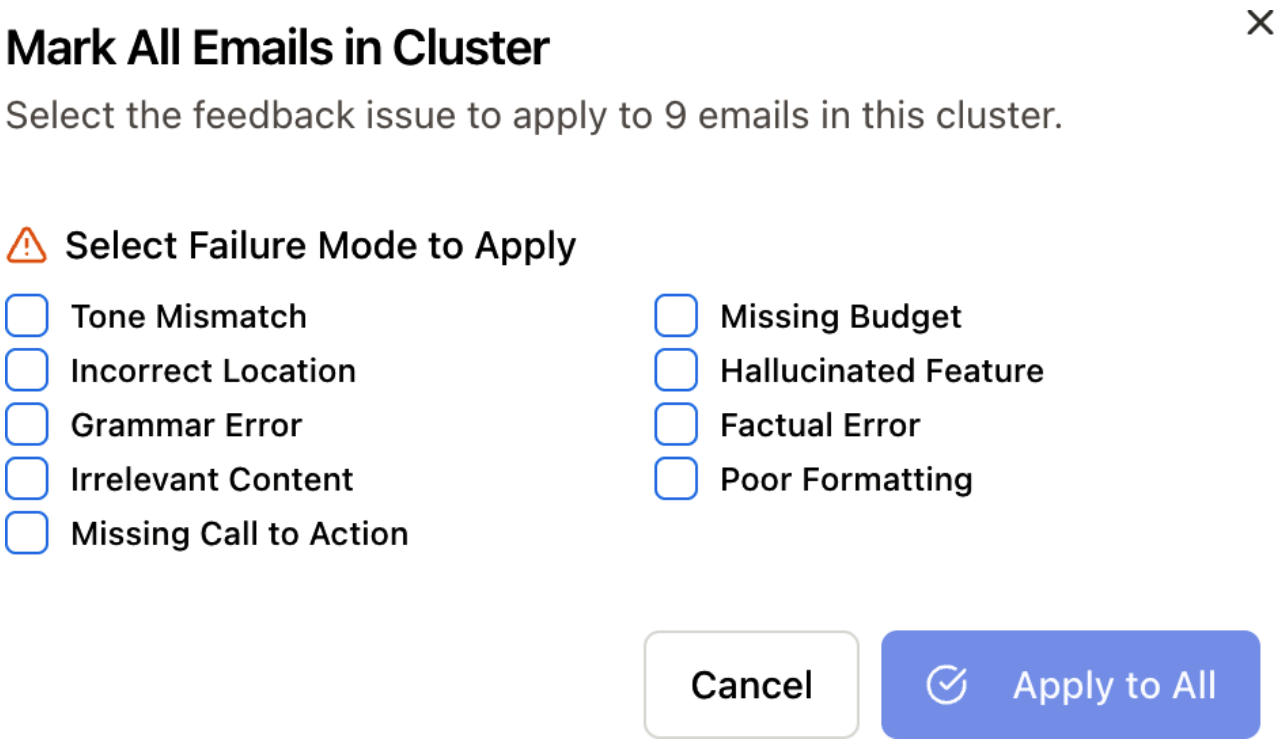


Figure 10-4. Bulk tagging interface for labeling all traces in a cluster with a shared failure mode (such as Tone Mismatch)

图 10-4. 批量标记接口，用于标记包含共同故障模式的所有轨迹（例如音质不匹配等故障模式）。

The view shown in Figure 10-5 presents client information, the full email for grading, and structured feedback options including predefined failure modes (such as Tone Mismatch, Missing Budget, Hallucinated Feature). Contextual information like Trace ID and Pipeline Version is also visible.

如图 10-5 所示，该界面展示了客户端信息、评分所需的完整电子邮件地址，以及包括预定义错误模式在内的结构化反馈选项（如语气不匹配、预算缺失、幻觉性功能等）。同时，上下文信息如跟踪 ID 和管道版本也可见。

The advanced interface brings all of this together. Individual traces appear with rich, readable layouts that show the input query, persona context, and the full output alongside any intermediate reasoning or tool usage. Emails render with clear visual formatting, highlighting key entities like price, location, or timing so errors stand out immediately.

先进的界面将所有这些信息整合在一起展示。各个跟踪记录以清晰易读的格式呈现，其中包括输入查询、角色背景以及完整输出结果，同时还会显示中间的推理过程或使用的工具。电子邮件则通过清晰的视觉格式呈现，能够突出显示诸如价格、地点或时间等关键信息，这样就能立即发现错误。

Reviewers select predefined failure tags (like “Tone Mismatch” or “Hallucinated Feature”), write free-form notes (the open codes from Chapter 3), and use hotkeys to speed up review (N for Next, D to Defer, T for Tone). Contextual metadata (trace ID,

pipeline version) appears directly in the interface to help with triage.

评审人员会选择预定义的故障标签（如“语气不符”或“幻觉效果”），撰写自由形式的评论（即第三章中的开放代码），并使用快捷键来加快评审速度（N 表示下一步，D 表示延迟处理，T 表示调整语气）。上下文相关的元数据（如跟踪 ID、管道版本）直接显示在界面上，以便更好地进行分类处理。

The interface also supports group-level exploration. Clustering groups similar emails, such as those for “investor clients” or “affordable condo searches.” Reviewers examine a few examples from each cluster to spot patterns and apply bulk labels to speed things up. Search tools find similar examples (for instance, “all emails that sound too apologetic”) across the dataset.

该界面还支持按群组进行搜索的功能。可以将类似的电子邮件分组，例如“投资者客户”或“价格合理的公寓搜索”等。查看者可以逐一检查每个群组中的示例，以发现规律，并应用批量标签来加快搜索速度。搜索工具还能在数据集中找到类似的示例（例如，“所有那些听起来过于道歉的电子邮件”）。

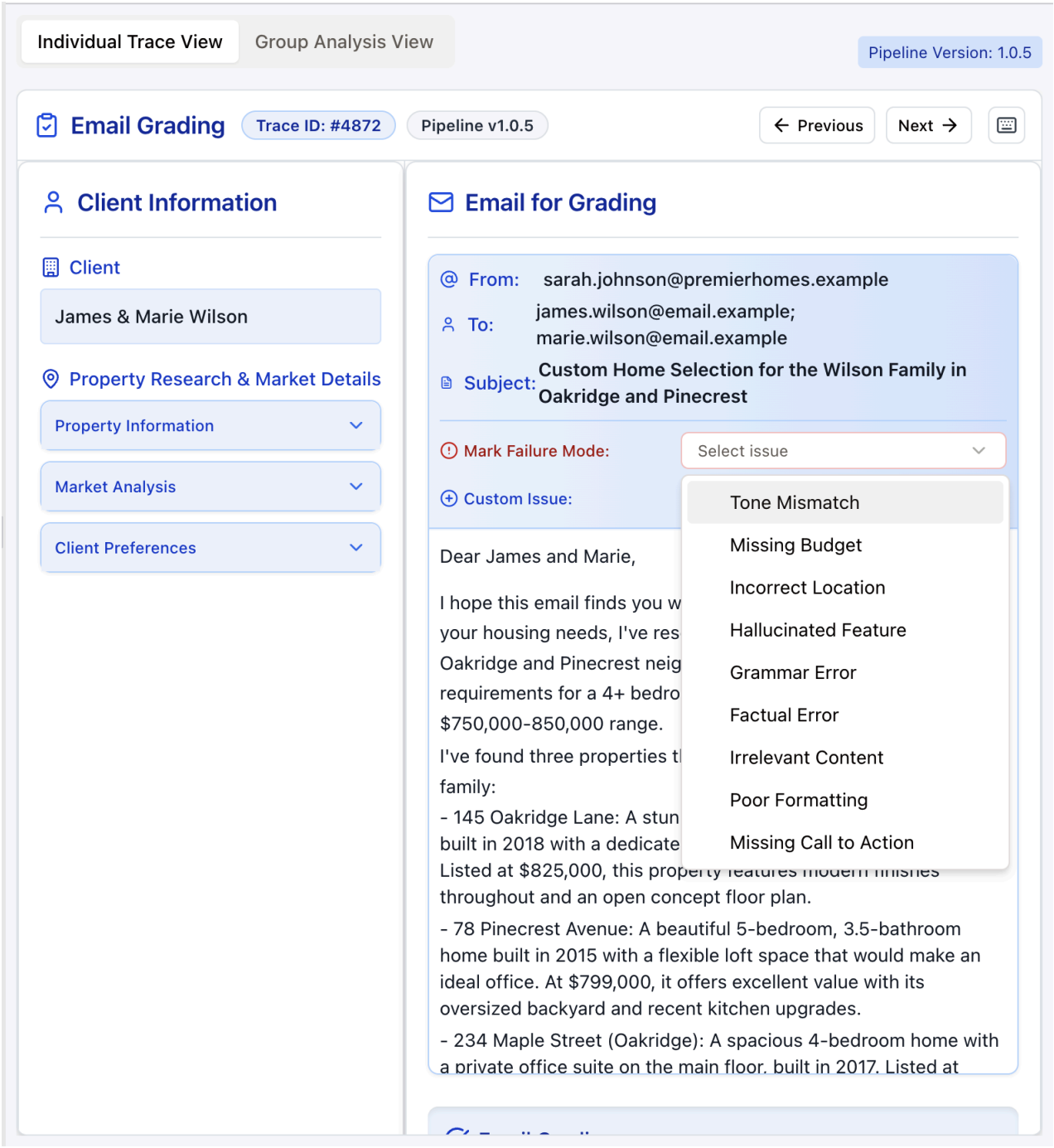


Figure 10-5. An advanced custom UI for individual trace review of the real estate assistant

图 10-5. 用于单独查看房地产相关数据的高级自定义用户界面，由房地产助手提供。

With the right interface, reviewing LLM agent outputs becomes faster, more consistent, and more useful. The highest-leverage features in practice are hotkeys for faster navigation, structured failure tagging to reduce ambiguity, and clustering to surface repeated failure modes.

通过合理的界面设计，审查大语言模型代理的输出变得更加快速、一致且有用。在实际应用中最有影响力的功能包括：快捷键以加快导航速度、结构化的错误标记以减少歧义，以及聚类功能以发现重复的故障模式。

For agentic systems, the review interface should also surface operational signals beyond the text output: how many tool calls the agent made, which tools it called and in what order, how many retrieval results were returned and whether the agent actually

used them, the rank of the retrieval results that ended up in the final answer, token counts and cost per trace, and latency breakdowns by step. These signals often reveal problems that the text output alone doesn't show, such as an agent that produces a correct answer, but at 10× the expected cost, because it looped unnecessarily.

对于代理式系统来说，评估界面还应能够显示除文本输出之外的其他操作信息：代理调用了多少个工具，这些工具以什么顺序被调用，返回了多少个检索结果，代理是否真的使用了这些结果，最终答案中检索结果的排名情况，每个跟踪的标记数量及成本，以及每步操作的延迟时间等。这些信息往往能够揭示出文本输出无法呈现的问题，比如某个代理虽然给出了正确的答案，但由于不必要的循环操作，导致成本达到了预期值的 10 倍。

Case Study: DocWrangler for Prompt Refinement

案例研究：使用 DocWrangler 进行 prompt 优化

Review interfaces can also influence how we improve our systems, not only how we evaluate them. *DocWrangler* is a purpose-built IDE for LLM-powered document processing (Shankar et al. 2025). It demonstrates many of the interface principles from this chapter, particularly how layout, interaction design, and lightweight feedback mechanisms accelerate iterative development.

评审界面也可以影响我们改进系统的方式，而不仅仅是评估方式。DocWrangler 是一款专为基于大语言模型的文档处理而设计的集成开发环境（Shankar 等人，2025 年）。它体现了本章所讨论的许多界面设计原则，尤其是布局、交互设计以及轻量级反馈机制如何促进迭代开发过程。

In DocWrangler, users author prompts to extract, reason about, and make sense of information from unstructured documents. The interface runs a prompt over a batch of documents, then lets users review the outputs one by one. Each screen shows the original document, the generated output, and the prompt that produced it, organized side by side to reduce friction when inspecting failures or inconsistencies. Figure 10-6

shows this review screen.

在 DocWrangler 中，用户可以编写提示语句来从非结构化文档中提取信息、分析数据并理解其中的内容。该工具会针对一批文档执行这些提示语句，然后让用户逐一查看生成的输出结果。每个界面都会同时显示原始文档、生成的输出结果以及产生这些结果的提示语句，这样在检查是否存在错误或不一致之处时就能更加方便。图 10-6 展示了这种查看结果的界面。

The interface encourages *open coding* (writing free-form notes about observed problems, as discussed in Chapter 3). Users quickly leave natural-language comments on any example, jotting down observations like "missed the date field," "output order is inconsistent," or "ignored second table." These lightweight notes are faster than formal labeling but still capture valuable signals. Over time, they help users notice recurring problems and refine their mental model of what is going wrong.

该界面鼓励开放式编码方式（即像第 3 章中所讨论的那样，对观察到的问题编写自由形式的注释）。用户可以在任何示例中快速添加自然语言注释，比如"遗漏了日期字段"、"输出顺序不一致"或"忽略了第二个表格"。这些简洁的注释比正式的标签更快速，但仍然能够捕捉到重要的信息。随着时间的推移，这些注释有助于用户发现反复出现的问题，并完善他们对问题发生机制的理解。

Once enough notes accumulate, users invoke the *prompt improvement* feature. This takes the current prompt and the accumulated reviewer comments and suggests an updated prompt aimed at fixing the issues. The interface presents both versions side by side for quick inspection, as in Figure 10-7.

当积累了足够的反馈意见后，用户可以调用"改进提示"功能。该功能会结合当前的提示信息以及用户积累的评论意见，提出一个更新的提示方案，以修复存在的问题。界面会同时展示两个版本的提示方案，以便用户快速查看，如图 10-7 所示。

While DocWrangler is tailored to document extraction, the same ideas apply broadly. Interfaces that streamline review and feed findings back into the system accelerate LLM agent development.

虽然 DocWrangler 是针对文档提取而设计的，但同样的理念也可以广泛应用于其他领域。能够简化审核流程并将分析结果反馈到系统中的界面设计，有助于加速大语言模型代理的开发进程。

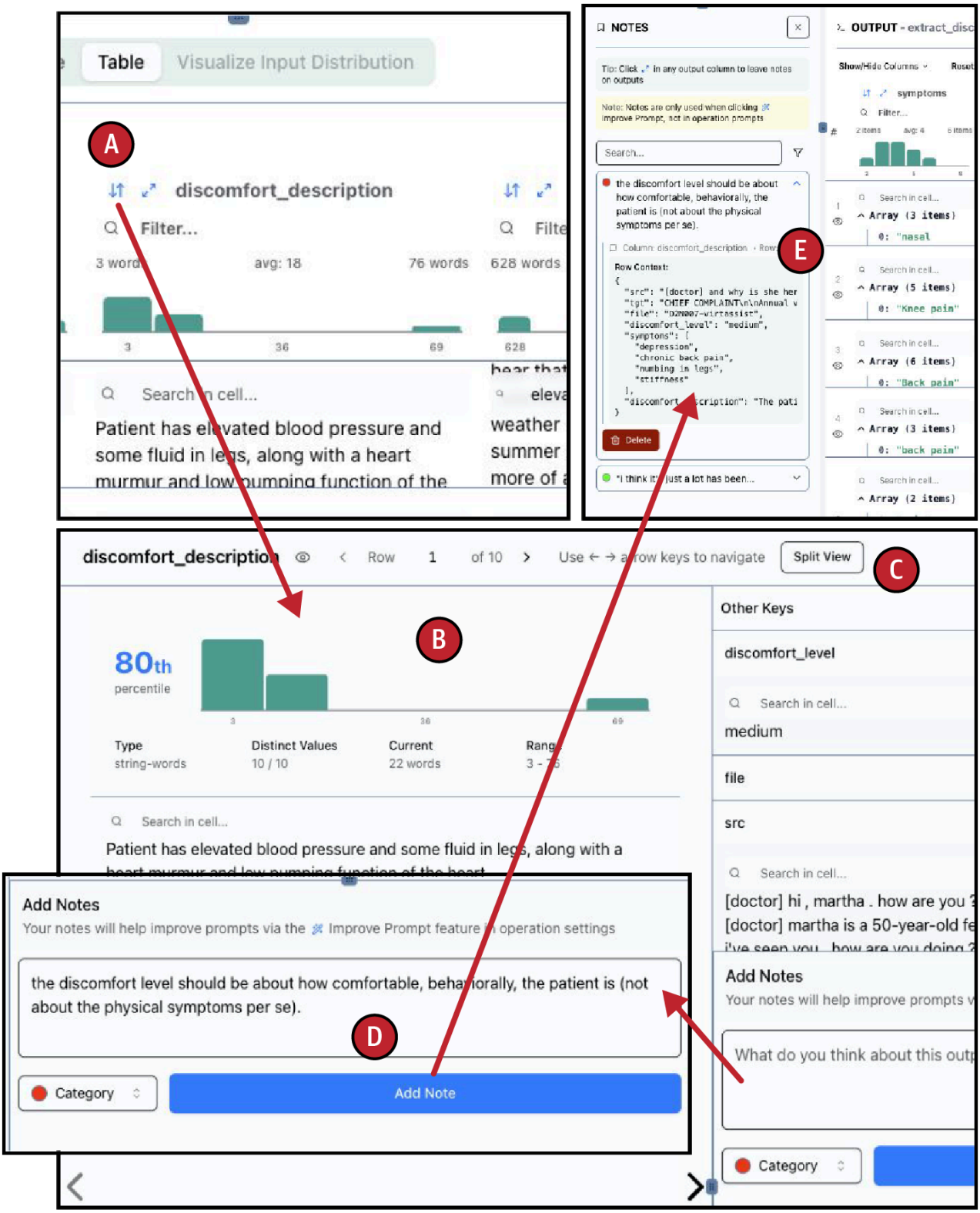


Figure 10-6. The DocWrangler interface for reviewing document-level outputs (full-sized version)

图 10-6. 用于查看文档级输出的 DocWrangler 界面（全尺寸版本）

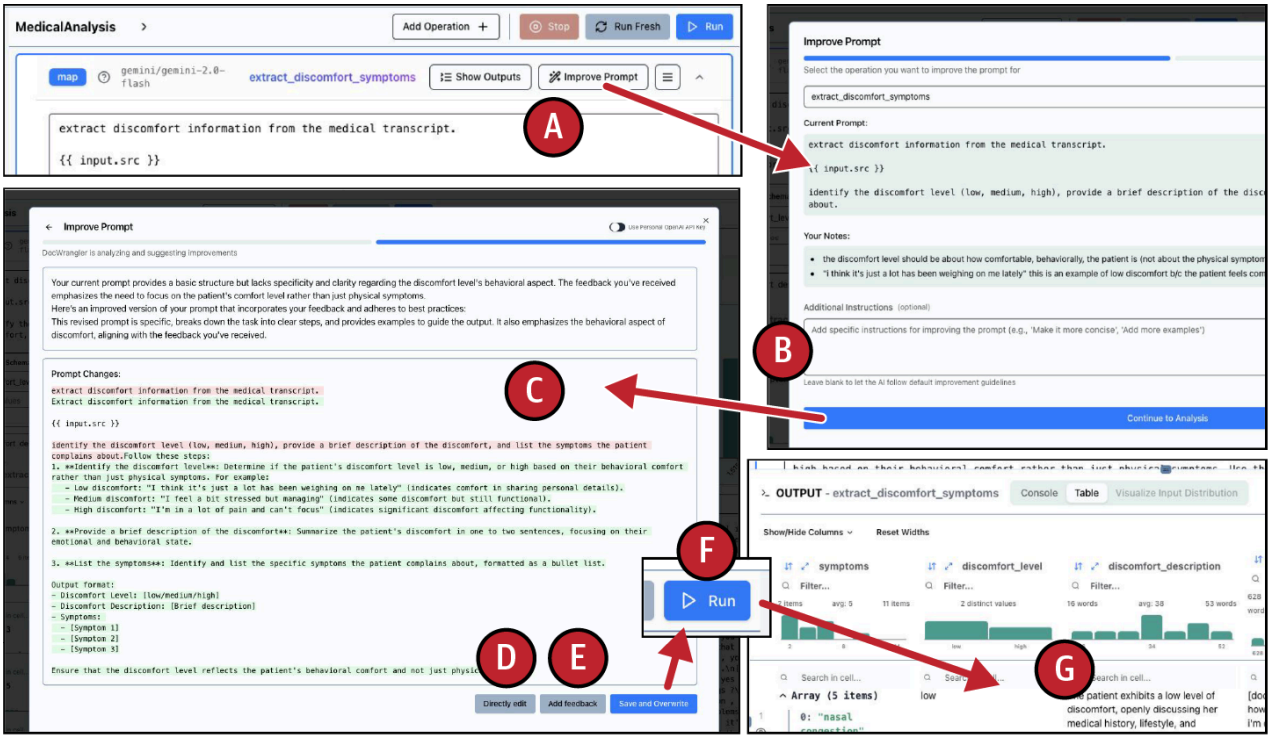


Figure 10-7. Prompt improvement workflow in DocWrangler (full-sized version)

图 10-7. DocWrangler 中的快速改进工作流程（全尺寸版本）

Common Pitfalls 常见陷阱

The most common mistake is not looking at enough traces. Teams often review a handful of examples after each change and move on, but subtle failure modes only appear after reading dozens or hundreds of traces, and rare failures (e.g., the agent sending an email to the wrong recipient) can have outsized impact even if they occur in fewer than 1% of sessions. Include a random sample of traces that passed all automated checks in every review session, because automated evaluators have blind spots and the only way to find them is to look at traces the evaluators thought were fine.

最常见的错误就是没有仔细检查足够的测试用例。团队通常每次修改后只会审查少数几个案例，而一些隐蔽的缺陷只有在阅读数十甚至数百个测试用例后才会被发现。而那些罕见的缺陷（例如，代理向错误的接收者发送邮件的情况）即使出现在不到 1%的案例中，也可能产生巨大的影响。因此，在每次评审中，应该随机选取一些通过所有自动化检查的测试用例，因为自动化评估工具也有其盲点，而发现这些盲点的唯一方法就是查看那些被评估者认为没有问题的测试用例。

A related issue is overrelying on evaluator disagreement for sampling. Sampling traces where automated evaluators disagree (as described in "Uncertainty Sampling") is powerful, but the approach is only as diverse as the evaluators in your pool are uncorrelated; if all your evaluators are LLM judges with similar prompts, they will agree on

the same cases and disagree on the same cases, leaving entire failure categories unsampled. Include at least one rule-based evaluator in the pool to add a different perspective.

另一个相关的问题是，人们过于依赖评估者之间的分歧来决定抽样方式。虽然由自动化评估者产生的分歧所带来的信息很有价值（如"不确定性抽样"中所描述的那样），但这种方法的有效性取决于你选择的评估者之间是否存在相关性；如果你的所有评估者都是使用类似提示的 LLM 评判员，那么他们就会对相同的案例持有相同的意见，从而无法对不同的案例进行区分，导致某些失败案例被完全忽略。因此，在评估者群体中至少应包含一个基于规则的评估者，以便引入不同的视角。

Finally, teams sometimes underestimate the value of a custom review interface. Off-the-shelf platforms (e.g., Langfuse, Braintrust, Arize Phoenix) provide trace visualization and annotation, and they are a reasonable starting point. But as your review process matures, a custom interface tailored to your specific workflow, with domain-specific views, keyboard shortcuts for your most common annotations, and inline diffs between prompt versions, can make reviewers significantly faster and more consistent.

最后，团队有时会低估了定制评审界面的价值。现成的平台（如 Langfuse、Braintrust、Arize Phoenix 等）提供了跟踪可视化和注释功能，它们是一个合理的起点。但随着评审流程的成熟，一个根据特定工作流程定制的界面，包含针对特定领域的视图、适用于最常见注释的快捷键，以及提示版本之间的对比功能，能够显著提升评审者的效率并确保一致性。

Putting It Together 整合起来

Automated evaluation scales oversight, but humans remain essential for catching subtle errors and refining what "quality" means. Custom review interfaces, designed with HCI principles, make human review faster and more consistent than spreadsheets or generic tools. Sampling strategies (random, uncertainty-based, failure-driven) focus limited reviewer attention on the most informative traces. Review only matters if it leads to action. Interfaces should make it easy to file bugs, update evaluation datasets, and trigger prompt improvements.

自动化评估可以替代人工审核，但人类在发现细微错误和明确"质量"标准方面仍然至关重要。基于人类交互设计的交互式审核界面能够比使用电子表格或通用工具更快速、更高效地完成审核工作。不同的抽样策略（如随机抽样、基于不确定性的抽样、以故障为驱动的抽样）可以将有限的审核注意力集中在最有价值的数据上。只有当审核能够促使实际行动时，审核才具有意义。界面应该能够方便用户提交缺陷、更新评估数据集以及触发改进措施。

In the next chapter, we zoom out from individual traces and look at how to analyze them in aggregate to find patterns that trace-by-trace review cannot surface.

在下一章中，我们将不再专注于单个痕迹，而是尝试从整体角度来分析这些痕迹，以发现那些单凭单个痕迹无法发现的模式。

Chapter 11. Data Analysis for Traces

第十一章 痕迹数据分析

Chapter 10 was about how we might use custom interfaces to accelerate our error analysis. This chapter is going to be about applying a broader *data analysis* mindset to reviewing traces. Error analysis tells us *why* something might look bad, and is actually a type of data analysis. We will cover other techniques you can use to spot patterns across many traces at once, and decide which ones to read closely.

第 10 章讨论了如何利用自定义接口来加速错误分析过程。本章将探讨如何运用更广泛的数据分析思维方法来审查各种轨迹数据。错误分析能够帮助我们理解为何某些情况看起来存在问题，实际上也是一种数据分析方式。我们还将介绍其他技术，这些技术可以帮助我们同时发现多个轨迹中的模式，并决定哪些模式需要特别关注。

In this chapter, you will learn:

在本章中，你将学到：

- How to use exploratory data analysis (EDA) and clustering to surface hypotheses about your traces for further investigation

  如何利用探索性数据分析（EDA）和聚类分析来揭示有关数据的假设，以便进行进一步的调查。

- How to compute and plot descriptive statistics over traces (turns, latency, tool calls, tokens, sentiment) and use them to spot patterns

  如何计算并绘制关于轨迹的描述性统计信息（如转弯次数、延迟时间、工具调用次数、标记数量、情感表达等），并利用这些统计信息来发现其中的模式。

- How to use semantic search to find more traces that look like a problem you have already spotted

  如何使用语义搜索来找到更多类似你已经发现的问题的痕迹？

- How to use AI coding assistants (Claude Code, Codex) to accelerate this work without letting them confabulate your analysis for you

如何使用 AI 编码助手（如 Claude Code、Codex）来加速这项工作，同时避免让这些工具自行分析你的数据。

EDA and Clustering to Surface Hypotheses

EDA 与聚类分析在提出假设中的作用

Exploratory data analysis (EDA) is the practice of "poking" at a dataset to generate hypotheses about it. The statistician John Tukey, who pioneered the box plot and many of the visual methods that data scientists still rely on today, introduced the term in the 1970s as a counterweight to *confirmatory* analysis, which is the familiar routine of fitting a model and running a significance test. Before we can confirm anything, we need to know what to confirm, and EDA is how we get there. We plot, group, filter, and read until we have a hypothesis sharp enough to test.

探索性数据分析（EDA）是一种对数据集进行探究以形成相关假设的方法。统计学家约翰·塔基在 20 世纪 70 年代提出了这一术语，他开创了箱形图以及许多数据科学家至今仍使用的可视化方法。EDA 作为一种补充手段，用于替代那种通过构建模型并进行显著性检验的确认性分析。在确认任何结果之前，我们必须先明确要确认什么，而 EDA 正是实现这一目标的方式。通过绘图、分组、筛选等方式，我们逐步得出足够有力的假设，从而可以进行验证。

For LLM traces, EDA is how we move from "something is off" to "here is a specific failure mode." Error analysis (Chapter 3) is itself a form of EDA: reading traces and grouping them into a taxonomy is the same loop applied to text. This section covers techniques that extend that practice beyond trace-by-trace reading.

对于 LLM 的跟踪数据，错误分析是一种将"有些地方存在问题"转化为"存在特定故障模式"的方法。错误分析（第三章已介绍）本身也是一种错误分析的形式：通过阅读跟踪数据并将它们分类，就像对文本进行分類一样。本节介绍了一些能够扩展这种分析方式的技术，使其不仅仅局限于逐条分析跟踪数据。

A huge mistake people make with EDA is treating a cluster as a failure mode. Clusters do not map cleanly to failure modes. The same goes for correlations, bar charts, and other EDA outputs. We treat them as prompts to go read the underlying traces. We start with a rough hypothesis, pick a slice of the data, read the traces in that slice, and

either sharpen the hypothesis or drop it. We expect to drop most of our first tries.

人们在执行异常数据分析时犯的一个重大错误是，将某个数据簇视为一种故障模式。实际上，数据簇并不能清晰地对应到具体的故障模式上。同样的情况也适用于相关性分析、条形图以及其他异常数据分析的结果。我们把这些结果当作提示，从而去阅读那些底层的数据记录。我们通常会先提出一个大致的假设，然后选取一部分数据来进行分析，再看看那些数据的记录情况，然后根据分析结果来修正或放弃这个假设。我们预计大多数情况下，第一次尝试就会失败。

Clustering 聚类分析

We introduced clustering in Chapter 10 as a way to surface patterns across groups of traces. Here we use it for hypothesis generation: we read a few representatives from each cluster and ask what their differences suggest about where the agent is going wrong.

我们在第 10 章中介绍了聚类技术，作为一种揭示不同轨迹群组之间模式的方法。在这里，我们运用这一技术来提出假设：我们从每个簇中选取一些代表性样本，分析它们之间的差异，从而判断代理在哪些方面出现了问题。

Sometimes we cluster and find nothing useful. We cannot tell the clusters apart, or we just see categories we already knew about (queries about condos, queries about houses). That is information too. Either our embedding is not capturing the variation we care about, or the variation we care about is not the dominant axis in the data. For our real estate assistant (see Example 3-1), we clustered on query embeddings and got the obvious groups (scheduling, search, email generation) but nothing about where the agent was failing. When we clustered the *final email* instead, we found a small group of messages that all opened with a long apology before getting to any listings. That was a tone problem we had missed in our earlier error analysis.

有时，我们会将信息归为不同的类别，但实际上并没有发现任何有用的信息。我们无法区分这些类别，或者看到的只是我们早已熟悉的类别（比如关于公寓的查询、关于房子的查询）。那些信息其实也是信息的一部分。要么是我们的嵌入方式没有捕捉到我们关心的各种差异，要么就是我们所关心的差异并不是数据中的主要特征。以我们的房地产助手为例（参见示例 3-1），我们按照查询嵌入进行了聚类，得到了一些明显的分组（如调度、搜索、生成电子邮件等），但并没有发现助手在哪些地方出现了错误。当我们以最终生成的电子邮件作为聚类对象时，我们发现了一组消息，这些消息都是以长篇道歉开始的，之后才涉及具体的房源信息。原来，这是一个我们在之前的错误分析中忽略的问题。

Beyond Clustering 超越聚类分析

Clustering is one tool in the EDA toolkit. We also look at per-trace metrics (number of turns, latency, tool calls, input and output tokens, sentiment, user segment, time of day) and ask how they are distributed, how they correlate, and how they break down by metadata. Here are a few data analysis techniques we have found useful:

聚类是 EDA 工具包中的一个功能。我们还研究了针对每条跟踪数据的各种指标（如转动次数、延迟时间、工具调用次数、输入输出标记、情感分析结果、用户分组、时间分布等）。我们探讨了这些指标的分布情况、它们之间的相关性，以及它们如何根据元数据进行分类。以下是我们认为一些有用的数据分析技术：

Plot the distribution of a metric.

绘制某个度量的分布图。

With a histogram or box plot of turns per session, latency, or tokens, we can see whether the metric is roughly unimodal, bimodal (two humps instead of one, often because we have two distinct groups of users or two failure modes mixed together), or long-tailed (a few traces much bigger than the rest). If we see a bimodal shape, we split the dataset and read each mode separately.

通过查看每会话的转数、延迟或代币数量的直方图或箱形图，我们可以判断该指标大致是单峰的、双峰的（有两个高峰，通常是因为存在两个不同的用户群体或两种故障模式相互交织），还是长尾分布的（少数数据点的数值远大于其他数据点）。如果我们看到双峰分布，那么就需要将数据集分成两部分，分别分析每个峰的情况。

Cross-tabulate metadata against outcomes.

将元数据与结果进行交叉分析。

If we log a thumbs-up/down signal, we can break it down by client persona, agent version, time of day, or session length. For example, we might chart thumbs-down rate by session length. If long sessions fail more often, we dig into that as our next hypothesis.

如果我们记录了点赞/反对的信号，那么我们可以根据不同的客户类型、代理版本、时间或会话时长来进行分析。例如，我们可以绘制出按会话时长划分的反对率图表。如果长会话的失败频率更高，那么我们就可以将这一结果作为我们的下一个假设进行深入探讨。

Look for correlations between metrics.

寻找各种指标之间的相关性。

We can plot tool calls against latency, or tool calls against thumbs-down rate, to see whether the two are related. If they are (e.g., traces with more tool calls also get more thumbs-down), we dig into that relationship.

我们可以绘制工具调用次数与延迟之间的关系，或者工具调用次数与拒绝率之间的关系，以判断这两者是否相关。如果它们确实存在关联（例如，工具调用次数较多的请求往往也会导致更多的拒绝情况），那么我们就可以进一步研究这种关系了。

Filter to a subset and read.

将过滤器应用于子集，然后进行读取。

Pick an interesting slice (thumbs-down traces, traces over five turns, traces with a specific tool call) and read 20 of them back to back. Reading in a concentrated batch is how we catch patterns we would miss reading one trace at a time.

选择一些有趣的类型来进行分析——比如那些包含多个轨迹的、超过五个转角的，或是使用了特定工具的轨迹。然后连续阅读其中的 20 个这样的轨迹。通过集中阅读这些轨迹，我们能够发现那些单独阅读时可能会忽略的模式。

Hunt for outliers. 寻找异常值。

An outlier is a trace whose value on some metric (latency, tokens, number of tool calls) is much larger or smaller than typical. We use two common rules for flagging them:

异常值指的是在某个指标上的值远高于或低于正常范围的记录。我们采用两条常见的规则来识别这些异常值：

- Interquartile range (IQR): let Q1 and Q3 be the 25th and 75th percentiles of the metric and IQR = Q3 − Q1. Flag any trace below Q1 − 1.5 × IQR or above Q3 + 1.5 × IQR. This rule is robust to skew, which trace metrics usually have (a few very slow or very chatty sessions pull the mean, but not the quartiles).

  四分位距（IQR）：设 Q1 和 Q3 分别为该指标的 25 百分位数和 75 百分位数。IQR 的计算公式为 Q3 - Q1。任何低于 Q1 - 1.5 × IQR 或高于 Q3 + 1.5 × IQR 的数据点都应被标记为异常值。这条规则对于存在偏态的数据集是有效的，因为通常会出现一些非常缓慢或异常大的会话，这些会话会影响均值，但不会影响四分位数值。

- Percentile cut: flag the top 1% and bottom 1%. This rule is simpler, and we end up with a fixed number of traces to read.

百分位切割：标记出最高的 1%和最低的 1%。这一规则更为简单，因为我们最终只需要处理固定数量的记录。

From Clusters to Confirmed Hypotheses

从聚类分析到已确认的假设

Clustering and distribution analysis surface patterns, but patterns are not conclusions. A cluster of traces that all seem to involve "location ambiguity" might actually contain several unrelated failure modes that happen to produce similar-looking outputs. Before acting on a pattern, confirm it with a lightweight protocol:

聚类和分析可以揭示一些表面上的模式，但这些模式并不能作为结论。一组似乎都涉及"位置模糊性"的异常记录，实际上可能包含多个无关的故障模式，而这些模式恰好产生了类似的结果。在采取行动之前，先用一个简单的协议来确认这些模式的真实性。

1. Read 20 traces from the candidate cluster carefully.

   仔细阅读来自候选节点簇中的 20 条记录。

2. Define a testable criterion based on what you observe (e.g., "the agent misinterprets a neighborhood name as a city name").

   根据你的观察结果，定义一个可验证的标准（例如："该代理将某个社区名称误认为是城市名称"）。

3. Check whether the criterion holds for 70% to 80% of the traces in the cluster. If it does, the pattern is real and can become a failure mode in your taxonomy. If it doesn't, the cluster is too heterogeneous and needs to be split or reexamined.

   请检查该标准是否适用于集群中 70%到 80%的实例。如果适用，那么这种模式是真实存在的，可能会成为你的分类中的一个故障模式。如果不适用，那么这个集群的异质性太大，需要对其进行拆分或重新评估。

When using an LLM to extract features from traces for clustering (e.g., "extract the failure type from each trace"), validate the extraction quality on a small manually labeled set before trusting the features at scale. If the LLM's extraction accuracy is below 90% on your validation set, the clusters you build on top of those features will be unreliable; fall back to rule-based feature extraction or manual labeling for the fea-

tures that matter most.

在使用大语言模型从跟踪数据中提取特征用于聚类时（例如"从每条跟踪数据中提取故障类型"），请先在一个小型的、由人工标记的数据集上验证这些提取结果的质量，然后再大规模使用这些特征。如果大语言模型在验证集上的提取准确率低于 90%，那么基于这些特征构建的聚类结果将不可靠。在这种情况下，建议采用基于规则的特征提取方法，或者为最重要的特征进行人工标记。

Semantic Search to Find Related Traces

语义搜索以查找相关的痕迹

One question we find ourselves repeatedly asking is: once we spot a failure mode, how do we find more traces like it? Error analysis often surfaces only one or two bad examples, but what we want is the full population (how common is this, and does it hit one user segment more than others?). Semantic search is our usual answer.

我们反复思考的一个问题是：一旦发现了某个故障模式，如何找到更多类似的故障现象呢？在错误分析中，通常只能发现一两个典型的例子，但我们希望了解的是整个群体的情况——这种故障有多常见，是否在某些用户群体中更为突出？语义搜索正是我们常用的解决方案。

We introduced embedding-based search in Chapter 10. Here we use it to expand a known failure mode into a larger set. Two query patterns we rely on:

我们在第 10 章中介绍了基于嵌入式的搜索方法。这里我们运用这一方法将一个已知的故障模式扩展为更庞大的集合。我们依赖两种查询模式来实现这一目标：

Seed with a problematic trace.

带有问题基因的种子。

Pick a trace that exhibits the failure (say, the apologetic email from the clustering example) and search for its nearest neighbors by embedding. We get back a ranked list of candidates and read them.

选择一个表现出该故障现象的实例（比如，在聚类示例中的那封表示歉意的邮件），然后通过嵌入技术找到其最近的邻居节点。最终，我们会得到一个包含候选节点的排名列表，并读取这些节点信息。

Query with a natural-language description.

带有自然语言描述的查询。

> Write out what we are looking for (“email that starts with an apology before listing any homes”) and embed that string. This is useful when we have a description of the pattern but not a canonical example.
>
> 请把我们要求的内容写下来（“在列出任何房屋之前，要有一封以道歉开头的邮件”），然后将这个字符串嵌入到文本中。当我们有关于这种模式的描述，但缺乏典型的例子时，这种方法非常有用。

Semantic search has one big caveat: embedding similarity is not the same as our notion of similarity. If you have a small set of labeled traces for the failure mode you're searching for, check the precision and recall of your semantic search against those labels; if too many irrelevant traces are showing up (low precision) or too many relevant ones are missing (low recall), adjust the similarity threshold or try a different embedding model.

语义搜索有一个重要的注意事项：嵌入的相似性与我们通常理解的相似性是不同的。如果你有少量用于搜索的标注轨迹数据，那么需要检查语义搜索的精确度和召回率。如果出现了太多不相关的轨迹数据（精确度过低），或者太多相关的轨迹数据缺失（召回率过低），那么就需要调整相似度阈值，或者尝试使用不同的嵌入模型。

Two traces can be near each other in embedding space because they share topic words (both mention “tour scheduling”), even when the failure we care about is narrower (the agent double-booked a showing in both).

在嵌入空间中，两条轨迹可以彼此靠近，因为它们共享了一些主题词（比如两者都提到了“行程安排”）。即使我们关心的故障类型更为具体（比如某个代理同时预约了两次展示服务），这两条轨迹仍然可以靠近在一起。

One workaround we have found useful is to ask an LLM agent to extract application-specific features from each trace (e.g., did the agent book a tour? did it mention a conflicting time? did it hedge?) and embed those features, or a short summary, instead of the raw trace. The agent itself can propose the features if we give it a handful of examples of the failure mode. With feature embeddings, we get closer to our notion of

similarity.

我们发现的一个有用的方法是将 LLM 代理请求从每条跟踪数据中提取与应用程序相关的特征（例如，代理是否预订了参观？它是否提到了时间上的冲突？是否进行了对冲操作？）。然后将这些特征或简短的摘要嵌入到原始跟踪数据中。如果我们给代理提供一些故障模式的示例，它自己就可以提出这些特征。通过使用特征嵌入，我们可以更接近地理解相似性的概念。

Using AI Coding Assistants

利用人工智能编程助手

The work in this chapter (loading traces, plotting distributions, clustering, running semantic search) is mostly code we would rather not write from scratch. AI coding assistants like Claude Code, Codex, and notebook-embedded agents accelerate the mechanics considerably. Here is how we use them without letting them take over the analysis.

在本章中，那些涉及加载数据、绘制分布图、进行聚类分析以及执行语义搜索等操作的工作，其实都是我们不太愿意从头开始编写代码的任务。像 Claude Code、Codex 这样的 AI 编程助手，以及嵌入在笔记本中的代理程序，都能显著加快这些操作的执行速度。下面就是我们在使用这些工具的方式，同时不会让它们完全主导我们的分析过程。

Our typical setup is to drop a file of traces (JSON or JSONL), a short description of the application and what each field means, and, where relevant, the application code itself into a working directory, and then open a Claude Code or Codex session pointed at that directory. Once the assistant has all of that in context, we find ourselves asking for the kinds of things we would otherwise script ourselves. That might be plots of distributions (a histogram of turns per session with a log y-axis, then split by user segment), a classifier rubric or regex drafted from a handful of examples of a failure mode, a summary of a cluster of traces we paste in, or a direct search of the traces for a pattern we describe (“find traces where the agent apologized before giving any listings”). We use

the session as our scratchpad for most of the work in this chapter.

我们的常规操作是：将包含追踪数据的文件（JSON 或 JSONL 格式）、应用程序的简要描述以及每个字段的含义放入工作目录中。在必要时，还会将应用程序的源代码也放入该目录。之后，打开一个与该目录关联的 Claude Code 或 Codex 会话。一旦助手了解了所有相关信息后，我们就会开始执行那些通常需要编写脚本才能完成的任务。比如，绘制分布图（例如，按用户分组展示每次会话的会话次数直方图）；根据一些故障示例来编写正则表达式；对粘贴进来的追踪数据进行总结；或者根据描述寻找特定的模式（“找到那些在显示任何列表之前先道歉的追踪数据”）。在这一章的大部分工作中，我们都将会话视为我们的临时存储工具。

Early in EDA especially, we find it useful to ask for many charts in one go and then read only the ones that look interesting. Most will be uninformative, and that is expected. For example, we might ask for thumbs-down rate broken down by session length, user segment, agent version, time of day, and tool call count all at once, and then only read traces behind the one breakdown where something looks off.

在 EDA 的初期阶段，我们发现一次性请求大量图表是非常有用的做法。之后，只需阅读那些看起来有趣的图表即可。大多数图表可能并不具有参考价值，这也是意料之中的事。例如，我们可以一次性请求关于各会话时长、用户分组、代理版本、时间区间以及调用次数等指标的反对率数据，然后只关注那些存在异常情况的图表背后的记录。

Then, once we are looking for a specific pattern, asking the AI agent to search the traces directly is a lightweight alternative to the semantic search from the previous section. The practical constraint is the agent's context window: if your traces fit within the token budget (a few thousand short traces typically do), direct search lets you skip the embedding-index setup entirely. Once the dataset exceeds the context window (tens of thousands of traces, or longer traces each), you need to fall back to embedding search or a proper index.

那么，当我们需要寻找某种特定的模式时，可以让 AI 代理直接搜索相关痕迹。这种方法的优点在于它比上一节中使用的语义搜索更为简单。不过，实际应用时存在一个限制，那就是代理的上下文窗口大小：如果痕迹数量在令牌预算范围内（通常几千个痕迹即可），那么可以直接使用直接搜索的方式，无需进行嵌入索引的设置。但如果数据集中的痕迹数量超过了上下文窗口的范围（即数万甚至更多个痕迹），那么就需要转向嵌入搜索或适当的索引结构了。

One pitfall to avoid is asking an AI agent like Claude Code to explain *why* traces are failing. The agent will produce a plausible-sounding explanation from whatever examples it has seen, but that explanation may be wrong, and it will be hard to tell because it sounds confident. Any causal claim should be treated as a hypothesis, not a conclu-

sion. To verify it, change one variable (e.g., modify the prompt, swap a tool, or adjust a parameter) and compare the results against a golden set before and after the change. If the hypothesis is correct, the failure rate for the relevant failure mode should drop. If it doesn't, the explanation was wrong regardless of how convincing it sounded.

需要避免的一个问题是，让像 Claude Code 这样的 AI 代理来解释为什么某些行为会失败。该代理可能会根据它所见过的例子给出一个看似合理的解释，但那个解释可能是错误的。而且，由于它听起来似乎很有信心，因此很难判断其正确性。任何关于因果关系的说法都应该被视为一种假设，而不是结论。为了验证这一假设，可以修改其中一个变量（例如，修改提示参数、更换工具或调整参数），然后比较修改前后的结果。如果假设正确，那么相关故障模式的失败率应该会下降。如果情况并非如此，那么无论这个解释听起来有多可信，它都是错误的。

Stick to asking the AI agent to find, summarize, search, or plot. These are activities to aid in your analysis, not to do your analysis.

尽量让人工智能助手来负责寻找、总结、搜索或绘制图表等工作。这些任务是为了辅助你的分析过程，而不是用来代替你的分析工作。

Common Pitfalls 常见陷阱

The most common mistake in data analysis is treating cluster labels as ground truth. Clustering groups traces by similarity, but the cluster boundaries are arbitrary, and the labels you assign to clusters are hypotheses until you confirm them by reading traces; a cluster labeled "tone mismatch" might contain traces where the real issue is missing information, not tone.

数据分析中最常见的错误是将聚类标签视为真实情况。聚类是根据相似性来划分数据组的，但聚类边界是任意设定的，而分配给各聚类的标签则只是假设而已，直到通过阅读数据组来确认这些标签的正确性。例如，一个被标记为"语调不匹配"的聚类，其内部的数据组可能包含的情况是：真正的问题并非在于语调上的差异，而是缺乏某些关键信息。

A related mistake is jumping straight to investigating a pattern without controlling for confounders. For example, if you notice that traces with more than five tool calls have a higher failure rate, the cause might not be the number of tool calls itself; it might be that complex queries require more tool calls and are also inherently harder to answer. Before drawing conclusions, stratify by likely confounders (e.g., query complexity, user

segment, time of day) to see whether the pattern holds within each stratum.

另一个常见的错误是，直接开始研究某个模式，而没有考虑到各种混杂因素。例如，如果你注意到那些包含超过五个工具调用的查询失败率更高，那么原因可能不是工具调用的数量本身；可能是复杂的查询需要更多的工具调用，因此本质上更难解决。在得出结论之前，应该根据可能的混杂因素进行分组分析（如查询的复杂性、用户群体、时间等因素），以了解该模式在不同分组中是否成立。

Another pitfall, discussed in the previous section, is asking an AI agent like Claude Code to explain failures and treating the explanation as a conclusion rather than a hypothesis. Every causal claim needs verification through a controlled change.

另一个陷阱在于，如前一节所讨论的，人们会让像 Claude Code 这样的 AI 代理来解释失败的原因，而将这些解释视为结论而非假设。任何因果声明都需要通过有控制的实验来验证。

Finally, if you only look at traces from clusters, outliers, and semantic search results, your view of the system is skewed toward the unusual cases. Include a random sample in every analysis session to calibrate your sense of how the system performs overall.

最后，如果你只关注来自集群的痕迹、异常值以及语义搜索结果，那么你对系统的理解就会偏向于那些不寻常的情况。在每次分析过程中，都包含一些随机样本，这样可以帮助你更全面地了解系统的整体表现。

Putting It Together 整合起来

Error analysis gives us a taxonomy of failure modes from reading traces closely. Data analysis adds aggregate techniques for seeing those failures across the full trace set at once. We use EDA and clustering to surface hypotheses about failure modes, per-trace metrics (distributions, correlations, outliers) to spot patterns we would miss reading one trace at a time, and semantic search to expand a known failure mode into the full population it affects. AI coding assistants like Claude Code and Codex accelerate the mechanics of all of this (e.g., loading traces, plotting distributions, drafting rubrics, searching for patterns), but we treat their outputs as hypotheses to check against the

underlying traces, not as the analysis itself.

错误分析通过仔细分析数据轨迹，为我们提供了各种故障模式的分类。数据分析则运用了综合技术，能够一次性识别出整个数据轨迹集合中的故障现象。我们利用 EDA 和聚类分析方法来发现关于故障模式的假设；通过每个数据轨迹的度量指标（如分布、相关性、异常值）来发现那些单独分析时可能忽略的模式；而语义搜索则能够将已知的故障模式扩展至其影响的全部人群。像 Claude Code 和 Codex 这样的 AI 编码助手可以加速所有这些过程（例如，加载数据轨迹、绘制分布图、制定评估标准、寻找模式等），但我们将它们的输出结果视为需要针对底层数据轨迹进行验证的假设，而不是分析本身的结果。

In the next chapter, we turn to the improvement work itself: the prompt, code, and model changes that fix the failures we have surfaced, along with the cost-side optimizations that keep production sustainable.

在下一章中，我们将讨论具体的改进措施：包括优化提示、代码以及模型等方面的调整，这些调整能够解决我们之前遇到的故障问题。同时，我们还会考虑一些成本方面的优化措施，以确保生产的可持续性。

Chapter 12. Improving LLM Agents

第十二章 改进大语言模型代理

We have built evaluators, set up CI to catch regressions, and deployed monitoring to track production quality. We have also surfaced failure modes through error analysis (Chapter 3) and data analysis (Chapter 11). Now we use all of that to make the agent better.

我们已经开发了评估工具，建立了持续集成机制来检测回归问题，并实施了监控机制以跟踪生产质量。我们还通过错误分析（第三章）和数据分析（第十一章）发现了系统中的故障模式。现在，我们利用所有这些手段来改进代理程序的功能。

In this chapter, you will learn:

在本章中，你将学到：

- How to improve accuracy through prompt refinement, structural changes, and heavier interventions like fine-tuning

  如何通过优化提示、进行结构调整，以及采用更精细的调整等更深入的干预措施来提高准确性。

- How to reduce cost through model selection, token reduction, caching, and model cascades

  如何通过模型选择、令牌减少、缓存技术以及模型级联等方式来降低成本。

- How to verify that each change actually helps using the evaluators and CI infrastructure from earlier chapters

  如何使用前几章中提到的评估工具和持续集成基础设施来验证每次变更确实能够带来帮助。

Both accuracy and cost improvements depend on reliable measurement. Without evaluators (Chapter 5) and monitoring (Chapter 9), we cannot tell whether a change helps

or hurts.

无论是精度的提升还是成本的降低，都依赖于可靠的测量数据。如果没有评估人员（参见第 5 章）和监控机制（参见第 9 章），我们就无法判断这种改变是带来好处还是弊害。

Accuracy Optimization 精度优化

When an agent fails evaluations or frustrates users, accuracy is usually the first thing to fix. We can organize the techniques by effort level: low, medium, and high.

当代理未能通过评估或让用户感到不满时，准确性通常是首先需要解决的问题。我们可以根据技术难度将相关方法分为三个等级：低难度、中等难度和高难度。

Low Effort: Prompt Refinement

低投入：快速改进

The fastest way to improve accuracy is to refine the prompt. Prompt refinement includes clarifying instructions, adding few-shot examples, and steering the model with explicit personas. These are low-effort changes that can have a big impact.

提高准确度的最快方法就是优化提示语。优化提示语包括明确说明指令、添加少量样本示例，以及使用明确的角色描述来引导模型的行为。这些都是无需花费太多精力就能实现的改进措施，但却能带来显著的提升。

Fix specification errors first. As discussed in “Defining the Right Metrics (What to Measure)”, many failures come from the Gulf of Specification: the prompt does not clearly say what we want, so the model makes a reasonable guess that happens to be wrong. These are the cheapest failures to fix because they only require editing the prompt, not improving the model's ability to generalize.

首先解决规范描述上的错误。正如在“确定合适的评估指标（需要衡量什么）”部分所讨论的，许多失败原因都源于规范描述的不清晰——提示语没有明确说明我们想要达到的目标，因此模型会做出一些合理的猜测，但这些猜测往往是错误的。这些错误是最容易修复的，因为只需要修改提示语即可，而不需要提升模型的整体性能。

For our real estate assistant from Example 3-1, suppose a user asks “Find me homes near downtown.” The model returns a mix of houses, condos, and townhomes. The user only wanted single-family houses. This is a specification error: our prompt says `Search for listings matching the user's preferences` but never defines what “homes” means. The fix is to add a clarification: `When the user says "homes," default to single-`

family houses unless they specify otherwise (e.g., "condos," "townhomes").

以示例 3-1 中的房地产助手为例，假设有用户询问“查找市中心附近的房屋”。模型返回了各种房屋、公寓和联排别墅的信息。不过，用户只想要独栋住宅。这里存在规格说明上的错误：我们的提示中只提到了 Search for listings matching the user's preferences ，但没有明确“房屋”具体指的是什么。解决这个问题的方法就是添加一些说明： When the user says "homes," default to single-family houses unless they specify otherwise (e.g., "condos," "townhomes").

We want to fix specification errors *before* building evaluators, because there is no point measuring whether the model generalizes on a task we have not clearly defined yet.

我们希望在构建评估器之前先修复规范上的错误，因为如果我们没有明确定义好某个任务，那么测量模型的泛化能力是没有意义的。

TIP 提示/窍门

Before writing any evaluator, skim your 10 worst traces and ask: “Did the prompt actually say what I wanted?” In our experience, half of early failures are specification errors that disappear with a one-line prompt edit, saving weeks of evaluator development on a problem that was never a model limitation.

在编写任何评估器之前，先仔细检查一下最糟糕的 10 个测试用例。问问自己：“这个提示是否真的符合我的需求？”根据我们的经验，大多数早期失败的原因都是由于规范错误导致的，而这些问题只需修改一行提示语句就能解决，这样就能避免花费数周时间来处理那些其实并非模型限制的问题。

Add a few examples. We can include two or three representative input/output pairs directly in the prompt. These examples should target failure cases we have observed during error analysis (Chapter 3), but they should be distinct from the test cases used in CI. Reusing CI test cases as few-shot examples inflates our evaluation scores without actually improving the agent's behavior on new inputs.

再添加一些例子吧。我们可以在提示中直接列出两到三个代表性的输入/输出对。这些例子应该针对我们在错误分析过程中观察到的失败案例（参见第 3 章），但它们需要与 CI 测试案例中使用的案例有所区别。如果重复使用 CI 测试案例作为少样本示例，只会让我们的评估分数上升，而实际上并不会真正改善代理在新输入上的表现。

If we have a library of many good examples, we can go further and select them dynamically. Instead of always including the same fixed examples, we can retrieve the most similar ones to the current query (using embedding similarity) and include those as few-shot examples. This gives the model more relevant guidance per query without

bloating every prompt with every example we have.

如果我们拥有一个包含许多优秀示例的库，我们就可以进一步进行动态选择。不必总是使用相同的固定示例，而是可以选择与当前查询最相似的示例（通过嵌入相似性进行筛选），将这些示例作为少样本示例来使用。这样就能让模型在每个查询中获得更相关的指导，而不会因为使用了所有示例而使得提示内容过于冗长。

Use role-based guidance. Giving the model a persona (e.g., "You are a careful tax advisor") can help guide tone and reasoning style, especially for open-ended tasks.

使用基于角色的指导方式。为模型赋予一个角色形象（例如："你是一个严谨的税务顾问"），这有助于确定语言的风格和推理方式，尤其是在处理开放式任务时。

Ask for step-by-step reasoning. For tasks involving logic or multiple steps, explicitly asking the model to explain its reasoning can improve correctness and completeness. This technique is sometimes called chain-of-thought (CoT) prompting. Note that newer reasoning models (e.g., OpenAI's o-series, DeepSeek R1) generate chains of thought internally, so adding explicit "think step by step" instructions can interfere with the model's own reasoning process and may hurt performance. For reasoning models, it's better to adjust the thinking budget parameter (if the provider exposes one) rather than prompting for step-by-step reasoning.

请要求模型提供逐步的推理过程。在涉及逻辑或需要多个步骤的任务中，明确要求模型解释其推理过程可以提高结果的准确性和完整性。这种技术有时被称为"思维链"提示。需要注意的是，一些较新的推理模型（如 OpenAI 的 o 系列、DeepSeek R1）能够内部生成思维链，因此添加明确的"逐步思考"指令可能会干扰模型自身的推理过程，从而影响性能。对于推理模型来说，最好调整"思维预算"参数（如果提供商提供了这个参数），而不是不断要求模型提供逐步的推理过程。

These changes are fast to try and easy to validate with the golden test sets from our CI setup (Chapter 9).

这些更改实施起来非常迅速，而且可以通过我们持续集成流程中的黄金测试集来轻松验证（详见第 9 章）。

Medium Effort: Changing How the Agent Works

中等难度：改变代理的工作方式

When prompt edits are not enough, we can change how the agent is put together.

当简单的编辑不足以解决问题时，我们可以改变代理的构造方式。

First, we would break the task into smaller steps. Instead of a single LLM call that does everything, we can decompose the task into a sequence of focused steps. For example, imagine we are building a research agent that looks for relevant information in a corpus of external data and synthesizes it into a report. Instead of one call to an LLM to produce the full report from a query, we could break the task down:

首先，我们将任务分解为多个较小的步骤。而不是依赖一个 LLM 来完成所有工作，我们可以将任务拆分成一系列有针对性的操作。例如，假设我们正在构建一个研究智能体，该智能体需要从外部数据集中提取相关信息，并将其整理成报告。而不是依赖一个 LLM 来生成完整的报告，我们可以将任务拆分成几个子任务来逐一处理。

1. Extract the user's preferences or constraints.

   提取用户的偏好或限制条件。

2. Query a database or search tool.

   查询数据库或搜索工具。

3. Filter or summarize the results.

   过滤或汇总结果。

4. Generate a final answer or message.

   请给出一个最终的答案或消息。

Each step is easier to evaluate independently. If the generated report contains errors, we can trace the failure to a specific step (e.g., bad retrieval or incorrect filtering) rather than debugging a single monolithic call.

每一步都可以独立地进行评估。如果生成的报告中存在错误，我们可以确定错误发生在特定的步骤上（例如，是检索错误还是过滤错误），而无需调试整个系统。

Next, we would tune RAG retrieval. In many agents, retrieval quality drives overall performance more than generation quality does. We can improve retrieval by reworking the query sent to the retriever, adjusting how source documents are chunked, tuning the number of chunks retrieved, and adding a lightweight reranking step before passing results to the LLM. Chapter 7 covers how to measure retrieval quality in detail.

接下来，我们将调整 RAG 检索的算法。在许多智能体中，检索质量对整体性能的影响远远大于生成质量。我们可以通过重新设计发送给检索器的查询语句、调整源文档的分割方式、控制检索到的块的数量，以及在将结果传递给 LLM 之前添加一个简单的重新排序步骤来提升检索效果。第 7 章详细介绍了如何衡量检索质量。

DoorDash reported a concrete case of this in a recent engineering review. Its customer support chatbot was hallucinating answers to questions from dashers (the drivers who deliver orders), for instance, inventing refund policies or misinterpreting details about an order. The engineers found that the chatbot was seeing too much raw information at once: every event in a delivery's history (pickup time, location pings, chat messages, status changes) was being dumped into the prompt, and the model was getting confused by the volume. Their fix was to add a layer that summarized the relevant delivery history into a clean, structured form before passing it to the chatbot. No model change, no retraining, just a better way of organizing what the model already had access to. They tested the change by running thousands of simulated conversations against the chatbot (each simulated conversation was generated by another LLM playing the role of a dasher) and measured hallucination rates in the results. The fix cut hallucinations by roughly 90% in those simulations, and the improvement held up in production.

DoorDash 在最近的一次工程审查中发现了这样一个案例。他们的客户支持聊天机器人会给出一些错误的回答，比如编造退款政策，或者误解订单的详细信息。工程师们发现，聊天机器人一次性接收了过多的原始信息：每次配送过程中的所有事件信息——包括取货时间、位置信息、聊天记录以及状态变化——都被一次性输入到机器人的输入中，导致模型难以处理如此庞大的信息量。他们的解决方案是添加一层过滤机制，将相关的配送历史信息整理成简洁、结构化的形式，然后再将其传递给聊天机器人。无需修改模型或重新训练模型，只需改进模型获取信息的处理方式即可。他们通过数千次模拟对话测试了这一方案，每次模拟对话都是由另一个大语言模型生成的。在实验中，该算法成功地将幻觉发生率降低了约 90%。这一改进在实际应用中也得以保持。

If the model repeatedly calls a tool incorrectly (wrong argument formats, missing parameters), we can revisit how we describe the tool in the prompt (see the tool calling evaluation stages in Chapter 8). For our real estate assistant from Example 3-1, suppose the `check_calendar` tool expects dates in `YYYY-MM-DD` format, but the model keeps passing strings like `"next Saturday"` or `"March 10th"`. The tool description says `date: string` but does not specify the format. Changing it to `date: string in YYYY-MM-DD format (e.g., "2025-03-10")` fixes the argument generation error for every query

that involves scheduling.

如果模型反复调用某个工具时出错（例如错误的参数格式、缺少某些参数），那么我们需要重新审视在提示中描述该工具的方式（详见第 8 章中的工具调用评估部分）。以示例 3-1 中的房地产助手为例，假设 check_calendar 工具期望接收 YYYY-MM-DD 格式的时间日期，但模型不断传递如 "next Saturday" 或 "March 10th" 这样的字符串。工具描述中使用了 date: string 作为格式，但没有明确说明具体格式。将格式改为 date: string in YYYY-MM-DD format (e.g., "2025-03-10") 后，就可以解决所有涉及日程安排的查询中的参数生成问题了。

For tasks that involve reasoning or judgment, we can run the same prompt multiple times (with temperature > 0) and compare the outputs. If most responses agree on an answer, that answer is more likely to be correct. This is called *self-consistency*. For example, if we ask our real estate assistant to classify a user's intent, we can sample three responses. If two out of three say "scheduling request" and one says "listing question," we go with "scheduling request." Self-consistency costs more per query (we pay for multiple LLM calls), so it is best reserved for high-stakes decisions where getting it right matters more than speed.

对于需要推理或判断的任务，我们可以多次运行相同的提示（温度大于 0），然后比较不同的输出结果。如果大多数回复都指向同一个答案，那么这个答案就更有可能是正确的。这被称为"一致性原则"。例如，如果我们让房地产助手来判断用户的意图，我们可以获得三个不同的回复。如果三个回复中有两个说是"预约请求"，而另一个说是"房源问题"，那么我们就选择"预约请求"作为答案。不过，每次使用"一致性原则"都会增加成本（我们需要多次调用大型语言模型），因此最好只在那些需要准确回答、速度并不重要的高风险决策中使用这种方法。

We can add a second LLM call that checks the first one's output. For example, after the agent drafts an email, a separate call can verify: "Does this email accurately reflect the listing data? Does it match the client's persona?" This catches errors that the first call missed. The trade-off is latency and cost (two calls instead of one), so we typically use verification on the most important outputs, not on every request.

我们可以添加第二个大型语言模型来验证第一个模型的输出结果。例如，在代理编写了电子邮件之后，可以再进行一次验证："这份电子邮件是否准确地反映了商品信息？它是否符合客户的需求？"这样就能发现第一个模型遗漏的错误。不过，这样做会伴随着延迟和成本增加（需要两次调用而不是一次），因此我们通常只对最重要的输出结果进行验证，而不是对所有请求都进行验证。

We can add synchronous checks that run before the output reaches the user (see the guardrails discussion in Chapter 9). For example, a regex can catch generated emails

that contain patterns resembling Social Security numbers. A SQL parser can reject syntactically invalid queries before they hit the database. These are cheap and fast, and they prevent the most embarrassing failures even when the model occasionally slips.

我们可以添加同步检查机制，这些检查在输出结果被用户看到之前就进行执行（有关详细信息，请参考第 9 章中的相关讨论）。例如，可以使用正则表达式来过滤那些包含类似社会安全号码的生成的电子邮件。此外，SQL 解析器可以在查询语法存在错误时拒绝执行这些查询，从而避免出现错误。这些检查既便宜又快速，而且即使模型偶尔出现错误，也能有效避免最令人尴尬的故障发生。

High Effort: Model-Level Changes

高投入：模型级别的修改

If prompt and architecture changes have not solved the problem, we can consider model-level interventions. These take more effort and typically require high-quality labeled data.

如果简单的提示和架构调整未能解决问题，我们可以考虑在模型层面的干预措施。这种干预需要更多的努力，通常需要高质量的标注数据来支持。

Fine-tuning means taking a pretrained model and continuing to train it on our own set of example inputs and outputs, so that the model learns to handle our specific task in a specific way. When we have hundreds of high-quality labeled examples and cannot fix the issue with prompting alone, fine-tuning is an option.

微调意味着我们采用一个已经预训练的模型，然后继续用我们自己的示例输入和输出数据来训练该模型，从而使模型能够以特定方式处理我们的特定任务。当我们有数百个高质量的标记示例，但仅靠改进提示方式仍然无法解决问题时，那么微调就成为一个可行的选择。

In practice, most teams start with *parameter-efficient fine-tuning* (PEFT) methods like LoRA or QLoRA rather than full fine-tuning. LoRA updates a small set of low-rank adapter matrices instead of every parameter in the model, which means you can fine-tune on a single GPU (e.g., an A100 or H100) in a few hours rather than needing a multi-GPU cluster for days. QLoRA goes further by quantizing the base model to 4-bit precision during training, which lets you fine-tune a 70B-parameter model on a single GPU with 48 GB of memory. Often 50 to 200 high-quality examples are enough for a

meaningful improvement on a narrow task.

在实践中，大多数团队都会采用参数高效微调方法，如 LoRA 或 QLoRA，而不是进行完全的微调。LoRA 只需要更新一小组低秩适配器矩阵，而不是模型的每个参数。这样一来，就可以在几小时内仅使用单个 GPU（例如 A100 或 H100）进行微调，而无需使用多 GPU 集群来花费数天时间。QLoRA 则更进一步，在训练过程中将基础模型精度降低到 4 位，这样你就可以在拥有 48GB 内存的单个 GPU 上，对包含 700 亿参数的模型进行微调。通常，对于特定任务而言，50 到 200 个高质量示例就足以实现显著的改进。

Beyond supervised fine-tuning on input-output pairs, *preference fine-tuning* methods like direct preference optimization (DPO), which trains on pairs of outputs where one is preferred over the other, and Kahneman-Tversky optimization (KTO), which needs only individual outputs labeled as desirable or undesirable, train the model on preference feedback. Preference fine-tuning is useful when the failure mode is about style or quality (e.g., "the model's answers are correct but too verbose") rather than about correctness.

除了对输入-输出对进行监督式微调之外，还有一些基于偏好反馈的微调方法。例如，直接偏好优化（DPO）方法，它基于两种输出之间的偏好来进行模型训练；而 Kahneman-Tversky 优化方法（KTO）则只需要标注为"理想"或"不理想"的单个输出，从而基于偏好反馈来训练模型。当问题涉及风格或质量时（例如，"模型的答案虽然正确，但过于冗长"），偏好微调方法就非常有用。而当问题与正确性无关时，这种微调方法则更为适用。

Fine-tuning also means you need to *serve* the resulting model, which is a separate infrastructure cost. You can run it on your own GPUs (e.g., through vLLM or TGI), or use an inference provider like Together AI, Fireworks, or Modal that hosts fine-tuned models for you. Before committing to fine-tuning, make sure the cost economics work out: the savings from using a smaller fine-tuned model instead of a larger general-purpose one need to exceed the cost of training, serving, and maintaining the fine-tuned model over time.

微调还意味着你需要为最终的模型提供服务，而这需要额外的基础设施成本。你可以在自己的 GPU 上运行该模型（例如通过 vLLM 或 TGI），或者使用像 Together AI、Fireworks 或 Modal 这样的推理提供商，他们会为你托管经过微调的模型。在决定进行微调之前，请确保成本效益是合理的：使用更小型的微调模型所节省的成本，应该能够超过训练、服务以及长期维护该模型所需的成本。

For tasks with high stakes or hard-to-detect failure modes, we can build a lightweight interface (Chapter 10) for human reviewers to check and annotate a small sample of outputs regularly. Annotations can feed back into fine-tuning datasets and can also re-

veal failure modes that automated evaluators miss.

对于那些风险较高或难以检测到的故障模式相关的任务，我们可以设计一个轻量级界面（参见第 10 章），让人类评估人员能够定期检查和注释少量的输出结果。这些注释可以用于数据集的微调，同时还能发现自动化评估工具可能忽略的故障模式。

For agents where prompts are reused across thousands of requests, even small wording changes can have a large aggregate impact. We can treat the prompt as a tunable artifact: try variations, test them against held-out golden sets, and keep what improves accuracy. Automated prompt optimization frameworks like DSPy (Khattab et al. 2024), TextGrad (Yuksekgonul et al. 2025), and OPRO (Yang et al. 2024) take this further by systematically searching over prompt variations, using the evaluation metrics from Chapter 5 as the optimization objective. These tools can find prompt wordings that a human would not think to try, but they require a well-defined evaluation function and enough labeled examples to measure improvement reliably. Some teams formalize this process using grid search over template variations or automated optimization frameworks.[1]

对于那些需要重复使用提示的代理来说，即使是很小的措辞修改也会产生巨大的影响。我们可以将提示视为一个可调整的变量：尝试不同的变体，将其与保留下来的标准模板进行对比，选择能提升准确度的方案。像 DSPy（Khattab 等人，2024 年）、TextGrad（Yuksekgonul 等人，2025 年）和 OPRO（Yang 等人，2024 年）这样的自动化提示优化工具通过系统地搜索各种变体来进一步改进这一过程，并以第 5 章中的评估指标作为优化的目标。这些工具能够找到人类可能想不到的措辞方案，但它们需要一套明确的评估函数以及足够的标注样本来可靠地衡量改进效果。一些团队通过针对模板变体的网格搜索或自动化优化框架来将这一过程具体化。[1]

Cost Optimization 成本优化

Once accuracy meets requirements, we can start reducing cost. The approach is the same as with accuracy: find the biggest cost drivers, make targeted changes, and verify with evaluators that quality stays intact.

一旦准确率符合要求，我们就可以开始降低成本。方法与准确率优化相同：找出最大的成本驱动因素，进行有针对性的更改，并请评估人员验证质量是否保持完好。

Match Model Size to Task Complexity

根据任务复杂度匹配模型大小

Not every step in an agent needs the most powerful (and most expensive) LLM. We can map each task to the smallest model that meets its accuracy requirements.

并非代理中的每一步都需要最强大（且最昂贵）的大型语言模型。我们可以将每个任务映射到符合其准确率要求的最小模型。

Lightweight models work well for simple, high-volume tasks: sentiment classification, extracting fields from structured text, or recognizing basic user intent. We can reserve the largest models for subtasks that require advanced reasoning, follow complex instructions, or where accuracy stakes are high.

轻量级模型非常适合处理简单且高容量的任务，例如情感分类、从结构化文本中提取字段信息，或者识别用户的基本意图。我们可以将更强大的模型用于那些需要高级推理能力、遵循复杂指令，或者对准确性要求较高的子任务上。

For example, in our real estate assistant (see Example 3-1), a small model could classify incoming messages into categories like “listing question,” “scheduling request,” or “general inquiry.” Only complex queries, like those requiring a customized explanation of how a property compares to recent sales, would be routed to a more capable model.

例如，在我们的房地产助手程序中（参见示例 3-1），一个简单的模型可以将收到的消息分类为不同的类别，如“房源问题”、“预约请求”或“一般咨询”。只有那些需要更复杂处理的查询，比如需要针对特定情况对房产与近期销售行情进行比对的分析，才会被提交给更强大的模型进行处理。

Cut Token Usage 减少代币使用量

Since most LLM providers charge based on the number of tokens processed (both input and output), reducing token count directly reduces cost.

由于大多数大语言模型提供商都是根据处理过的令牌数量来收取费用的（无论是输入还是输出），因此减少令牌的数量就能直接降低成本。

Make sure instructions are concise. For RAG-based agents, tune the number of documents retrieved and placed in the prompt. If the agent makes multiple LLM calls that analyze the same long document, consider adding an initial step where a cheaper model summarizes the document first. Subsequent calls then operate on the shorter summary, which reduces input tokens for each downstream call. For instance, when analyzing a lengthy legal contract to extract various clauses and obligations, summa-

rizing the contract once upfront can be much cheaper than feeding the full text to multiple specialized extraction prompts.

请确保翻译说明简洁明了。对于基于 RAG 的代理程序，需要调整在提示中返回的文档数量。如果代理需要多次调用 LLM 来分析同一份长文档，可以考虑添加一个初始步骤：先用成本较低的模型对文档进行总结。之后，再基于已简化的文档内容进行分析，这样就能减少每次调用的输入数据量。例如，在分析一份冗长的法律合同以提取其中的各项条款和义务时，如果先对合同进行总结，那么相比直接将所有内容输入给多个专门的提取工具，成本会更低。

Explicitly instruct the LLM to be concise when appropriate. For tasks where the output follows a known structure, requesting output in a compact format like YAML (rather than natural language prose or JSON, which includes many braces and brackets) can reduce the number of generated tokens.

明确指示大语言模型在适当的时候保持简洁。对于输出遵循已知结构的任务，要求以紧凑的格式（如 YAML）输出（而不是自然语言散文或 JSON，后者包含许多括号和方括号），可以减少生成的 token 数量。

Make Prompts Cache-Friendly

使提示缓存友好

Many teams overlook caching as a cost reduction technique. Many LLM API providers (including OpenAI, Google, and Anthropic) use caching mechanisms that reduce both latency and cost for repeated requests. When Anthropic introduced prompt caching for Claude in 2024 (Anthropic 2024b), it reported that cache hits cost 10% of the base input token price (a 90% reduction) and that latency on long prompts dropped by up to 85%. These numbers only apply when the *prefix* of the prompt matches byte for byte across requests, which is why the way we structure our prompts matters.

许多团队忽视了缓存作为一种降低成本的方法。许多大语言模型 API 提供商（包括 OpenAI、Google 和 Anthropic）都采用了缓存机制，这样可以减少重复请求时的延迟和成本。在 2024 年，Anthropic 为 Claude 引入了提示缓存功能，他们报告称，缓存命中时的成本仅为基础令牌价格的 10%（降低了 90%），而长提示的延迟则减少了高达 85%。这些数值仅适用于请求中的提示内容逐字节匹配的情况，因此我们如何构造提示方式非常重要。

Modern LLMs are based on the Transformer architecture. At each step, a Transformer computes a set of “key” and “value” vectors for every token processed so far. Keys serve as lookup signals to decide which past tokens matter most. Values carry the ac-

tual information those tokens contribute. When generating long sequences or handling many similar prompts, recomputing all KV states from scratch is expensive.

现代的大型语言模型基于 Transformer 架构构建。在每一个步骤中，Transformer 都会为每一个处理过的令牌计算出一组“键”和“值”向量。这些键作为查找信号，用于确定哪些之前的令牌最为重要。而值则包含了这些令牌所携带的实际信息。在生成较长的序列或处理大量类似的提示时，重新计算所有的键值对是非常昂贵的操作。

LLM inference engines mitigate this redundant computation by storing intermediate KV states in a cache. When a new prompt begins with the exact same token sequence (the *prefix*), the engine reuses the cached KV states for that prefix and computes only the states for the new suffix. This reuse reduces both latency and compute cost.

LLM 推理引擎通过将中间 KV 状态存储在缓存中，从而避免了这种冗余计算。当新的提示以相同的标记序列开头时，引擎会重用缓存中的 KV 状态，只计算新后缀对应的状态。这种重用方式降低了延迟和计算成本。

Caching applies only when the prefix matches exactly. A single-token change invalidates the cache from that point forward and forces full recomputation. Consider a simple task: “Think of a friendlier way to say: X,” where X is any piece of text. If we place the instruction *before* the varying data, then every request shares the same long prefix (the instruction), and the provider can cache and reuse it. If instead the instruction comes *after* the data, only the first few tokens of the data (which may happen to overlap between requests) remain in the shared prefix, so there is minimal cache reuse.

缓存仅会在前缀完全匹配时生效。一旦令牌发生变化，缓存就会失效，从而需要重新计算整个结果。以这样一个简单的任务为例：“想出一种更友好的表达方式来表示 X”，其中 X 可以是任何文本片段。如果我们把指令放在数据之前，那么每次请求都会使用相同的长前缀（即指令），从而可以缓存并重复使用它。而如果将指令放在数据之后，那么只有数据的开头几个字符会保留在共享前缀中，因此缓存的利用率非常低。

Figure 12-1 illustrates this. Requests that share a constant prefix (A) get a cache hit on the second request. Putting varying data first (B) prevents caching. In panel A, Request 1 computes everything from scratch. Request 2 starts with the same system instruction, so the provider can reuse the cached KV states from Request 1 and only compute the new user query. In panel B, each request puts the user query first, which changes

the prefix, so there is no cache hit and everything is recomputed.

图 12-1 展示了这一现象。那些具有相同前缀的请求在第二次请求时仍能获得缓存结果。而如果将不同的数据作为前缀（即采用 B 的方式），则无法实现缓存。在 A 面板中，请求 1 会重新计算所有数据。请求 2 使用相同的系统指令开始计算，因此提供者可以复用请求 1 中缓存的 KV 数据，只需重新计算新的用户查询。在 B 面板中，每个请求都会以用户查询数据作为前缀，这改变了前缀的构成，因此无法获得缓存结果，所有数据都需要重新计算。

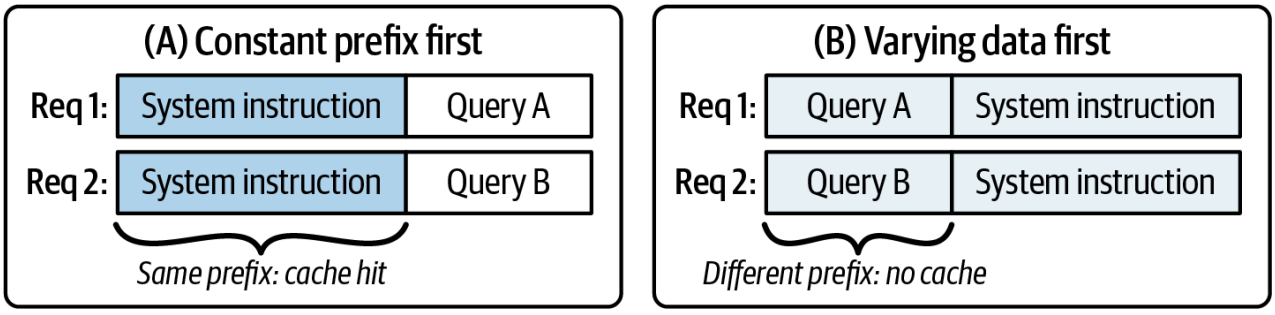


Figure 12-1. KV cache prefix reuse

图 12-1. KV 缓存前缀的重复使用情况

NOTE 注意/备注

Even whitespace differences break the cache prefix. If your prompt templates insert variable-length content (e.g., a username) before the main instructions, every request computes from scratch. Move all static instructions to the top and all variable data to the bottom.

即使是空白字符的差异也会破坏缓存前缀的有效性。如果你的提示模板在主要指令之前插入了可变长度的内容（例如用户名），那么每次请求都会重新计算。因此，应该将所有固定的指令放在顶部，而所有可变的数据则放在底部。

To take advantage of caching:

为了利用缓存功能：

- Put constant instructions before any user-provided or changing data.

  请将所有常量指令放在任何用户提供的或需要修改的数据之前。

- Keep formatting and phrasing consistent across requests.

  请保持所有请求中格式和措辞的一致性。

- Consult your LLM provider's documentation for the associated cost savings, as pricing for cached versus noncached tokens varies between providers.

请参考您选择的法学硕士课程提供商的文档，以获取相关的成本节省信息。因为不同提供商对于缓存与非缓存令牌的定价方式有所不同。

Batching 分批处理

If some of our LLM calls are not time-sensitive, we can batch them. Instead of sending one request at a time, we group multiple requests into a single API call and process them together. This reduces per-request overhead. Some providers offer explicit discounts for batched workloads. For example, OpenAI's Batch API processes requests within a 24-hour window at half the cost of synchronous calls.

如果我们的一些大型语言模型调用并不需要及时处理，那么我们可以将这些调用合并在一起进行处理。无需每次只发送一个请求，而是将多个请求合并到一个 API 调用中，然后一起进行处理。这样就能降低每个请求的处理成本。一些提供商为批量处理的工作负载提供专门的折扣优惠。例如，OpenAI 的 Batch API 可以在 24 小时内的时间内处理多个请求，但成本仅为同步调用的半价。

Batching works well for offline tasks: running evaluators on a day's worth of production traces, generating summaries for a backlog of documents, or scoring a dataset for CI. It does not work for real-time user-facing requests where latency matters.

对于离线任务来说，分批处理的方式非常有效：可以在一天内的生产数据上运行评估器，为大量文档生成摘要，或者对数据集进行评分以用于代码审查。不过，这种方式并不适用于需要实时处理的用户请求，因为实时性对延迟有着严格的要求。

Distillation 蒸馏

If we have a large model that performs well on a task but costs too much to run at scale, we can try *distillation*. The idea is to use the large model (the "teacher") to generate labeled outputs on a set of examples, then fine-tune a smaller model (the "student") on those outputs. The student learns to mimic the teacher's behavior at a fraction of the inference cost.

如果我们有一个性能良好的大型模型，但运行成本过高而无法大规模应用，我们可以尝试蒸馏技术。其原理是利用大型模型作为"教师"，生成一组带有标签的输出结果；然后利用这些输出结果来微调小型模型，使小型模型能够以较低的推理成本模仿大型模型的行为。

Distillation is different from standard fine-tuning. In standard fine-tuning, we train on human-labeled data. In distillation, we train on the teacher model's outputs. This means we can generate training data cheaply by running the teacher on unlabeled in-

puts, which makes it easier to scale up the training set.

蒸馏技术与传统的微调方法不同。在传统的微调中，我们是在人类标记的数据上进行训练的。而在蒸馏方法中，我们则是在教师模型的输出结果上进行训练。这意味着我们可以通过在未标记的输入数据上运行教师模型来低成本地生成训练数据，从而更容易地扩展训练集的规模。

The catch is that the student will not exceed the teacher's quality. If the teacher hallucinates on certain inputs, the student will learn those same mistakes. Students also tend to show worse calibration on rare or distribution-shifted inputs, because those cases are underrepresented in the teacher's output and the student doesn't learn them well. We evaluate the distilled model with the same rigor we apply to any other change (golden dataset checks, online monitoring), and we compare its error profile to the teacher's to make sure it has not picked up new failure modes.

问题是，学生的水平不会超过老师的水平。如果老师在面对某些输入时产生错误，那么学生也会重复犯同样的错误。此外，学生在处理那些罕见或分布发生变化的输入时，表现也会更差，因为这些情况在老师的输出中缺乏代表性，因此学生也无法很好地掌握这些输入。我们采用与处理其他变化时相同的严格标准来评估这个简化模型（包括使用黄金数据集进行验证、在线监控等），并将它的错误情况与老师的错误情况进行比较，以确保它没有引入新的错误模式。

Distillation also requires the same serving infrastructure as fine-tuning (dedicated GPUs or an inference provider), so it's most practical for high-volume tasks where the per-query cost savings justify the setup cost. For teams with lower query volumes, the economics may not work out, and a model cascade (described next) may be a better fit because it doesn't require training or hosting a custom model.

蒸馏算法同样需要与微调操作相同的服务基础设施（即专用的 GPU 或推理服务）。因此，这种方法最适合处理高量级的任务，因为每次查询所节省的成本足以弥补设置成本。对于查询量较小的团队来说，这种方案可能并不经济划算。在这种情况下，采用模型级联策略可能更为合适，因为这种方法不需要进行训练或托管自定义模型。

Model Cascades Cascades 模型

Suppose we are running an LLM-as-judge evaluator in production and it costs $0.02 per trace. On 10,000 traces a day, that is $200/day. We want to keep the same accu-

racy but spend less.

假设我们在生产环境中使用 LLM 作为裁判器，每个轨迹的评估成本为 0.02 美元。那么，在每天 10,000 个轨迹的情况下，每天的评估成本就是 200 美元。我们希望在保持同样准确性的前提下，降低评估成本。

A *model cascade* does this by setting a *target accuracy*: the minimum agreement rate between the cascade's output and the expensive model's output (e.g., 95% agreement). We then route each query through a cheap model first. If the cheap model is confident in its answer, we accept it. If it is not confident, we escalate to the expensive model. The goal is to meet the target accuracy while sending as few queries as possible to the expensive model.

一个模型级联通过设置一个目标精度来实现这一点：级联的输出和昂贵模型输出的最小协议率（例如，95%的协议率）。然后我们将每个查询首先通过一个廉价模型。如果廉价模型对其答案有信心，我们就接受它。如果它没有信心，我们就升级到昂贵模型。目标是在将尽可能少的查询发送到昂贵模型的同时达到目标精度。

Cascades require two things from the proxy: a binary verdict (pass / fail , true / false) and a confidence score we can threshold on. That is why an LLM-as-judge is a natural example: in Chapter 5, we define judges as binary classifiers. If the underlying task is open-ended, ask for structured output that includes both the verdict and a score—not a free-form paragraph alone.

Cascades 需要从代理那里获得两样东西：一个二元判决结果（ pass / fail ， true / false ），以及一个可以据此进行筛选的置信度评分。这就是为什么使用 LLM 作为评判器是一个很好的例子：在第五章中，我们将评判器定义为二元分类器。如果底层任务具有开放性，那么就需要要求得到包含判决结果和评分的结构化输出——而不是仅仅一个自由形式的段落。

We call the expensive model the *oracle* (the best system we currently have) and the cheap model the *proxy*. Figure 12-2 shows the flow.

我们将那个成本较高的模型称为“权威模型”（即目前最好的系统），而那个成本较低的模型则被称为“替代模型”。图 12-2 展示了这种流程的示意图。

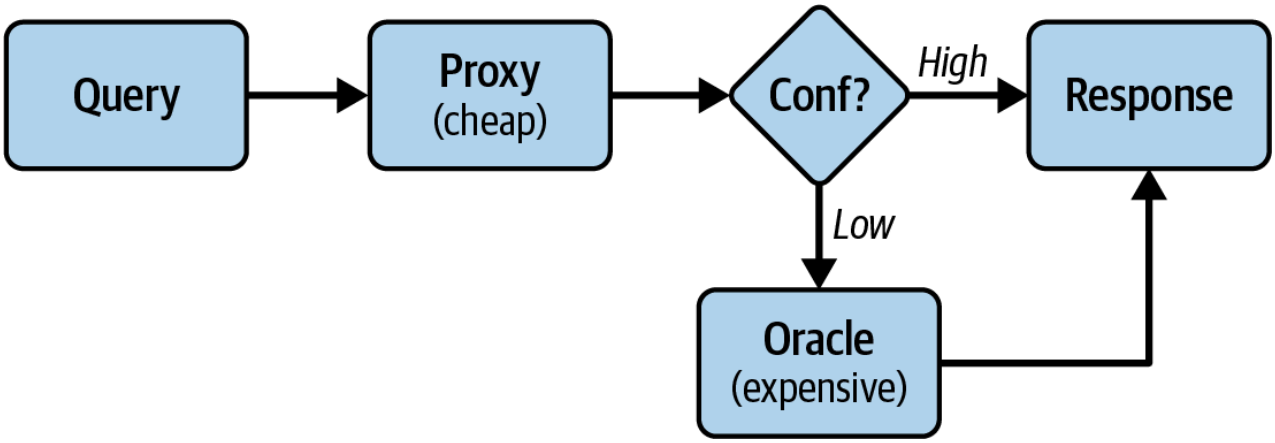


Figure 12-2. A model cascade

图 12-2. 模型级联图

The oracle does not need to be perfect. It is simply our best current system. The cascade tries to reproduce the oracle's answers at closer to the proxy's cost.

这个预言机不需要做到完美无缺。它只是我们目前最好的系统罢了。Cascade 系统试图以接近代理的成本来复制预言机的答案。

ESTIMATING CONFIDENCE 估算置信度

Do not take model confidence at face value. Whether we read it from log probabilities or ask the model to verbalize it (discussed later in this section), it is easy—even tempting—to treat confidence as a reliable measure of correctness. It is not. Confidence is just another signal the model produces, and it is often noisy.

不要轻信模型的置信度数值。无论是从对数概率的角度来理解，还是让模型自行表述置信度（本节后面会讨论），都将置信度视为正确性的可靠指标——这其实是很自然的，甚至有些诱人。但实际上并非如此。置信度只是模型产生的另一个信号而已，而且往往带有噪声。

The goal here is not to validate confidence against human labels. We run the proxy and the oracle on the same held-out examples and ask a simpler question: when the proxy reports high confidence, does its answer usually match the oracle's? When they match, does the proxy usually report high confidence? That correlation tells us where to set the threshold—the cutoff above which we can trust the proxy to replicate the oracle's output without calling it. A smaller proxy may look confident far more often than it deserves. Fortunately, the threshold-finding process is robust to poor calibration: if the confidence scores don't separate correct from incorrect answers, the calibration step will simply fail to find a valid threshold, and the cascade will route everything to the oracle, which is the safe default. Figure 12-2 shows the decision we are calibrating; we walk through how to find that threshold later in this section.

这里的目标不是来验证模型对人类标签的置信度。我们在相同的保留样本上运行代理模型和真实模型，然后提出一个更简单的问题：当代理模型报告出较高的置信度时，其预测结果通常会与真实模型的结果一致吗？当两者一致时，代理模型是否也通常会表现出较高的置信度？这种相关性可以帮助我们确定阈值——即我们可以信赖代理模型来复制真实模型输出的阈值。较小的代理模型可能经常表现出较高的置信度，但实际上它并不值得如此。幸运的是，确定阈值的流程对糟糕的校准情况具有鲁棒性：如果置信度分数无法区分正确和错误的答案，那么校准步骤将无法找到有效的阈值，此时系统会将所有输入数据都传递给真实模型，这才是安全的默认选择。图 12-2 展示了我们正在校准的决策过程；在本节的后续部分，我们将详细介绍如何找到这个阈值。

The cascade depends on knowing how confident the proxy is in each answer. To understand how we can get this, recall from Chapter 1 that LLMs generate text one token at a time, assigning a probability to every token in the vocabulary at each step. When the model generates `true` for a classification task, it has internally computed a probability for `true` and a probability for every other token (`false` , `maybe` , `yes` , etc.). If the probability of `true` is 0.90 and the probability of `false` is 0.10, the model is very con-

fident. If `true` is 0.55 and `false` is 0.45, it is close to a coin flip.

这种概率的分配取决于模型对每个答案的置信程度。要理解这一点，可以回想一下第一章的内容：大语言模型每次生成一个文本令牌，并为词汇表中每个令牌分配一个概率值。当模型为分类任务生成答案时，它会内部计算出每个令牌的概率值。例如，如果 `true` 的概率为 0.90， `false` 的概率为 0.10，那么模型对 `true` 的置信度非常高。而如果 `true` 的概率为 0.55， `false` 的概率为 0.45，那么这种情况就接近于随机情况了。

This is why *classification tasks* (where the answer is a single token like `true` or `false`) are the best case for cascades. We can use the probabilities of the valid answers as confidence estimates for the cascade.

这就是为什么分类任务（例如，答案是一个单独的标记，如 `true` 或 `false` ）是最适合用于级联模型的情况。我们可以将有效答案的概率作为级联模型的置信度估计。

Approach 1: Logprobs for classification tasks

方法 1：用于分类任务的 Logprobs 方法

Most LLM APIs do not return probabilities directly. They return *log probabilities* (logprobs), which are the natural logarithm of the probabilities.[2] A probability of 0.90 becomes a logprob of about -0.10. A probability of 0.01 becomes a logprob of about -4.60. Logprobs are always negative or zero, and closer to zero means higher probability.

大多数大语言模型 API 并不直接返回概率值。它们返回的是对数概率，即概率的自然对数。例如，0.90 的概率对应的对数概率约为-0.10；而 0.01 的概率对应的对数概率则约为-4.60。对数概率总是负数或零，数值越接近零，表示概率越高。

When we request logprobs from the API, most providers return the logprobs for the top 5 to 20 most likely tokens. For a prompt that ends with "Answer true or false," both `true` and `false` almost always appear in that top list. We convert both logprobs back to probabilities with `exp()`, normalize so they sum to 1, and use the higher one as the confidence score. Figure 12-3 walks through an example where the model is about 90% confident in `true`.

当我们通过 API 请求 logprobs 时，大多数提供商会返回最有可能出现的 5 到 20 个关键词对应的 logprobs。对于一个以"回答是或不是"结尾的提示， `true` 和 `false` 几乎总是会出现在排名前名单中。我们会将这两个 logprobs 转换回概率值，并进行归一化处理，使得总和为 1。然后选择其中概率较高的那个作为置信度得分。图 12-3 展示了一个例子：模型对 `true` 的置信度约为 90%。

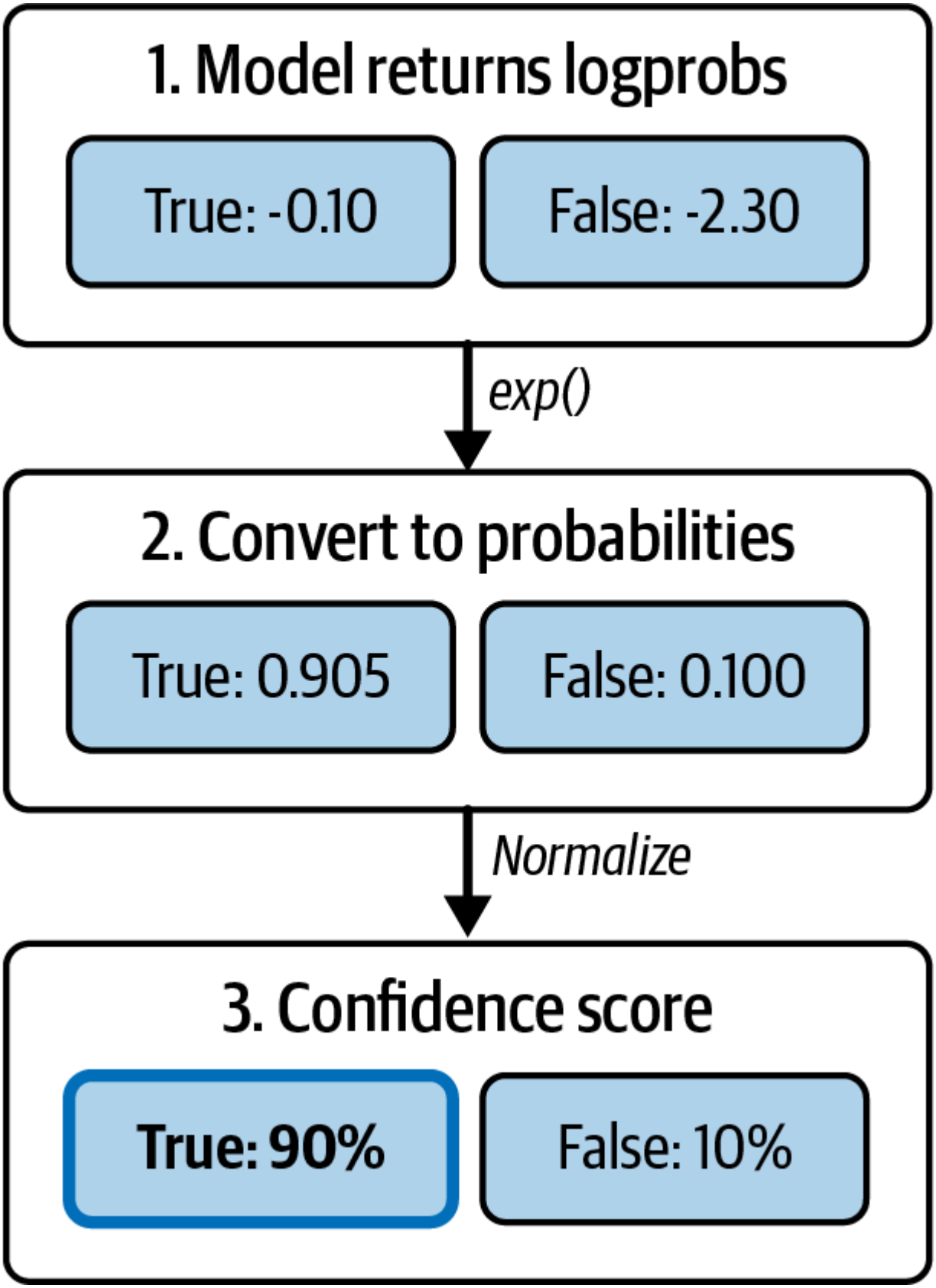


Figure 12-3. Turning logprobs into a confidence score for a binary classification task

图 12-3. 将 logprobs 转换为置信度得分，用于二元分类任务

For open-ended tasks where the answer is a full sentence or paragraph, this approach does not work. The answer is many tokens long, and the top-token logprobs only tell us about one token at a time. There is no way to compare "the probability of this full answer" against "the probability of a different full answer" because the API does not

give us that.

对于需要回答完整句子或段落的开放式任务来说，这种方法是不可行的。回答通常包含许多词汇，而顶级词汇的 logprobs 只能告诉我们每个词汇的概率。我们无法将“这个完整回答的概率”与“另一个完整回答的概率”进行比较，因为该 API 并不提供这种功能。

Approach 2: Verbalized confidence for open-ended tasks (use with caution)

方法二：在开放式任务中使用口头表达式的自信度评估（使用时要谨慎）

In practice, LLMs are rarely used as pure classifiers. When LLMs produce open-ended text, Approach 1 leaves us missing two things:

在实践中，大语言模型很少被用作纯粹的分类器。当大语言模型生成开放式文本时，第一种方法就存在两个不足之处：

1. We cannot easily tell whether the proxy's output agrees with the oracle's. A string match fails when two valid answers use different words.

   我们无法轻易判断代理的输出是否与预言机的输出一致。当两个有效的答案使用不同的词语时，字符串匹配就会失败。

2. We cannot use logprobs as a confidence estimate. The answer spans many tokens, and the API does not expose one probability for the entire response.

   我们无法将 logprobs 用作置信度的估计指标。该答案包含多个标记，而该 API 并没有提供整个响应的单一概率值。

Instead, we ask the proxy to return *structured output*: the open-ended answer in one field, and a separate verbalized confidence score in another. We can add this to the prompt (e.g., “On a scale from 0 to 10, how confident are you in your answer?”), or enforce it with JSON schema through the provider's structured output API.[3] Example 12-1 shows an OpenAI Chat Completions request that requires both fields.

相反，我们要求代理返回结构化输出：一个字段包含开放式回答，另一个字段则包含明确的置信度评分。我们可以在提示中加入这一信息（例如：“在 0 到 10 的范围内，你对自己的回答有多自信？”），或者通过提供商的结构化输出 API 来强制执行这一要求。示例 12-1 展示了一种需要同时包含这两个字段的 OpenAI Chat Completions 请求。[3]

Example 12-1. Requesting structured output with an open-ended answer and a confidence score (OpenAI Chat Completions API)

示例 12-1：使用 OpenAI Chat Completions API 请求结构化输出，包括开放式答案和置信度评分

```
response = client.chat.completions.create(
    model="gpt-4o-2024-08-06",
    messages=[
        {
            "role": "system",
            "content": (
                "Evaluate the trace. Return JSON with 'answer' "
                "(your evaluation) and 'confidence' (0-10, how "
                "confident you are in your answer)."
            ),
        },
        {"role": "user", "content": trace_text},
    ],
    response_format={
        "type": "json_schema",
        "json_schema": {
            "name": "evaluator_response",
            "strict": True,
            "schema": {
                "type": "object",
                "properties": {
                    "answer": {"type": "string"},
                    "confidence": {
                        "type": "integer",
                        "minimum": 0,
                        "maximum": 10,
                    },
                },
                "required": ["answer", "confidence"],
                "additionalProperties": False,
            },
        },
    },
)
```

The response format looks roughly like this:

响应格式大致如下：

```
{
  "answer": "The email tone is professional and appropriate for a luxury buyer.",
  "confidence": 8
}
```

WARNING 警告

Verbalized confidence calibration is improving (Tian et al. 2023), and in our experiments it works reasonably well for classification tasks. For open-ended outputs, however, verbalized scores are much less reliable. Base models also tend to be better calibrated than instruction-tuned or fine-tuned models (Tian et al. 2023), because RLHF training biases models toward expressing confidence even when it isn't warranted.

通过语言描述来校准信心度已经取得了不错的进展（Tian 等人，2023 年）。在我们的实验中，这种方法在分类任务上表现相当不错。不过，对于需要开放式回答的任务来说，通过语言描述获得的分数则缺乏可靠性。基础模型通常比经过微调或优化的模型更能适应这种校准方式（Tian 等人，2023 年），因为强化学习下的信心校准训练会促使模型在无需表达信心时也会主动表达出信心。

To learn whether the verbalized confidence score is useful at all, we still need to know when the proxy's output agrees with the oracle's. If we have a high-quality LLM-as-judge we already trust (Chapter 5), we can use it offline to score agreement between proxy and oracle outputs. If we do not have such a judge, we fall back to human labels.

要判断所得到的置信度评分是否真的有用处，我们还需要知道代理的输出与预言器的输出何时是一致的。如果我们有一个高质量的、可以作为评判标准的 LLM（参见第 5 章），我们就可以用它来评估代理输出与预言器输出之间的一致性。如果我们没有这样的评判标准，那我们就不得不依赖人类提供的标签了。

Just as in Approach 1, we then look for confidence thresholds where proxy-oracle agreement is high enough to meet our target. The exact tuning procedure is described in "Offline Calibration for a Model Cascade".

与方法 1 类似，我们继续寻找那些代理与决策树之间的一致性足够高的置信度阈值。具体的调参过程可以在"模型级联的离线校准"一文中找到详细说明。

NOTE 注意/备注

If the proxy's confidence scores (whether from logprobs or verbalized) do not correlate with whether its answers match the oracle, we will not find a threshold that meets the target accuracy. The cascade then defaults to sending everything to the oracle. We have not saved any money, but nothing has broken. Better-calibrated proxies save more because we can trust them on a larger fraction of queries.

如果代理程序的置信度得分（无论是基于对数概率还是以自然语言表述）与它的答案是否与预言结果一致无关，那么我们将无法找到满足目标准确率的阈值。在这种情况下，系统会默认将所有查询结果都提交给预言程序来判断。虽然我们没有节省任何成本，但也没有出现任何问题。那些校准更精确的代理程序能够节省更多资源，因为我们可以在更大比例的查询上信任它们的结果。

Tuning the cascade threshold

调整级联阈值的阈值

Before deploying the cascade, we calibrate it offline to find the confidence threshold.

在部署级联模型之前，我们先在离线环境中对其进行校准，以确定置信度阈值。

We take a held-out evaluation set of *N* examples and run both the proxy and the oracle on all of them. For each example, we record the proxy's answer, its confidence score, and the oracle's answer (which we treat as ground truth). We then find the lowest confidence threshold where the proxy's answers agree with the oracle at least as often as our target accuracy requires (e.g., 95% of the time). The following sidebar shows the procedure.

我们选取了 N 个样本进行评估，并对每个样本同时使用代理模型和真实值进行预测。对于每个样本，我们记录代理模型的预测结果、其置信度以及真实值（作为基准真值）。然后，我们找到最低的置信度阈值，使得代理模型的预测结果与真实值一致的频率至少达到我们设定的目标准确率要求（例如，95%的情况下）。下图展示了这一过程的详细步骤。

We find the confidence threshold where the proxy agrees with the oracle at the target rate:

我们找到了一个置信度阈值，在这个阈值下，该代理指标与目标值一致：

Inputs 输入内容

Held-out evaluation set of *N* examples. Target agreement rate *r* (e.g., 0.95).

包含 N 个样本的评估数据集。目标一致性率 r（例如，0.95）。

Procedure 程序/步骤

1. Run the proxy on all *N* examples. Record each answer and confidence score.

   在所有 N 个例子中运行代理服务器。记录每个例子的回答结果以及置信度得分。

2. Run the oracle on all *N* examples. Record each answer.

   对所有 N 个例子运行预言程序，并记录每个问题的答案。

3. Mark whether the proxy answer matches the oracle answer for each example.

   请标记每个例子中，代理回答是否与预言答案一致。

4. Sort examples by proxy confidence, highest first.

   按照代理置信度对示例进行排序，从高到低排列。

5. Walk down the sorted list. At each position *i*, compute the match rate among examples 1 through *i*.

   沿着排序后的列表向下遍历。在每个位置，计算从 1 到 n 的样本之间的匹配率。

6. Set the threshold *t* to the confidence score at the last position where the match rate is at least *r*.

   将阈值 t 设定为最后一个位置的置信度得分，该位置的匹配率至少为 r。

Output 输出结果

Threshold *t*. 阈值 t。

At runtime, every production query goes to the proxy first. If the proxy's confidence is at or above *t*, we return the proxy's answer. If it is below *t*, we escalate to the oracle.

在运行时，每个实际查询都会先经过代理服务器处理。如果代理服务器的判断置信度达到或高于 t 值，我们就采用其提供的答案；如果置信度低于 t 值，我们就需要将其问题提交给数据库服务器处理。

One consequence of this two-step routing is that escalated queries are slower than if we had called the oracle directly, because they pass through both the proxy and the oracle. This is acceptable when most queries (70%–80%) are handled by the proxy alone, since the average latency across all queries still drops.

这种两步处理流程的一个后果是，对于需要升级处理的查询来说，其执行速度会比直接调用数据库服务器要慢一些，因为查询需要经过代理服务器和数据库服务器的双重处理。不过，当大部分查询（70%–80%）都能由代理服务器直接处理时，这种延迟是可以接受的，因为所有查询的平均执行时间仍然会有所降低。

If we have more than two models at different cost levels, we can chain them: try the cheapest model first, escalate to the next one if confidence is low, and so on until one is confident enough or we reach the oracle. Each model gets its own threshold, calibrated the same way.

如果我们拥有多个成本不同的模型，就可以将它们串联起来使用：先尝试成本最低的模型，如果信心不足，则选择下一个模型继续尝试，如此反复直到足够信任某个模型，或者达到“神谕式”的解决方案。每个模型的阈值都是独立设定的，采用相同的方法进行校准。

For more on model cascades, see the FrugalGPT paper (Chen et al. 2023), which introduced LLM cascades and showed they can match GPT-4 quality at a fraction of the cost. Task cascades (Shankar et al. 2026) extend this idea by varying what the model is asked to do at each stage, not just which model is used. For example, a cheap model might answer a simplified version of the question, and only queries it cannot handle get passed to a more expensive model with the full, detailed prompt.

有关模型级联的更多信息，请参阅 FrugalGPT 论文（Chen 等人，2023 年）。该论文介绍了 LLM 级联技术，并证明以较低的成本就能获得与 GPT-4 相当的效果。而任务级联技术（Shankar 等人，2026 年）则在此基础上进一步扩展，通过在不同阶段要求模型执行不同的任务，而不仅仅是更换使用的模型。例如，一个成本较低的模型可以回答问题的简化版本，而那些无法处理复杂问题的任务则会被交给更昂贵的模型来处理，让该模型能够处理完整的、详细的输入。

Routing 路由

Routing is different from cascading. In a cascade, every query starts at the cheap model and escalates if confidence is low. In routing, a classifier decides *up front* which model to send each query to, without trying the cheap model first. For example, a router might send simple queries ("what's the listing price?") directly to a fast, cheap model, and send complex queries ("compare these three properties across school districts, commute times, and price trends") directly to a more capable and expensive one.

路由与级联不同。在级联模式中，每个查询都从成本较低的模型开始处理，如果置信度较低，则会将查询升级到成本较高的模型进行处理。而在路由模式中，分类器可以预先决定每个查询应该被发送到哪个模型进行处理，而无需先尝试成本较低的模型。例如，路由器可能会将简单的查询（"比如价格是多少？"）直接发送给成本较低且效率较高的模型，而将复杂的查询（"比较不同学区之间的三种属性，包括通勤时间以及价格趋势"）直接发送给更强大且成本更高的模型。

Routing avoids the latency penalty of cascading (where escalated queries pass through two models), but it requires a reliable classifier, and misrouted queries get no second chance. In practice, some teams combine both: a router handles the obvious cases, and a cascade handles the ambiguous middle.

路由方式避免了级联查询所带来的延迟问题（即查询会经过两个模型进行处理），但它需要一个可靠的分类器来正确识别查询的方向。而一旦查询方向被错误导向，就没有第二次机会了。实际上，有些团队会结合这两种方式：路由器处理那些较为明确的情况，而级联查询则用于处理那些模糊不清的情况。

Several off-the-shelf routing solutions are available. RouteLLM (Ong et al. 2024) is an open source router trained on human preference data from Chatbot Arena, which achieved 85% cost reduction on MT-Bench while maintaining 95% of GPT-4 quality. Microsoft Foundry's Model Router provides managed routing as a service, dynamically selecting models per query based on complexity and cost. Martian offers a similar managed routing product that adapts to your traffic over time. These tools let you add routing without building a classifier from scratch.

目前有几种现成的路由解决方案可供选择。RouteLLM（由 Ong 等人于 2024 年提出）是一种开源路由器，其训练数据来自 Chatbot Arena 的人类偏好数据。该路由器在保持与 GPT-4 相当的性能的同时，将计算成本降低了 85%。微软 Foundry 的 Model Router 则提供托管路由服务，能够根据查询的复杂性和成本动态选择合适的模型。Martian 则提供了一种类似的托管路由产品，可以随着流量需求的变化而进行调整。这些工具让你无需从头开始构建分类器，就能轻松实现路由功能的添加。

Closing: The Lifecycle Continues

结尾：生命循环持续下去

This book followed the *Analyze-Measure-Improve* lifecycle introduced in Chapter 1. We started by reading traces and identifying failure modes (Chapter 3). We built evaluators that quantify those failures at scale (Chapter 5). We integrated those evaluators into CI so that regressions are caught before they reach users (Chapter 9). In this chapter, we used the evidence from those earlier phases to make concrete improvements in both accuracy and cost.

这本书遵循了第一章中提到的分析-测量-改进生命周期。我们首先分析了各种痕迹，并识别出故障模式（第三章）。随后，我们开发了能够大规模量化这些故障的评估工具（第五章）。接着，我们将这些评估工具整合到持续集成过程中，以便在故障影响用户之前及时发现问题（第九章）。在本章中，我们利用了前面几个阶段积累的经验，从而在准确性和成本方面取得了实质性的改进。

Every improvement can change the failure distribution. A prompt change that fixes one class of errors may introduce another. A cost optimization that routes queries to a cheaper model may degrade accuracy in subtle ways. The response is to keep the loop running: analyze the new behavior, measure the impact, and improve again.

每一个改进都可能改变错误的分布。那种能解决一类错误的快速解决方案，可能会引入另一类问题。而成本优化措施，虽然能将查询分配到成本更低的模型上，但却可能会以微妙的方式降低系统的准确性。因此，应对的方式是持续循环这个过程：分析新的行为模式，评估其影响，然后再次进行改进。

Evaluation datasets go stale as products evolve and user behavior shifts. Error taxonomies need periodic revisiting. New evaluators appear when new failure modes surface. Improvements go through CI so they persist across model updates and code changes.

随着产品的发展和用户行为的变化，评估数据集也会逐渐过时。因此，需要定期重新评估错误分类。当新的故障模式出现时，就需要新的评估人员来处理这些问题。改进措施需要经过持续性的测试验证，以确保它们能够在模型更新和代码更改中保持有效性。

Even an informal version of this loop (review some traces each week, update the golden set, fix the worst prompt) compounds over time.

即使只是以非正式的方式重复执行这个循环——每周回顾一些线索，更新“黄金集”，修复最糟糕的提示问题——随着时间的推移，其效果也会逐渐显现。

Choosing an Improvement Strategy

选择改进策略

With many improvement options available, it can be hard to know where to start. Table 12-1 maps each strategy to its typical effort, data requirements, and best-suited failure type.

由于有多种改进方案可供选择，因此很难确定从哪个方面开始着手。表 12-1 详细列出了每种策略对应的典型工作量、数据需求，以及最适合的失败类型。

Table 12-1. Decision framework for choosing an improvement strategy

表 12-1. 选择改进策略的决策框架

Strategy 策略/战略	Effort 努力/付出	Data needed 需要的数据	Best for 最适合……的情况/用途
Prompt refinement 及时改进	Low 低	A few failing examples 几个失败的例子	Specification failures, formatting issues, missing constraints 规范不符合要求、格式问题、缺失约束条件
Architectural changes (task decomposition, retrieval, tools) 建筑上的变化（任务分解、检索方式、工具的应用）	Medium 中等	Understanding of failure patterns 对失败模式的理解	Multi-step reasoning errors, missing information, complex workflows 多步骤推理中的错误、信息缺失、复杂的工作流程
Automated prompt optimization (DSPy, TextGrad) 自动化提示优化（DSPy、TextGrad）	Medium 中等	50+ labeled examples + evaluation function 50 个以上的标注示例 + 评估函数	Systematic quality improvements when manual prompt iteration plateaus 在手动提示迭代阶段，系统实现了有计划的品质提升
Parameter-efficient fine-tuning (LoRA/QLoRA) 参数高效微调（LoRA/QLoRA）	High 很高	50 to 200+ labeled examples 50 到 200 个带有标注的示例	Narrow, well-defined tasks where prompting alone can't fix generalization failures 那些需要明确界定的狭窄任务，这类任务仅靠提示本身是无法解决泛化问题的
Full fine-tuning/preference tuning (DPO) 完全调优/偏好调整（DPO）	High 很高	500+ labeled examples or preference pairs 500 多个带有标签的示例或偏好对	Style, tone, or format issues across many tasks;

Strategy 策略/战略	Effort 努力/付出	Data needed 需要的数据	Best for 最适合……的情况/用途
			when PEFT isn't enough 在许多任务中都会出现风格、语气或格式方面的问题；当单纯使用 PEFT 方法已经不够时
Distillation + cascade 蒸馏 + 级联处理	High 很高	Large labeled dataset + oracle model 大型标注数据集 + 基于真实数据的模型	Cost reduction while maintaining accuracy on high-volume, well-understood tasks 在保持高准确率的同时降低成本，适用于那些需求量大且任务较为明确的场景。

Start at the top of the table and work down. Each row requires more effort and data than the one above it, so exhaust the cheaper options first before investing in heavier interventions.

从表格的顶部开始，逐渐向下处理。每一行所需的处理时间和精力都比上一行要多，因此先处理那些成本较低的选项，然后再考虑采用更昂贵的方法。

Next Steps 下一步

This book covers what we consider the durable core of evaluation practice. The specific models, APIs, and tools will keep changing; by the time you read this, some of the examples may already reference older model versions, and new evaluation platforms and techniques will have appeared. But the practice of systematically understanding how your system fails, measuring those failures, and fixing them is not going away. The Analyze-Measure-Improve lifecycle is a framework for thinking, not a set of frozen instructions, and it adapts to whatever the next generation of models and tools looks

like.

这本书涵盖了我们认为的评估实践的核心内容。具体的模型、接口和工具会不断演变；当你阅读这本书时，一些示例可能会引用更老版本的模型，而新的评估平台和技术也会不断涌现。不过，系统地理解系统为何会失败、衡量这些失败情况并对其进行修复的做法是不会消失的。分析-测量-改进的生命周期是一种思考方式，而不是一套固定的指令集。它会随着下一代模型和工具的发展而不断调整自身。

Several areas are evolving quickly and are likely to change how evaluation works in the coming years. Reasoning models that produce long internal chains of thought raise new questions about how to evaluate intermediate reasoning steps, not just final answers. Multi-agent systems, where several agents coordinate to complete a task, introduce evaluation challenges around inter-agent communication and collective behavior that single-agent evaluation doesn't cover. Process reward models, which score each step of a solution rather than just the final output, may change how we build automated evaluators. And as agents take on longer-running, more autonomous workloads, the line between evaluation and monitoring will continue to blur.

许多领域正在快速发展，在未来几年内，评估的方式可能会发生变革。那些能够产生复杂内部思维链的逻辑模型，引发了关于如何评估中间推理步骤的问题，而不仅仅是最终答案。多智能体系统需要多个智能体协同完成任务，这就带来了关于智能体间沟通以及集体行为的评估挑战，而这些挑战是单智能体评估无法解决的。过程奖励模型会针对解决方案的每一个步骤进行评分，而不仅仅是最终输出结果，这可能会改变我们构建自动化评估工具的方式。随着智能体承担越来越长且更自主的工作任务，评估与监控之间的界限将继续模糊下去。

We don't cover these topics in depth here because the community is still working out the practical guidance. But the foundations in this book, namely reading traces, building taxonomies, calibrating judges, running CI, and reviewing outputs, will apply to all of them.

我们在这里不会深入讨论这些主题，因为相关社区仍在努力完善这些实用指南。不过，这本书中所介绍的基础内容——比如读取痕迹、构建分类体系、调整评判器参数、执行 CI 操作以及审查输出结果——都是适用于所有相关场景的。

You can follow our ongoing work as the field moves. Hamel writes practical notes on his blog, Shreya writes about research and systems on her site, and Hamel posts video

walkthroughs on YouTube.

随着这一领域的发展，你可以持续关注我们的工作进展。Hamel 在他的博客上撰写实用性的文章，Shreya 则在她的网站上分享关于研究和系统的内容，而 Hamel 还会在 YouTube 上发布视频教程。

The live course based on this book is for teams actively shipping AI features who want to apply the framework faster: you bring your own evals and get direct feedback on them, see how others adapt the same framework to systems unlike yours, and work through hard cases together.

这本书籍所介绍的实战课程适用于那些正在积极开发人工智能功能的团队。这些团队希望更快地应用该框架。你们可以带上自己的评估代码，并获得直接的反馈；同时，还可以了解其他人是如何将同一框架应用到与你们不同的系统中的。此外，你们还可以一起解决一些复杂的案例。

1 We recommend waiting to try prompt optimization or fine-tuning until you have exhausted the quicker methods described earlier in this chapter. These heavier techniques are harder to debug and maintain.

我们建议等到你尝试了本章前面提到的那些较为简单的优化方法之后，再考虑使用更精细的优化技术。因为这些更复杂的优化方法更难调试和维护。

2 For OpenAI, see the logprobs parameter in the Chat Completions API.

对于 OpenAI 来说，可以在 Chat Completions API 中的 logprobs 参数を見到相关信息。

3 For OpenAI, see Structured Outputs.

关于 OpenAI 的相关内容，请参考"结构化输出"部分。

Bibliography 参考文献

Amershi, Saleema, Andrew Begel, Christian Bird, et al. 2019. "Software Engineering for Machine Learning: A Case Study." In *2019 IEEE/ACM 41st International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, Montreal, QC. 291–300.

阿姆尔希、萨伊玛、安德鲁·贝格尔、克里斯蒂安·伯德等。2019 年。"机器学习软件工程：一个案例研究"。载于 2019 年 IEEE/ACM 第 41 届国际软件工程会议：实践中的软件工程（ICSE-SEIP）会议论文集，蒙特利尔，QC。第 291-300 页。

Anthropic. 2024a. "Create Strong Empirical Evaluations." *https://docs.anthropic.com/en/docs/build-with-claude/develop-tests*.

人本性特性。2024a。"创建可靠的实证评估。" https://docs.anthropic.com/en/docs/build-with-claude/develop-tests

Anthropic. 2024b. "Prompt Caching with Claude." *https://www.anthropic.com/news/prompt-caching*.

人本性技术。2024b。"使用 Claude 进行提示缓存。" https://www.anthropic.com/news/prompt-caching

Arawjo, Ian. 2025a. "EvalGen: Helping Developers Create LLM Evals Aligned to Their Preferences." Medium. *https://ianarawjo.medium.com/evalgen-helping-developers-create-llm-evals-aligned-to-their-preferences-26757f7e145d*.

Arawjo, Ian. 2025a. "EvalGen：帮助开发者创建符合他们偏好的大语言模型评估方案。" Medium 网站文章。https://ianarawjo.medium.com/evalgen-helping-developers-create-llm-evals-aligned-to-their-preferences-26757f7e145d

Arawjo, Ian. 2025b. "What AI Engineers Can Learn from Qualitative Research Methods in HCI." Medium. *https://ianarawjo.medium.com/what-ai-engineers-can-learn-from-qualitative-research-methods-in-hci-5b29b9b7465a*.

Arawjo, Ian. 2025b. "人工智能工程师可以从定性研究方法中学习到什么。" Medium 网站文章。https://ianarawjo.medium.com/what-ai-engineers-can-learn-from-qualitative-research-methods-in-hci-5b29b9b7465a

Arawjo, Ian, Chelse Swoopes, Priyan Vaithilingam, Martin Wattenberg, and Elena Glassman. 2024. "ChainForge: A Visual Toolkit for Prompt Engineering and LLM

Hypothesis Testing." In *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems*. arXiv:2309.09128.

Arawjo, Ian, Chelse Swoopes, Priyan Vaithilingam, Martin Wattenberg 和 Elena Glassman。2024 年。"ChainForge：一种用于提示生成和大型语言模型假设检验的可视化工具包。"发表于《2024 年计算机系统中人类因素会议》论文集。arXiv:2309.09128。

Arize AI. 2025. "Phoenix: Open-Source LLM Tracing & Evaluation." Accessed 2025-05-04. *https://phoenix.arize.com*.

Arize AI. 2025 年。"凤凰：开源大语言模型跟踪与评估系统。"访问日期：2025 年 5 月 4 日。https://phoenix.arize.com

Artstein, Ron, and Massimo Poesio. 2008. "Inter-Coder Agreement for Computational Linguistics." *Computational Linguistics* 34: 555–596.

Artstein, Ron 和 Massimo Poesio。2008 年。"计算语言学中的编码者一致性问题"。《计算语言学》杂志，第 34 卷，第 555-596 页。

Ball, Thomas. 1999. "The Concept of Dynamic Analysis." *ACM SIGSOFT Software Engineering Notes* 24: 216–234.

托马斯·鲍尔。1999 年。《动态分析的概念》。ACM SIGSOFT 软件工程笔记第 24 期：216-234 页。

Berryman, John, Shawn Simister, and Hamel Husain. 2025. "How Evals Made GitHub Copilot Work." *https://maven.com/p/da8264/how-evals-made-github-copilot-work*.

贝里曼、约翰、肖恩·西米斯特和哈梅尔·侯赛因。2025 年。"评估如何优化 GitHub Copilot 的功能。"https://maven.com/p/da8264/how-evals-made-github-copilot-work

Bischof, Bryan. 2025. "The Map Is Not the Territory." YouTube video. *https://www.youtube.com/watch?v=zqjnEptOn4k*.

比肖夫，布莱恩。2025 年。"地图并非领土。" YouTube 视频。https://www.youtube.com/watch?v=zqjnEptOn4k

Bommasani, Rishi, Drew A. Hudson, Ehsan Adeli, et al. 2021. "On the Opportunities and Risks of Foundation Models." arXiv:2108.07258.

Bommasani, Rishi, Drew A. Hudson, Ehsan Adeli 等。2021 年。"基础模型的机会与风险"。arXiv:2108.07258。

Bordes, Florian, Richard Yuanzhe Pang, Anurag Ajay, et al. 2024. "An Introduction to Vision-Language Modeling." arXiv:2405.17247.

博德斯、弗洛里安、理查德·袁哲·庞、阿努拉格·阿贾伊等。2024 年。“视觉与语言建模简介”。arXiv:2405.17247。

Breck, Eric, Shanqing Cai, Eric Nielsen, Michael Salib, and D. Sculley. 2017. “The ML Test Score: A Rubric for ML Production Readiness and Technical Debt Reduction.” *In 2017 IEEE International Conference on Big Data (Big Data)*. 1123–1132.

布雷克、埃里克、蔡善清、埃里克·尼尔森、迈克尔·萨利布和 D·斯卡利。2017 年。“机器学习测试分数：评估机器学习项目准备程度及技术债务减少程度的指标”。载于 2017 年 IEEE 大数据国际会议论文集，第 1123-1132 页。

Bringhurst, Robert. 2005. *The Elements of Typography*. Hartley & Marks.

布里恩赫斯特，罗伯特。2005。《排版要素》。哈特利与马克斯出版社。

Brown, Lawrence D., T. Tony Cai, and Anirban DasGupta. 2001. “Interval Estimation for a Binomial Proportion.” *Statistical Science* 16: 101–133.

布朗、劳伦斯·D.，蔡唐尼，和阿尼尔班·达斯古普塔。2001 年。“二项分布的区间估计”。《统计科学》杂志，第 16 卷，第 101-133 页。

Brown, Tom, Benjamin Mann, Nick Ryder, et al. 2020. “Language Models Are Few-Shot Learners.” *Advances in Neural Information Processing Systems* 33: 1877–1901.

布朗，汤姆；本杰明·曼；尼克·莱德等。2020 年。“语言模型是少样本学习器”。《神经信息处理系统进展》第 33 卷：1877–1901 页。

Cemri, Mert, Melissa Z. Pan, Shuyi Yang, et al. 2025. “Why Do Multi-Agent LLM Systems Fail?” arXiv:2503.13657. *https://arxiv.org/abs/2503.13657*.

Cemri、Mert、Melissa Z. Pan、Shuyi Yang 等人。2025 年。“为什么多智能体 LLM 系统会失败？”arXiv:2503.13657。https://arxiv.org/abs/2503.13657

Chen, Boyuan, Zhuo Xu, Sean Kirmani, et al. 2024. “SpatialVLM: Endowing Vision-Language Models with Spatial Reasoning Capabilities.” In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, Seattle, WA. 14455–14465.

陈博远、卓旭、Sean Kirmani 等。2024 年。“SpatialVLM：为视觉语言模型赋予空间推理能力。”发表于《IEEE/CVF 计算机视觉与模式识别会议论文集》，西雅图，华盛顿州。14455–14465 页。

Chen, Lingjiao, Matei Zaharia, and James Zou. 2023. “FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance.” arXiv:2305.05176.

陈凌娇、Matei Zaharia 和 James Zou。2023 年。“FrugalGPT：如何在降低成本的同时提升大型语言模型的性能。”arXiv:2305.05176。

Chen, Lingjiao, Matei Zaharia, and James Zou. 2024. “How Is ChatGPT’s Behavior Changing over Time?” *Harvard Data Science Review* 6.

陈凌娇、Matei Zaharia 和 James Zou。2024 年。“ChatGPT 的行为如何随时间而变化？”《哈佛数据科学评论》第 6 期。

Chen, Mark, Jerry Tworek, Heewoo Jun, et al. 2021. “Evaluating Large Language Models Trained on Code.” arXiv:2107.03374.

陈、马克、杰里·托雷克、俊熙等。2021 年。“评估基于代码训练的大型语言模型”。arXiv:2107.03374。

Chiang, Wei-Lin, Lianmin Zheng, Ying Sheng, et al. 2024. “Chatbot Arena: An Open Platform for Evaluating LLMs by Human Preference.” In *Forty-First International Conference on Machine Learning (ICML)*. 8359–8388. *https://arxiv.org/abs/2403.04132*.

Chiang, Wei-Lin, Lianmin Zheng, Ying Sheng 等。2024 年。“聊天机器人竞技场：一个用于通过人类偏好来评估大语言模型的开放平台。”载于第四十一届国际机器学习会议（ICML）。8359–8388 页。https://arxiv.org/abs/2403.04132

Chiang, Wei-Lin, Zhuohan Li, Zi Lin, et al. 2023. “Vicuna: An Open-Source Chatbot Impressing GPT-4 with 90% ChatGPT Quality.” *https://lmsys.org/blog/2023-03-30-vicuna*.

Chiang, Wei-Lin, Zhuohan Li, Zi Lin 等。2023 年。“Vicuna：一个开源聊天机器人，其聊天体验达到了与 GPT-4 相当的水平，准确率高达 90%。” https://lmsys.org/blog/2023-03-30-vicuna

Clavié, Benjamin. 2024. “rerankers: A Lightweight Python Library to Unify Ranking Methods.” arXiv:2408.17344. *https://arxiv.org/abs/2408.17344*.

克利亚埃，本杰明。2024 年。“rerankers：一个轻量级的 Python 库，用于统一排名方法。”arXiv:2408.17344。https://arxiv.org/abs/2408.17344

Clavié, Benjamin. 2025. “RAGatouille: Easily Use and Train State-of-the-Art Late-Interaction Retrieval Methods (ColBERT) in Any RAG Pipeline.” GitHub (Version 0.0.9). *https://github.com/AnswerDotAI/RAGatouille*.

克利亚埃，本杰明。2025 年。“RAGatouille：在任何 RAG 管道中轻松使用并训练先进的后期交互式信息检索方法（ColBERT）。” GitHub（版本 0.0.9）。https://github.com/AnswerDotAI/RAGatouille。

Cobbe, Karl, Vineet Kosaraju, Mohammad Bavarian, et al. 2021. “Training Verifiers to Solve Math Word Problems.” arXiv:2110.14168.

Cobbe, Karl, Vineet Kosaraju, Mohammad Bavarian 等。2021 年。“训练验证器以解决数学文字题。”arXiv:2110.14168。

Cohen, Jacob. 1960. “A Coefficient of Agreement for Nominal Scales.” *Educational and Psychological Measurement* 20: 37–46.

科恩，雅各布。1960 年。“名义尺度下的一致性系数”。《教育与心理学测量》第 20 卷，第 37-46 页。

Colvin, Samuel. 2017. “Pydantic.” GitHub. Accessed 2025-05-03. *https://github.com/pydantic/pydantic*.

科尔文，塞缪尔。2017 年。“Pydantic”项目。GitHub 网站。访问日期：2025 年 5 月 3 日。https://github.com/pydantic/pydantic

Cormack, Gordon V., Charles L. A. Clarke, and Stefan Büttcher. 2009. “Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods.” In *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*. 758–759. *https://dl.acm.org/doi/10.1145/1571941.1572114*.

Cormack, Gordon V., Charles L. A. Clarke, 和 Stefan Büttcher. 2009. “互惠排序融合方法优于康德拉雪特法和个体排序学习方法。” 载于第 32 届国际 ACM SIGIR 信息检索研究与发展会议论文集。第 758-759 页。https://dl.acm.org/doi/10.1145/1571941.1572114

DeepSeek-AI. 2025. “DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning.” *Nature* 645: 633–638. *https://arxiv.org/abs/2501.12948*.

DeepSeek-AI. 2025. “DeepSeek-R1：通过强化学习激发大语言模型推理能力.” 《自然》杂志，第 645 期，第 633-638 页。https://arxiv.org/abs/2501.12948

Dohan, David, Winnie Xu, Aitor Lewkowycz, et al. 2022. “Language Model Cascades.” arXiv:2207.10342.

多汉、大卫，徐维妮，艾托尔·莱夫科维奇等。2022 年。“语言模型级联”。arXiv:2207.10342。

DoorDash Engineering. 2026. “A Simulation and Evaluation Flywheel to Develop LLM Chatbots at Scale.” *DoorDash Engineering* (blog). *https://careersatdoordash.com/blog/doordash-simulation-evaluation-flywheel-to-develop-llm-chatbots-at-scale*.

DoorDash Engineering。2026 年。“一种用于大规模开发 LLM 聊天机器人的仿真与评估方案。” DoorDash Engineering（博客）。https://careersatdoordash.com/blog/doordash-simulation-evaluation-flywheel-to-develop-llm-chatbots-at-scale

Epperson, Will, Gagan Bansal, Victor C. Dibia, et al. 2025. “Interactive Debugging and Steering of Multi-Agent AI Systems.” In *Proceedings of the 2025 CHI Conference on*

Human Factors in Computing Systems. 1–15.

埃珀森、威尔、加甘·班萨尔、维克多·C·迪比亚等。2025 年。“多智能体 AI 系统的交互式调试与操控”。载于《2025 年计算机系统中人类因素会议论文集》。第 1-15 页。

Es, Shahul, Jithin James, Luis Espinosa Anke, and Steven Schockaert. 2024. “RAGAS: Automated Evaluation of Retrieval Augmented Generation.” In *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations*. 150–158.

Es、Shahul、Jithin James、Luis Espinosa Anke 和 Steven Schockaert。2024 年。“RAGAS：自动评估检索增强生成系统。”载于《第 18 届欧洲计算语言学协会系统演示会议论文集》，第 150-158 页。

Fernandez, Tomas. 2024. “Handling Flaky Tests in LLM-Powered Applications.” *https://semaphore.io/blog/llms-flaky-tests*.

费尔南德斯，托马斯。2024 年。“在基于大语言模型的应用程序中处理不稳定的测试问题”。https://semaphore.io/blog/llms-flaky-tests

Fink, George, and Matt Bishop. 1997. “Property-Based Testing: A New Approach to Testing for Assurance.” *ACM SIGSOFT Software Engineering Notes* 22: 74–80.

芬克、乔治和马特·毕晓普。1997 年。《基于属性的测试：一种新的质量保证测试方法》。ACM SIGSOFT 软件工程笔记第 22 卷：74-80 页。

Fleiss, Joseph L. 1971. “Measuring Nominal Scale Agreement Among Many Raters.” *Psychological Bulletin* 76: 378.

弗莱斯，约瑟夫·L. 1971 年。《测量多评分者之间名义尺度一致性》。《心理学公报》第 76 卷，第 378 页。

Fourrier, Clémentine, and The Hugging Face Community. 2024. “LLM Evaluation Guidebook.” GitHub. *https://github.com/huggingface/evaluation-guidebook*.

Fourrier、Clémentine 和 Hugging Face 社区。2024 年。“大型语言模型评估指南”。GitHub 网站：https://github.com/huggingface/evaluation-guidebook

Friedman, Dan, and Adji Bousso Dieng. 2023. “The Vendi Score: A Diversity Evaluation Metric for Machine Learning.” *Transactions on Machine Learning Research 2023*. *https://openreview.net/forum?id=g97OHbQyk1*.

弗里德曼，丹，以及阿吉·布索·迪恩。2023 年。“Vendi 评分：一种用于机器学习领域多样性评估的指标。”机器学习研究交易会议论文集，2023 年。https://openreview.net/forum?id=g97OHbQyk1

Glaser, Barney, and Anselm Strauss. 2017. *Discovery of Grounded Theory: Strategies for Qualitative Research*. Routledge.

格莱瑟、巴尼和安塞尔姆·斯特劳斯。2017 年。《扎根理论的研究方法：定性研究的策略》。罗奇德出版社。

Gligorić, Kristina, Tijana Zrnic, Cinoo Lee, Emmanuel J. Candès, and Dan Jurafsky. 2024. "Can Unconfident LLM Annotations Be Used for Confident Conclusions?" arXiv:2408.15204.

格利戈里奇、克里斯蒂娜、蒂亚娜·兹尔尼茨、李晨奥、埃曼努尔·J·坎德斯和丹·朱拉夫斯基。2024 年。"不自信的大语言模型注释能否用于得出可靠的结论？" arXiv:2408.15204。

Hahn, Michael. 2020. "Theoretical Limitations of Self-Attention in Neural Sequence Models." *Transactions of the Association for Computational Linguistics* 8: 156–171.

Hahn, Michael. 2020. "神经序列模型中自注意力的理论限制。"《计算语言学协会会刊》第 8 卷，第 156-171 页。

Hendrycks, Dan, Collin Burns, Steven Basart, et al. 2021. "Measuring Massive Multitask Language Understanding." In *International Conference on Learning Representations*. *https://openreview.net/forum?id=d7KBjmI3GmQ*.

Hendrycks, Dan, Collin Burns, Steven Basart 等。2021 年。"大规模多任务语言理解能力的测量"。发表于《国际学习表示会议》论文集。https://openreview.net/forum?id=d7KBjmI3GmQ

Hohman, Fred, Chaoqun Wang, Jinmook Lee, et al. 2024. "Talaria: Interactively Optimizing Machine Learning Models for Efficient Inference." In *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems*. 1–19.

霍曼、弗雷德，王超群，李锦穆等。2024 年。"Talaria：交互式优化机器学习模型以实现高效推理。"载于《2024 年计算机系统人类因素会议论文集》。1-19 页。

Hsieh, Cheng-Ping, Simeng Sun, Samuel Kriman, et al. 2024. "RULER: What's the Real Context Size of Your Long-Context Language Models?" In *First Conference on Language Modeling*. *https://openreview.net/forum?id=kIoBbc76Sy*.

谢成平、孙胜民、Samuel Kriman 等。2024 年。"RULER：长上下文语言模型的真实上下文尺寸是多少？"发表于《第一届语言建模会议》。https://openreview.net/forum?id=kIoBbc76Sy

Hunter, John D. 2007. "Matplotlib: A 2D Graphics Environment." *Computing in Science & Engineering* 9: 90–95.

亨特，约翰·D. 2007 年。"Matplotlib：一种二维图形环境。"《科学工程计算》杂志，第 9 期，第 90-95 页。

Husain, Hamel. 2025. "A Field Guide to Rapidly Improving AI Products." *Hamel.dev* (blog). *https://hamel.dev/blog/posts/field-guide*.

胡赛因，哈梅尔。2025 年。“快速改进的人工智能产品指南”。哈梅尔开发博客。https://hamel.dev/blog/posts/field-guide

Jain, Naman, King Han, Alex Gu, et al. 2025. “LiveCodeBench: Holistic and Contamination Free Evaluation of Large Language Models for Code.” In *The Thirteenth International Conference on Learning Representations*. *https://openreview.net/forum?id=chfJJYC3iL*.

Jain, Naman, King Han, Alex Gu 等。2025 年。“LiveCodeBench：对大型语言模型进行全面且无污染的性能评估工具。”发表于《第十三届国际学习表示会议》。https://openreview.net/forum?id=chfJJYC3iL

Järvelin, Kalervo, and Jaana Kekäläinen. 2002. “Cumulated Gain-Based Evaluation of IR Techniques.” *ACM Transactions on Information Systems* 20 (4): 422–446.

Järvelin, Kalervo 和 Jaana Kekäläinen。2002 年。“基于累积收益的红外技术评估”。ACM 信息系统交易杂志，第 20 卷，第 4 期，第 422-446 页。

Jimenez, Carlos E., John Yang, Alexander Wettig, et al. 2024. “SWE-bench: Can Language Models Resolve Real-World GitHub Issues?” In *International Conference on Learning Representations*, Vienna, Austria.

Jimenez, Carlos E., John Yang, Alexander Wettig 等。2024 年。“SWE-bench：语言模型能否解决现实世界中的 GitHub 问题？”发表于《国际学习表示会议》，奥地利维也纳。

Kalai, Adam Tauman, and Santosh S. Vempala. 2024. “Calibrated Language Models Must Hallucinate.” In *Proceedings of the 56th Annual ACM Symposium on Theory of Computing*, Vancouver, BC. 160–171.

Kalai、Adam Tauman 和 Santosh S. Vempala。2024 年。“经过校准的语言模型必然会产生幻觉。”载于第 56 届 ACM 计算理论年会论文集，温哥华，不列颠哥伦比亚省。第 160–171 页。

Khattab, Omar, and Matei Zaharia. 2020. “ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT.” In *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*. 39–48. *https://arxiv.org/abs/2004.12832*.

Khattab, Omar, 和 Matei Zaharia。2020 年。“ColBERT：通过上下文化的后期交互来实现高效有效的检索方法。”载于第 43 届国际 ACM SIGIR 信息检索研究与发展会议论文集。39–48 页。https://arxiv.org/abs/2004.12832

Khattab, Omar, Arnav Singhvi, Paridhi Maheshwari, et al. 2024. “DSPy: Compiling Declarative Language Model Calls into State-of-the-Art Pipelines.” In *The Twelfth*

International Conference on Learning Representations, Vienna, Austria.

Khattab, Omar, Arnav Singhvi, Paridhi Maheshwari 等。2024 年。“DSPy：使用声明性语言模型来调用最先进的管道系统。”发表于《第十二届国际学习表示会议》，奥地利维也纳。

Kim, Seungone, Jamin Shin, Yejin Cho, et al. 2023. “Prometheus: Inducing Fine-Grained Evaluation Capability in Language Models.” In *The Twelfth International Conference on Learning Representations*, Vienna, Austria.

Kim, Seungone, Jamin Shin, Yejin Cho 等人。2023 年。“Prometheus：提升语言模型的细粒度评估能力”。发表于《第十二届国际表示学习会议》，奥地利维也纳。

Krippendorff, Klaus. 2011. “Computing Krippendorff's Alpha-Reliability.” Citeseer.

克里普恩多夫，克劳斯。2011 年。《计算克里普恩多夫的 Alpha 可靠性指标》。Citeseer 网站。

Kulal, Sumith, Panupong Pasupat, Kartik Chandra, et al. 2019. “SPoC: Search-Based Pseudocode to Code.” *Advances in Neural Information Processing Systems* 32.

Kulal, Sumith, Panupong Pasupat, Kartik Chandra 等。2019 年。“SPoC：基于搜索的伪代码到真代码的转换方法”。《神经信息处理系统进展》第 32 卷。

Laban, Philippe, et al. 2025. “LLMs Get Lost in Multi-Turn Conversation.” arXiv:2505.06120. *https://arxiv.org/abs/2505.06120*.

拉班、菲利普等，2025 年。“大型语言模型在多轮对话中的表现问题”。arXiv:2505.06120。https://arxiv.org/abs/2505.06120

Landis, J. Richard, and Gary G. Koch. 1977. “The Measurement of Observer Agreement for Categorical Data.” *Biometrics* 33 (1): 159–174.

兰迪斯、J.理查德和加里·G.科赫。1977 年。“分类数据的观察者一致性测量”。《生物统计学》杂志，第 33 卷，第 1 期，第 159-174 页。

LangChain. 2025. “LangSmith.” Accessed 2025-05-04. *https://www.langchain.com/langsmith*.

LangChain. 2025. “LangSmith”. 访问日期：2025-05-04. https://www.langchain.com/langsmith

Lauro, Quentin Romero, Shreya Shankar, Sepanta Zeighami, and Aditya Parameswaran. 2025. “RAG Without the Lag: Interactive Debugging for Retrieval-Augmented Generation Pipelines.” arXiv:2504.13587.

Lauro、Quentin Romero、Shreya Shankar、Sepanta Zeighami 和 Aditya Parameswaran。2025 年。“无需等待的交互式调试方法：用于检索增强生成模型的调试工具”。arXiv:2504.13587。

Li, Junnan, Dongxu Li, Caiming Xiong, and Steven Hoi. 2022. “BLIP: Bootstrapping Language-Image Pre-training for Unified Vision-Language Understanding and Generation.” In *International Conference on Machine Learning, Baltimore, MD*. 12888–12900.

李俊南、董旭李、熊彩明和史蒂文·霍伊。2022 年。“BLIP：用于统一视觉-语言理解与生成的语言-图像预训练方法。”发表于《国际机器学习会议》，马里兰州巴尔的摩市。页码 12888–12900。

Li, Tianle, Ge Zhang, Quy Duc Do, Xiang Yue, and Wenhu Chen. 2024. “Long-Context LLMs Struggle with Long In-Context Learning.” arXiv:2404.02060.

李天乐、葛张、杜奎德、薛颖、陈文虎。2024 年。“长上下文大语言模型在长上下文学习方面存在困难。”arXiv:2404.02060。

Liang, Percy, Rishi Bommasani, Tony Lee, et al. 2023. “Holistic Evaluation of Language Models.” *Transactions on Machine Learning Research*. *https://openreview.net/forum?id=iO4LZibEqW*.

梁、珀西、里希·博马萨尼、托尼·李等。2023 年。“语言模型的全面评估”。机器学习研究交易。https://openreview.net/forum?id=iO4LZibEqW

Liang, Weixin, Yaohui Zhang, Mihai Codreanu, Jiayu Wang, Hancheng Cao, and James Zou. 2025. “The Widespread Adoption of Large Language Model-Assisted Writing Across Society.” arXiv:2502.09747.

梁伟新、张耀辉、米哈伊·科德鲁安、王佳宇、曹汉成和詹姆斯·邹。2025 年。“大型语言模型辅助写作方式在社会的广泛应用”。arXiv:2502.09747。

Lin, Chin-Yew. 2004. “ROUGE: A Package for Automatic Evaluation of Summaries.” In *Text Summarization Branches Out*. 74–81.

林锦伟。2004 年。“ROUGE：一种用于自动摘要评估的工具包。”载于《文本摘要技术发展综述》，第 74–81 页。

Lin, Yiming, Mawil Hasan, Rohan Kosalge, Alvin Cheung, and Aditya G. Parameswaran. 2025. “TWIX: Automatically Reconstructing Structured Data from Templatized Documents.” arXiv:2501.06659.

林一鸣、马维尔·哈桑、罗汉·科萨尔格、阿尔文·张和阿迪蒂亚·G·帕拉梅斯瓦兰。2025 年。“TWIX：从模板化的文档中自动重构结构化数据”。arXiv:2501.06659。

Liu, Chunwei, Matthew Russo, Michael Cafarella, et al. 2025. “Palimpzest: Optimizing AI-Powered Analytics with Declarative Query Processing.” In *Proceedings of the*

Conference on Innovative Database Research (CIDR), Amsterdam, the Netherlands.

刘春伟、马修·鲁索、迈克尔·卡法雷拉等。2025 年。“Palimpzest：利用声明式查询处理优化人工智能驱动的分析技术。”发表于《创新数据库研究会议》论文集，荷兰阿姆斯特丹。

Liu, Michael Xieyang, Frederick Liu, Alexander J. Fiannaca, et al. 2024. "'We Need Structured Output': Towards User-Centered Constraints on Large Language Model Output." *In Extended Abstracts of the CHI Conference on Human Factors in Computing Systems*. 1–9.

刘、迈克尔·谢阳、弗雷德里克·刘、亚历山大·J·菲安纳卡等。2024 年。“我们需要结构化的输出：针对大型语言模型输出的用户中心约束。”载于《计算机系统中人类因素会议扩展摘要集》。1–9 页。

Liu, Nelson F., Kevin Lin, John Hewitt, et al. 2024. "Lost in the Middle: How Language Models Use Long Contexts." *Transactions of the Association for Computational Linguistics*. 12.

刘、纳尔逊·F.、林凯文、约翰·休伊特等。2024 年。《计算语言学协会会刊》。12 卷。

Liu, Shu, Asim Biswal, Audrey Cheng, et al. 2024. "Optimizing LLM Queries in Relational Workloads." arXiv:2403.05821.

刘 Shu、Asim Biswal、Audrey Cheng 等。2024 年。“在关系型负载中优化大语言模型的查询操作”。arXiv:2403.05821。

Min, Sewon, Xinxi Lyu, Ari Holtzman, et al. 2022. "Rethinking the Role of Demonstrations: What Makes In-Context Learning Work?" In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, Abu Dhabi, United Arab Emirates. 11048–11064.

Min, Sewon, Xinxi Lyu, Ari Holtzman 等人。2022 年。“重新思考示范的作用：是什么让上下文学习能够发挥作用？”发表于《2022 年自然语言处理实证方法会议论文集》，阿布扎比，阿拉伯联合酋长国。11048–11064 页。

Mitra, Arindam, Luciano Del Corro, Guoqing Zheng, et al. 2024. "AgentInstruct: Toward Generative Teaching with Agentic Flows." arXiv:2407.03502. *https://arxiv.org/abs/2407.03502*.

米特拉、阿林丹姆、卢西亚诺·德尔·科罗、郑国清等。2024 年。“AgentInstruct：通过智能体流实现生成式教学。”arXiv:2407.03502。https://arxiv.org/abs/2407.03502

Mittelbach, Frank, and Michel Goossens. 2004. *The LaTeX Companion*. Addison-Wesley.

米特尔巴赫、弗兰克和米歇尔·戈森斯。2004 年。《LaTeX 伴侣手册》。艾迪森-韦斯利出版社。

Moffatt v. Air Canada. 2024. 2024 BCCRT 149 (British Columbia Civil Resolution Tribunal). *https://www.canlii.org/en/bc/bccrt/doc/2024/2024bccrt149/2024bccrt149.html*.

Moffatt 诉 Air Canada. 2024. 2024 BCCRT 149 (不列颠哥伦比亚民事解决法庭). https://www.canlii.org/en/bc/bccrt/doc/2024/2024bccrt149/2024bccrt149.html.

Morse, Janice M. 1995. "The Significance of Saturation." *Qualitative Health Research* 5: 147–149.

Morse, Janice M. 1995. "饱和的重要性." 定性健康研究 5: 147–149.

Nagaonkar, Sankalp, Augustya Sharma, Ashish Choithani, and Ashutosh Trivedi. 2025. "Benchmarking Vision-Language Models on Optical Character Recognition in Dynamic Video Environments." arXiv:2502.06445.

纳加昂卡尔、桑卡尔普、奥塞亚·夏尔马、阿希什·乔蒂亚尼和阿舒托什·特里维迪。2025 年。"在动态视频环境中对视觉语言模型进行基准测试，基于光学字符识别技术。" arXiv:2502.06445。

Nakkiran, Preetum, Arwen Bradley, Hattie Zhou, and Madhu Advani. 2025. "Step-by-Step Diffusion: An Elementary Tutorial." Foundations and Trends in Computer Graphics and Vision 17. *https://doi.org/10.1561/0600000114*.

Nakkiran、Preetum、Arwen Bradley、Hattie Zhou 和 Madhu Advani。2025 年。"逐步扩散：一个基础教程。"计算机图形学与视觉领域的基础与趋势杂志，第 17 期。https://doi.org/10.1561/0600000114

Ni, Jinjie, Fuzhao Xue, Xiang Yue, et al. 2024. "MixEval: Deriving Wisdom of the Crowd from LLM Benchmark Mixtures." In *The Thirty-Eighth Annual Conference on Neural Information Processing Systems*. *https://openreview.net/forum?id=6A29LUZhfv*.

Ni, Jinjie, Fuzhao Xue, Xiang Yue 等。2024 年。"MixEval：从 LLM 基准混合模型中提取群体智慧的方法。"发表于《第三十八届神经信息处理系统年会》。https://openreview.net/forum?id=6A29LUZhfv

Nielsen, Jakob. 1994a. "Enhancing the Explanatory Power of Usability Heuristics." In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*. 152–158.

尼尔森，雅各布。1994a。"提升可用性启发式算法的解释能力。"载于《SIGCHI 计算机系统中人类因素会议论文集》，第 152–158 页。

Nielsen, Jakob. 1994b. *Usability Engineering*. Morgan Kaufmann.

尼尔森，雅各布。1994b。《可用性工程》。摩根考夫曼出版社。

Norman, Donald A. 1988. *The Design of Everyday Things*. Basic Books.

诺曼，唐纳德·A. 1988.《日常事物的设计》。基础图书出版社。

Norman, Donald A. 2013. *The Design of Everyday Things: Revised and Expanded Edition*. Basic Books.

诺曼，唐纳德·A. 2013.《日常事物的设计：修订扩展版》。基础图书出版社。

Nye, Maxwell, Anders Johan Andreassen, Guy Gur-Ari, et al. 2021. "Show Your Work: Scratchpads for Intermediate Computation with Language Models." arXiv:2112.00114.

尼耶、马克斯韦尔、安德斯·约翰·安德雷亚森、盖伊·古尔-阿里等，2021 年。"展示你的工作：利用语言模型进行中级计算操作。"arXiv:2112.00114。

Ong, Isaac, Amjad Almahairi, Vincent Wu, et al. 2024. "RouteLLM: Learning to Route LLMs with Preference Data." In *The Thirteenth International Conference on Learning Representations*. *https://arxiv.org/abs/2406.18665*.

Ong, Isaac, Amjad Almahairi, Vincent Wu 等。2024 年。"RouteLLM：利用偏好数据学习为大型语言模型制定路径策略。"发表于《第十三届国际表示学习会议》。https://arxiv.org/abs/2406.18665

OpenAI. 2025. "Memory and New Controls for ChatGPT." Web Archive snapshot from 2025-04-27; accessed 2025-05-03. *https://web.archive.org/web/20250427165326/https://openai.com/index/memory-and-new-controls-for-chatgpt*.

OpenAI. 2025 年。"ChatGPT 的记忆功能与新的控制方式。"该网页存档的快照日期为 2025 年 4 月 27 日；访问时间截至 2025 年 5 月 3 日。https://web.archive.org/web/20250427165326/https://openai.com/index/memory-and-new-controls-for-chatgpt

Opsahl-Ong, Krista, Michael J. Ryan, Josh Purtell, et al. 2024. "Optimizing Instructions and Demonstrations for Multi-Stage Language Model Programs." In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. 9340–9366. *https://aclanthology.org/2024.emnlp-main.525*.

Opsahl-Ong、Krista、Michael J. Ryan、Josh Purtell 等。2024 年。"为多阶段语言模型程序优化指令与演示方式"。载于《2024 年自然语言处理实证方法会议论文集》。第 9340–9366 页。https://aclanthology.org/2024.emnlp-main.525

Ouyang, Long, Jeffrey Wu, Xu Jiang, et al. 2022. "Training Language Models to Follow Instructions with Human Feedback." *Advances in Neural Information Processing*

Systems 35: 27730–27744.

欧阳、龙、Jeffrey Wu、徐江等。2022 年。“利用人类反馈训练语言模型以遵循指令”。《神经信息处理系统进展》第 35 卷：27730–27744 页。

Packer, Charles, Vivian Fang, Shishir G. Patil, Kevin Lin, Sarah Wooders, and Joseph E. Gonzalez. 2023. “MemGPT: Towards LLMs as Operating Systems.” arXiv:2310.08560.

帕克、查尔斯、方维安、吉里·帕蒂尔、凯文·林、莎拉·伍德尔斯和约瑟夫·E·冈萨雷斯。2023 年。“MemGPT：迈向将大语言模型作为操作系统的发展。”arXiv:2310.08560。

Papineni, Kishore, Salim Roukos, Todd Ward, and Wei-Jing Zhu. 2002. “BLEU: A Method for Automatic Evaluation of Machine Translation.” In *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics*, Philadelphia, PA. 311–318.

帕皮内尼、基肖尔、萨利姆·鲁科斯、托德·沃德和朱伟静。2002 年。“BLEU：一种自动评估机器翻译的方法。”载于《第 40 届计算语言学协会年会论文集》，费城，宾夕法尼亚州。第 311-318 页。

Parcalabescu, Letitia, Michele Cafagna, Lilitta Muradjan, Anette Frank, Iacer Calixto, and Albert Gatt. 2022. “VALSE: A Task-Independent Benchmark for Vision and Language Models Centered on Linguistic Phenomena.” In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Dublin, Ireland. 8253–8280.

Parcalabescu, Letitia, Michele Cafagna, Lilitta Muradjan, Anette Frank, Iacer Calixto 和 Albert Gatt。2022 年。“VALSE：一个基于语言现象的视觉与语言模型任务独立基准测试。”载于《第 60 届计算语言学协会年会论文集》（第 1 卷：长文），爱尔兰都柏林。8253–8280 页。

Parameswaran, Aditya G., Shreya Shankar, Parth Asawa, Naman Jain, and Yujie Wang. 2024. “Revisiting Prompt Engineering via Declarative Crowdsourcing.” In *14th Conference on Innovative Data Systems Research*, CIDR 2024, Chaminade, HI, USA, January 14–17, 2024. *https://www.cidrdb.org/cidr2024/papers/p67-parameswaran.pdf*.

帕拉梅斯瓦兰、阿迪蒂亚·G.、施雷亚·尚卡尔、帕尔思·阿萨瓦、纳曼·贾恩和王宇杰。2024 年。“通过声明式众包重新思考提示生成方法。”发表于第 14 届创新数据系统研究会议，CIDR 2024，美国夏威夷州查米纳德，2024 年 1 月 14 日至 17 日。https://www.cidrdb.org/cidr2024/papers/p67-parameswaran.pdf

Patil, Shishir G., Tianjun Zhang, Vivian Fang, et al. 2024a. “GoEx: Perspectives and Designs Towards a Runtime for Autonomous LLM Applications.” arXiv:2404.06921.

Patil, Shishir G., Tianjun Zhang, Vivian Fang 等。2024 年。“GoEx：构建自主大语言模型应用的运行时环境的前景与设计”。arXiv:2404.06921。

Patil, Shishir G., Tianjun Zhang, Xin Wang, and Joseph E. Gonzalez. 2024b. “Gorilla: Large Language Model Connected with Massive APIs.” *Advances in Neural Information Processing Systems* 37: 126544–126565.

帕蒂尔、希希尔·G.张、张天俊、王欣和约瑟夫·E.冈萨雷斯。2024 年。“大猩猩：与大规模应用程序接口相连的大型语言模型”。神经信息处理系统进展杂志，第 37 卷：126544-126565 页。

Pedro, Rodrigo, Daniel Castro, Paulo Carreira, and Nuno Santos. 2023. “From Prompt Injections to SQL Injection Attacks: How Protected Is Your LLM-Integrated Web Application?” arXiv:2308.01990.

Polyzotis, Neoklis, Martin Zinkevich, Sudip Roy, Eric Breck, and Steven Whang. 2019. “Data Validation for Machine Learning.” *Proceedings of Machine Learning and Systems* 1: 334–347.

Qian, Jing, Hong Wang, Zekun Li, Shiyang Li, and Xifeng Yan. 2022. “Limitations of Language Models in Arithmetic and Symbolic Induction.” arXiv:2208.05051.

Rafailov, Rafael, Archit Sharma, Eric Mitchell, Christopher D. Manning, Stefano Ermon, and Chelsea Finn. 2023. “Direct Preference Optimization: Your Language Model Is Secretly a Reward Model.” *Advances in Neural Information Processing Systems* 36: 53728–53741.

Reimers, Nils, and Iryna Gurevych. 2019. “Sentence-BERT: Sentence Embeddings Using Siamese BERT-Networks.” In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, Hong Kong. 3982–3992.

Rein, David, Betty Li Hou, Asa Cooper Stickland, et al. 2024. “GPQA: A Graduate-Level Google-Proof Q&A Benchmark.” In *First Conference on Language Modeling*, Philadelphia, PA.

Robertson, Stephen, and Hugo Zaragoza. 2009. “The Probabilistic Relevance Framework: BM25 and Beyond.” *Foundations and Trends in Information Retrieval* 3: 333–389.

罗伯逊、斯蒂芬和乌戈·萨拉戈萨。2009 年。“概率相关框架：BM25 及其扩展。”《信息检索的基础与趋势》第 3 卷，第 333-389 页。

Rogan, Walter J., and Beth Gladen. 1978. “Estimating Prevalence from the Results of a Screening Test.” *American Journal of Epidemiology* 107: 71–76.

罗根、沃尔特·J.和贝丝·格莱登。1978 年。“从筛查测试的结果中估计患病率”。《美国流行病学杂志》第 107 卷，第 71-76 页。

Saad-Falcon, Jon, Omar Khattab, Christopher Potts, and Matei Zaharia. 2024. "'ARES': An Automated Evaluation Framework for Retrieval-Augmented Generation Systems." In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies* (Volume 1: Long Papers). 338–354. *https://aclanthology.org/2024.naacl-long.20*.

Saad-Falcon、Jon、Omar Khattab、Christopher Potts 和 Matei Zaharia。2024 年。"ARES：一种用于检索增强生成系统的自动化评估框架"。载于 2024 年北美计算语言学协会人类语言技术分会会议论文集（第 1 卷：长文）。338–354 页。https://aclanthology.org/2024.naacl-long.20.

Sachs, Sarah. 2024. "How Notion Evaluates AI at Scale Across 70 Engineers." *Braintrust* (blog). *https://www.braintrust.dev/blog/notion*.

萨克斯，莎拉。2024 年。"How Notion 评估 70 名工程师在大规模应用人工智能方面的表现。"Braintrust 博客。https://www.braintrust.dev/blog/notion

Satyanarayan, Arvind, Dominik Moritz, Kanit Wongsuphasawat, and Jeffrey Heer. 2016. "Vega-Lite: A Grammar of Interactive Graphics." *IEEE Transactions on Visualization and Computer Graphics* 23: 341–350.

萨塔纳拉亚纳、阿尔文德、多米尼克·莫里茨、卡尼特·翁苏帕萨瓦特和杰弗里·希尔。2016 年。"Vega-Lite：一种交互式图形的语法规范"。《IEEE 可视化与计算机图形学汇刊》第 23 卷，第 341-350 页。

Schick, Timo, Jane Dwivedi-Yu, Roberto Dessì, et al. 2023. "Toolformer: Language Models Can Teach Themselves to Use Tools." *Advances in Neural Information Processing Systems* 36: 68539–68551.

Schick, Timo, Jane Dwivedi-Yu, Roberto Dessì等。2023 年。"Toolformer：语言模型能够自我学习如何使用工具。"《神经信息处理系统进展》杂志，第 36 卷，第 68539-68551 页。

Sclar, Melanie, Yejin Choi, Yulia Tsvetkov, and Alane Suhr. 2024. "Quantifying Language Models' Sensitivity to Spurious Features in Prompt Design or: How I Learned to Start Worrying About Prompt Formatting." In *The Twelfth International Conference on Learning Representations*. *https://openreview.net/forum?id=RIu5lyNXjT*.

Sclar, Melanie, Yejin Choi, Yulia Tsvetkov, 和 Alane Suhr。2024 年。"量化语言模型对提示设计中虚假特征的敏感性：我是如何开始担心提示格式问题的。"发表于《第十二届国际学习表示会议》。https://openreview.net/forum?id=RIu5lyNXjT

Sculley, David, Gary Holt, Daniel Golovin, et al. 2015. "Hidden Technical Debt in Machine Learning Systems." *Advances in Neural Information Processing Systems* 28.

Sculley, David, Gary Holt, Daniel Golovin 等。2015 年。《机器学习系统中的隐藏技术缺陷》。神经信息处理系统进展杂志，第 28 卷。

Settles, Burr. 2009. “Active Learning Literature Survey.” University of Wisconsin-Madison Department of Computer Sciences.

塞特尔，伯尔。2009 年。《主动学习相关文献综述》。威斯康星大学麦迪逊分校计算机科学系。

Shankar, Shreya, Tristan Chambers, Tarak Shah, Aditya G. Parameswaran, and Eugene Wu. 2024a. “DocETL: Agentic Query Rewriting and Evaluation for Complex Document Processing.” arXiv:2410.12189.

尚卡尔、施雷亚、特里斯坦·钱伯斯、塔拉克·沙阿、阿迪蒂亚·G·帕拉梅斯瓦兰和尤金·吴。2024 年。“DocETL：用于复杂文档处理的代理式查询重写与评估方法”。arXiv:2410.12189。

Shankar, Shreya, Haotian Li, Parth Asawa, et al. 2024b. “SPADE: Synthesizing Data Quality Assertions for Large Language Model Pipelines.” *Proceedings of the VLDB Endowment* 17: 4173–4186.

Shankar、Shreya、Haotian Li、Parth Asawa 等学者在 2024 年发表的研究论文中提出了 SPADE 算法。该论文题为“SPADE：为大型语言模型管道整合数据质量断言”，发表于《VLDB 年鉴》第 17 期，页码为 4173 至 4186 页。

Shankar, Shreya, Rolando Garcia, Joseph M Hellerstein, and Aditya G. Parameswaran. 2024c. “‘We Have No Idea How Models Will Behave in Production Until Production’: How Engineers Operationalize Machine Learning.” *Proceedings of the ACM on Human-Computer Interaction* 8: 1–34.

Shankar、Shreya、Rolando Garcia、Joseph M Hellerstein 和 Aditya G. Parameswaran。2024c。“在实际生产之前，我们根本无法预知模型会在实际运行中如何表现：工程师们是如何将机器学习技术应用于实际工作的。”《ACM 人机交互会议论文集》第 8 卷，第 1-34 页。

Shankar, Shreya, J. D. Zamfirescu-Pereira, Björn Hartmann, Aditya Parameswaran, and Ian Arawjo. 2024d. “Who Validates the Validators? Aligning LLM-Assisted Evaluation of LLM Outputs with Human Preferences.” In *Proceedings of the 37th Annual ACM Symposium on User Interface Software and Technology*, Pittsburgh, PA. 1–14.

尚卡尔、施雷亚、J.D.扎梅菲尔斯库-佩雷拉、比约恩·哈特曼、阿迪蒂亚·帕拉梅斯瓦兰和伊恩·阿拉乔。2024 年。“谁来验证验证者？将大语言模型辅助的输出评估与人类偏好相结合。”发表于第 37 届 ACM 用户界面软件与技术研讨会论文集，匹兹堡，宾夕法尼亚州。第 1-14 页。

Shankar, Shreya, Bhavya Chopra, Mawil Hasan, et al. 2025. “Steering Semantic Data Processing with DocWrangler.” arXiv:2504.14764.

Shankar、Shreya、Bhavya Chopra、Mawil Hasan 等，2025 年。“使用 DocWrangler 进行语义数据处理指南”。arXiv:2504.14764。

Shankar, Shreya, Sepanta Zeigham, and Aditya G. Parameswaran. 2026. “Task Cascades for Efficient Unstructured Data Processing.” *Proceedings of the ACM on Management of Data (PACMMOD)* 4, no. 1 (SIGMOD). arXiv:2601.05536.

尚卡尔、施雷亚、塞帕塔·泽格姆和阿迪蒂亚·G·帕拉梅斯瓦兰。2026 年。“用于高效处理非结构化数据的任务级并行处理方案”。载于《ACM 数据管理会议论文集》第 4 卷，第 1 期（SIGMOD）。arXiv:2601.05536。

Shneiderman, Ben. 1997. *Designing the User Interface: Strategies for Effective Human-Computer Interaction*. Addison-Wesley Longman.

施奈德曼，本。1997 年。《设计用户界面：实现有效人机交互的策略》。艾迪森-韦斯利朗曼出版社。

Strauss, Anselm and Juliet Corbin. 1990. *Basics of Qualitative Research*. Sage.

斯特劳斯、安塞尔姆和朱丽叶·科尔宾。1990 年。《定性研究基础》。萨奇出版社。

Sun, Yuchong, Che Liu, Kun Zhou, et al. 2024. “Parrot: Enhancing Multi-Turn Instruction Following for Large Language Models.” *In Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 9729–9750. *https://aclanthology.org/2024.acl-long.525*.

孙宇聪、车刘、周坤等。2024 年。“Parrot：提升大型语言模型的多轮指令遵循能力”。载于《第 62 届计算语言学协会年会论文集》（第 1 卷：长文）。9729–9750 页。https://aclanthology.org/2024.acl-long.525.

Tay, Yi, Mostafa Dehghani, Samira Abnar, et al. 2021. “Long Range Arena: A Benchmark for Efficient Transformers.” In *International Conference on Learning Representations*. *https://openreview.net/forum?id=qVyeW-grC2k*.

泰、易、莫斯塔法·德赫加尼、萨米拉·阿布纳等，2021 年。“长距离竞技场：高效变换器的基准测试”。发表于《国际学习表示会议》论文集。https://openreview.net/forum?id=qVyeW-grC2k

Tian, Katherine, Eric Mitchell, Allan Zhou, et al. 2023. “Just Ask for Calibration: Strategies for Eliciting Calibrated Confidence Scores from Language Models Fine-Tuned with Human Feedback.” In *Proceedings of the 2023 Conference on Empirical*

Methods in Natural Language Processing. *https://arxiv.org/abs/2305.14975*.

田、凯瑟琳、埃里克·米切尔、周安等。2023 年。“只需请求校准：利用经过人类反馈优化的语言模型获得校准后的置信度评分的策略”。载于《2023 年自然语言处理实证方法会议论文集》。https://arxiv.org/abs/2305.14975

Touvron, Hugo, Louis Martin, Kevin Stone, et al. 2023. “Llama 2: Open Foundation and Fine-Tuned Chat Models.” arXiv:2307.09288.

Touvron, Hugo, Louis Martin, Kevin Stone 等。2023 年。“Llama 2：开放基础框架与微调聊天模型”。arXiv:2307.09288。

Tufte, Edward R. 1990. *Envisioning Information*. Graphics Press.

塔夫特，爱德华·R. 1990。《信息的呈现方式》。图形出版社。

Tufte, Edward R. 1997. *Visual Explanations*. Graphics Press.

塔夫特，爱德华·R. 1997.《视觉说明》。图形出版社。

Tufte, Edward R. 2001. *The Visual Display of Quantitative Information*. Graphics Press.

Tufte, Edward R. 2001.《定量信息的视觉呈现》。Graphics Press 出版社。

Tufte, Edward R. 2006. *Beautiful Evidence*. Graphics Press.

塔夫特，爱德华·R. 2006.《美丽的证据》。图形出版社。

Uber Engineering. 2024. “QueryGPT—Natural Language to SQL Using Generative AI.” *Uber Engineering* (blog). *https://www.uber.com/blog/query-gpt*.

Uber 工程部门。2024 年。“QueryGPT——利用生成式人工智能将自然语言转换为 SQL 语句。” Uber 工程部门博客。https://www.uber.com/blog/query-gpt

Umeki, Hideo. 2008. “The geometry Package.” *http://ctan.org/pkg/geometry*.

梅木，秀雄。2008 年。“几何包”。http://ctan.org/pkg/geometry

Vaswani, Ashish, Noam Shazeer, Niki Parmar, et al. 2017. “Attention Is All You Need.” *Advances in Neural Information Processing Systems* 30.

Vaswani, Ashish, Noam Shazeer, Niki Parmar 等。2017 年。“注意力就是你所需要的全部。”《神经信息处理系统进展》第 30 卷。

Vir, Reya, Shreya Shankar, Harrison Chase, William Hinthorn, and Aditya Parameswaran. 2025. “‘PROMPTEVALS’: A Dataset of Assertions and Guardrails for Custom Production Large Language Model Pipelines.” In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies* (Volume 1: Long Papers).

4225–4245. *https://aclanthology.org/2025.naacl-long.213*.

Vir、Reya、Shreya Shankar、Harrison Chase、William Hinthorn 和 Aditya Parameswaran。2025 年。“PROMPTEVALS：一个用于自定义生产大型语言模型管道的断言与约束规则数据集。”载于 2025 年美洲国家分会与计算语言学协会人类语言技术研讨会论文集第 1 卷：长文部分。4225–4245 页。https://aclanthology.org/2025.naacl-long.213

Voiceflow. 2023. “How Much Do ChatGPT Versions Affect Real World Performance?” *https://www.voiceflow.com/pathways/how-much-do-chatgpt-versions-affect-real-world-performance*.

Voiceflow. 2023 年。“ChatGPT 的版本对实际性能产生多大影响？” https://www.voiceflow.com/pathways/how-much-do-chatgpt-versions-affect-real-world-performance

Ward, Emma, and Steven Feldstein. 2024. “Evaluating Large Language Models.” *Center for Security and Emerging Technology*. *https://cset.georgetown.edu/article/evaluating-large-language-models*.

沃德、艾玛和史蒂文·费尔德斯坦。2024 年。“评估大型语言模型”。安全与新兴技术中心。https://cset.georgetown.edu/article/evaluating-large-language-models

Webber, Emily, and Andrea Olgiati. 2023. *Pretrain Vision and Large Language Models in Python: End-to-End Techniques for Building and Deploying Foundation Models on AWS*. Packt Publishing.

韦伯、艾米丽和安德烈亚·奥尔吉亚蒂。2023 年。《用 Python 预训练视觉模型和大型语言模型：在 AWS 上构建和部署基础模型的方法》。Packt 出版公司出版。

Wei, Jason, Xuezhi Wang, Dale Schuurmans, et al. 2022. “Chain-of-Thought Prompting Elicits Reasoning in Large Language Models.” *Advances in Neural Information Processing Systems* 35: 24824–24837.

魏杰森、王学智、Dale Schuurmans 等。2022 年。《思维链提示在大型语言模型中的推理能力提升》。神经信息处理系统进展杂志，第 35 卷，第 24824 至 24837 页。

Weights & Biases. 2025. “Weave: A Toolkit for Developing Generative AI Applications.” Accessed 2025-05-04. *https://wandb.ai/site/weave*.

权重与平衡计划：2025 年。“Weave：用于开发生成式人工智能应用的工具包。”访问日期：2025 年 5 月 4 日。https://wandb.ai/site/weave

Weiss, Gail, Yoav Goldberg, and Eran Yahav. 2021. “Thinking Like Transformers.” In *International Conference on Machine Learning*. 11080–11090.

魏斯、盖尔、约阿夫·戈德堡和埃兰·亚哈夫。2021 年。“像变形金刚一样思考”。载于《国际机器学习会议论文集》。第 11080–11090 页。

Yan, Ziyou. 2021. “System Design for Recommendations and Search.” *https://eugeneyan.com/writing/system-design-for-discovery*.

严子尧. 2021. “推荐与搜索系统的设计”. https://eugeneyan.com/writing/system-design-for-discovery

Yan, Ziyou. 2023. “Evaluation & Hallucination Detection for Abstractive Summaries.” *https://eugeneyan.com/writing/abstractive*.

严子尧. 2023. “抽象性摘要的评估与幻觉检测”. https://eugeneyan.com/writing/abstractive.

Yan, Ziyou. 2024. “Evaluating the Effectiveness of LLM-Evaluators (aka LLM-as-Judge).” *https://eugeneyan.com/writing/llm-evaluators*.

严子尧. 2024. “评估大语言模型评估器的有效性（即作为评估者的大语言模型）”. https://eugeneyan.com/writing/llm-evaluators

Yang, Chengrun, Xuezhi Wang, Yifeng Lu, et al. 2024. “Large Language Models as Optimizers.” In *The Twelfth International Conference on Learning Representations*, Vienna, Austria. *https://proceedings.iclr.cc/paper_files/paper/2024/hash/3339f19c5fcee3ad74502947a32be9e6-Abstract-Conference.html*.

杨成伦、王雪芝、陆一丰等。2024 年。“大型语言模型作为优化器”。发表于《第十二届国际表示学习会议》，奥地利维也纳。https://proceedings.iclr.cc/paperfiles/paper/2024/hash/3339f19c5fcee3ad74502947a32be9e6-Abstract-Conference.html

Yang, John, Kilian Lieret, Jeffrey Ma, et al. 2025. “ProgramBench: Can Language Models Rebuild Programs from Scratch?” arXiv:2605.03546. *https://arxiv.org/abs/2605.03546*.

杨约翰、基利安·莱雷特、杰弗里·马等。2025 年。“ProgramBench：语言模型能否从零开始重建程序？”arXiv:2605.03546。https://arxiv.org/abs/2605.03546

Yao, Shunyu, Jeffrey Zhao, Dian Yu, et al. 2023. “ReAct: Synergizing Reasoning and Acting in Language Models.” *In The Eleventh International Conference on Learning Representations*. *https://openreview.net/forum?id=WE_vluYUL-X*.

姚顺宇、Jeffrey Zhao、杜彦等。2023 年。“ReAct：在语言模型中协同推理与行动。”发表于《第十一届国际学习表示会议》。https://openreview.net/forum?id=WE_vluYUL-X

Yuksekgonul, Mert, Federico Bianchi, Joseph Boen, et al. 2025. “Optimizing Generative AI by Backpropagating Language Model Feedback.” *Nature* 639: 609–616.

https://doi.org/10.1038/s41586-025-08661-4.

Yuksekgonul, Mert, Federico Bianchi, Joseph Boen 等。2025 年。“通过反向传播语言模型反馈来优化生成式人工智能”。《自然》杂志，第 639 期，609-616 页。https://doi.org/10.1038/s41586-025-08661-4

Zaharia, Matei, Omar Khattab, Lingjiao Chen, et al. 2024. “The Shift from Models to Compound AI Systems.” *https://bair.berkeley.edu/blog/2024/02/18/compound-ai-systems*.

扎哈里亚、马泰、奥马尔·哈特巴特、陈凌娇等。2024 年。“从模型到复合人工智能系统的转变”。https://bair.berkeley.edu/blog/2024/02/18/compound-ai-systems

Zheng, Lianmin, Wei-Lin Chiang, Ying Sheng, et al. 2023. “Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena.” *Advances in Neural Information Processing Systems* 36: 46595–46623.

郑连敏、魏林·江、盛颖等。2023 年。“使用 MT-Bench 和聊天机器人竞技场来评估大语言模型作为裁判的能力”。神经信息处理系统进展杂志，第 36 卷：46595–46623 页。

Zhou, Hattie, Arwen Bradley, Etai Littwin, et al. 2024. “What Algorithms Can Transformers Learn? A Study in Length Generalization.” In *The Twelfth International Conference on Learning Representations*. *https://openreview.net/forum?id=AssIuHnmHX*.

周、哈蒂、阿文·布拉德利、埃泰·利特温等。2024 年。“哪些算法可以被 Transformer 学习到？关于长度泛化的研究。”发表于《第十二届国际表示学习会议》。https://openreview.net/forum?id=AssIuHnmHX

Zhou, Yiyang, Chenhang Cui, Jaehong Yoon, et al. 2024. “Analyzing and Mitigating Object Hallucination in Large Vision-Language Models.” In *The Twelfth International Conference on Learning Representations*. *https://openreview.net/forum?id=oZDJKTlOUe*.

周一阳、陈航、崔杰宏、尹杰宏等。2024 年。“分析与缓解大型视觉语言模型中的物体幻觉问题”。发表于《第十二届国际表示学习会议》。https://openreview.net/forum?id=oZDJKTlOUe

Zhu, Yaoming, Sidi Lu, Lei Zheng, et al. 2018. “Texygen: A Benchmarking Platform for Text Generation Models.” In *The 41st International ACM SIGIR Conference on Research & Development in Information Retrieval*, *Ann Arbor, MI*. 1097–1100.

朱耀明、Sidi Lu、雷正等。2018 年。“Texygen：文本生成模型的基准测试平台”。发表于第 41 届国际 ACM SIGIR 信息检索研究与发展会议，密歇根州安阿伯市。1097–1100 页。

Index

A

absolute evaluation
- definition of, LLMs and Evaluation Basics
- reference-based versus reference-free, Absolute Evaluation, Defining the Right Metrics (What to Measure)

accuracy estimates, Data Splits for Designing and Validating LLM-as-Judge

accuracy optimization
- changing how agents work, Medium Effort: Changing How the Agent Works
- model-level changes, High Effort: Model-Level Changes
- prompt refinement, Low Effort: Prompt Refinement

active learning, Uncertainty Sampling

adversarial evaluation, Adversarial and Security Evaluation

agency, The Spectrum of Agency

agent loops, Agent Loops

agentic RAG, Overview

agentic systems, Error Analysis

agents, definition of, Tools and Agents

AI coding assistants, The Gulf of Generalization (Between the Data and the LLM Agent), Using AI Coding Assistants

Air Canada example, Introduction

alignment sessions
- datasets and documentation, Save Everything
- escalation, Escalation
- preparing for, Preparing for the Session
- purpose of, Facilitating Alignment Sessions
- rubric improvement, Techniques for Improving the Rubric
- running, Running the Session
- things to avoid, What to Avoid

ambiguity, Tools and Agents

Analyze phase
- about, Analyze, Error Analysis

- collaborative evaluation, Collaborative Evaluation Practices-Putting It Together
- end results of, Putting It Together
- error analysis, Error Analysis-Putting It Together

Analyze-Measure-Improve lifecycle
- applying, Putting It Together
- benefits of, What This Book Covers
- bridging the gulfs with, The LLM Evaluation Lifecycle: Bridging the Gulfs with Evaluation-Improve
- overview of, Closing: The Lifecycle Continues

anchoring failures, Conversational Coherence and Memory

answer correctness, Common Pitfalls

application stage, Application

application-centric evaluation, What This Book Doesn't Cover

argument generation, Stage 2: Argument Generation

Arize Phoenix, Observability, The Case for Custom Review Interfaces

assistant role, Components of LLM Agents

automated evaluators (see also evaluation)
- common pitfalls, Common Pitfalls
- data splits for design and validation, Data Splits for Designing and Validating LLM-as-Judge
- estimating true success rates, Estimating True Success Rates with Imperfect Judges-Python Code for Estimating Success Rates
- group-wise metrics for multiple outputs, Optional: Group-wise Metrics for Evaluating Multiple Outputs
- how to measure, Implementing Metrics (How to Measure)
- implementing, Implementing Automated Evaluators
- iterative prompt refinement for, Iterative Prompt Refinement for the LLM-as-Judge-If Alignment Stalls
- for multi-turn conversations, Automated Evaluation of Multi-Turn Traces
- prompts for, Writing LLM-as-Judge Prompts-Data Splits for Designing and Validating LLM-as-Judge
- Python code for success rates, Python Code for Estimating Success Rates-Optional: Group-wise Metrics for Evaluating Multiple Outputs
- running in production, Running Automated Evaluators in Production
- using without manual evaluation, Common Pitfalls
- what to measure, Defining the Right Metrics (What to Measure)-Implementing Metrics (How to Measure)
- workflow for, Putting It Together

autonomy, Components of LLM Agents, Tools and Agents, The Spectrum of Agency
axial coding, Open Coding: Reading and Labeling Traces, Axial Coding: Structuring Failure Modes

B

batching, Batching
benchmarks, Post-training, Comparative Evaluation, Writing LLM-as-Judge Prompts
benevolent dictator model, When a Single Expert Is Enough
bias, Writing LLM-as-Judge Prompts
bimodal metrics, Beyond Clustering
binary pass/fail judgements
- assigning to each trace, Open Coding: Reading and Labeling Traces
- benefits of, Common Pitfalls
- confidence scores and, Approach 1: Logprobs for classification tasks
- for feedback collection, Essential Interface Elements, Case Study: EvalGen
- LLM judges and, Defining the Right Metrics (What to Measure), Writing LLM-as-Judge Prompts
- model cascades and, Model Cascades
- percent agreement and, Cohen's Kappa: Adjusting for Chance
- recommendations for, Ordinal ratings, Filtering Synthetic Questions
- relevance and, Precision@k and Recall@k

BM25 algorithm, Retrieval, Retrieval Fundamentals, Overview
bootstrap confidence intervals, Putting It Together
box plots, Beyond Clustering
Braintrust, The Case for Custom Review Interfaces
brittleness scores, Perturbation Testing

C

caching, Make Prompts Cache-Friendly
calibration (see verbalized confidence calibrations)
cascades, Model Cascades-Tuning the cascade threshold
case studies (see also examples)
- DocWrangler interface, Case Study: DocWrangler for Prompt Refinement
- EvalGen interface, Case Study: EvalGen

chain-of-thought (CoT) prompting, Low Effort: Prompt Refinement
ChainForge, Case Study: EvalGen

challenges, of evaluation, Preface, Introduction, Evaluation in Applications, Why LLM Agent Evaluation Is Challenging (see also common pitfalls)
chapter overviews, How This Book Is Organized
chunks
- contextual augmentation, Other Chunking Approaches
- evaluating and optimizing chunking strategies, Evaluating and Optimizing Chunking Strategies
- finding largest size for reliable output, Long Documents
- semantic or structural chunking, Other Chunking Approaches
- synthetically generating query-answer pairs, Synthetically Generating Query-Answer Pairs
- tuning chunk size and overlap, Tuning Chunk Size and Overlap
- variables introduced by, Common Pitfalls
CI/CD (see continuous integration and deployment)
classification tasks, Model Cascades
clustering, Clustering, EDA and Clustering to Surface Hypotheses, Common Pitfalls
code generation assistants, The Gulf of Generalization (Between the Data and the LLM Agent), Using AI Coding Assistants
code-based evaluators, Implementing Metrics (How to Measure)
coding, open and axial, Open Coding: Reading and Labeling Traces, Axial Coding: Structuring Failure Modes, Case Study: DocWrangler for Prompt Refinement
Cohen's Kappa, Cohen’s Kappa: Adjusting for Chance-Missing data
coherence failures, Conversational Coherence and Memory-Conversational Coherence and Memory
ColBERT model, Overview
collaborative evaluation
- alignment sessions, Facilitating Alignment Sessions
- common pitfalls, Common Pitfalls
- defined, Collaborative Evaluation Practices
- end results of, Putting It Together
- inter-annotator agreement (IAA), Measuring Inter-Annotator Agreement-Missing data
- interfaces for, Case Study: EvalGen
- versus single experts, When a Single Expert Is Enough
- workflow for, A Collaborative Annotation Workflow-A Collaborative Annotation Workflow
column-prune agent, Why Evaluate Each Stage Separately
common pitfalls

  - automated evaluators, Common Pitfalls
  - collaborative evaluation, Common Pitfalls
  - continuous integration (CI), Common Mistakes with CI
  - continuous integration and deployment, Common Pitfalls
  - data analysis for traces, Common Pitfalls
  - error analysis, Common Pitfalls
  - human review interfaces, Common Pitfalls
  - multi-turn conversations, Common Pitfalls
  - retrieval-augmented generation (RAG), Common Pitfalls
  - tool use evaluation, Common Pitfalls

comparative evaluation, LLMs and Evaluation Basics, Comparative Evaluation

component-level evaluations, Traceability

components, evaluating individual, Traceability

comprehension, The Gulf of Comprehension (Between the Developer and Data)

confidence intervals, Estimating True Success Rates with Imperfect Judges-Python Code for Estimating Success Rates

confidence scores, Model Cascades, Approach 1: Logprobs for classification tasks

confidence, estimating, Model Cascades

confirmatory analysis, EDA and Clustering to Surface Hypotheses

confounders, controlling for, Common Pitfalls

constant output tasks, Long Documents

context swapping, Evaluating Generation Quality

context truncation, Conversational Coherence and Memory

context windows, Normalized Discounted Cumulative Gain (NDCG@k), Long Documents

continuous integration and deployment (CI/CD)

  - applying to LLM agents, Putting It Together
  - building safety nets against regression, CI: Building a Safety Net Against Regressions-Revisiting Prompts When Models Improve
  - challenges of, for LLM agents, Continuous Integration and Deployment for LLM Agents
  - CI flywheel, The Continuous Improvement Flywheel
  - common pitfalls, Common Pitfalls
  - key concepts for LLM agents, Continuous Integration and Deployment for LLM Agents
  - key differences from traditional ML, Continuous Integration and Deployment for LLM Agents
  - online monitoring and, CD and Online Monitoring-Judge Drift

- pipeline for LLM agents, Continuous Integration and Deployment for LLM Agents
conversation length
- failures correlating with, Common Pitfalls
- stratifying by, Stratifying by Conversation Length
conversational coherence, Conversational Coherence and Memory
conversations, Conversations
cost optimization
- batching, Batching
- caching, Make Prompts Cache-Friendly
- distillation, Distillation
- model cascades, Model Cascades-Tuning the cascade threshold
- model size matched to task complexity, Match Model Size to Task Complexity
- routing, Routing
- token usage, Cut Token Usage
CoT (chain-of-thought) prompting, Low Effort: Prompt Refinement
counting, incorrect, Images
course work, Common Pitfalls, Next Steps
criteria drift, Case Study: EvalGen
CSV format, The Case for Custom Review Interfaces
custom review interfaces, The Case for Custom Review Interfaces-Features That Improve Review Speed and Quality, A Simple Custom UI-An Advanced Review Interface, Common Pitfalls
customer support chatbots, The Gulf of Generalization (Between the Data and the LLM Agent)

D

data
- data analysis for traces, Data Analysis for Traces-Putting It Together
- data analysis versus error analysis, Putting It Together
- data leakage, Data Splits for Designing and Validating LLM-as-Judge, Common Mistakes with CI
- data splits, Data Splits for Designing and Validating LLM-as-Judge
- handling complex input data, Handling Complex Input Data-PDF Documents
- missing data, Missing data
- overfitting to synthetic evaluation data, Common Pitfalls
datasets
- and documentation for alignment sessions, Save Everything

- creating starting datasets of traces, Creating a Starting Dataset of Traces-Generating Tuples and Queries
- golden datasets, Continuous Integration and Deployment for LLM Agents-Putting It Together, Integrating Human Review into Engineering Workflows

DCG@k, Normalized Discounted Cumulative Gain (NDCG@k)
details, ignoring, Stage 4: Output Handling
development (dev) sets, Data Splits for Designing and Validating LLM-as-Judge
dimensions, defining, Defining Dimensions
direct preference optimization (DPO), Post-training, High Effort: Model-Level Changes
distillation, Distillation
diversity, Optional: Group-wise Metrics for Evaluating Multiple Outputs
document chunks (see chunks)
document-level outputs, Case Study: DocWrangler for Prompt Refinement
documents
- handling long, Long Documents
- PDF documents, PDF Documents

DocWrangler interface (for prompt refinement), Case Study: DocWrangler for Prompt Refinement
domain experts, When a Single Expert Is Enough
DoorDash example, Medium Effort: Changing How the Agent Works
DPO (see direct preference optimization)

E

EDA (exploratory data analysis), EDA and Clustering to Surface Hypotheses-Beyond Clustering
email processing example, The Gulf of Comprehension (Between the Developer and Data)
email summarizer example, Components of LLM Agents
embedding-based retrieval, Retrieval, Retrieval Fundamentals, Optional: Group-wise Metrics for Evaluating Multiple Outputs, Semantic Search to Find Related Traces
end-to-end evaluations, Traceability
error analysis (see also traces)
- about, Error Analysis-Error Analysis
- common pitfalls, Common Pitfalls
- creating a starting dataset of traces, Creating a Starting Dataset of Traces-Generating Tuples and Queries
- versus data analysis, Putting It Together

- as exploratory data analysis, EDA and Clustering to Surface Hypotheses
- human review and, Collaborative Evaluation Practices
- iteration and refinement, Iteration and Refinement
- labeling traces with structured failure modes, Labeling Traces with Structured Failure Modes
- reading and labeling traces, Open Coding: Reading and Labeling Traces-Open Coding: Reading and Labeling Traces
- structuring failure modes, Axial Coding: Structuring Failure Modes
- terminology used in, Establishing Terminology

EvalGen interface (for human evaluation of LLM outputs), Case Study: EvalGen
evaluation (see also automated evaluators)

- adversarial and security evaluation, Adversarial and Security Evaluation
- Analyze-Measure-Improve lifecycle, The LLM Evaluation Lifecycle: Bridging the Gulfs with Evaluation-Putting It Together
- application-centric, What This Book Doesn't Cover
- challenges of, Preface, Introduction, Evaluation in Applications, Why LLM Agent Evaluation Is Challenging
- definition of, What Is Evaluation?
- developing robust processes for, Error Analysis
- evaluating and optimizing chunking strategies, Evaluating and Optimizing Chunking Strategies-Other Chunking Approaches
- evaluating tool calls, Evaluating Tool Calls-Why Evaluate Each Stage Separately
- of generation quality in RAG, Evaluating Generation Quality
- of individual agent components, Traceability
- integration of, What Is Evaluation?
- memory evaluation, Conversational Coherence and Memory
- modes of, LLMs and Evaluation Basics
- of multi-turn conversations, Practical Strategies for Multi-Turn Evaluation-Stratifying by Conversation Length, Common Pitfalls
- next steps, Next Steps
- online monitoring/evaluation, CD and Online Monitoring-Judge Drift
- overrelying on evaluator disagreement, Common Pitfalls
- pre- and post-launch, Introduction
- roles of, Evaluation Throughout the Lifecycle of an LLM
- setups for, Evaluation Setups-Comparative Evaluation
- stage-by-stage evaluation, Why Evaluate Each Stage Separately
- Three Gulfs model, The Three Gulfs of LLM Application Development-Why LLM Agent Evaluation Is Challenging

- at turn level, Turn Level
- workflow for RAG, Putting It Together

examples
- Air Canada chatbot malfunction, Introduction
- code generation assistants, The Gulf of Generalization (Between the Data and the LLM Agent)
- customer support chatbots, The Gulf of Generalization (Between the Data and the LLM Agent)
- DocWrangler case study, Case Study: DocWrangler for Prompt Refinement
- DoorDash, Medium Effort: Changing How the Agent Works
- email processing, The Gulf of Comprehension (Between the Developer and Data)
- email summarizer, Components of LLM Agents
- EvalGen case study, Case Study: EvalGen
- human review interface, Example Walkthrough: Reviewing Real Estate Assistant Emails
- real estate assistant, Error Analysis, Defining the Right Metrics (What to Measure), What Are Tools and Why Do LLMs Use Them?, CI: Building a Safety Net Against Regressions, The Case for Custom Review Interfaces, Medium Effort: Changing How the Agent Works, Match Model Size to Task Complexity
- Uber, Why Evaluate Each Stage Separately

executability checks, Defining the Right Metrics (What to Measure)
execution success, Why Evaluate Each Stage Separately
exploratory data analysis (EDA), EDA and Clustering to Surface Hypotheses-Beyond Clustering

F

F1 score, Continuous Integration and Deployment for LLM Agents
failure modes
- building taxonomy of, Creating a Starting Dataset of Traces, Putting It Together
- clusters and, EDA and Clustering to Surface Hypotheses
- dedicated evaluators for, Implementing Metrics (How to Measure)
- defining the right metrics, Defining the Right Metrics (What to Measure)-Implementing Metrics (How to Measure)
- hard-to-detect, High Effort: Model-Level Changes
- initial frequency count, Putting It Together
- isolating failures in multi-turn conversations, Isolating Failures

- labeling traces with structured failure modes, Labeling Traces with Structured Failure Modes
- missing subtle, Common Pitfalls
- multi-turn versus single-turn, Common Pitfalls
- structuring, Axial Coding: Structuring Failure Modes

failure-driven sampling, Failure-Driven Sampling
faithfulness, Evaluating Generation Quality
false passes and false fails (see TPR/TNR)
feedback collection, Essential Interface Elements, Integrating Human Review into Engineering Workflows
few-shot examples
- data leakage and, Common Pitfalls, Common Mistakes with CI
- data split and, Data Splits for Designing and Validating LLM-as-Judge
- definition of term, What Is Evaluation?
- filtering synthetic questions, Filtering Synthetic Questions
- LLM-as-judge prompts, Writing LLM-as-Judge Prompts-Data Splits for Designing and Validating LLM-as-Judge
- prompt refinement and, Iterative Prompt Refinement for the LLM-as-Judge-Iterative Prompt Refinement for the LLM-as-Judge, Low Effort: Prompt Refinement
- revalidation and, Judge Drift

fine-tuning, High Effort: Model-Level Changes
first state/first failure, From Traces to Systemic Patterns
first-pass retrieval, Overview
fixed-size chunking, Evaluating and Optimizing Chunking Strategies
flaky tests, CI: Building a Safety Net Against Regressions
Fleiss' Kappa, More than two annotators
freedom, Tools and Agents
future concerns, Next Steps

G

generalization, The Gulf of Generalization (Between the Data and the LLM Agent), Putting It Together
golden datasets, Continuous Integration and Deployment for LLM Agents-Putting It Together, Integrating Human Review into Engineering Workflows
graded relevance, Normalized Discounted Cumulative Gain (NDCG@k)
Gradio, The Case for Custom Review Interfaces
grid search, Tuning Chunk Size and Overlap
ground truth, Absolute Evaluation

grounding, Common Pitfalls
guardrails, Guardrails, Medium Effort: Changing How the Agent Works
Gulf of Comprehension, The Gulf of Comprehension (Between the Developer and Data), Putting It Together
Gulf of Generalization, The Gulf of Generalization (Between the Data and the LLM Agent), Putting It Together
Gulf of Specification, The Gulf of Specification (Between the Developer and the LLM Agent), Putting It Together

H

hallucinations
- Air Canada incident, Introduction
- of objects or details not actually present, Images
- RAG evaluation, Evaluating Generation Quality
- reading and labeling traces, Open Coding: Reading and Labeling Traces, Common Pitfalls
- real estate assistant example, Axial Coding: Structuring Failure Modes

held-out sets (see test sets)
Helicone, The Case for Custom Review Interfaces
high-agency, The Spectrum of Agency
histograms, Beyond Clustering
hit rate@k, Mean Reciprocal Rank
hotkeys, Features That Improve Review Speed and Quality
hotspots, investigating, Investigating Hotspots
human review
- common pitfalls, Common Pitfalls
- custom review interfaces, The Case for Custom Review Interfaces-Features That Improve Review Speed and Quality, A Simple Custom UI-An Advanced Review Interface, Common Pitfalls
- DocWrangler case study, Case Study: DocWrangler for Prompt Refinement
- in error analysis, Collaborative Evaluation Practices
- essential nature of, Putting It Together
- hard-to-detect failure modes, High Effort: Model-Level Changes
- integrating into engineering workflows, Integrating Human Review into Engineering Workflows
- interfaces for, Interfaces for Human Review
- measuring agreement with, Missing data
- navigating trace groups and discovering patterns, Navigating Trace Groups and Discovering Patterns

  - real estate assistant walkthrough, Example Walkthrough: Reviewing Real Estate Assistant Emails
  - selecting traces for, Selecting Traces for Human Review

hybrid search/retrieval, Retrieval Fundamentals, Overview

hypotheses

  - versus conclusions, Common Pitfalls
  - confirming, From Clusters to Confirmed Hypotheses
  - generating, Clustering

I

IAA (see inter-annotator agreement)

Ideal DCG (IDCG@k), Normalized Discounted Cumulative Gain (NDCG@k)

images

  - evaluating image generation, Images
  - vision-language models and, Images

Improve phase

  - about, Improve
  - accuracy optimization, Accuracy Optimization-High Effort: Model-Level Changes
  - continuous integration and deployment (CI/CD), Continuous Integration and Deployment for LLM Agents-Putting It Together
  - cost optimization, Cost Optimization-Routing
  - data analysis for traces, Data Analysis for Traces-Putting It Together
  - human review, Interfaces for Human Review-Putting It Together

improvement strategies, selecting, Choosing an Improvement Strategy

incorrect values, Stage 2: Argument Generation

indirect injections, Adversarial and Security Evaluation

individual components, evaluating, Traceability

input data, handling complex, Handling Complex Input Data-PDF Documents

instruction drift, Conversational Coherence and Memory

intent agent, Why Evaluate Each Stage Separately

intent classifiers, Revisiting Prompts When Models Improve

inter-annotator agreement (IAA)

  - Cohen's Kappa, Cohen’s Kappa: Adjusting for Chance
  - Kappa scores interpretation, Interpreting Kappa Scores
  - missing data, Missing data
  - with more than two annotators, More than two annotators
  - ordinal ratings, Ordinal ratings
  - percent agreement, Percent Agreement: Simple but Misleading

- Python implementation of Cohen's Kappa, Python Implementation
intrinsic evaluation, Pretraining

J

JSON schema, Tools and Agents, Guardrails, Approach 2: Verbalized confidence for open-ended tasks (use with caution)
judge drift, Judge Drift
judges (see LLM-as-judge evaluators)

K

Kahneman-Tversky optimization (KTO), High Effort: Model-Level Changes
kappa (see Cohen's Kappa)
keyboard shortcuts, Features That Improve Review Speed and Quality
keyword search, Overview
keyword-based retrieval, Retrieval, Search and Similarity Tools
Krippendorff's Alpha, Missing data
KTO (Kahneman-Tversky optimization), High Effort: Model-Level Changes
KV cache, Make Prompts Cache-Friendly

L

Langfuse, The Case for Custom Review Interfaces
LangSmith, Observability
latency
- batching and, Batching
- cascade threshold and, Tuning the cascade threshold
- due to accuracy optimization, Medium Effort: Changing How the Agent Works
- in exploratory data analysis, Beyond Clustering
- reducing with cache-friendly prompts, Make Prompts Cache-Friendly
- review interfaces and, An Advanced Review Interface
- routing and, Routing
- when using LLM-as-judges for guardrails, Guardrails

leaderboards, Comparative Evaluation
Likert scales, Common Pitfalls
LiteLLM library, Components of LLM Agents
live course, Common Pitfalls, Next Steps
LLM agents
- applying CI/CD to, Putting It Together

- building blocks for, Putting It Together
- changing how agents work, Medium Effort: Changing How the Agent Works
- components of, Components of LLM Agents-Agent Loops
- definition of, Components of LLM Agents
- improving, Improving LLM Agents-Choosing an Improvement Strategy
- interacting with programmatically, Components of LLM Agents
- key differences from traditional ML, Continuous Integration and Deployment for LLM Agents
- lifecycle of an LLM, Evaluation Throughout the Lifecycle of an LLM
- versus LLM applications, Introduction
- model-level interventions, High Effort: Model-Level Changes
- pace of change, A Note on the Pace of Change
- rapid adoption of, Preface, Introduction
- Three Gulfs model, The Three Gulfs of LLM Application Development-Why LLM Agent Evaluation Is Challenging
- tool use and complex agents, Evaluating Tool Use and Complex Agents-Putting It Together

LLM-as-judge evaluators
- agreement with ground truth, Missing data
- calibrating, Integrating Human Review into Engineering Workflows
- characteristics of, Running Automated Evaluators in Production
- data splits for design and validation, Data Splits for Designing and Validating LLM-as-Judge
- estimating true success rates, Estimating True Success Rates with Imperfect Judges-Python Code for Estimating Success Rates
- iterative prompt refinement for, Iterative Prompt Refinement for the LLM-as-Judge-If Alignment Stalls
- prompts for, Writing LLM-as-Judge Prompts-Data Splits for Designing and Validating LLM-as-Judge
- Python code for success rates, Python Code for Estimating Success Rates-Optional: Group-wise Metrics for Evaluating Multiple Outputs
- subtle failures missed by, Common Pitfalls
- when to use, Implementing Metrics (How to Measure)
- workflow for, Putting It Together

location ambiguity errors, Iteration and Refinement

log probabilities (logprobs), Approach 1: Logprobs for classification tasks

logs
- exporting into spreadsheets, The Case for Custom Review Interfaces, Starting with a Spreadsheet

- of intermediate tool-use steps, Traceability
- online evaluations using, Observability
- sampling traces from production logs, Creating a Starting Dataset of Traces, Common Pitfalls

long document handling, Long Documents
LoRA (low-rank adaptation), High Effort: Model-Level Changes
lost in the middle effect, Mean Reciprocal Rank
low-agency, The Spectrum of Agency

M

MCP (see Model Context Protocol)
mean reciprocal rank (MRR), Mean Reciprocal Rank
Measure phase
- about, Measure
- automated evaluators, Implementing Automated Evaluators-Putting It Together
- multi-turn conversations, Evaluating Multi-Turn Conversations-Putting It Together
- retrieval-augmented generation, Evaluating Retrieval-Augmented Generation-Putting It Together
- tool use and complex agents, Evaluating Tool Use and Complex Agents-Putting It Together

memory evaluation, Conversational Coherence and Memory
metadata search, Search and Similarity Tools
metrics
- choosing incorrect, Common Pitfalls
- conflicting, Common Pitfalls
- defining the right metrics, Defining the Right Metrics (What to Measure)-Implementing Metrics (How to Measure)
- group-wise metrics for multiple outputs, Optional: Group-wise Metrics for Evaluating Multiple Outputs
- implementing, Implementing Metrics (How to Measure)
- per-trace metrics, Beyond Clustering
- perplexity, Pretraining
- relying solely on end-to-end, Common Pitfalls
- for retrieval quality, Metrics for Retrieval Quality-Normalized Discounted Cumulative Gain (NDCG@k)

misinterpretations, Evaluating Generation Quality, Stage 4: Output Handling
model cascades, Model Cascades-Tuning the cascade threshold

Model Context Protocol (MCP), Tools, Why Evaluate Each Stage Separately
model versions, pinning, CI: Building a Safety Net Against Regressions-Revisiting Prompts When Models Improve
MRR (mean reciprocal rank), Mean Reciprocal Rank
multi-hop questions, Mean Reciprocal Rank
multi-step reasoning, Components of LLM Agents
multi-step traces, debugging, Debugging Multi-Step Traces-Evolving the Heatmap over Time
multi-turn conversations
- about, Evaluating Multi-Turn Conversations
- automated evaluation of, Automated Evaluation of Multi-Turn Traces
- common pitfalls, Common Pitfalls
- defined, Evaluating Multi-Turn Conversations
- evaluating at different levels, Evaluating at Different Levels-Conversational Coherence and Memory
- evaluation strategies, Practical Strategies for Multi-Turn Evaluation-Perturbation Testing
- workflow for evaluating, Putting It Together

N

NDCG@k, Normalized Discounted Cumulative Gain (NDCG@k)-Normalized Discounted Cumulative Gain (NDCG@k)
needle-in-a-haystack tasks, Long Documents
next steps, Next Steps
next-token prediction, Pretraining
Nielsen, Jakob, The Case for Custom Review Interfaces
nondeterminism, Single LLM Call, Establishing Terminology
normalization, of phone numbers, Tools and Agents
Notion, Collaborative Evaluation Practices

O

observability, Observability
OCR (optical character recognition), PDF Documents
omissions, Evaluating Generation Quality
online monitoring/evaluation, CD and Online Monitoring-Judge Drift
open coding, Open Coding: Reading and Labeling Traces, Axial Coding: Structuring Failure Modes, Case Study: DocWrangler for Prompt Refinement

open-ended tasks, Approach 2: Verbalized confidence for open-ended tasks (use with caution)
OpenTelemetry, Observability
optical character recognition (OCR), PDF Documents
oracles, Model Cascades
outliers, hunting for, Beyond Clustering
outputs
  - comparing, Medium Effort: Changing How the Agent Works
  - constant output tasks, Long Documents
  - evaluating solely the final, Common Pitfalls
  - structured, Approach 2: Verbalized confidence for open-ended tasks (use with caution)
  - variable-size output tasks, Long Documents
overconfidence, Common Pitfalls
overfitting, Common Pitfalls
overlap strategies, Evaluating and Optimizing Chunking Strategies

P

pairwise comparisons, Comparative Evaluation
parameter-efficient fine-tuning (PEFT), High Effort: Model-Level Changes
parametric knowledge, Evaluating Generation Quality
pass rates (see success rates)
Pass@k, Optional: Group-wise Metrics for Evaluating Multiple Outputs
patterns, discovering, From Traces to Systemic Patterns, Navigating Trace Groups and Discovering Patterns
PDF documents, PDF Documents
PEFT (parameter-efficient fine-tuning), High Effort: Model-Level Changes
per-trace metrics, Beyond Clustering
percent agreement, Percent Agreement: Simple but Misleading, Using Only Percent Agreement
perplexity, Pretraining
persistent state, Conversational Coherence and Memory
persona
  - tone mismatch, Writing LLM-as-Judge Prompts
personas
  - accuracy optimization and, Low Effort: Prompt Refinement
  - collecting traces with, Collecting Traces
  - generating test conversations, User Simulators for Generating Test Conversations

  - misidentifying client personas, Common Pitfalls
  - tone mismatch, Defining the Right Metrics (What to Measure)

perturbation testing, Perturbation Testing

phone numbers, normalizing, Tools and Agents

position bias, Writing LLM-as-Judge Prompts

post-training stage, Post-training

Precision@k, Optional: Group-wise Metrics for Evaluating Multiple Outputs, Precision@k and Recall@k

predictability, Tools and Agents

preference fine-tuning, High Effort: Model-Level Changes

prelaunch evaluation, Introduction

prerequisite knowledge, Who Should Read This Book

pretraining stage, Pretraining

prevalence, of failure modes, Putting It Together

principles, A Note on the Pace of Change

privilege escalation, Adversarial and Security Evaluation

process reward models, Post-training

product metrics, Common Pitfalls

production and improvement

  - continuous integration and deployment (CI/CD), Continuous Integration and Deployment for LLM Agents-Putting It Together
  - data analysis for traces, Data Analysis for Traces-Putting It Together
  - human review, Interfaces for Human Review-Putting It Together
  - improving LLM agents, Improving LLM Agents-Choosing an Improvement Strategy

production logs, sampling traces from, Creating a Starting Dataset of Traces, Common Pitfalls

production quality, Common Mistakes with CI

prompt caching, Make Prompts Cache-Friendly

prompt injection attacks, Adversarial and Security Evaluation

prompts

  - agent loops, Agent Loops
  - chain-of-thought prompting, Low Effort: Prompt Refinement
  - conversational, Conversations
  - DocWrangler for refining, Case Study: DocWrangler for Prompt Refinement
  - essential components of, Writing LLM-as-Judge Prompts
  - fundamentals of, Fundamentals of Prompting
  - iterative prompt refinement, Iterative Prompt Refinement for the LLM-as-Judge

- for LLM-as-judge evaluators, Writing LLM-as-Judge Prompts-Data Splits for Designing and Validating LLM-as-Judge
- optimizing, High Effort: Model-Level Changes
- refining, Low Effort: Prompt Refinement
- retrieval, Retrieval, Retrieval Fundamentals
- revisiting when models improve, Revisiting Prompts When Models Improve
- single LLM calls, Single LLM Call, Prompts for Single LLM Calls
- templates for, Prompts for Single LLM Calls
- tool descriptions as, Why Evaluate Each Stage Separately
- tool-calling, Tools, Tools and Agents-Tools and Agents

proxies, Model Cascades
Pydantic, Stage 2: Argument Generation

Q

QLoRA technique, High Effort: Model-Level Changes
queries
- defined, Establishing Terminology
- generating, Generating Tuples and Queries
- similarity of, Why Evaluate Each Stage Separately
- synthetic queries, Creating a Starting Dataset of Traces, Synthetically Generating Query-Answer Pairs-Filtering Synthetic Questions

QueryGPT, Why Evaluate Each Stage Separately
quick overall ratings, Essential Interface Elements

R

RAG (see retrieval-augmented generation)
random sampling, Random Sampling
ranking, Comparative Evaluation
ratings, quick overall, Essential Interface Elements
ReAct loop, Tools and Agents
real estate assistant example, Error Analysis, Defining the Right Metrics (What to Measure), What Are Tools and Why Do LLMs Use Them?, CI: Building a Safety Net Against Regressions, The Case for Custom Review Interfaces, Medium Effort: Changing How the Agent Works, Match Model Size to Task Complexity
reasoning models (see chain-of-thought prompting)
recall, Overview
Recall@k, Optional: Group-wise Metrics for Evaluating Multiple Outputs, Precision@k and Recall@k

reciprocal rank fusion (RRF), Retrieval Fundamentals, Overview
reference-based versus reference-free evaluation, Absolute Evaluation, Defining the Right Metrics (What to Measure)
regressions, building safety net against, CI: Building a Safety Net Against Regressions-Revisiting Prompts When Models Improve
regular expressions (regex), Defining the Right Metrics (What to Measure), Putting It Together, Using AI Coding Assistants, Medium Effort: Changing How the Agent Works (see also code-based evaluators)
reinforcement learning with verifiable rewards (RLVR), Post-training
relevance, Evaluating Generation Quality
reranking strategies, Overview
retrieval correctness, Common Pitfalls
retrieval-augmented generation (RAG)
- agentic RAG, Overview
- approaches to retrieval, Retrieval
- common pitfalls, Common Pitfalls
- evaluating, Evaluating Retrieval-Augmented Generation-Overview
- evaluating and optimizing chunking strategies, Evaluating and Optimizing Chunking Strategies-Other Chunking Approaches
- evaluating generation quality, Evaluating Generation Quality
- hybrid retrieval, Overview
- metrics for retrieval quality, Metrics for Retrieval Quality-Normalized Discounted Cumulative Gain (NDCG@k)
- retrieval fundamentals, Retrieval Fundamentals
- synthetically generating query-answer pairs, Synthetically Generating Query-Answer Pairs-Filtering Synthetic Questions
- tuning, Medium Effort: Changing How the Agent Works, Cut Token Usage
- two-stage retrieval, Overview
- workflow for evaluating, Putting It Together
review interfaces, custom, The Case for Custom Review Interfaces-Features That Improve Review Speed and Quality, A Simple Custom UI-An Advanced Review Interface, Common Pitfalls (see also human review)
RLVR (reinforcement learning with verifiable rewards), Post-training
Rogan-Gladen correction, Putting It Together
role field, Components of LLM Agents
role-based guidance, Low Effort: Prompt Refinement
routing, Routing
RRF (reciprocal rank fusion), Retrieval Fundamentals, Overview
rubrics

- alignment sessions and, Facilitating Alignment Sessions-Save Everything
- and human review interfaces, Case Study: EvalGen
- benefits of clear, Percent Agreement: Simple but Misleading, Images
- consistent application of, Measuring Inter-Annotator Agreement
- developing, Missing data
- effects of underspecified, Common Pitfalls, Interpreting Kappa Scores
- and human review interfaces, The Case for Custom Review Interfaces
- pinpointing application of, Interpreting Kappa Scores
- rubric iteration, Interpreting Kappa Scores
- updating, Case Study: EvalGen
- used for evaluation, What Is Evaluation?, Collaborative Evaluation Practices
- used in data analysis, Using AI Coding Assistants
- workflow for applying, A Collaborative Annotation Workflow-A Collaborative Annotation Workflow

rule-based evaluators, Common Pitfalls

runtime errors, Stage 3: Tool Execution

S

sampling
- failure-driven, Failure-Driven Sampling
- overrelying on evaluator disagreement for, Common Pitfalls
- random versus uncertainty, Random Sampling
- traces from production logs, Creating a Starting Dataset of Traces

saturation, Creating a Starting Dataset of Traces

schema violations, Stage 2: Argument Generation

search
- hybrid search, Retrieval Fundamentals
- tools for search and similarity, Search and Similarity Tools, Semantic Search to Find Related Traces

security evaluation, Adversarial and Security Evaluation

self-consistency, Medium Effort: Changing How the Agent Works

self-enhancement bias, Writing LLM-as-Judge Prompts

semantic closeness/similarity, Optional: Group-wise Metrics for Evaluating Multiple Outputs, Search and Similarity Tools, Semantic Search to Find Related Traces

semantic consistency, Stage 4: Output Handling

sessions
- defined, Evaluating Multi-Turn Conversations
- evaluating at session level, Session Level

  - in online monitoring, Observability
  - operationalizing session-level success, Operationalizing Session-Level Success
similarity
  - semantic closeness/similarity, Optional: Group-wise Metrics for Evaluating Multiple Outputs, Semantic Search to Find Related Traces
  - tools to discover, Search and Similarity Tools
single-turn interactions, Evaluating Multi-Turn Conversations, Common Pitfalls
smoke tests, Common Mistakes with CI
spans, in online monitoring, Observability
spatial reasoning, Images
specialized subagents, Why Evaluate Each Stage Separately
specification, The Gulf of Specification (Between the Developer and the LLM Agent), Putting It Together
spectrum of agency, The Spectrum of Agency
spreadsheets, The Case for Custom Review Interfaces, Starting with a Spreadsheet
stage-by-stage evaluation, Why Evaluate Each Stage Separately
staleness, Conversational Coherence and Memory, Common Pitfalls
states, From Traces to Systemic Patterns
step-by-step reasoning, Post-training, Low Effort: Prompt Refinement
Streamlit, The Case for Custom Review Interfaces
structured output, Approach 2: Verbalized confidence for open-ended tasks (use with caution)
subagents, Why Evaluate Each Stage Separately
success rates, estimating, Estimating True Success Rates with Imperfect Judges-Python Code for Estimating Success Rates
Success@k, Optional: Group-wise Metrics for Evaluating Multiple Outputs, Mean Reciprocal Rank
supplemental material, obtaining, Common Pitfalls, Next Steps
synthetic queries, Creating a Starting Dataset of Traces
synthetic question-answer-context triples, Synthetically Generating Query-Answer Pairs
synthetic-annotator collusion, Synthetic-Annotator Collusion
system role, Components of LLM Agents
systematic bias, Writing LLM-as-Judge Prompts
systemic patterns, discovering, From Traces to Systemic Patterns, Navigating Trace Groups and Discovering Patterns

T

table agent, Why Evaluate Each Stage Separately
target accuracy, Model Cascades
target audiences, Who Should Read This Book
tasks
- breaking into smaller steps, Medium Effort: Changing How the Agent Works
- classification tasks, Model Cascades
- completion rates, Common Pitfalls
- constant output tasks, Long Documents
- model size matched to complexity of, Match Model Size to Task Complexity
- open-ended tasks, Approach 2: Verbalized confidence for open-ended tasks (use with caution)

techniques, A Note on the Pace of Change
temperature parameter, Single LLM Call
templates, for prompting, Prompts for Single LLM Calls
temporal drift, Ignoring Temporal Drift
test conversations, User Simulators for Generating Test Conversations
test sets, Data Splits for Designing and Validating LLM-as-Judge
text, failure or difficulty recognizing, Images
threat categories, for tool-using agents, Adversarial and Security Evaluation
Three Gulfs model, The Three Gulfs of LLM Application Development-Why LLM Agent Evaluation Is Challenging
tokens
- decreasing usage of, Cut Token Usage
- definition of, Pretraining

tool role, Components of LLM Agents
tool selection errors, Evaluating Tool Calls
tools
- adversarial and security evaluation, Adversarial and Security Evaluation
- agency and, The Spectrum of Agency
- common evaluation pitfalls, Common Pitfalls
- debugging multi-step traces, Debugging Multi-Step Traces-Evolving the Heatmap over Time
- definition of, Tools, What Are Tools and Why Do LLMs Use Them?
- effect of names, descriptions, and parameter specifications, Why Evaluate Each Stage Separately
- evaluating tool calls, Evaluating Tool Calls-Why Evaluate Each Stage Separately

- evaluating tool use, Evaluating Tool Use and Complex Agents
- fixing incorrect calls for, Medium Effort: Changing How the Agent Works
- four stages of tool calls, Evaluating Tool Calls
- handling complex input data, Handling Complex Input Data-PDF Documents
- Model Context Protocol (MCP), Tools, Why Evaluate Each Stage Separately
- search and similarity tools, Search and Similarity Tools
- threat categories for tool-using agents, Adversarial and Security Evaluation
- tool calling cycle, What Are Tools and Why Do LLMs Use Them?
- tool sequencing, Common Pitfalls
- use by LLMs, What Are Tools and Why Do LLMs Use Them?
- workflow for tool-using agent evaluation, Putting It Together

topics covered, What This Book Covers

TPR/TNR (true positive/true negative rates)
- correcting raw predictions with, Estimating True Success Rates with Imperfect Judges
- iterative prompt refinement, Iterative Prompt Refinement for the LLM-as-Judge
- versus precision and recall, Iterative Prompt Refinement for the LLM-as-Judge

traceability, Traceability

traces
- collecting for multi-turn evaluation, Collecting Traces
- creating a starting dataset of, Creating a Starting Dataset of Traces-Generating Tuples and Queries, Putting It Together
- custom review interfaces for, The Case for Custom Review Interfaces-Features That Improve Review Speed and Quality, Common Pitfalls
- data analysis for, Data Analysis for Traces-Putting It Together
- debugging multi-step, Debugging Multi-Step Traces-Evolving the Heatmap over Time
- defined, Establishing Terminology
- failure to examine closely, Common Pitfalls
- labeling with structured failure modes, Labeling Traces with Structured Failure Modes
- navigating trace groups and discovering patterns, Navigating Trace Groups and Discovering Patterns
- in online monitoring, Observability
- reading and labeling, Open Coding: Reading and Labeling Traces-Open Coding: Reading and Labeling Traces
- selecting for human review, Selecting Traces for Human Review
- structuring failure modes, Axial Coding: Structuring Failure Modes

training sets, Data Splits for Designing and Validating LLM-as-Judge
Transformer architecture, Make Prompts Cache-Friendly
transition failure heatmaps, The Transition Failure Heatmap
true success rates, estimating, Estimating True Success Rates with Imperfect Judges-Python Code for Estimating Success Rates
tuples, generating, Generating Tuples and Queries
turns
  – defined, Evaluating Multi-Turn Conversations
  – evaluating at turn-level, Turn Level
two-stage retrieval, Overview
type violations, Stage 2: Argument Generation

U

Uber, Why Evaluate Each Stage Separately
uncertainty sampling, Uncertainty Sampling, Common Pitfalls
uncertainty, quantifying, Step 4: Quantify Uncertainty with a Bootstrap
unimodal metrics, Beyond Clustering
usability heuristics, The Case for Custom Review Interfaces
user role, Components of LLM Agents
user satisfaction, Common Pitfalls
user simulators, User Simulators for Generating Test Conversations

V

values, incorrect, Stage 2: Argument Generation
variable-size output tasks, Long Documents
verbalized confidence calibrations, Approach 2: Verbalized confidence for open-ended tasks (use with caution)-Tuning the cascade threshold
verbosity bias, Writing LLM-as-Judge Prompts
verification, Medium Effort: Changing How the Agent Works
vision-language models (VLMs), Images
Voiceflow, Revisiting Prompts When Models Improve

W

W&B Weave, Observability, The Case for Custom Review Interfaces
whitespace, Make Prompts Cache-Friendly

About the Authors 关于作者

Shreya Shankar is an incoming assistant professor in the computer science department at Carnegie Mellon. Her research focuses on effectively integrating LLMs into database systems and business intelligence tools. Some of the open source systems she built include DocETL, a system for LLM-powered document processing, and Quail, an execution engine for AI functions in SQL. Her work has received paper awards at multiple conferences (SIGMOD, CHI, UIST) and runs in production environments.

Shreya Shankar 是卡内基梅隆大学计算机科学系的助理教授。她的研究重点是将大语言模型有效地整合到数据库系统和商业智能工具中。她开发的一些开源系统包括 DocETL——一种基于大语言模型文档处理系统，以及 Quail——一种用于 SQL 中的人工智能功能的执行引擎。她的研究成果已在多个会议上获得奖项（如 SIGMOD、CHI、UIST 等），并且这些系统已经在实际环境中运行。

Hamel Husain is an ML engineer with 20 years of experience, including roles at innovative companies like Airbnb and GitHub where he contributed to early LLM research used by OpenAI and led popular open source ML tools. Currently an independent consultant, Hamel has extensive hands-on experience helping numerous companies (35+ implementations cited) build and evaluate real-world AI products using the techniques described in this book.

Hamel Husain 是一名拥有 20 年经验的机器学习工程师，曾在 Airbnb 和 GitHub 等创新公司工作，期间参与了 OpenAI 所开发的先进语言模型研究，并主导了多项流行的开源机器学习工具的开发。目前作为独立顾问，Hamel 拥有丰富的实践经验，曾帮助众多公司构建并评估实际应用的 AI 产品，这些公司的数量超过 35 家。这些经验都基于本书中描述的技术。

Shreya and Hamel teach a live course based on the material in this book.

Shreya 和 Hamel 根据本书的内容教授一门在线课程。